

药材的烘干问题

摘 要

本文把药材热风烘干抽象为圆柱体内热质耦合扩散与表面对流交换的初边值问题：由能量与水分守恒律导出一维径向变系数抛物方程组，中心取对称条件，表面取题给参数确定的 Robin 条件 $-k \partial_r T = h(T - T_{\text{air}})$ 与 $-D \partial_r C = h_m(C - C_{\text{air}})$ ，烘房状态由附件 1 逐点插值给出。四个问题共用同一套模型与程序，差别只在物性公式、时间范围与边界是否移动。数值上采用顶点型有限体积离散（ r 加权控制体、界面状态法计算非线性系数、Robin 通量直接施加于表面半控制体）配合稀疏雅可比自适应 BDF 积分，并在每个数据折点重启积分器；生产网格 $\Delta r = 0.0031 \text{ cm}$ （641 节点，与 0.1 cm 输出网格重合：问题 1–3 无插值，问题 4 的固定物理位置由 $\xi = r/R(t)$ 网格上的三次样条取得），容差 $\text{rtol} = 1 \times 10^{-9}$ 、 $\text{atol} = 1 \times 10^{-11}$ 。

问题一（预热平衡阶段，附录 2 物性）：1800 s 时中心温度 33.5753°C 、表面 36.7856°C （比烘房低 4.727 K ），中心水分浓度仍为 2.5500 kg/kg 、表面降至 1.5102 kg/kg ，体积平均失水 10.06% 。这一“温度已穿透、水分仅在表层”的格局由热、质傅里叶数 0.76 与 0.022 （扩散长度仅 0.30 cm ）解释。

问题二（全过程，附录 3 变物性）：3 h 时中心温度 49.8495°C 、中心水分 1.7662 kg/kg 、表面 1.0081 kg/kg ，平均失水 45.78% ；中心温度在 2.27 h 后与烘房相差不足 1 K ，此后扩散系数由含水率单独控制。

问题三：以“各处水分浓度低于 0.15 kg/kg ”即 $\max_r C(r, t) < 0.15$ 为判据、用事件求根精确定位，得烘干时间 57.48 h （生产网格值；网格收敛外推 57.4737 h ，离散不确定度 0.41 min ，故两位小数应报为 $57.47\text{--}57.48 \text{ h}$ ，即 2.40 天 ），落在题述“2–3 天”之内；此时表面水分已降至 0.0526 kg/kg ，中心成为控制点。

问题四（尺寸变化）：按干物质守恒在 Landau 物质坐标下求解收缩域问题（附件 2 的 $R(t)$ 用保单调 PCHIP 插值），烘干时长 51.09 h ；与同物性、半径固定 2 cm 的 129.85 h 相比缩短为 $1/2.54$ ，与扩散时间尺度之比 $(R_0/R_\infty)^2 = 2.79$ 相符，说明收缩通过缩短扩散路径主导了加速。

模型检验（校验器与求解器代码分离）：问题 1 的温度场用 Duhamel 叠加求得真解，在 `result 1.xlsx` 的全部 37800 个输出单元上与数值解最大相差 $2.0 \times 10^{-6}^\circ\text{C}$ ；与常物性 Robin 圆柱贝塞尔级数解的最大误差为 $1.0 \times 10^{-5}^\circ\text{C}$ 与 $1.1 \times 10^{-5} \text{ kg/kg}$ 。用不受参照偏置影响的三网格估计量，问题 1、2 的空间观测阶为 $1.99\text{--}2.00$ ，问题 3、4 的烘干终点量为 $1.67\text{--}1.91$ 。在结果文件的完整 1 s 输出网格上逐格评估离散误差：温度无一单元超过四位小数的半个末位，水分仅 42（问题 1， $t \leq 42 \text{ s}$ ）与 29（问题 2， $t \leq 29 \text{ s}$ ）个表面单元超过。离散水分平衡（常物性下还有能量平衡）是半离散系统的精确线性不变量（残差 $\sim 10^{-15}$ ）；极值原理与径向单调性全程成立；更严格容差、Radau 与限制最大步长的解相差不超过 $4.2 \times 10^{-9} \text{ kg/kg}$ ；独立编写的二维轴对称程序在端面绝热时与一维解相差 $6.1 \times 10^{-9} \text{ kg/kg}$ ，端部效应仅 -0.004% 。灵敏度给出烘干时间对烘房温度平台、扩散前因子与收缩幅度的弹性 -1.603 、 -0.893 、 -1.130 ，而对换热系数几乎不敏感（ -0.001 ）——过程由内部扩散控制。解释性选择中最大的两项是水分守恒的写法（ -18.71% ）与判据表述（ -37.28% ），均已显式声明并定量；模型评价一节以湿球能量上限与平衡含水率驱动力界定了题给模型省略的蒸发潜热与吸附平衡的影响范围（ $+17.3\%\text{--}26.3\%$ ）。

关键词：热质耦合扩散；对流边界条件；有限体积法；移动边界与 Landau 变换；验证与确认（V&V）；灵敏度分析

1 问题重述

1.1 问题背景与数据

干燥是决定中药成品品质的关键工序，热风烘干通过调控烘房温湿环境使药材内部水分向表面迁移并被气流带走，过程包含**预热平衡**与**恒温干燥**两个阶段。待干燥药材近似为圆柱体，长 $L = 25\text{ cm}$ 、初始半径 $R_0 = 2\text{ cm}$ ；初始时刻温度均匀为 $T_0 = 28^\circ\text{C}$ 、水分浓度（干基含水率）均匀为 $C_0 = 2.55\text{ kg/kg}$ 。烘房温度 $T_{\text{air}}(t)$ 与水分浓度 $C_{\text{air}}(t)$ 由附件 1 给出（0–14400 s，每 60 s 一个样本，共 241 个）；药材半径 $R(t)$ 由附件 2 给出（0–72 h，每 1800 s 一个样本，共 145 个，由 2.000 cm 单调减至 1.198 cm）。密度 ρ 、比热容 c_p 、热传导系数 k 与水分浓度扩散系数 D 由附录 2–4 的经验公式给出，其中 D 随含水率 C （问题 2–4 中还随温度 T ）呈指数变化。

1.2 待解决的问题

问题 1（预热平衡阶段，附录 2 常物性 + $D(C)$ ）：建立 0–1800 s 内温度与水分浓度的变化模型，按表 2、表 3 的格式给出 $t \in \{100, \dots, 1800\}\text{ s}$ 、 $r \in \{0, 0.5, 1, 1.5, 2\}\text{ cm}$ 处的结果，完整结果（每 1 s、每 0.1 cm）写入 result1.xlsx。**问题 2**（整个烘干过程，统一采用附录 3 的变物性公式）：给出 3 h 内每 0.5 h 的结果（表 4、表 5），完整结果写入 result2.xlsx。**问题 3**（烘干时长）：按“药材各处水分浓度低于 0.15 kg/kg ”确定烘干所需时间，给出每 6 h、每 0.5 cm 的水分浓度（表 6，含“烘干结束时间”行），完整结果（每 60 s、每 0.1 cm）写入 result3.xlsx。**问题 4**（尺寸变化）：按附件 2 的 $R(t)$ 与附录 4 的物性确定烘干时长，按表 7 的格式给出结果（含“药材表面”列），完整结果写入 result4.xlsx。

所有结果保留四位小数；结果文件的 A 列为时间（s），第 1 行为到药材中心的距离（cm）。本文的结果文件均由附件 3 的模板生成，格式与模板逐格一致（第 8 节与附录 D）。

1.3 题目中需要解释的五处不确定性

题目给出了全部物性与传递参数，但未写出控制方程与边界条件的具体形式，也未规定若干实现细节。本文对以下五处作出明确解释，并对每一处给出备选方案的定量对照（第 8.9 小节，表 9）：

- 边界条件的形式**：把“对流换热系数 h ”与“对流传质系数 h_m ”按其量纲唯一地解释为表面 Robin 条件的系数，即 $-k \partial_r T = h(T_s - T_{\text{air}})$ 与 $-D \partial_r C = h_m(C_s - C_{\text{air}})$ 。这也使题给参数恰好构成一个封闭的定解问题（见假设 4）。
- 附件 1 之外的烘房状态**：问题 3、4 的烘干持续数十小时，超出附件 1 的 4 h 覆盖范围。本文在样本之间线性插值，在 14400 s 之后取恒温干燥阶段的平台值 $T_{\text{air}} = 49.999^\circ\text{C}$ 、 $C_{\text{air}} = 0.04999\text{ kg/kg}$ （末 1 h 样本均值）。
- 收缩域上方程的写法**：附件 2 的 $R(t)$ 只给出几何，未说明被边界“扫过”的物质如何处理。本文以干物质守恒的物质（Lagrangian）坐标为主模型，并给出实验室坐标（Eulerian）写法的对照结果。

- (d) **水分守恒的写法**：题目给出“水分浓度扩散系数” D 却未写出水分方程。本文取干基 Fick 形式 $\partial_t C = \nabla \cdot (D \nabla C)$ ；它在固定体积、干物质守恒的前提下**就是**守恒律本身（第 4.1 小节）。若改把附录给出的 $\rho(C)$ 读作真实湿密度、从而令干物质表观密度 $\rho_s = \rho/(1+C)$ 随 C 变化，则应取守恒型写法 $\partial_t(\rho_s C) = \nabla \cdot (\rho_s D \nabla C)$ ，烘干时间变为 46.73 h（-18.71%，MDR-0010）——这是除判据表述外最大的单项解释不确定性。
- (e) **result2.xlsx 的时间范围**：问题 2 要求“整个烘干过程”的模型，但在“完整结果”前未像问题 1 那样写出时间上限。本文按与表 3、表 4 一致的 3 h 每 1 s 交付 result2.xlsx，并**另行交付**result2_全过程.xlsx（同一模板、温度与水分浓度两张表、覆盖 $0-t_{\text{end}}$ 、分辨率 60 s），使两种读法都有对应的交付物（第 5.1 小节）。

2 问题分析

2.1 问题的共同结构

四个问题问的是同一个物理过程在不同参数体系与不同时间尺度上的表现，因此不应当分别建立四个模型，而应当建立一个带参数的初边值问题，用同一套离散与积分方法求解，四个问题只在下列三处不同：

问题	物性公式	时间范围与输出	计算域
1	附录 2（常物性 + $D(C)$ ）	0–1800 s，每 1 s	固定， $r \in [0, R_0]$
2	附录 3（ ρ, c_p, k 随 C ； $D(C, T)$ ）	0–3 h，每 1 s	固定
3	附录 3	0– t_{end} ，每 60 s	固定
4	附录 4	0– t_{end} ，每 60 s	收缩， $r \in [0, R(t)]$

2.2 关键困难与对策

- 三个量级的时间尺度分离**。以附录 2 的参数计，热扩散时间尺度 $R_0^2/\alpha = 2369$ s，而水分扩散时间尺度 $R_0^2/D(C_0) = 22.5$ h，两者相差 34 倍；温度与水分又通过 $D(C, T)$ 强耦合。这使方程组**刚性**：显式格式的**稳定步长**由温度方程决定（ $\Delta t \lesssim \Delta r^2/2\alpha$ ，在生产网格上约 3×10^{-3} s），而需要积分的总时长达 206929.7 s。对策是全隐式、变阶变步长的 BDF 方法（第 4.4 小节）。
- 扩散系数跨越三到四个数量级**。 $D \propto e^{-a/C}$ 在干燥后期急剧变小：以附录 3 计， D 由初始（28°C、 C_0 ）的 5.642×10^{-9} m²/s 先随升温增大到过程最大值 1.35×10^{-8} m²/s（恒温平台、 C_0 ），再降到烘干结束时表面的 3.1×10^{-12} m²/s，最大值与末值之比为 4320（图 4）。表面因此形成低扩散率的“硬壳”，同一参数体系下的传质 Biot 数 $h_m R_0/D$ 由初始的 2.84 增至约 5131，过程由**内部扩散**而非外部对流控制——这一判断在灵敏度分析中被定量证实（弹性 -0.001 对 h 、-0.893 对 D_0 ）。
- 边界数据是折线与样条**。附件 1 每 60 s 一个样本，线性插值后斜率在每个样本处跳变；附件 2 的 PCHIP 插值在每个节点处曲率跳变。多步法跨过折点时的误差不受容差控制（这一现象在初期实验中被观测到，见 MDR-0004 补充），对策是在**每个数据折点重启积分器**，段内强迫项光滑。

4. “各处低于 0.15”是一个停止时刻问题。判据定义的时刻不落在输出网格上。对策是把 $g(t) = \max_i C_i(t) - 0.15$ 作为积分器的事件函数精确求根，并同时报告 60 s 网格上的值以与结果文件对照。
5. 问题 4 是移动边界问题。计算域随 $R(t)$ 收缩，而附件 2 只给几何、未给物理原则。本文以干物质守恒的物质坐标 (Lagrangian) 为主模型，实验室坐标写法作为定量对照 (第 7.1 小节)。
6. 四位小数的精度要求。离散误差必须远小于 5×10^{-5} 。对策是把输出点取在网格节点上 ($\Delta r = 0.1/2^k$ cm)，并在结果文件的完整输出网格上逐格评估误差 (第 8.2 小节)。

2.3 技术路线

图 1 给出总体技术路线。所有计算在同一云端运行 (run) 内完成，每个阶段的输入输出散列、参数与环境写入清单，论文中的每一个数字都追溯到某个阶段的产物 (附录 D)。

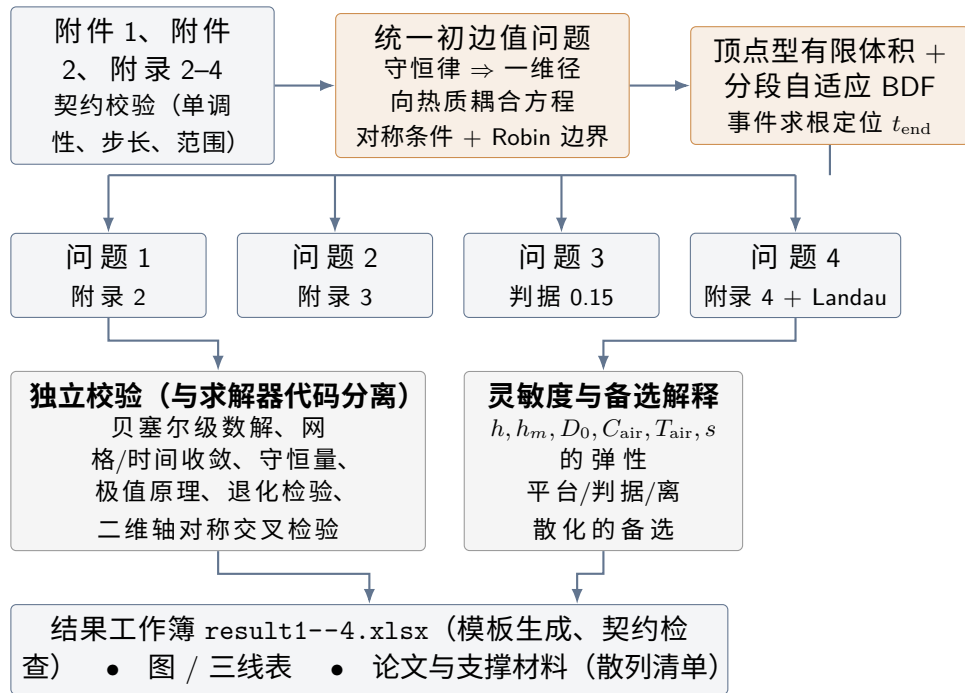


图 1 技术路线：由守恒律建立统一的一维径向热质耦合初边值问题，四个问题从同一条离散与积分总线分出，只改变物性、时间范围与边界是否移动；每一组解都经与求解器分离的独立校验，并给出灵敏度与备选解释。

3 模型假设与符号说明

3.1 模型假设

下列假设分为两类：**题给假设**直接来自题目与附录，**附加假设**是本文为闭合模型而引入的。每条给出依据，并说明被违反时结果会向哪个方向偏移；其中可量化的偏差在第 8 节或第 10 节给出。

假设 1 (圆柱几何与一维径向化, 附加). 药材为半径 R 、长 L 的正圆柱, 温度与水分浓度只依赖到轴心的距离 r 与时间 t , 即 $T = T(r, t)$ 、 $C = C(r, t)$ 。依据: 长径比 $L/R_0 = 12.5$; 在整个烘干时长内, 水分的轴向傅里叶数上界为 $D_{\max}t_{\text{end}}/(L/2)^2 = 0.18 < 1$, 端面影响不能扩散到中截面; 热的轴向傅里叶数虽为 1.92, 但药材温度在 3 h 内即与烘房接近一致 (问题 2), 此后不存在可被端面放大的轴向温差。验证: 独立编写的二维轴对称程序给出端部效应对烘干时间的影响为 -0.004% (第 8.6 小节), 故一维化带来的偏差可忽略。

假设 2 (连续介质、各向同性与均匀初值, 题给 + 附加). 药材是各向同性的连续介质, 物性只通过局部的 C 与 T 依赖于位置; 初始时刻 $T \equiv T_0 = 28^\circ\text{C}$ 、 $C \equiv C_0 = 2.55 \text{ kg/kg}$ (干基含水率)。依据: 题给经验公式均为 C 、 T 的显函数, 未区分方向。后果: 真实药材 (纤维结构) 的轴向扩散通常快于径向, 忽略各向异性会略微高估烘干时间。

假设 3 (内部水分迁移服从 Fick 扩散, 题给). 药材内部的水分通量 (相对于固体骨架) 为 $j = -\rho_s D(C, T) \nabla C$, ρ_s 为干物质表观密度; 液态水与水蒸气的迁移合并为一个有效扩散系数 D 。依据: 题目给出的正是“水分浓度扩散系数”的经验公式, 其指数形式 $D = D_0 e^{-a/C} e^{-E/T_K}$ 是干燥文献中有效扩散率的常用拟合形式 [1-2]。后果: 不区分毛细流动与蒸汽扩散, 也不含总压梯度驱动的渗流 [3]。

假设 4 (表面为对流型 Robin 条件, 传递系数恒定, 题给 + 附加). 在表面 $r = R(t)$ 处

$$-k \partial_r T = h (T_s - T_{\text{air}}(t)), \quad -D \partial_r C = h_m (C_s - C_{\text{air}}(t)),$$

其中 $h = 25 \text{ W}/(\text{m}^2 \text{ K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 在四个问题中一律沿用 (附录 3、4 未另给 h 、 h_m)。依据: h 、 h_m 的量纲唯一地确定了上述形式; h 、 h_m 刻画烘房气流与表面之间的外部传递, 不属于附录 3、4 所描述的药材内部物性 (MDR-0002)。后果: h_m 的取值误差对烘干时间的弹性为 -0.114 , h 的为 -0.001 (表 18), 故该假设不是结果的敏感来源。

假设 5 (烘房状态空间均匀、时间上分段线性并在恒温阶段取平台, 附加). $T_{\text{air}}(t)$ 、 $C_{\text{air}}(t)$ 在药材表面处处相同; 在附件 1 的样本之间线性插值; $t > 14400 \text{ s}$ 后取恒温干燥阶段的平台值 (末 1 h 样本均值) $T_{\text{air}} = 49.999^\circ\text{C}$ 、 $C_{\text{air}} = 0.04999 \text{ kg/kg}$ 。依据: 附件 1 的温度在约 2 h 后围绕 50°C 波动 (末 1 h 内取值范围 $49.76\text{--}50.24^\circ\text{C}$ 、标准差 0.15 K 、相邻样本最大差 0.45 K , 为测量噪声量级), “恒温干燥阶段”的物理含义是烘房保持设定值, 取末 1 h 均值是对设定值的无偏且抗单点噪声的估计 (MDR-0003)。注意平台值与末样本相差 0.17 K , 故 T_{air} 在 14400 s 处有一处 0.17 K 的**跳跃** (积分器在该时刻重启, 见第 4.4 小节)。备选: 取末样本值或名义值 $50^\circ\text{C}/0.05 \text{ kg/kg}$ 的结果见表 9 (相差 -0.53% 以内, 其中“取末样本”一行同时消去了该跳跃)。

假设 6 (能量方程不含蒸发潜热源, 题给). 能量方程只含导热项, 表面只通过对流换热与烘房交换热量, 不扣除表面蒸发所需的潜热。依据: 题目给出的参数体系中没有潜热、蒸气压或平衡含水率, 表面传质律 $h_m(C_s - C_{\text{air}})$ 也不含湿球反馈; 强行加入潜热项会使该传质律给出的蒸发速率把表面温度压到 0°C 以下 (不适定, MDR-0008)。后果: 前十余小时药材温度被高估, 因而 D 被高估、烘干时间被低估。第 10 节以“潜热等效温降”作后验诊断 (峰值 54 K , 降到 1 K 以下需 16.1 h), 并以湿球能量上限给出界定性结果 ($+17.3\%$)。

假设 7 (问题 1–3 体积不变; 问题 4 径向均匀收缩 (轴向长度不变) 且单位长度干物质守恒, 题给 + 附加). 问题 1–3 中 $R \equiv R_0$; 问题 4 中半径按附件 2 的 $R(t)$ 变化 (保单调三次 Hermite 插值 PCHIP[4], 72 h 之后保持末值 1.198 cm), 收缩只发生在径向且是均匀的: 每个物质点的径向坐标与 $R(t)$ 成比例, 轴向长度 L 不变, 故单位长度的干物质质量守恒 ($\rho_s R^2$ 与 t 无关)。依据: 题目只给出径向的 $R(t)$, 未给出 L 的变化; 均匀收缩是干燥文献处理收缩的标准出发点 [5]。若改设各向同性收缩 ($L \propto R$), 守恒式应为 $\rho_s R^3 = \text{常数}$, 方程中的 $1/R^2$ 因子与守恒律都要改写, 烘干时间会进一步缩短。与数据的相容性: 题给的 $R(t)$ 与题给的 $\rho(C)$ 在干物质守恒下并不严格相容 (按 $\rho_s R^2$ 计相差 13.4%, 第 7.1 小节), 本文以附件 2 的几何为准。备选: 实验室坐标写法 (物质被边界扫出计算域) 的结果相差 3.06% (表 9)。

假设 8 (忽略辐射、收缩功与内部对流, 附加). 不计烘房壁面与药材之间的辐射换热、收缩过程的机械功与孔隙内的对流。依据: 题目只给出对流换热系数; 50 °C 量级下线性化辐射系数 $h_r = 4\varepsilon\sigma T_K^3$ 只有 h 的四分之一量级, 而烘干时间对 h 的弹性仅 -0.001 , 故影响可忽略。

假设 9 (烘干结束判据, 题给). 烘干结束时刻为 $t_{\text{end}} = \inf\{t > 0 : \max_{0 \leq r \leq R(t)} C(r, t) < C^*\}$, $C^* = 0.15 \text{ kg/kg}$ 。依据: 题目要求 “各处的水分浓度应低于 0.15 kg/kg”。备选: 若按体积平均含水率判断, 烘干时间为 36.05 h (-37.28% , 表 9) ——判据的选择比任何参数扰动都更显著地改变答案。

3.2 符号说明

表 1 列出全文使用的主要符号; 其余符号在首次出现处定义。除特别说明外一律使用国际单位制, 含水率为干基 (kg 水/kg 干物质)。

表 1 全文使用的主要符号、含义、单位与取值来源

符号	含义	单位	取值/来源
t	时间	s	—
r	到药材中心 (轴) 的距离	m	$0 \leq r \leq R(t)$
$R(t)$	药材半径; $R_0 = R(0)$, R_∞ 为末值	m	附件 2; $R_0 = 0.02 \text{ m}$
L	圆柱长度	m	0.25 m
ξ	无量纲径向坐标 $r/R(t)$ (Landau 坐标)	—	$\xi \in [0, 1]$
T	药材温度; $T_s = T(R, t)$ 为表面温度	°C	$T(r, 0) = 28 \text{ °C}$
C	药材水分浓度 (干基含水率); $C_s = C(R, t)$	kg/kg	$C(r, 0) = 2.55$
$\bar{C}$	体积平均水分浓度	kg/kg	$\frac{2}{R^2} \int_0^R C r \text{ d}r$
$T_{\text{air}}, C_{\text{air}}$	烘房温度与水分浓度	°C, kg/kg	附件 1 + 平台值
ρ, c_p, k	密度、比热容、热传导系数	kg/m ³ , J/(kg K), W/(m K)	附录 2–4
D	水分浓度扩散系数 $D_0 e^{-a/C} e^{-E/T_K}$	m ² /s	附录 2–4
α	热扩散率 $k/(\rho c_p)$	m ² /s	1.689×10^{-7} (附录 2)
h, h_m	对流换热系数、对流传质系数	W/(m ² K), m/s	$25, 8 \times 10^{-7}$
Bi, Bi_m	热、质 Biot 数 $hR_0/k, h_m R_0/D$	—	1.389, 3.240
Fo, Fo_m	热、质 Fourier 数 $\alpha t/R_0^2, Dt/R_0^2$	—	第 4.2 小节
C^*	烘干结束的水分浓度阈值	kg/kg	0.15

表 1 (续)

符号	含义	单位	取值/来源
t_{end}	烘干结束时间	s (报告为 h)	求解结果
$n, \Delta\xi$	径向控制体个数、无量纲步长 $1/n$	—	节点数 641
Δr	径向网格步长 R_0/n	cm	0.0031
ρ_s	干物质表观密度 $\rho/(1+C)$	kg/m ³	注 4.1
s	收缩幅度: $R(t; s) = R_0 - s[R_0 - R_{\text{附件 2}}(t)]$	—	$s = 1$ 为题给数据
L_v, ρ_{s0}	汽化潜热、按初态折算的干物质表观密度	J/kg, kg/m ³	第 10 节
$T_{\text{wb}}, C_{\text{eq}}$	烘房湿球温度、平衡含水率	°C, kg/kg	第 10 节

4 问题一的模型建立与求解

本节从守恒律出发建立四个问题共用的控制方程，给出离散格式及其守恒与极值性质，并给出问题 1（预热平衡阶段、附录 2 物性）的结果。

4.1 由守恒律到一维径向方程

水分。 以干物质骨架为参照系。记干物质表观密度为 ρ_s (kg 干物质每 m³ 湿物料)，干基含水率为 C ，则单位体积含水量为 $\rho_s C$ 。由假设 3，水分相对骨架的质量通量为 $\mathbf{j} = -\rho_s D(C, T) \nabla C$ 。在任一固定物质区域 Ω 上作水分衡算并用散度定理，得 $\partial_t(\rho_s C) + \nabla \cdot \mathbf{j} = 0$ 。问题 1–3 的计算域体积固定（假设 7），而干物质既不产生也不迁移，故单位体积干物质质量 ρ_s 与时间无关，可从两端约去，得（干燥过程中的热质迁移理论见文献 [6, 2]）

$$\frac{\partial C}{\partial t} = \nabla \cdot (D(C, T) \nabla C). \quad (1)$$

注 4.1 ($\rho(C)$ 的读法与守恒型写法). 式 (1) 的前提是 ρ_s 在固定域上为常数，这在“体积不变 + 干物质守恒”下是定理而非假设。需要说明的是，若把附录 3、4 给出的 $\rho(C)$ 读作**真实湿密度**，则 $\rho_s(C) = \rho(C)/(1+C)$ 随 C 变化（附录 3：由 275 增到 582 kg/m³），与“体积不变”互相矛盾——除非干物质本身在变，而这是不可能的。本文因此把 $\rho(C)$ 理解为用于**体积热容 ρc_p 的经验相关式**（能量方程中正是这样使用），水分方程取(1)。作为解释不确定性，若坚持前一种读法而采用守恒型写法 $\partial_t(\rho_s C) = \nabla \cdot (\rho_s D \nabla C)$ （表面通量相应为 $\rho_s h_m (C_s - C_{\text{air}})$ ，贮存系数 $\sigma = d(\rho_s C)/dC$ ），问题 3 的烘干时间变为 46.73 h (−18.71%，表 9，MDR-0010)。

能量。 忽略蒸发潜热源与对流焓输运（假设 6、8），以 Fourier 定律 $\mathbf{q} = -k \nabla T$ 作能量衡算，得

$$\rho(C) c_p(C) \frac{\partial T}{\partial t} = \nabla \cdot (k(C) \nabla T). \quad (2)$$

一维径向化。 把三维方程(1)、(2) 约化到柱坐标: 由假设 1, 场量只依赖 (r, t) , $\nabla \cdot (a \nabla u) = \frac{1}{r} \frac{\partial}{\partial r} (a r \frac{\partial u}{\partial r})$, 于是得到本文的**控制方程**: 对 $0 < r < R(t)$ 、 $t > 0$,

$$\rho(C)c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(k(C) r \frac{\partial T}{\partial r} \right), \quad (3)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(D(C, T) r \frac{\partial C}{\partial r} \right), \quad (4)$$

配以中心对称条件、表面 Robin 条件 (假设 4) 与均匀初值 (假设 2):

$$\frac{\partial T}{\partial r} \Big|_{r=0} = \frac{\partial C}{\partial r} \Big|_{r=0} = 0, \quad (5)$$

$$-k \frac{\partial T}{\partial r} \Big|_{r=R} = h (T_s - T_{\text{air}}(t)), \quad -D \frac{\partial C}{\partial r} \Big|_{r=R} = h_m (C_s - C_{\text{air}}(t)), \quad (6)$$

$$T(r, 0) = T_0 = 28^\circ\text{C}, \quad C(r, 0) = C_0 = 2.55 \text{ kg/kg}. \quad (7)$$

量纲检查: (3) 两端为 W/m^3 , (4) 两端为 $(\text{kg/kg})\text{s}^{-1}$, (6) 两端分别为 W/m^2 与 $(\text{kg/kg})\text{m s}^{-1}$, 自洽。式 (3)–(7) 即四个问题共用的定解问题; 问题 1–3 取 $R \equiv R_0$, 问题 4 取 $R = R(t)$ (第 7.1 小节)。

问题 1 的特殊结构。 附录 2 的 ρ, c_p, k, h 为常数, 仅 $D = 7 \times 10^{-9} e^{-0.89/C}$ 依赖 C 。于是(3) 与式 (5)–(7) 一起退化为**常系数线性问题**, 与 C 无关; 而(4) 是拟线性方程, 单向地由 C 自身决定。注意问题 1 的烘房温度**并非常数** (1800 s 内由 28.0°C 升到 41.513°C), 因此分离变量级数不是它的解; 但由于方程线性, 对分段线性的 $T_{\text{air}}(t)$ 可用 Duhamel 叠加写出**闭式级数解**, 第 8.1 小节以它作为问题 1 温度场的真解基准 (MDR-0012)。

4.2 无量纲数与两个时间尺度

以附录 2 的参数与初始状态计算: 热扩散率 $\alpha = k/(\rho c_p) = 1.689 \times 10^{-7} \text{ m}^2/\text{s}$, 初始扩散系数 $D(C_0) = 4.938 \times 10^{-9} \text{ m}^2/\text{s}$,

$$\text{Bi} = \frac{hR_0}{k} = 1.389, \quad \text{Bi}_m = \frac{h_m R_0}{D(C_0)} = 3.240, \quad \frac{\tau_{\text{热}}}{\tau_{\text{质}}} = \frac{R_0^2/\alpha}{R_0^2/D} = \frac{1}{34}.$$

$\text{Bi} \approx 1.4$ 说明内、外热阻相当, 集总参数法 ($\text{Bi} \ll 1$) 不适用 [7], 必须解分布参数模型; $\text{Bi}_m > 3$ 说明传质的内阻已占优。两个时间尺度为 $\tau_{\text{热}} = R_0^2/\alpha = 2369 \text{ s}$ 与 $\tau_{\text{质}} = R_0^2/D = 22.5 \text{ h}$: 在 1800 s 的预热阶段内, 热 Fourier 数 $\text{Fo} = \alpha t/R_0^2 = 0.76$ 已是 $O(1)$, 而质 Fourier 数只有 0.022, 对应的扩散长度 $\sqrt{Dt} = 0.30 \text{ cm}$ 。这一预判与计算结果完全一致 (图 2): 1800 s 时温度已渗透到中心, 而水分的变化仍限制在表层约 0.5 cm 内。

4.3 有限体积离散

为了让同一套离散同时适用于固定域与收缩域, 在无量纲坐标 $\xi = r/R(t) \in [0, 1]$ 上离散 (固定域时 $R \equiv R_0$, $\xi = r/R_0$)。采用有限体积法 [8-9], 取**顶点型** (vertex-centred) 网格

$$\xi_i = i \Delta\xi, \quad i = 0, 1, \dots, n, \quad \Delta\xi = 1/n,$$

控制体为 $[\xi_{i-1/2}, \xi_{i+1/2}]$, 其中 $\xi_{i\pm 1/2} = \xi_i \pm \Delta\xi/2$, 并约定 $\xi_{-1/2} = 0$ 、 $\xi_{n+1/2} = 1$, 即轴心与表面处为**半控制体**。控制体的径向测度 (已除以 $2\pi R^2$) 为

$$V_i = \int_{\xi_{i-1/2}}^{\xi_{i+1/2}} \xi d\xi = \frac{1}{2}(\xi_{i+1/2}^2 - \xi_{i-1/2}^2), \quad (8)$$

$$V_0 = \frac{\Delta\xi^2}{8}, \quad V_i = \xi_i \Delta\xi \quad (0 < i < n), \quad V_n = \frac{\Delta\xi}{2} - \frac{\Delta\xi^2}{8}.$$

把(4) 在控制体上积分并用散度定理, 内部界面通量取二阶中心差分

$$\mathcal{F}_{i+1/2} = \xi_{i+1/2} D_{i+1/2} \frac{C_{i+1} - C_i}{\Delta\xi}, \quad i = 0, \dots, n-1, \quad (9)$$

轴心处由对称性取 $\mathcal{F}_{-1/2} = 0$, 表面处**直接施加** Robin 通量 $\mathcal{F}_{n+1/2} = -R h_m (C_n - C_{\text{air}})$ (不引入虚拟节点, 因而不损失精度也不破坏守恒)。于是得到半离散系统

$$R^2(t) V_i \frac{dC_i}{dt} = \mathcal{F}_{i+1/2} - \mathcal{F}_{i-1/2}, \quad \rho_i c_{p,i} R^2(t) V_i \frac{dT_i}{dt} = \mathcal{G}_{i+1/2} - \mathcal{G}_{i-1/2}, \quad (10)$$

其中 $\mathcal{G}$ 与 $\mathcal{F}$ 同构 (把 D 换成 k 、 h_m 换成 h 、 C 换成 T), $\rho_i = \rho(C_i)$ 、 $c_{p,i} = c_p(C_i)$ 。

界面系数。 非线性系数在界面上取**界面状态法** $D_{i+1/2} = D(\frac{C_i+C_{i+1}}{2}, \frac{T_i+T_{i+1}}{2})$ 、 $k_{i+1/2} = k(\frac{C_i+C_{i+1}}{2})$ 。对 $D \propto e^{-a/C}$ 这类强非线性系数, 界面状态法比系数的算术平均更接近 Kirchhoff 变换给出的精确界面通量 (算术平均会系统性高估、调和平均会低估)。三种取法的对照见表 9: 烘干时间之差不超过 0.11%, 说明结果对这一离散化选择不敏感。

两个累积状态与离散守恒。 把边界损失本身作为两个附加的常微分方程状态并入系统:

$$\frac{dQ_m}{dt} = \frac{h_m (C_n - C_{\text{air}})}{R(t)}, \quad \frac{dQ_h}{dt} = \frac{h (T_n - T_{\text{air}})}{R(t)}, \quad Q_m(0) = Q_h(0) = 0. \quad (11)$$

未知量为 $\mathbf{y} = (T_0, \dots, T_n, C_0, \dots, C_n, Q_m, Q_h)^T \in \mathbb{R}^N$, $N = 2(n+1) + 2 = 1284$ 。

命题 4.2 (离散水分守恒是精确的线性不变量). 记离散水分存量 $W(t) = \sum_{i=0}^n V_i C_i(t)$ 。则沿着半离散系统(10)–(11) 的任意解,

$$W(t) + Q_m(t) \equiv W(0) \quad \text{对一切 } t \text{ 成立.} \quad (12)$$

证明. 由(10) 求和, 内部通量成对相消 (telescoping) : $\frac{dW}{dt} = \sum_i \frac{\mathcal{F}_{i+1/2} - \mathcal{F}_{i-1/2}}{R^2} = \frac{\mathcal{F}_{n+1/2} - \mathcal{F}_{-1/2}}{R^2} = \frac{-R h_m (C_n - C_{\text{air}})}{R^2} = -\frac{dQ_m}{dt}$. $\square$

W 即 (除以 $2\pi\rho_s R^2$ 后的) 单位长度含水量, 故(12) 就是“含水量的减少等于表面通量的时间积分”。由于它是**线性**不变量, BDF 与 Radau 都精确保持它 (直到舍入误差); 因此把它作为校验量, 检验的是离散格式本身而不是时间积分的精度——实测相对残差约 10^{-15} (表 8)。对附录 2 的常物性, 温度方程同样给出 $\rho c_p \sum_i V_i T_i + Q_h \equiv \text{常数}$ 。

命题 4.3 (离散极值原理). 记 $M(t) = \max_i C_i(t)$ 、 $\bar{C}_{\text{air}} = \sup_s C_{\text{air}}(s)$ 。在使 $M(t) > C_{\text{air}}(t)$ 的每个时刻, M 的右上 Dini 导数满足 $D^+ M(t) \leq 0$; 从而对一切 t

$$\max_i C_i(t) \leq \max\{C_0, \bar{C}_{\text{air}}\}, \quad \min_i C_i(t) \geq \min\{C_0, \inf_s C_{\text{air}}(s)\}.$$

温度分量同理。

证明. 取最大值的下标可能随时间更替, 故 M 只是 Lipschitz 的, 用 Dini 导数叙述: $D^+M(t) = \max\{dC_i/dt : C_i(t) = M(t)\}$ (Danskin 定理)。设 i^* 满足 $C_{i^*} = M(t)$ 。 $D_{i+1/2} > 0$ 、 $\xi_{i+1/2} \geq 0$ 。当 $0 < i^* < n$ 时 $C_{i^*\pm 1} \leq C_{i^*}$, 故(9) 给出 $\mathcal{F}_{i^*+1/2} \leq 0$ 且 $-\mathcal{F}_{i^*-1/2} \leq 0$, 右端非正; $i^* = 0$ 时只剩 $\mathcal{F}_{1/2} \leq 0$; $i^* = n$ 时边界项为 $-Rh_m(C_n - C_{\text{air}})$, 在所讨论的时刻 $C_n = M(t) > C_{\text{air}}(t)$, 故亦非正。于是 $D^+M \leq 0$ 。再用标准的微分不等式论证: 若某时刻 $M > \max\{C_0, \overline{C}_{\text{air}}\}$, 则由 $M(0) = C_0$ 与连续性存在最后一个使 $M = \max\{C_0, \overline{C}_{\text{air}}\}$ 的时刻, 其后 $M > \overline{C}_{\text{air}} \geq C_{\text{air}}$ 从而 $D^+M \leq 0$, 与 M 的上升矛盾。下界同理 (此时 C_{i^*} 为最小分量, 边界项非负)。□

命题 4.3 保证离散解不会出现非物理的负含水率或超过初值的“回潮”, 也保证了 D 的计算始终在其定义域内。校验器在全部输出时刻独立检查该性质 (表 8)。

精度。 在均匀网格上, (9) 是界面导数的二阶近似, V_i 取自控制体的精确积分, 故内部格式二阶; 轴心与表面的半控制体上 Robin 通量直接施加, 不引入额外误差。实测观测阶为 2.0–2.3 (表 11、表 12)。

4.4 时间积分、输出采样与算法

半离散系统刚性 ($\tau_{\text{热}}/\tau_{\text{质}} \sim 10^{-2}$, 且空间加密使刚性比再放大 n^2 倍), 必须用全隐式方法。本文采用 SciPy[10] 与 NumPy[11] 实现的变阶变步长 BDF[12–13], 并提供块三对角的雅可比稀疏结构 (每个节点只与自身及左右邻居的 T, C 耦合), 使每步的线性代数为 $O(N)$ 。三个实现要点:

1. **在数据折点重启。** $T_{\text{air}}, C_{\text{air}}$ 是折线、 R 是 PCHIP 样条, 其导数在样本点处跳变。多步法跨过折点时的局部误差不受容差控制。因此把 $[0, t_{\text{终}}]$ 按附件 1 的全部样本时刻、切换到平台的时刻以及附件 2 的全部节点分段, 逐段积分、段间重启 (问题 3 共 241 段)。
2. **输出用稠密插值精确采样。** 所需时刻 (每 1 s 或每 60 s) 由积分器的稠密输出在段内插值得到, 不使用最近邻。
3. **事件求根定位 t_{end} 。** 以 $g(t) = \max_i C_i(t) - C^*$ 为事件函数 (连续、在干燥过程中单调下降穿零), 由积分器在稠密输出上求根, 得到不受输出网格分辨率限制的 t_{end} 。

复杂度与参数。 每个 BDF 步的代价为 $O(N)$ (带状 LU), 总代价 $O(N \cdot N_{\text{步}})$ 。生产网格共 641 个节点, $\Delta r = 0.0031 \text{ cm} = 0.1/2^5 \text{ cm}$, 使 0.1 cm 的输出点恰好落在节点上; 容差 $\varepsilon_r = 1 \times 10^{-9}$ 、 $\varepsilon_a = 1 \times 10^{-11}$ 。问题 3 的完整求解共 22510 次右端项调用、3205 次 LU 分解, 单次求解在 8 核云端实例上不足 30 s。

4.5 问题一的结果与分析

表 2 与表 3 给出题目要求格式的结果, 图 2、图 3 给出对应的剖面与时程; 每 1 s、每 0.1 cm 的完整结果见 result1.xlsx (时间 1 s–1800 s 每 1 s、位置 0–2 cm 每 0.1 cm, 温度与水分浓度两张工作表)。

Algorithm 1: 径向有限体积 + 分段自适应 BDF 求解 (含烘干结束时刻)

输入: 物性公式集 $\mathcal{P}$ (附录 2/3/4)、烘房数据 $(T_{\text{air}}, C_{\text{air}})$ 、半径数据 $R(\cdot)$ 、网格数 n 、容差 $(\varepsilon_r, \varepsilon_a)$ 、输出时刻集 $\mathcal{T}$

输出: $\{T_i(t), C_i(t)\}_{t \in \mathcal{T}}$ 、烘干结束时刻 t_{end} 、校验量 Q_m, Q_h

- 1 构造网格 $\{\xi_i\}$ 与控制体测度 $\{V_i\}$ (式 (8)); $\mathbf{y}(0) \leftarrow (T_0 \mathbf{1}, C_0 \mathbf{1}, 0, 0)$;
 - 2 $\mathcal{K} \leftarrow$ 附件 1 的样本时刻 $\cup$ 平台切换时刻 $\cup$ 附件 2 的 PCHIP 节点 (升序去重);
 - 3 构造雅可比稀疏模板 (块三对角, $O(N)$ 个非零);
 - 4 **foreach** 相邻折点区间 $[\tau_j, \tau_{j+1}] \subset [0, t_{\text{终}}]$ **do**
 - 5 在该区间上用 BDF 积分 $\dot{\mathbf{y}} = \mathbf{f}(t, \mathbf{y})$, 容差 $(\varepsilon_r, \varepsilon_a)$, 稀疏雅可比;
 - 6 **if** $\mathcal{T} \cap [\tau_j, \tau_{j+1}] \neq \emptyset$ **then** 由稠密输出取出这些时刻的状态;
 - 7 **if** 尚未定位 t_{end} 且 $\max_i C_i(\tau_{j+1}) < C^*$ **then**
 - 8 在 $[\tau_j, \tau_{j+1}]$ 上对 $g(t) = \max_i C_i(t) - C^*$ 求根, 得 t_{end} 与该时刻的剖面;
 - 9 $\mathbf{y}(\tau_{j+1})$ 作为下一段的初值 (重启多步法);
 - 10 **return** 采样结果、 t_{end} 、 Q_m, Q_h
-

表 2 30 分钟内药材的温度 ($^{\circ}\text{C}$; 列为到药材中心的距离/cm)

时间/s	0	0.5	1	1.5	2
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 3 30 分钟内药材的水分浓度 (kg/kg; 列为到药材中心的距离/cm)

时间/s	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2471
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

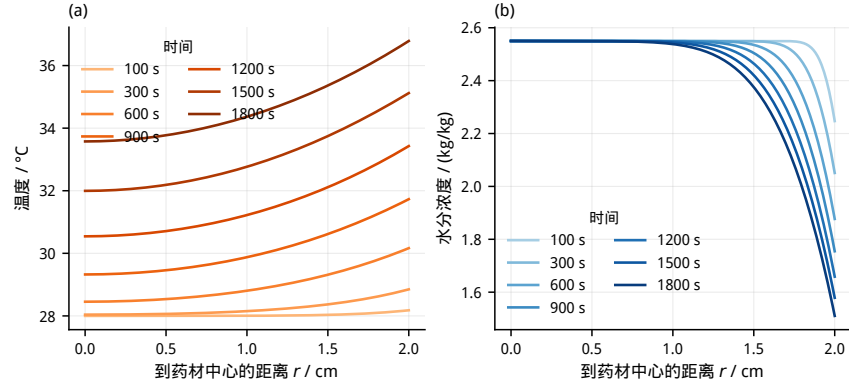


图 2 问题 1: 表 2、表 3 各时刻的温度 (a) 与水分浓度 (b) 径向剖面。1800 s 内热扩散已渗透到中心 (中心温度明显高于初值), 而水分的变化仍限制在表层约 0.5 cm, 与 $Fo = 0.76$ 、 $Fo_m = 0.022$ 的量级判断一致。

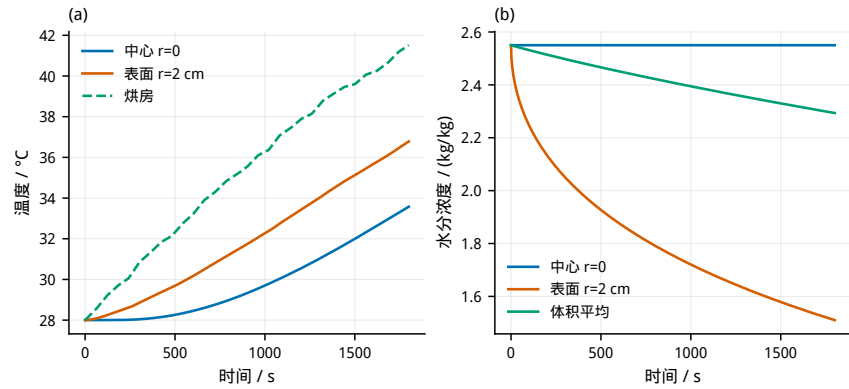


图 3 问题 1: 中心、表面与烘房温度 (a) 以及中心、表面、体积平均水分浓度 (b) 的时程。表面温度始终低于烘房温度 (1800 s 时相差 4.727 K), 中心又低于表面, 与 $Bi = 1.389$ (内阻与外阻相当) 一致; 中心水分浓度在全程保持初值, 体积平均含水率的下降速率随表层变干而逐渐减慢。

结果分析。 (i) 1800 s 时中心温度 33.5753 °C、表面温度 36.7856 °C，同时烘房温度为 41.513 °C：表面比烘房低 4.727 K，中心又明显低于表面（表 2）。截面上的不均匀度（中心与表面之差）与表面到烘房的温差是同一量级，这正是 $Bi = 1.389 \approx 1$ 的含义：内部导热热阻与外部对流热阻相当，二者共同决定预热速率，因而既不能用集总参数法、也不能假定表面温度等于烘房温度。(ii) 水分浓度在中心 ($r = 0$) 四位小数下仍为初值 2.5500， $r = 1$ cm 处也只降到 2.5383（降幅 0.5%），而表面已降至 1.5102，体积平均为 2.2936，即 30 min 内失水 10.06%（表 3）；水分的扩散长度 $\sqrt{Dt} = 0.30$ cm 正与这一格局相符。表面附近出现陡峭的边界层：这正是 $Bi_m = 3.240 > 1$ 与 $Fo_m \ll 1$ 的表现。(iii) 该结构对后续问题有直接含义：干燥初期的失水几乎全部来自表层，表层含水率迅速下降使 $D \propto e^{-0.89/C}$ 显著减小，从而自发形成低扩散率的“硬壳”，把整个过程推向内部扩散控制。

5 问题二的模型建立与求解

本节回答：把物性换成附录 3 的变系数公式后，预热与恒温两个阶段的温度与水分如何演化，3 h 内的结果是多少，以及交付文件覆盖多长时间。模型、离散与积分方法与第 4 节完全相同，只更换物性函数与积分区间。

5.1 模型的改变

问题 2 覆盖预热平衡与恒温干燥两个阶段，统一采用附录 3 的经验公式（题目要求“相关经验公式统一采用附录 3 中的公式”）：

$$\begin{aligned} \rho &= 650 + 128C, & c_p &= 1450 + 2736 \frac{C}{C+1}, & k &= 0.21 + 0.38 \frac{C}{C+1}, \\ D &= 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T_K}, \end{aligned} \quad (13)$$

其中 $T_K = T + 273.15$ 为绝对温度。控制方程、边界条件与初值仍为(3)–(7)， h 、 h_m 沿用附录 2 的值（假设 4）。与问题 1 相比有三点实质变化：

1. **双向耦合。** ρ, c_p, k 依赖 C ， D 依赖 C 与 T ，温度场与水分场**双向耦合**，不再有解析解。
2. **扩散系数的温度激活。**式 (13) 中的 D 含 Arrhenius 因子 e^{-3850/T_K} ， $d \ln D / dT = E / T_K^2$ 。在同一含水率 C_0 下，温度由 28 °C 升到恒温阶段的平台值使 D 由 5.642×10^{-9} 增大到 1.35×10^{-8} m²/s，即增大一倍以上。因此预热阶段虽然失水不多，却通过升温为后续干燥“解锁”了扩散能力。
3. **与含水率的竞争。**因子 $e^{-0.45/C}$ 随 C 减小而减小。干燥初期升温占优（ D 增大），后期失水占优（ D 急剧减小）。图 4 给出三组附录物性的 $D(C, T)$ 与热扩散率。

数值方法与第 4.3、4.4 小节完全相同，只更换物性函数。

交付文件的时间范围（第 1.3 小节第 (e) 处解释）。 问题 2 要求“整个烘干过程”的模型，但在“完整结果”前未像问题 1 那样写出时间上限，故有两种读法。本文的模型对两种读法是同一个：问题 2 与问题 3 是**同一个**初边值问题（附录 3 物性、固定半径）在不同积分区间上的解，校验器在两次求解共有的 181 个 60 s 整点上实测其最大差异为 0.0

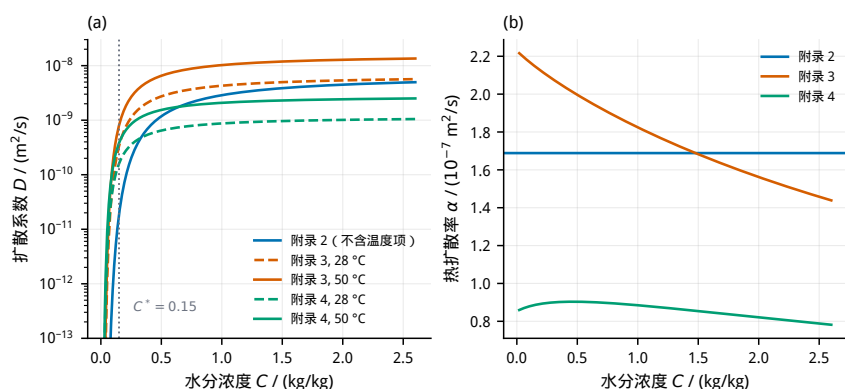


图 4 附录 2–4 的物性：水分浓度扩散系数随含水率与温度的变化 (a, 纵轴对数坐标) 与热扩散率 $\alpha = k/(\rho c_p)$ (b)。 D 在干燥过程中跨越三到四个数量级，是刚性与“表面硬壳”的来源；附录 4 的 D 在高含水率段比附录 3 小约 5.4 倍。

kg/kg (水分) 与 $0.0\text{ }^{\circ}\text{C}$ (温度)，即积分容差量级。交付上：result2.xlsx 按与表 3、表 4 一致的 0–3 h 每 1 s 给出 (10800 行 $\times$ 两张表)；若按全过程每 1 s 输出则为 206940 行、实测 57.3 MB，超出提交与支撑材料的规模预算，因此另行交付 result2_全过程.xlsx——同一模板、同样的“温度”“水分浓度”两张表、覆盖 $0-t_{\text{end}}$ 、分辨率 60 s (即题目在问题 3、4 中对全过程所采用的分辨率)。两种读法都有对应的交付物 (MDR-0005)。

5.2 结果与分析

表 4 3 小时内药材的温度 ($^{\circ}\text{C}$ ；列为到药材中心的距离/cm)

时间/h	0	0.5	1	1.5	2
0.5	32.1892	32.3821	32.9659	33.9608	35.4130
1.0	40.3816	40.5536	41.0605	41.8782	42.9976
1.5	45.8468	45.9348	46.1930	46.6051	47.1401
2.0	48.4502	48.4881	48.5984	48.7736	49.0033
2.5	49.4670	49.4792	49.5136	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

结果分析。 (剖面与时程见图 5 与图 6。) (i) 3 h 时中心温度 $49.8495\text{ }^{\circ}\text{C}$ 、表面 $49.9664\text{ }^{\circ}\text{C}$ (表 4)，截面温差已进入小数点后第一位；中心温度达到烘房温度 1 K 以内所需时间为 2.27 h，与热扩散时间尺度 $\tau_{\text{热}}$ 及烘房自身的升温时间相当。此后可以认为药材是等温的，这为后续把注意力集中在水分方程提供了依据。 (ii) 3 h 时中心水分 1.7662、表面 1.0081、体积平均 1.3825 kg/kg，累计失水 45.78%——远高于问题 1 的 10.06%，因为温度升高把 D 整体抬高。 (iii) 一个值得注意的现象：3 h 时表面的扩散系数 $D = 1.027 \times 10^{-8} \text{m}^2/\text{s}$ ，大于初始值 $5.642 \times 10^{-9} \text{m}^2/\text{s}$ ，尽管表面含水率已从 2.55 降到 1.0081。这说明在前 3 h 内温度激活仍压过含水率下降的影响；转折发生在含水率降到约 $0.5 \text{kg}/\text{kg}$ 之后 (图 4(a))。相应地，附录 3 参数体系下的传质 Biot 数 $h_m R_0/D$ 由初始的 2.84 降到

表 5 3 小时内药材的水分浓度 (kg/kg; 列为到药材中心的距离/cm)

时间/h	0	0.5	1	1.5	2
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2.0	2.1709	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

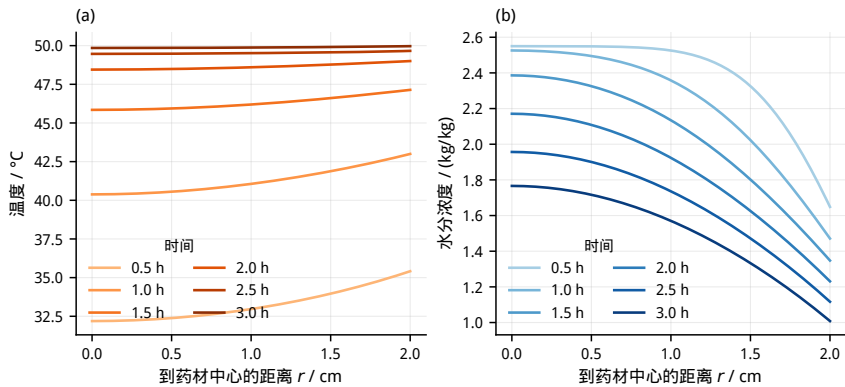


图 5 问题 2: 表 4、表 5 各时刻的温度 (a) 与水分浓度 (b) 径向剖面。温度剖面在 2 h 后几乎拉平 (表 4 各行内的差已进入小数点后第一位), 水分剖面则持续变陡。

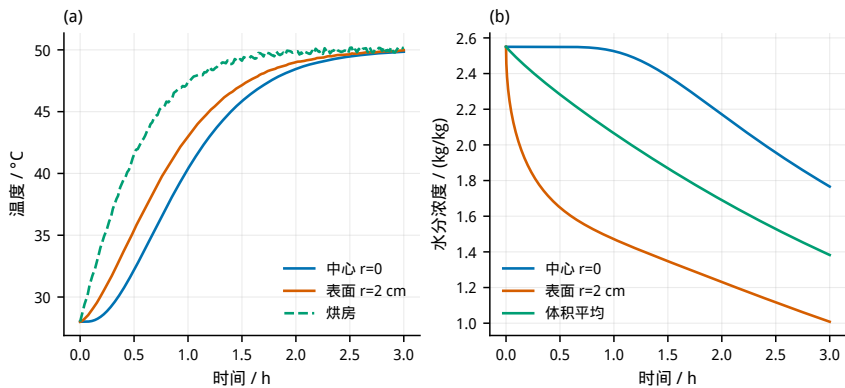


图 6 问题 2: 3 h 内温度 (a) 与水分浓度 (b) 的时程。中心温度在 2.27 h 后与烘房相差不足 1 K、2.69 h 后不足 0.5 K; 此后 D 的变化完全由含水率决定。

1.56, 随后在干燥后期又升到 10^3 量级。(iv) 表 3、表 4 的时刻均落在附件 1 的覆盖范围内, 因此问题 2 的结果不依赖假设 5 中的平台外推。

6 问题三的模型建立与求解

本节回答：在与问题 2 完全相同的模型下，“药材各处水分浓度低于 0.15 kg/kg”这一要求何时首次满足，以及这个时刻对数值细节与判据表述有多敏感。结论先行： $t_{\text{end}} = 57.48 \text{ h}$ (2.40 天)，由中心点控制，过程完全由内部扩散而非表面对流决定。

6.1 烘干结束时间的定义与求解

模型与问题 2 完全相同（附录 3 物性、固定半径），只是把积分区间延长到烘干结束。按假设 9，

$$t_{\text{end}} = \inf \left\{ t > 0 : \max_{0 \leq r \leq R_0} C(r, t) < C^* \right\}, \quad C^* = 0.15 \text{ kg/kg}. \quad (14)$$

由命题 4.3 与数值上验证的径向单调性 ($\partial_r C \leq 0$, 表 8)，最大值在中心取得，故(14)等价于 $C(0, t_{\text{end}}) = C^*$ ；但程序不假定这一点，仍以 $\max_i C_i$ 作事件函数（算法 1），从而对任何剖面形状都成立。

为什么必须精确求根。 若只在 60 s 的输出网格上判断，得到的是 t_{end} 向上取整到 60 s 的值：事件求根给出 57.4805 h，60 s 网格给出 57.4833 h，与 result3.xlsx 的最后一行一致。

注 6.1 (四位小数的显示与判据). 判据在 t_{end} 处恰好取等号，因此其后第一个 60 s 整点 (57.4833 h) 的中心水分浓度精确值只有 0.149997，按题目要求四舍五入到四位小数后显示仍为 0.1500——只翻结果文件的读者会误判为未达判据。为避免这一歧义，result3.xlsx 与 result4.xlsx 的行一直写到四位小数下全场均不超过 0.1499 的首个 60 s 整点（分别为 57.53 h 与 51.12 h，即比 t_{end} 多 3.2 与 1.6 min，MDR-0005）。烘干时间的答案仍是精确的 t_{end} ，与末行时刻无关；表 6、表 7 的“烘干结束时间”行给出的正是 t_{end} 处的剖面，其中心值同样显示为 0.1500。判据是否成立由未舍入的解按(14)判定，并由与求解器分离的校验器独立确认（第 8 节）： $t < t_{\text{end}}$ 的全部输出行满足 $\max_i C_i \geq C^*$ ， $t \geq t_{\text{end}}$ 的全部行满足 $\max_i C_i < C^*$ 。

6.2 结果与分析

结果分析。（剖面与时程见图 7。）(i) **烘干所需时间为 $t_{\text{end}} = 57.48 \text{ h}$ (2.40 天)**，落在题目所述“一般持续 2–3 天”的范围内——这是一个独立于数值细节的合理性旁证。此处报告的是生产网格值（与 result3.xlsx 一致）；网格收敛外推给出 57.4737 h，两者之差 0.41 min 恰好跨越两位小数的舍入边界，故严格的表述是 $t_{\text{end}} = 57.47\text{--}57.48 \text{ h}$ ，三位有效数字为 57.5 h（注 8.1）。(ii) 过程高度不均衡：体积平均含水率在 3.42 h 内即损失一半，而把**中心**降到 C^* 需要 57.48 h。烘干结束时表面含水率仅 0.0526、体积平均为 0.1243 kg/kg，即为了满足“各处低于 0.15”，大部分物料被过度干燥。若改用体积平均含水率作判据，只需 36.05 h (–37.28%)——判据的表述是本问题中最敏感的建模决策，远超任何参数的 $\pm 20\%$ 扰动（表 18、表 9）。(iii) 机理上，长尾来自扩散系数的自反馈：表面含水率降到 0.05 量级后 D 降至 $3.1 \times 10^{-12} \text{ m}^2/\text{s}$ ，比初始值小 4320 倍，传质 Biot 数升到约 5131。此时表面对流早已不是瓶颈（对 h_m 的弹性仅 –0.114），**过程完全由内部扩散控制**。(iv) 因此工艺上缩短烘干时间的有效手段是提高烘房温度（弹性 –1.603，因为 D 的 Arrhenius 因子）或减小特征尺寸（问题 4），而不是加大风速（提高 h 、 h_m ）。

表 6 药材烘干过程的水分浓度 (kg/kg; 固定半径 2 cm, 附录 3 物性; 列为到药材中心的距离/cm)

时间/h	0	0.5	1	1.5	2
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2916	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1856	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0548
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1515	0.1436	0.1277	0.0530
烘干结束时间 (57.48 h)	0.1500	0.1477	0.1402	0.1249	0.0526

烘干结束时间为 $\max_r C(r, t) < 0.15$ 首次成立的精确时刻 (事件求根), 结果文件按 60 s 分辨率给出。

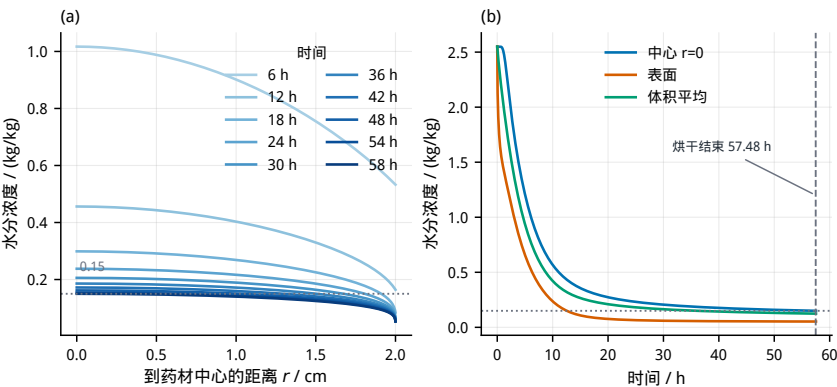


图 7 问题 3: 每 6 h 与烘干结束时刻的水分浓度剖面 (a) 以及中心、表面、体积平均水分浓度的时程 (b); 点线为判据 $C^* = 0.15$ kg/kg, 虚线为烘干结束时刻 $t_{\text{end}} = 57.48$ h。表面在十余小时内即降到判据以下 (表 6), 其后的绝大部分时间完全用于把中心的水分排出。

7 问题四的模型建立与求解

本节回答：当药材按附件 2 的 $R(t)$ 收缩时烘干需要多久，以及“收缩”这一几何信息如何转化为方程。难点不在求解而在建模——附件 2 只给了几何，必须补一条物理原则；本节把这一步显式化，并对备选写法给出定量对照。结论先行： $t_{\text{end}} = 51.09 \text{ h}$ 。

7.1 收缩域上的方程：物质坐标表述

问题 4 中计算域 $0 < r < R(t)$ 随附件 2 给出的 $R(t)$ 收缩 ($2.000 \text{ cm} \rightarrow 1.198 \text{ cm}$, 主要发生在前 12 h, 图 11(c), 附录 B)。附件 2 只给出几何，未说明被边界“扫过”的物质如何处理，因此必须补充一条物理原则。本文采用**干物质守恒** (假设 7)：收缩是均匀的，每个物质点的径向坐标与 $R(t)$ 成比例，于是 $\xi = r/R(t)$ 是**物质坐标** (Lagrangian 坐标, Landau 变换 [14-15])。

命题 7.1 (物质坐标下的方程). 在均匀收缩与干物质守恒的假设下，以 $\xi = r/R(t)$ 为自变量、 $\hat{C}(\xi, t) = C(R(t)\xi, t)$ ，有

$$\left. \frac{\partial \hat{C}}{\partial t} \right|_{\xi} = \frac{1}{R^2(t)} \cdot \frac{1}{\xi} \frac{\partial}{\partial \xi} \left(D \xi \frac{\partial \hat{C}}{\partial \xi} \right), \quad -\left. \frac{D}{R(t)} \frac{\partial \hat{C}}{\partial \xi} \right|_{\xi=1} = h_m (\hat{C}(1, t) - C_{\text{air}}), \quad (15)$$

温度方程同理 (把 D 换成 $k/(\rho c_p)$ 并保留 ρc_p 因子)。方程中**不出现对流项**，收缩的全部作用体现为因子 $1/R^2(t)$ 。此外，以干基计的水分总量满足

$$\frac{d}{dt} \int_0^1 \hat{C} \xi d\xi = -\frac{h_m (\hat{C}(1, t) - C_{\text{air}})}{R(t)}, \quad (16)$$

即水分守恒。

证明要点. 干物质守恒与均匀收缩意味着每个物质环 $[\xi, \xi + d\xi]$ 包含的干物质 $\propto \xi d\xi$ 与时间无关；水分相对骨架的通量为 $-\rho_s D \partial_r C = -\rho_s D R^{-1} \partial_{\xi} \hat{C}$ ，其通过半径 $r = R\xi$ 处环面 (面积 $\propto R\xi$) 的总量 $\propto -\rho_s D \xi \partial_{\xi} \hat{C}$ 中 R 恰好相消。对物质环作衡算并除以其干物质质量 ($\propto R^2 \xi d\xi$) 即得(15)；对 $\xi \in [0, 1]$ 积分并用对称条件即得(16)。□

题给数据的自治性检验. 上述推导只用到“均匀收缩 + 单位长度干物质守恒”，即 $\rho_s R^2$ 与时间无关。值得指出的是，题目给出的两组数据在这一原则下**并不严格相容**：由附录 4 的 $\rho = 760 + 90C$ 得 $\rho_s = \rho/(1 + C)$ ，初态 279、末态 ($C = C^*$) 673 kg/m^3 ，若 $\rho_s R^2$ 守恒则应有 $R_{\infty} = R_0 \sqrt{\rho_s(C_0)/\rho_s(C^*)} = 1.287 \text{ cm}$ ，而附件 2 给出 1.198 cm (半径差 7.5%，按 $\rho_s R^2$ 计的干物质差 13.4%)。本文的处理是：**以附件 2 的几何为准** (题目要求“根据附件 2 确定烘干时长”)，把 ρ_s 视为随 R^{-2} 演化的模型量，从而水分方程严格守恒； $\rho(C)$ 只用于能量方程的体积热容 ρc_p (注 4.1)。由于药材在 2.27 h 后已与烘房接近等温 (第 5 节)， ρc_p 的偏差对 t_{end} 的影响可忽略；这一 13.4% 的数据内部不一致记入第 10 节的局限。

备选：实验室坐标写法. 若把附录方程视为实验室坐标 (r, t) 下的方程并在 $0 < r < R(t)$ 上求解，同一变换会**额外**产生一个对流项：

$$\left. \frac{\partial \hat{C}}{\partial t} \right|_{\xi} = \frac{1}{R^2} \cdot \frac{1}{\xi} \frac{\partial}{\partial \xi} \left(D \xi \frac{\partial \hat{C}}{\partial \xi} \right) + \xi \frac{\dot{R}}{R} \frac{\partial \hat{C}}{\partial \xi}. \quad (17)$$

此时水分总量的变化等于表面通量积分再加上 $C_s R \dot{R}$ 一项——被退缩的边界扫出计算域的水分被“丢弃”，总量不守恒（校验器测得这一伪损失占总失水的三成以上）。因此本文以(15)为主模型，把(17)的结果作为对照（表 9、表 16）。

实现。 第 4.3 小节的离散本来就写在 ξ 上，因此只需把常数 R_0^2 换成 $R^2(t)$ (Lagrangian) 或再加上(17)的对流项 (Eulerian, 界面用中心差分、表面用三点单侧差分)。 $R(t)$ 由附件 2 的样本作保单调 PCHIP 插值 [4]（保证 R 不出现非物理的回升），72 h 之后保持末值。物性改用附录 4 的式 (18)：

$$\begin{aligned} \rho &= 760 + 90C, & c_p &= 1850 + 2150 \frac{C}{C+1}, & k &= 0.12 + 0.20 \frac{C}{C+1}, \\ D &= 4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/T_K}. \end{aligned} \quad (18)$$

退化检验。 令 $R \equiv R_0$ （但仍走移动边界的代码路径），两种写法都应退化为固定域格式。实测与固定域求解器之差为 9.4×10^{-14} ，为舍入误差量级，说明移动边界的实现没有引入额外误差。

7.2 结果与分析

表 7 考虑尺寸变化的药材烘干过程水分浓度 (kg/kg；附录 4 物性；列为到药材中心的距离/cm，超出当前半径处为“—”)

时间/h	0	0.5	1	1.5	2	药材表面
6	1.7188	1.5370	1.0221	—	—	0.4205
12	0.7374	0.6544	0.4082	—	—	0.1669
18	0.4086	0.3686	0.2397	—	—	0.0892
24	0.2851	0.2610	0.1790	—	—	0.0673
30	0.2263	0.2094	0.1493	—	—	0.0595
36	0.1928	0.1796	0.1317	—	—	0.0560
42	0.1712	0.1604	0.1200	—	—	0.0541
48	0.1561	0.1468	0.1116	—	—	0.0530
烘干结束时间 (51.09 h)	0.1500	0.1413	0.1081	—	—	0.0526

固定物理位置的值由 $\xi = r/R(t)$ 网格上的三次样条取得；“药材表面”为 $\xi = 1$ 的节点值。

结果分析。 (i) 考虑尺寸变化后的烘干时长为 $t_{\text{end}} = 51.09 \text{ h}$ (51.0893 h, 2.13 天)，此时半径已收缩到 1.200 cm；表 7 所列的 6 h 及以后各时刻， $r = 1.5, 2 \text{ cm}$ 均已超出当前半径，故该两列留空；result4.xlsx 则按每一时刻的 $R(t)$ 逐格判断（例如 $t = 1800 \text{ s}$ 时 $R = 1.873 \text{ cm}$ ， $r \leq 1.873$ 的单元仍有数值），并另设“药材表面”列给出 $\xi = 1$ 处的值。(ii) 收缩的作用可以定量解释。保持附录 4 的物性而把半径固定在 2 cm，烘干需要 129.85 h，是移动边界结果的 2.54 倍；而扩散时间尺度 $\propto R^2$ ，末半径对应的比值为 $(R_0/R_\infty)^2 = 2.79$ 。两个数字量级一致（收缩需要时间完成，故实际加速略小于上界），说明收缩通过缩短扩散路径主导了加速，末半径处的截面积只剩初始的 35.9%。(iii) 与

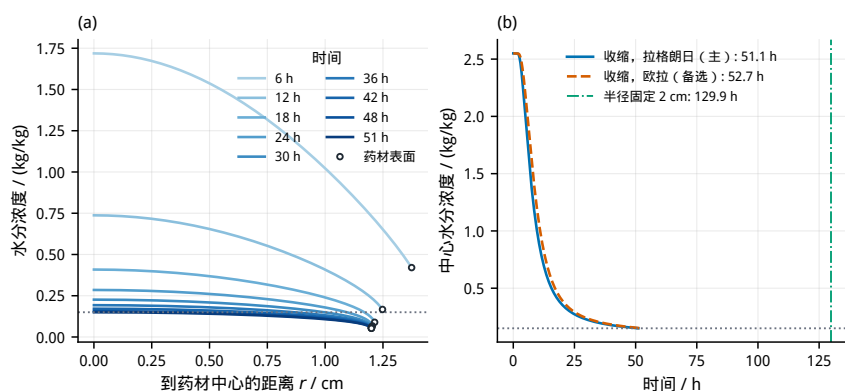


图 8 问题 4: 物理坐标下每 6 h 的水分浓度剖面与表面位置 (a, 空心圆为 $\xi = 1$ 的表面值, 超出当前半径处无定义); 中心水分浓度时程在主模型 (物质坐标)、实验室坐标备选与半径固定 2 cm 参照下的比较 (b)。

问题 3 的 57.48 h 相比, 问题 4 更短 (51.09 h), 但这是两个效应叠加的结果: 附录 4 的扩散系数在高含水率段比附录 3 小约 5.4 倍 (使干燥变慢), 而收缩使之变快且占优。因此**不能**把问题 3 与问题 4 的时间差简单归因于“尺寸变化”, 必须用同物性对照 (第 (ii) 点) 才能分离两种效应——这是本文把固定半径参照一并计算的原因。(iv) 实验室坐标备选写法给出 52.65 h, 比主模型长 3.06% (表 16, 附录 A)。差异方向可以解释: 该写法丢弃了被退缩边界扫出的水分, 却同时丢弃了这部分物质所占的体积, 净效果是早期剖面偏湿 (图 8(b))。两者之差 3% 量级给出了“收缩如何处理”这一解释不确定性的规模。

8 模型检验

本节按验证与确认 (verification & validation) 的标准框架 [16-17] 检验前四节的结果: 验证回答“方程解对了吗”, 确认回答“解对了方程吗”。所有检验由**与求解器代码分离**的校验器完成——它只读取求解结果数组, 自行重算平衡量与解析解, 不调用求解器的任何内部函数 (附录 E 的 pipelines/a/verify.py 与 pipelines/a/solver.py)。表 8 汇总四个问题的校验结论, 下面逐项说明。

表 8 各问题的独立校验: 水分守恒的两种相对残差、极值原理、沿径向单调性、综合结论

问题	不变量残差	重采样残差	极值原理	单调性	通过
问题 1	6.74e-15	1.92e-06	是	是	是
问题 2	1.97e-15	4.04e-07	是	是	是
问题 3	2.68e-15	9.48e-05	是	是	是
问题 4	3.84e-15	2.09e-04	是	是	是

“不变量残差”比较离散存量与积分器内累积的边界通量 (半离散系统的精确线性不变量, 检验格式本身); “重采样残差”由输出样本按梯形法独立重算通量积分, 受输出间隔的梯形误差限制, 检验的是实际通量。

8.1 与解析级数解的对照

取常物性、恒定环境的无限长圆柱，则(3)–(6)有分离变量的解析解：记 $\theta = (u - u_\infty)/(u_0 - u_\infty)$ 、 $Fo = at/R_0^2$ 、 $Bi = \beta R_0/a'$ (a 为扩散系数、 β 为传递系数)，则

$$\theta(\xi, Fo) = \sum_{m=1}^{\infty} \frac{2 Bi}{(\lambda_m^2 + Bi^2) J_0(\lambda_m)} J_0(\lambda_m \xi) e^{-\lambda_m^2 Fo}, \quad \lambda_m J_1(\lambda_m) = Bi J_0(\lambda_m), \quad (19)$$

特征值 λ_m 位于 J_0 的相邻零点之间，由 Brent 法求得（取 400 项，截断误差远小于比较量级）[18–19]。分别以 $Bi = 1.389$ （温度， $a = \alpha$ ）与 $Bi_m = 3.240$ （把 D 冻结为 $D(C_0)$ 的水分方程）作对照，结果见表 12（附录 A）与图 9(b)：误差随 Δr 干净地二阶下降，观测阶 2.00（温度）与 2.00（水分）；在生产网格上最大绝对误差为 $1.0 \times 10^{-5}^\circ\text{C}$ 与 $1.1 \times 10^{-5} \text{ kg/kg}$ ，比四位小数的舍入尺度 5×10^{-5} 小约一个数量级。这同时确认了离散格式的实现（控制体测度、半控制体上的 Robin 通量、轴心对称条件）是正确的。

问题 1 本身的真解对照（Duhamel 叠加）。 上述算例的环境是人为取为常数的；问题 1 的烘房温度实际上由 28.0°C 升到 41.513°C ，因而(19)并不是它的解。但附录 2 的温度方程是常系数线性方程，对分段线性的 $T_{\text{air}}(t)$ 可由 Duhamel 叠加给出闭式级数解：以 $\mu_m = \alpha \lambda_m^2 / R_0^2$ 、 $c_m = 2Bi/[(\lambda_m^2 + Bi^2)J_0(\lambda_m)]$ 记，

$$T(\xi, t) = T_{\text{air}}(t) + \sum_{m \geq 1} c_m \left[(T_0 - T_{\text{air}}(0)) e^{-\mu_m t} - \int_0^t e^{-\mu_m(t-s)} T'_{\text{air}}(s) ds \right] J_0(\lambda_m \xi), \quad (20)$$

其中最后一个积分在附件 1 的每个样本区间上是初等的（ T'_{air} 为常数），故(20)除级数截断外是精确的。这给出了问题 1 温度场的真解基准（MDR-0012）：表 12 第三组显示生产网格上表格时刻的最大误差为 $2.0 \times 10^{-6}^\circ\text{C}$ 、观测阶 1.97；在 result1.xlsx 温度工作表的完整 37800 个输出单元上最大误差为 $2.0 \times 10^{-6}^\circ\text{C}$ ，比四位小数的半个末位 5×10^{-5} 小一个半数量级。交付的温度数据因此不是与另一套数值解、而是与解析解逐格吻合的。

8.2 网格收敛、观测阶与四位小数的可靠性

取 $\Delta r = 0.1/2^k \text{ cm}$ ， $k = 0, \dots, 6$ ($k = 5$ 为生产网格)，在各问题表格所要求的时刻与位置上计算误差与观测阶，结果见表 11（附录 A）与图 9(a)。

两种观测阶，以及为什么必须并列给出。 自收敛研究只能以最细网格为参照， $e_k = \|u_k - u_m\|_\infty$ 、 $p_k = \log_2(e_k/e_{k+1})$ 。对真实误差 $\propto h^2$ 的格式，这一估计量在最后几对网格上被系统性抬高：

$$\frac{e_k}{e_{k+1}} = 4 \cdot \frac{1 - 4^{-(m-k)}}{1 - 4^{-(m-k-1)}}, \quad m - k = 2 \Rightarrow \log_2 5 = 2.32.$$

因此表 11 中出现的 2.31–2.32 并非超收敛，而是参照偏置的确切数值。本文同时给出不需要参照解的三网格估计量 $p_k = \log_2(\|u_k - u_{k+1}\|_\infty / \|u_{k+1} - u_{k+2}\|_\infty)$ （MDR-0011）：问题 1、2 的场量给出 1.99–2.00，与解析解对照的 2.00（表 12）一致，即格式确为二阶；问题 3、4 的水分给出 1.65 与 1.91，低于 2——原因是这两问的取样时刻绑定在随网格移动的 t_{end} 上，误差里混入了穿零时刻的移动，这是量的性质而不是格式的缺陷。

生产网格上的误差。表格时刻与位置上 ($k = 5$): 问题 1 温度 1.4×10^{-6} 、问题 1 水分 2.2×10^{-5} 、问题 2 温度 8.6×10^{-7} 、问题 2 水分 6.9×10^{-7} 、问题 3 水分 9.5×10^{-6} 、问题 4 水分 1.9×10^{-6} 。

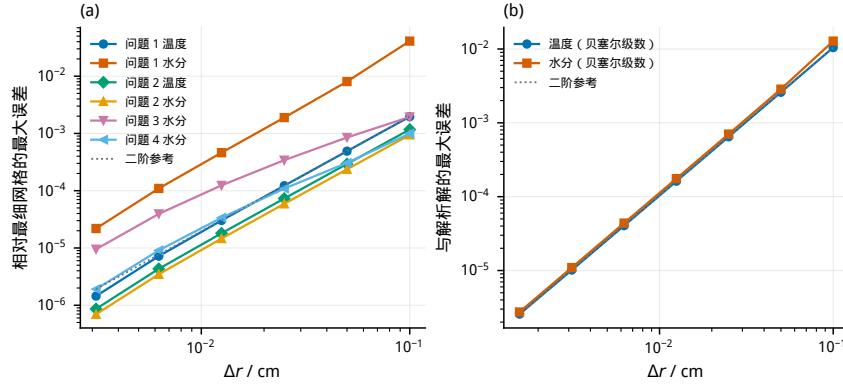


图 9 网格收敛: 各问题在表格时刻与位置上相对最细网格 ($\Delta r = 0.1/2^6$ cm) 的最大误差 (a) 与常物性算例相对贝塞尔级数解(19) 的最大误差 (b); 点线为二阶参考斜率。

结果文件完整输出网格上的误差: 四位小数在哪里有意义。 上述取样只覆盖论文表格要求的 7 个时刻, 而 result1.xlsx、result2.xlsx 自 $t = 1$ s 起每 1 s 输出。 t 很小时表面边界层厚度 $\sqrt{Dt}$ ($t = 1$ s 时约 0.007 cm) 只跨越两三个生产网格单元, 误差必然大于表格时刻处的误差。为此在 $k = 6, 7$ 两级网格上重解问题 1、2 的完整输出网格, 按实测的三网格阶作 Richardson 外推得极限, 再逐格评估交付数组的误差 (表 14, 附录 A):

- **温度:** 问题 1 最大误差 1.9×10^{-6} 、问题 2 1.7×10^{-6} °C, 全网格无一单元超过 5×10^{-5} ; 结合(20) 的真解对照 (2.0×10^{-6} °C), 温度的四位小数全部有意义。
- **水分:** 最大误差 4.0×10^{-4} (问题 1) 与 3.2×10^{-4} (问题 2), 均出现在 $t = 1$ s, $r = 2$ cm, 即边界层最薄的一格。误差超过 5×10^{-5} 的单元在问题 1 共 42 个 (占 0.11%, 全部位于 $t \leq 42$ s), 在问题 2 共 29 个 (占 0.01%, $t \leq 29$ s); 两者都只出现在表面最外侧的一两列。

结论应当如实表述: result1.xlsx 的 42 个单元 ($t \leq 42$ s 的表面附近) 与 result2.xlsx 的 29 个单元 ($t \leq 29$ s) 的第四位小数受离散误差影响 (量级 10^{-4} , 第三位小数仍可靠); 其余 99.9% 以上的单元以及全部温度单元, 四位小数都有意义。论文正文的表格起自 $t = 100$ s (问题 1) 与 $t = 0.5$ h (问题 2), 已在该区域之外。

8.3 烘干时间的收敛与外推

烘干时间本身的收敛单列于表 13 (附录 A): 生产网格上的 t_{end} 与最细网格相差 0.28 min (问题 3) 与 0.03 min (问题 4)。 t_{end} 的三网格观测阶为 1.67 (问题 3) 与 1.91 (问题 4), 均低于场量本身的 2; 因此 Richardson 外推 [20] 按实测阶而非 $p = 2$ 进行, $t^* = t_6 + (t_6 - t_5)/(2^p - 1)$, 得极限估计 57.4737 h 与 51.0885 h, 生产网格的离散误差为 0.41 min 与 0.05 min。

注 8.1 (t_{end} 的第二位小数). 问题 4 的两位小数是稳定的 (生产网格 51.0893 h 与外推 51.0885 h 都给出 51.09 h)。问题 3 不然: 生产网格给出 57.4805 h (四舍五入为 57.48

h), 外推极限 57.4737 h (四舍五入为 57.47 h), 二者恰好落在 57.475 这一舍入边界的两侧, 离散误差 0.41 min 大于两位小数的分辨率 0.005 h。因此本文把问题 3 的答案报告为

$t_{\text{end}} = 57.48 \text{ h}$ (生产网格值, 与 result3.xlsx 一致), 网格收敛极限 57.4737 h,

即 57.47–57.48 h; 三位有效数字 57.5 h 是无歧义的。这一 0.4 min 的离散不确定度远小于解释性选择 (表 9) 与参数不确定性带来的变化。

8.4 时间积分的无关性

表 15 (附录 A) 给出在生产网格上改变时间积分设置的结果: 把容差收紧一个量级、改用隐式 Runge–Kutta 方法 Radau IIA[12]、以及把最大步长强制限制为 $\Delta t \leq 3600, 900, 300 \text{ s}$ 。水分浓度的最大差异不超过 4.2×10^{-9} , 烘干时间的差异不超过 0.01 s; 把容差放松两个量级 (rtol= 10^{-7} , 表 15 第三行) 时差异仍远小于四位小数的舍入尺度。这说明结果与时间步长序列无关, 生产容差有充分余量。注: $\Delta t \leq 3600 \text{ s}$ 一行的解与生产设置逐位相同 (水分最大差 0、 t_{end} 差 0 s), 但右端项调用次数比生产设置多 18 次——这来自限制 Δt 后 BDF 初始步长试探的额外求值, 而非不同的步序列; 该行证明的是“步长限制未被触发”。

8.5 守恒量与定性性质

离散守恒 (两个互补的残差)。 按命题 4.2, $W + Q_m$ 是半离散系统的精确线性不变量。校验器算两种残差 (表 8):

1. **不变量残差:** 由输出数组重算 $W(t) - W(0)$ 并与积分器内累积的 $-Q_m(t)$ 比较, 四个问题均在 10^{-15} 量级 (图 12), 达到双精度舍入极限。因为 Q_m 本身是同一常微分系统的一个状态, 这一项检验的是**离散格式** (控制体测度、界面通量的成对相消、边界通量的施加方式): 任何一处不一致都会立即表现为 $O(1)$ 的残差; 它不检验时间积分的精度, 也无法单独检出扩散系数的错误。
2. **重采样残差:** 完全抛开 Q_m , 直接由输出样本上的表面通量 $h_m(C_s - C_{\text{air}})$ 按梯形法重算时间积分, 与存量变化比较。它与时间积分解耦, 但受输出间隔 (60 s) 的梯形误差限制, 四个问题为 1.9×10^{-6} – 9.5×10^{-5} 。两者一起说明: 格式严格守恒, 而实际通量的时间积分也在输出分辨率允许的精度内一致。

问题 1 的常物性温度方程另有能量不变量, 残差同量级。

极值原理与单调性。 校验器在全部输出时刻检查三项 (命题 4.3): (i) T 、 C 分别落在由初值与环境值张成的包络 $[\min(T_0, \min T_{\text{air}}), \max(T_0, \max T_{\text{air}})]$ 、 $[\min(C_0, \min C_{\text{air}}), C_0]$ 之内; (ii) C 沿 r 单调不减; (iii) $\max_r C$ 在中心取得。三项在四个问题中全部成立 (表 8), 因此不存在非物理的负含水率、超调或内部极值。

判据的一致性。 校验器独立检查: 在 $t < t_{\text{end}}$ 的全部输出行上 $\max_i C_i \geq C^*$, 在 $t \geq t_{\text{end}}$ 的全部行上 $\max_i C_i < C^*$, 且 t_{end} 处的剖面最大值等于 C^* (误差 $< 10^{-12}$)。这确认事件求根定位的确实是判据首次成立的时刻。

退化检验。 问题 4 的移动边界代码在 $R \equiv R_0$ 时应退化为固定域格式。两种写法与固定域求解器之差为 9.4×10^{-14} ，为舍入量级。

8.6 独立实现的交叉检验与一维化的端部效应

为检验实现本身以及假设 1，本文另行编写了一个二维轴对称 (r, z) 有限体积程序，直接在物理坐标下离散，不共享一维求解器的任何代码，轴向网格 $\Delta z = 0.50 \text{ cm}$ ：

1. **代码独立性：**端面取绝热/不透条件时，二维解在中截面上应与一维解完全一致。实测最大差异为 6.1×10^{-9} （水分）与 $5.8 \times 10^{-8} \text{ }^\circ\text{C}$ （温度），为积分容差量级——两套独立编写的程序给出同一答案。
2. **端部效应：**端面同样施加 Robin 条件时，中截面的水分浓度与一维解相差 2.3×10^{-5} ，中心点在烘干时刻的差异为 2.4×10^{-6} ，烘干时间由 58.50 h 变为 58.50 h，相对变化 -0.004% 。

端部效应之小与第 4.2 小节的量级判断一致：水分的轴向 Fourier 数上界仅 0.18，端面的影响来不及扩散到中截面。图 13（附录 B）显示轴向水分分布只在端面附近的薄层内偏离中截面值。因此假设 1（一维径向化）被定量确认。

8.7 结果文件与表格的一致性检查

四个结果工作簿（以及补充交付的result2_全过程.xlsx）由附件 3 的模板生成（不重建工作簿结构），并由自动契约逐项检查：工作表名称与模板一致、A1 文字与模板一致、首行的距离刻度落在 0, 0.5, 1, 1.5, 2 cm 的应有位置、表头单元数（result4.xlsx 为 23，含“药材表面”列）、A 列时间的步长与起点（1s/60s）、行数区间、数值区无非数字单元、小数位不超过四位、问题 4 中且仅在超出当前半径处留空。全部契约检查通过（附录 D）。论文表格与结果文件由同一份求解结果生成，因此逐位一致：result3.xlsx 共 3452 个时间行、result4.xlsx 共 3067 行。

8.8 灵敏度与不确定性

对六个可能存在不确定性的量分别取 $\times 0.8, 0.9, 1.1, 1.2$ 重新求解，以烘干时间 t_{end} 为指标，用 $\pm 10\%$ 的中心差分估计弹性 $E_p = (\Delta t_{\text{end}}/t_{\text{end}})/(\Delta p/p)$ （表 18，附录 A；图 10）：

1. **烘房温度平台的弹性最大**（ -1.603 ）： D 的 Arrhenius 因子把温度的相对变化放大了 E/T_K 倍；但实际工艺中提高温度受有效成分热稳定性的限制（第 10 节）。
2. **扩散系数前因子 D_0 的弹性**为 -0.893 （问题 3）与 -0.889 （问题 4），接近 -1 ，正是扩散控制过程的特征（ $t \propto 1/D$ ）； D_0 是经验拟合量，其不确定性几乎等比例地传递到烘干时间，是最主要的参数不确定性来源。
3. **收缩幅度 s 定义为对收缩量的缩放**： $R(t; s) = R_0 - s[R_0 - R_{\text{附件 2}}(t)]$ ， $s = 1$ 即题给数据， $s = 1.2$ 表示收缩量放大 20%（末半径由 1.198 降到 1.038 cm）， $s = 0.8$ 则升到 1.358 cm。实测弹性 -1.130 ，而由 $t \propto R_\infty^2$ 给出的标度预测为 $d \ln t / d \ln s = -2s(R_0 - R_\infty)/R_\infty = -1.34$ ；实测略小于预测，因为收缩需要时间完成，前期仍以较大的半径扩散。二者同号同量级，再次确认收缩通过缩短扩散路径起作用。

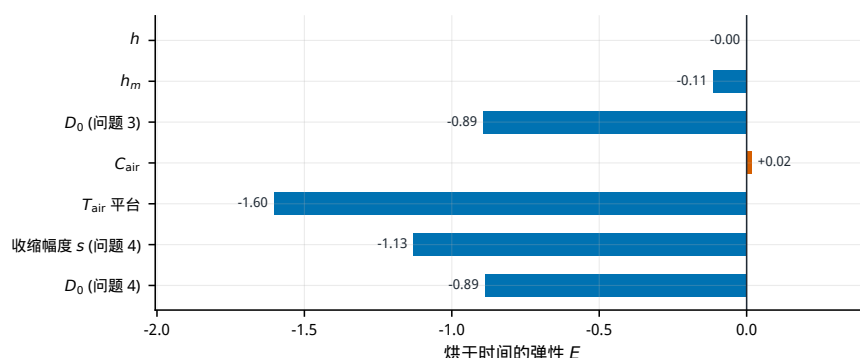


图 10 烘干时间对各参数的弹性 ($\pm 10\%$ 中心差分)。负值表示参数增大使烘干变快。烘房温度平台、扩散系数前因子与收缩幅度是主导因素，对流换热系数几乎无影响。

4. 传质系数 h_m 的弹性只有 -0.114 ，换热系数 h 几乎为零 (-0.001)，烘房湿度 C_{air} 为 0.019 。结论：在题给参数下，过程由内部扩散控制，强化外部对流（加大风速）对缩短烘干时间几乎无效。

不确定性预算。 把上述结果整理为 t_{end} 的误差来源：数值离散（网格 ≤ 0.41 min，时间积分 ≤ 0.01 s，一维化 -0.004% ）合计远小于 0.1% ；离散化与插值的解释性选择（平台取法 -0.53% 、界面平均 0.11% ）不超过 1% ，收缩域写法 3.06% ；守恒写法（是否按 ρ_s 加权，注 4.1） -18.71% ；参数不确定性（若 D_0 有 $\pm 20\%$ 的拟合误差）使 t_{end} 在表 18 的 $\times 0.8$ 与 $\times 1.2$ 两行之间变动，约为两成；而判据的表述（“各处” vs “平均”）可带来 -37.28% 的改变。因此本文报告的 t_{end} 在题给模型内部是精确到 0.4 min 的，其现实不确定性主要来自经验公式的拟合精度、水分守恒的写法与工艺判据的定义，而非数值方法。

8.9 备选解释的对照

第 1.3 小节列出的每一处解释都给出了备选结果（表 9）：恒温阶段平台值的三种取法相差不超过 -0.53% ；界面系数三种取法相差不超过 0.11% ；问题 4 的实验室坐标写法相差 3.06% ；半径固定的对照相差 154.17% （用于分离收缩与物性两种效应）；守恒型水分方程相差 -18.71% ；判据改用体积平均相差 -37.28% 。可见数值与插值层面的解释只带来百分之几以内的变化；真正主导答案的是两处建模层面的表述——水分守恒的写法（约 -19% ）与烘干判据的定义（约 -37% ），两者都已在第 1.3 小节明确声明并在此定量给出。

9 结论

表 10 汇总四个问题的答案；每个数字都可在对应小节与结果文件中逐格核对。
三条跨问题的结论：

1. **过程由内部扩散控制。** 传质 Biot 数由 2.84 升到 10^3 量级，烘干时间对 h 的弹性只有 -0.001 、对 h_m 为 -0.114 ，而对扩散前因子 D_0 为 -0.893 、对烘房温度平台为 -1.603 。工艺上应提温或减小尺寸，加大风速无效。
2. **“各处低于 0.15 ”是最敏感的一处表述。** 满足它需要 57.48 h，而此时体积平均含水率已低至 0.1243 kg/kg；改用平均含水率判据只需 36.05 h (-37.28%)。判据的定义

表 9 备选解释与离散化选择下的烘干时间（相对主结果的变化）

备选	t_{end}/h	变化/%
平台取末样本（MDR-0003 方案 2）	57.18	-0.53
平台取 50 °C、0.05（方案 3）	57.48	-0.00
界面系数算术平均	57.46	-0.03
界面系数调和平均	57.54	0.11
守恒型水分方程 $\partial_t(\rho_s C) = \nabla \cdot (\rho_s D \nabla C)$	46.73	-18.71
问题 4 欧拉写法（含对流项）	52.65	3.06
问题 4 物性、半径固定 2 cm	129.85	154.17
以平均含水率 < 0.15 为判据	36.05	-37.28

表 10 四个问题的主要结果汇总（详见对应小节；结果文件由附件 3 的模板生成）

问题	主要答案	交付文件	详见
1（预热平衡，附录 2）	1800 s: 中心 33.5753 °C / 2.5500, 表面 36.7856 °C / 1.5102; 失水 10.06%	result1.xlsx	第 4 节
2（全过程，附录 3）	3 h: 中心 49.8495 °C / 1.7662, 表面 1.0081; 失水 45.78%	result2.xlsx、 result2_全过程.xlsx	第 5 节
3（烘干时长）	$t_{\text{end}} = 57.48 \text{ h}$ (2.40 天; 外推 57.4737 h), 由中心控制	result3.xlsx	第 6 节
4（尺寸变化）	$t_{\text{end}} = 51.09 \text{ h}$, 末半径 1.200 cm; 同物性固定半径需 129.85 h	result4.xlsx	第 7 节

比任何参数的 $\pm 20\%$ 扰动都更决定答案。

3. **数值上的答案是可核查的，物理上的答案还有已知的偏差方向。**离散与时间积分误差合计远小于 0.1% (第 8 节)，解释性选择中最大的两项为守恒写法 (-18.71%) 与判据表述 (-37.28%)；题给模型省略了蒸发潜热，界定分析表明真实烘干时间很可能比主结果长 17.3% – 26.3% (第 10.2 小节)。

10 模型评价与推广

本节回答三个问题：本文的模型好在哪里；题给参数体系省略了哪些物理机制、它们能把答案推到什么范围；以及在此基础上能给出什么工艺建议。第 10.2 小节的全部计算都在题给模型之外进行，不改变第 4–7 节的任何结果。

10.1 模型的优点

1. **一个模型覆盖四个问题。**从守恒律出发建立的(3)–(7) 是唯一的定解问题，四个问题只更换物性、时间范围与边界是否移动；所有结果由同一套程序、同一次云端运行产生。
2. **离散格式具有可证明的结构性质。**有限体积离散使水分（及常物性下的能量）平衡成为半离散系统的**精确线性不变量**（命题 4.2），并满足离散极值原理（命题 4.3）。这两条不是事后的数值观察，而是格式的性质，保证了长时间积分不会累积质量误差、不会产生非物理的负含水率。
3. **精度可控且被独立确认。**问题 1–3 的输出点与网格节点重合（无插值误差；问题 4 的固定物理位置需在 ξ 网格上作三次样条重采样，其误差已含在表 11 的问题 4 行内）；收敛阶、解析解误差与 Richardson 外推共同界定了四位小数的有效范围（第 8.2 小节，包括诚实地指出最初数十秒表面单元的例外）；校验器与求解器代码分离，二维轴对称程序是完全独立的第二实现，问题 1 的温度场另有 Duhamel 真解对照。题目未规定之处（第 1.3 小节的五处，加上判据分辨率与界面平均）都给出了备选结果与差异幅度（表 9），使结论的稳健性可检验而不是靠断言。

10.2 题给模型省略的两项物理机制及其影响范围

题给参数体系把表面传质写成 $-D\partial_r C = h_m(C_s - C_{\text{air}})$ ，即把药材的**干基含水率**与烘房空气的“水分浓度”放在同一量纲下比较，并且能量方程不含蒸发潜热。主结果必须、也确实严格按题给参数计算；本小节在题给模型之外量化这两项被省略的机制能把答案推到什么范围。全部计算在问题 3 的设定下进行（固定半径、附录 3 物性），不改变第 4–7 节的任何结果。

(a) **蒸发潜热的后验一致性诊断。** 由问题 3 的解可以反算题给传质律隐含的蒸发质量通量 $j_w = \rho_{s0} h_m (C_s - C_{\text{air}})$ ，其中干物质表观密度 $\rho_{s0} = \rho(C_0)/(1 + C_0) = 275 \text{ kg/m}^3$ （按初始状态折算，仅用于把 (kg/kg) 的通量折算成 $\text{kg}/(\text{m}^2 \text{ s})$ ）。把其潜热负荷折算为“等效表面温降” $\Delta T_{\text{lat}} = L_v(T_s) j_w / h$ （即若要在能量上支付这一蒸发速率，表面需要比烘房低多少度）：初始时 $\Delta T_{\text{lat}} = 54 \text{ K}$ ，3 h 时 20.1 K ，12 h 时 2.4 K ，降到 1 K 以下需要 16.1 h （图 14(a)，附录 B）。全过程蒸发的水量为 0.838 kg 每米长度，对应潜热 2.01

MJ/m, 而题给能量边界条件全程供入的净对流热量只有 84.1 kJ/m (等于显热储量的增加), 二者相差 24 倍。**结论:** 题给模型在最初十余小时显著高估药材温度 (真实过程中表面接近湿球温度), 因而高估 D 、低估烘干时间; 16.1 h 之后潜热的等效温降已不足 1 K, 模型自愈。直接把潜热项加入能量边界条件是不可行的——题给传质律没有湿球反馈 (蒸发速率不随表面降温而减小), 会把表面温度压到 0 °C 以下, 问题不适定。

(b) 能量限制 (恒速干燥段) 的界定分析。 把烘房的“水分浓度”按湿空气含湿量 (kg 水/kg 干空气) 解读 (饱和蒸气压用 Magnus 公式 [21], 湿球温度由绝热饱和方程求根 [22]), 则初始状态 (28 °C 与附件 1 的首个含湿量样本) 对应相对湿度 82%、湿球温度 25.5 °C; 恒温阶段 (49.999 °C、0.04999 kg/kg) 对应相对湿度 61%、湿球温度 41.7 °C (湿球温差 8.3 K), 数值上合理。经典干燥理论 [23, 2] 指出, 恒速段的蒸发通量不能超过对流供热所能维持的湿球极限式 (21)

$$j_{\max}(t) = \frac{h[T_{\text{air}}(t) - T_{\text{wb}}(t)]}{L_v}, \quad (21)$$

初始时刻 $j_{\max} = 0.091$ 、恒温阶段 $j_{\max} = 0.312$ kg/(m² h), 而题给模型的初始蒸发通量为 2.00 kg/(m² h)——是**同时刻**上限的 22 倍, 即使与恒温阶段的上限相比也还有 6.4 倍。把表面损失律改为 $\min\{h_m(C_s - C_{\text{air}}), j_{\max}/\rho_s\}$ (这保持了命题 4.2 的守恒不变量, 且当表面变干后自动退化为题给模型), 得烘干时间 67.40 h, 比主结果长 17.3%; 上限在 17.0 h 后不再起作用。此处的换算密度 ρ_s 有两种取法: 用初态常数 $\rho_{s0} = 275$ 得上述 17.3%; 用局部的 $\rho_s(C_s(t))$ (干燥后期升到 582 kg/m³, 因而上限更紧) 得 72.59 h (+26.3%)。把上限取 $\times 0.5$ 与 $\times 2$, 结果分别为 85.19 h 与 59.96 h (表 17, 附录 A)。因此潜热被省略所带来的影响范围应表述为 +17.3% 至 +26.3% 量级, 而不是单一数值。另一方面, 若全程按湿球极限蒸发, 得到纯能量下界 21.4 h (−62.7%) ——主结果远高于该下界, 再次说明**题给设定下的烘干时长由内部扩散控制而非能量供给控制**。

(c) 平衡含水率驱动力。 物理上传质的驱动力应为 $C_s - C_{\text{eq}}$, C_{eq} 是药材在烘房温湿度下的平衡含水率 (由等温吸附曲线给出, 题目未提供)。以常数 $C_{\text{eq}} \in \{0.075, 0.100, 0.125\}$ 替换 C_{air} , 烘干时间分别为 58.45、61.31、69.82 h (+1.7%、+6.7%、+21.5%)。当 $C_{\text{eq}} \rightarrow C^* = 0.15$ 时烘干时间发散: **终点判据必须与烘房湿度相匹配**, 否则物理上不可达——这是本模型给出的最有工艺价值的结论之一。值得注意的是, 这一单调性并非普遍成立: 驱动力减小使表面更湿, 而 $D \propto e^{-a/C}$ 使湿表面的扩散系数大得多, 两种效应竞争; 在附录 2 的律 ($a = 0.89$) 下较大的 C_{eq} 反而可能略快 (表面结壳减轻, case hardening), 附录 3 ($a = 0.45$) 下则单调变慢。

10.3 局限

- 表面条件的物理内涵。** 主模型按题给参数把烘房含湿量与药材干基含水率直接相减, 未使用等温吸附曲线; 第 10.2 小节的 (c) 给出了这一简化的影响区间 (+1.7% 至 +21.5%)。
- 蒸发潜热未耦合。** 见 (a)、(b); 界定性结果表明真实烘干时间很可能比主结果长 17.3% 至 26.3% 量级 (表 17)。注意情形 (b) 只限制质量通量而未改变能量方程 (温度仍按题给模型演化), 因此是**界定分析**而非完整的耦合模型; 把烘房“水分浓度”读作湿空气含湿量也是一种解释, 主结果不依赖它。

3. **水分守恒的写法**。主模型的干基 Fick 形式在“固定体积 + 干物质守恒”下是守恒律本身；若把 $\rho(C)$ 读作真实湿密度而采用 ρ_s 加权的守恒型写法，烘干时间变为 46.73 h (−18.71%)。这是除判据表述外**最大的单项解释不确定性**，已在第 1.3 小节声明、在注 4.1 论证、在表 9 量化。
4. **收缩数据的自治性与已知性**。问题 4 直接使用给定的 $R(t)$ ，未建立含水率 $\rightarrow$ 体积的本构关系；更进一步，题给的 $R(t)$ 与题给的 $\rho(C)$ 在干物质守恒下**相差 13.4%**（由 $\rho(C)$ 反推的末半径为 1.287 cm，附件 2 给出 1.198 cm），这是题给数据的内部不一致，本文以几何为准并把 $\rho(C)$ 限用于 ρ_{c_p} 。轴向收缩、开裂以及表面硬壳下均匀收缩假设的失真均未建模。
5. **物性与传递系数的常数化**。 h 、 h_m 全程恒定（收缩使表面积减小、气流条件改变时它们会变化）； D 的经验公式在 $C \rightarrow 0$ 时的外推缺乏数据支撑，而正是这一区间决定了烘干末段的长尾。
6. **一维与均匀性**。端部效应已量化为 −0.004%（可忽略），但各向异性、非均质与批次内个体差异未建模。
7. **烘房条件的外推**。14 400 s 之后的烘房状态由平台值外推（假设 5），虽经三种取法对照（ $\leq -0.53\%$ ），但若实际工艺存在升温/降温程序，结果会改变——由弹性 −1.603 可直接估算。

10.4 推广与工艺含义

1. **模型的推广**。本文的离散框架（Landau 坐标 + 有限体积 + 分段自适应 BDF + 事件求根）可直接推广到球形与板状药材（更换几何权 ξ^{d-1} ）、由含水率驱动的自适应收缩（把 $R(t)$ 换成由 $\bar{C}$ 决定的本构关系）、含潜热耦合与吸附平衡的完整模型，以及以能耗或品质为目标的**烘房温湿度轨迹优化**（把 $T_{\text{air}}(t)$ 作为控制变量，本文求解器作为状态方程反复调用）。
2. **工艺含义**。灵敏度给出明确的优先级：(i) 提高烘房温度最有效 (−1.603)，但受热敏成分降解约束，宜先低温排湿、后升温；(ii) **减小特征尺寸最可靠** ($t_{\text{end}} \propto R^2$ ，问题 4 中收缩本身就把时间压缩到 1/2.54)；(iii) 加大风速几乎无效 (−0.001、−0.114) 反而增加能耗；(iv) 降温收益也小 (0.019)，但须保证 C_{eq} 显著低于 C^* ，否则终点不可达；(v) 判据定义值得与工艺方商榷：按“各处低于 0.15”需 57.48 h，而此时体积平均含水率已低至 0.1243 kg/kg，大量物料被过度干燥。

AI 工具使用声明

本研究在问题分析、方案比较、程序编写与调试、云端实验组织、数值核验、图表绘制以及论文文字组织等环节使用了生成式 AI 工具（Anthropic Claude Code，模型 Claude Fable 5.1）；所有数据处理、计算与排版在 Modal 云平台执行。使用过程与人工核验方式详见支撑材料中的《AI 工具使用详情.pdf》。

参考文献

- [1] ZOGZAS N P, MAROULIS Z B, MARINOS-KOURIS D. Moisture diffusivity data compilation in foodstuffs [J/OL]. Drying Technology, 1996, 14(10): 2225-2253. DOI: 10.1080/07373939608917205.

- [2] MUJUMDAR A S. Handbook of industrial drying[M]. 4th ed. Boca Raton, FL: CRC Press, 2014.
- [3] WHITAKER S. Simultaneous heat, mass, and momentum transfer in porous media: a theory of drying [J/OL]. *Advances in Heat Transfer*, 1977, 13: 119-203. DOI: 10.1016/S0065-2717(08)70223-5.
- [4] FRITSCH F N, CARLSON R E. Monotone piecewise cubic interpolation[J/OL]. *SIAM Journal on Numerical Analysis*, 1980, 17(2): 238-246. DOI: 10.1137/0717021.
- [5] RATTI C. Shrinkage during drying of foodstuffs[J/OL]. *Journal of Food Engineering*, 1994, 23(1): 91-105. DOI: 10.1016/0260-8774(94)90125-2.
- [6] LUIKOV A V. Heat and mass transfer in capillary-porous bodies[M]. Oxford: Pergamon Press, 1966.
- [7] INCROPERA F P, DEWITT D P, BERGMAN T L, et al. Fundamentals of heat and mass transfer[M]. 6th ed. Hoboken, NJ: John Wiley & Sons, 2007.
- [8] PATANKAR S V. Numerical heat transfer and fluid flow[M]. Washington, D.C.: Hemisphere Publishing Corporation, 1980.
- [9] QUARTERONI A, SACCO R, SALERI F. Numerical mathematics[M]. 2nd ed. Berlin: Springer, 2007.
- [10] VIRTANEN P, GOMMERS R, OLIPHANT T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python[J/OL]. *Nature Methods*, 2020, 17(3): 261-272. DOI: 10.1038/s41592-019-0686-2.
- [11] HARRIS C R, MILLMAN K J, VAN DER WALT S J, et al. Array programming with NumPy[J/OL]. *Nature*, 2020, 585(7825): 357-362. DOI: 10.1038/s41586-020-2649-2.
- [12] HAIRER E, WANNER G. Solving ordinary differential equations II: Stiff and differential-algebraic problems [M]. 2nd ed. Berlin: Springer, 1996.
- [13] SHAMPINE L F, REICHEL T M W. The MATLAB ODE suite[J/OL]. *SIAM Journal on Scientific Computing*, 1997, 18(1): 1-22. DOI: 10.1137/S1064827594276424.
- [14] LANDAU H G. Heat conduction in a melting solid[J/OL]. *Quarterly of Applied Mathematics*, 1950, 8(1): 81-94. DOI: 10.1090/qam/33441.
- [15] CRANK J. Free and moving boundary problems[M]. Oxford: Clarendon Press, 1984.
- [16] ROACHE P J. Verification and validation in computational science and engineering[M]. Albuquerque, NM: Hermosa Publishers, 1998.
- [17] OBERKAMPF W L, ROY C J. Verification and validation in scientific computing[M]. Cambridge: Cambridge University Press, 2010.
- [18] CARSLAW H S, JAEGER J C. Conduction of heat in solids[M]. 2nd ed. Oxford: Clarendon Press, 1959.
- [19] CRANK J. The mathematics of diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [20] RICHARDSON L F, GAUNT J A. The deferred approach to the limit[J/OL]. *Philosophical Transactions of the Royal Society A*, 1927, 226: 299-361. DOI: 10.1098/rsta.1927.0008.
- [21] ALDUCHOV O A, ESKRIDGE R E. Improved Magnus form approximation of saturation vapor pressure [J/OL]. *Journal of Applied Meteorology*, 1996, 35(4): 601-609. DOI: 10.1175/1520-0450(1996)035<0601:IMFAOS>2.0.CO;2.
- [22] ASHRAE. ASHRAE handbook—fundamentals, chapter 1: Psychrometrics[M]. Atlanta, GA: American Society of Heating, Refrigerating and Air-Conditioning Engineers, 2017.
- [23] SHERWOOD T K. The drying of solids—I[J/OL]. *Industrial & Engineering Chemistry*, 1929, 21(1): 12-16. DOI: 10.1021/ie50229a004.

A 补充表格

本附录收录正文引用的大型表格：网格收敛表（表 11）、与级数解的对照表（表 12）、烘干时间的网格收敛表（表 13）、结果文件完整输出网格上的误差表（表 14）、时间积分无关性表（表 15）问题 4 备选写法的结果表（表 16）模型评价扩展的结果表（表 17）以及灵敏度重解表（表 18）。

表 11 网格收敛：以 $\Delta r = 0.1/2^6$ cm 为参照的最大绝对误差与两种观测阶

量	k	$\Delta r/\text{cm}$	最大误差	自收敛阶	三网格阶
问题 1 温度	0	0.1000	1.96e-03	2.00	2.00
问题 1 温度	1	0.0500	4.90e-04	2.00	2.00
问题 1 温度	2	0.0250	1.22e-04	2.02	2.00
问题 1 温度	3	0.0125	3.02e-05	2.07	2.00
问题 1 温度	4	0.0063	7.20e-06	2.31	1.99
问题 1 温度	5	0.0031	1.45e-06	—	—
问题 1 水分	0	0.1000	4.10e-02	2.35	2.42
问题 1 水分	1	0.0500	8.05e-03	2.09	2.11
问题 1 水分	2	0.0250	1.89e-03	2.03	2.02
问题 1 水分	3	0.0125	4.61e-04	2.07	2.01
问题 1 水分	4	0.0063	1.10e-04	2.32	2.00
问题 1 水分	5	0.0031	2.19e-05	—	—
问题 2 温度	0	0.1000	1.17e-03	2.00	2.00
问题 2 温度	1	0.0500	2.93e-04	2.00	2.00
问题 2 温度	2	0.0250	7.30e-05	2.02	2.00
问题 2 温度	3	0.0125	1.81e-05	2.06	1.99
问题 2 温度	4	0.0063	4.32e-06	2.32	2.00
问题 2 温度	5	0.0031	8.64e-07	—	—
问题 2 水分	0	0.1000	9.45e-04	2.00	1.99
问题 2 水分	1	0.0500	2.37e-04	2.00	2.00
问题 2 水分	2	0.0250	5.90e-05	2.02	2.00
问题 2 水分	3	0.0125	1.46e-05	2.07	2.00
问题 2 水分	4	0.0063	3.47e-06	2.32	2.00
问题 2 水分	5	0.0031	6.95e-07	—	—
问题 3 水分	0	0.1000	1.94e-03	1.19	1.10
问题 3 水分	1	0.0500	8.50e-04	1.32	1.23
问题 3 水分	2	0.0250	3.41e-04	1.46	1.36
问题 3 水分	3	0.0125	1.24e-04	1.66	1.50
问题 3 水分	4	0.0063	3.94e-05	2.05	1.65
问题 3 水分	5	0.0031	9.51e-06	—	—
问题 4 水分	0	0.1000	9.96e-04	1.71	1.92
问题 4 水分	1	0.0500	3.06e-04	1.49	1.39
问题 4 水分	2	0.0250	1.09e-04	1.69	1.60
问题 4 水分	3	0.0125	3.38e-05	1.89	1.78
问题 4 水分	4	0.0063	9.09e-06	2.25	1.91
问题 4 水分	5	0.0031	1.91e-06	—	—

“自收敛阶” $p_k = \log_2(e_k/e_{k+1})$ 以最细网格为参照，最后一对网格被系统性抬高 $\log_2[(1 - 4^{-(m-k)})/(1 - 4^{-(m-k-1)})]$ （二阶格式在 $m - k = 2$ 时为 2.32）；“三网格阶” $p_k = \log_2(\|u_k - u_{k+1}\|_\infty / \|u_{k+1} - u_{k+2}\|_\infty)$ 不需要参照解，无此偏置（MDR-0011）。

表 12 与 Robin 圆柱级数解的最大绝对误差及观测阶数（前两组为人为构造的常环境算例，第三组为问题 1 本身）

量	k	$\Delta r/\text{cm}$	最大误差	观测阶
常环境温度 (Bi=1.389)	0	0.1000	1.05e-02	2.01
常环境温度 (Bi=1.389)	1	0.0500	2.59e-03	2.00
常环境温度 (Bi=1.389)	2	0.0250	6.47e-04	2.00
常环境温度 (Bi=1.389)	3	0.0125	1.62e-04	2.00
常环境温度 (Bi=1.389)	4	0.0063	4.05e-05	2.00
常环境温度 (Bi=1.389)	5	0.0031	1.01e-05	1.98
常环境温度 (Bi=1.389)	6	0.0016	2.57e-06	—
常环境水分 ($\text{Bi}_m=3.240$)	0	0.1000	1.28e-02	2.16
常环境水分 ($\text{Bi}_m=3.240$)	1	0.0500	2.86e-03	2.02
常环境水分 ($\text{Bi}_m=3.240$)	2	0.0250	7.05e-04	2.00
常环境水分 ($\text{Bi}_m=3.240$)	3	0.0125	1.76e-04	2.00
常环境水分 ($\text{Bi}_m=3.240$)	4	0.0063	4.39e-05	2.00
常环境水分 ($\text{Bi}_m=3.240$)	5	0.0031	1.10e-05	2.00
常环境水分 ($\text{Bi}_m=3.240$)	6	0.0016	2.74e-06	—
问题 1 温度、Duhamel (Bi=1.389)	0	0.1000	1.96e-03	2.00
问题 1 温度、Duhamel (Bi=1.389)	1	0.0500	4.91e-04	2.00
问题 1 温度、Duhamel (Bi=1.389)	2	0.0250	1.23e-04	2.00
问题 1 温度、Duhamel (Bi=1.389)	3	0.0125	3.07e-05	1.99
问题 1 温度、Duhamel (Bi=1.389)	4	0.0063	7.73e-06	1.97
问题 1 温度、Duhamel (Bi=1.389)	5	0.0031	1.98e-06	1.90
问题 1 温度、Duhamel (Bi=1.389)	6	0.0016	5.30e-07	—

第三组把附件 1 的分段线性烘房温度用 Duhamel 叠加代入级数解，因而是问题 1 温度场的真解对照，而非环境算例。

表 13 烘干时间随网格加密的收敛（与最细网格之差，单位 min）与三网格观测阶

问题	k	$\Delta r/\text{cm}$	t_{end}/h	与最细网格之差/min	三网格阶
问题 3	0	0.1000	58.5042	61.702	1.16
问题 3	1	0.0500	57.9133	26.249	1.26
问题 3	2	0.0250	57.6483	10.350	1.38
问题 3	3	0.0125	57.5377	3.711	1.52
问题 3	4	0.0063	57.4952	1.166	1.67
问题 3	5	0.0031	57.4805	0.279	—
问题 3	6	0.0016	57.4758	—	—
问题 4	0	0.1000	51.3337	14.702	1.24
问题 4	1	0.0500	51.1852	5.791	1.43
问题 4	2	0.0250	51.1224	2.021	1.63
问题 4	3	0.0125	51.0990	0.620	1.80
问题 4	4	0.0063	51.0914	0.165	1.91
问题 4	5	0.0031	51.0893	0.035	—
问题 4	6	0.0016	51.0887	—	—

t_{end} 的收敛阶低于场量本身：判据时刻随网格移动，误差同时包含解误差与穿零时刻的移动。Richardson 外推按该观测阶进行（正文第 8.2 小节）。

表 14 结果文件完整输出网格（每 1 s × 每 0.1 cm）上的离散误差与四位小数的可靠性

量	单元数	最大误差	误差最大处	三网格阶	误差 $> 5 \times 10^{-5}$ 的单元	最后一处/s	此后最大误差
问题 1 温度	37800	1.93e-06	300 s / 2 cm	1.98	0	0	1.9e-06
问题 1 水分	37800	3.97e-04	1 s / 2 cm	2.03	42	42	5.0e-05
问题 2 温度	226800	1.66e-06	2940 s / 2 cm	2.02	0	0	1.7e-06
问题 2 水分	226800	3.23e-04	1 s / 2 cm	2.02	29	29	5.0e-05

误差相对由 $k = 6, 7$ 两级网格作 Richardson 外推得到的极限。 5×10^{-5} 是四位小数的半个末位：误差低于它的单元，第四位小数由离散误差决定的可能性已被排除；“最后一处”之后的全部输出时刻上误差见末列。

表 15 时间积分无关性（问题 3，生产网格；与生产设置 BDF, rtol=1e-09, atol=1e-11、步长不限的差异）

积分器、容差与最大步长	水分最大差	t_{end} 差/s	右端项调用次数
BDF, rtol=1e-10, atol=1e-12	3.61e-09	-0.01	32741
Radau, rtol=1e-09, atol=1e-11	4.24e-09	-0.01	34619
BDF, rtol=1e-07, atol=1e-09	2.55e-07	0.31	10523
BDF, rtol=1e-09, atol=1e-11, $\Delta t \leq 3600$ s	0.00e+00	0.00	22528
BDF, rtol=1e-09, atol=1e-11, $\Delta t \leq 900$ s	1.66e-09	-0.01	23048
BDF, rtol=1e-09, atol=1e-11, $\Delta t \leq 300$ s	2.28e-09	-0.01	24917

$\Delta t \leq$ 行为限制最大步长的自适应 BDF（其余设置同生产）；步长限制改变的只是步序列，结果差异应在容差量级。

表 16 备选写法（eulerian, MDR-0006）下的问题 4 水分浓度（kg/kg）

时间/h	0	0.5	1	1.5	2	药材表面
6	1.9518	1.7824	1.2569	—	—	0.5669
12	0.8734	0.7763	0.4859	—	—	0.2144
18	0.4593	0.4130	0.2652	—	—	0.1020
24	0.3072	0.2805	0.1902	—	—	0.0713
30	0.2380	0.2198	0.1555	—	—	0.0610
36	0.2000	0.1861	0.1356	—	—	0.0567
42	0.1761	0.1647	0.1226	—	—	0.0545
48	0.1596	0.1499	0.1135	—	—	0.0532
烘干结束时间（52.65 h）	0.1500	0.1413	0.1081	—	—	0.0526

表 17 模型评价扩展（MDR-0008，问题 3 设定）：能量限制与平衡含水率驱动力下的烘干时间及相对主结果的变化

情形	参数	t_{end}/h	变化/%
题给模型（主结果）	—	57.48	0.0
蒸发通量受能量限制（湿球极限 $\times 0.5$ ）	$j_{\text{max}} = 0.156 \text{ kg}/(\text{m}^2 \cdot \text{h})$	85.19	48.2
蒸发通量受能量限制（湿球极限 $\times 1$ ）	$j_{\text{max}} = 0.312 \text{ kg}/(\text{m}^2 \cdot \text{h})$	67.40	17.3
蒸发通量受能量限制（湿球极限 $\times 2$ ）	$j_{\text{max}} = 0.623 \text{ kg}/(\text{m}^2 \cdot \text{h})$	59.96	4.3
蒸发通量受能量限制（换算用局部 $\rho_s(C_s)$ ）	$j_{\text{max}} = 0.312 \text{ kg}/(\text{m}^2 \cdot \text{h})$	72.59	26.3
驱动力取 $C_s - C_{\text{eq}}$ （平衡含水率）	$C_{\text{eq}} = 0.075 \text{ kg/kg}$	58.45	1.7
驱动力取 $C_s - C_{\text{eq}}$ （平衡含水率）	$C_{\text{eq}} = 0.1 \text{ kg/kg}$	61.31	6.7
驱动力取 $C_s - C_{\text{eq}}$ （平衡含水率）	$C_{\text{eq}} = 0.125 \text{ kg/kg}$	69.82	21.5
纯能量下界（全程按湿球极限蒸发）	$j_{\text{max}} = 0.312 \text{ kg}/(\text{m}^2 \cdot \text{h})$	21.42	-62.7

湿球极限 $j_{\text{max}} = h(T_{\text{air}} - T_{\text{wb}})/L_v$ 随烘房状态变化，表中给出恒温阶段的值。

表 18 参数 $\pm 10\%$ 、 $\pm 20\%$ 时的烘干时间 (h) 与弹性 $E = (\Delta t_{\text{end}}/t_{\text{end}})/(\Delta p/p)$

参数	基准	$\times 0.8$	$\times 0.9$	$\times 1.1$	$\times 1.2$	弹性
h	57.48	57.50	57.49	57.47	57.47	-0.001
h_m	57.48	59.18	58.22	56.90	56.44	-0.114
D_0 (问题 3)	57.48	70.25	63.14	52.87	49.05	-0.893
C_{air}	57.48	57.31	57.38	57.60	57.74	0.019
T_{air} 平台	57.48	80.60	67.72	49.29	42.71	-1.603
收缩幅度 s (问题 4)	51.09	63.48	57.07	45.53	40.40	-1.130
D_0 (问题 4)	51.09	62.35	56.09	47.01	43.62	-0.889

B 补充图形

本附录收录正文引用的四张支撑性图形：边界数据、离散守恒残差、端部效应与模型评价扩展。

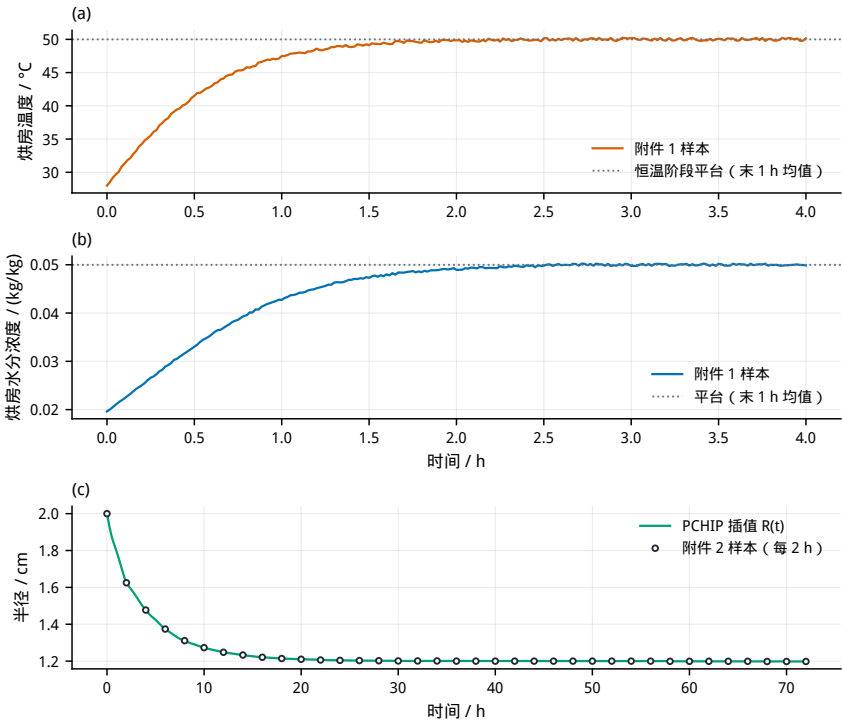


图 11 边界数据：附件 1 的烘房温度 (a) 与水分浓度 (b) 及恒温干燥阶段的平台值 (末 1 h 样本均值，假设 5)；附件 2 的半径样本与保单调 PCHIP 插值 $R(t)$ (c，为清晰起见每 2 h 标一个样本点)。

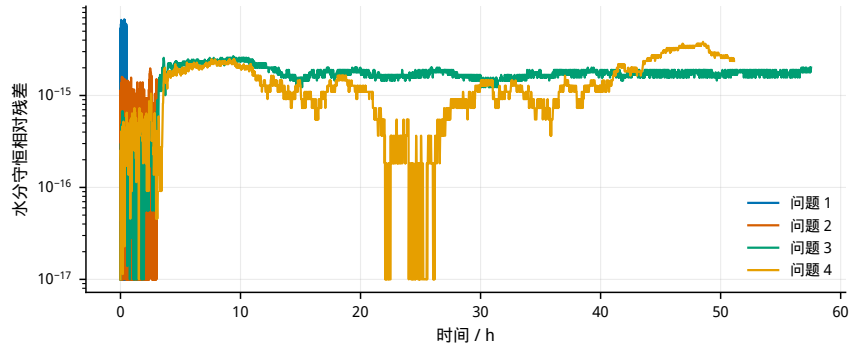


图 12 离散水分存量的变化与积分器内累积的表面通量之差，按最终失水量归一：四个问题的相对残差全程保持在 10^{-15} 量级（双精度舍入），确认离散格式严格守恒。

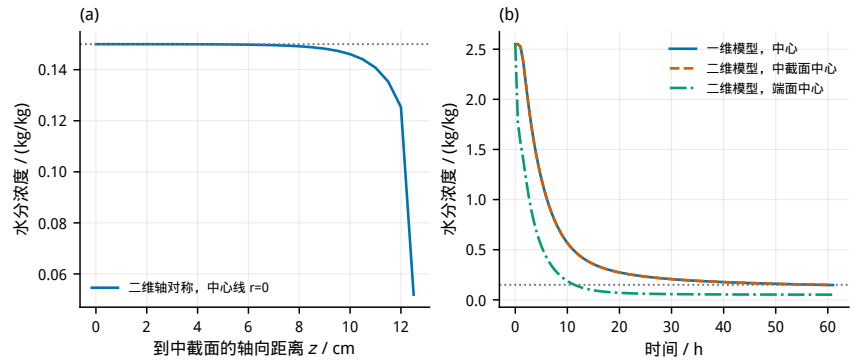


图 13 端部效应：一维烘干结束时刻二维模型中心线 ($r = 0$) 上沿轴向的水分浓度分布 (a)；一维解与二维解在中截面与端面处的中心水分时程 (b)。端面附近的额外干燥只影响端部薄层，不影响中截面，烘干时间的相对变化仅 -0.004% 。

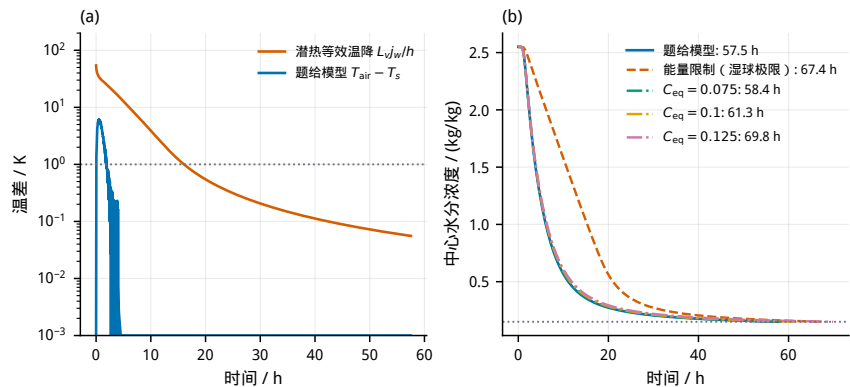


图 14 模型评价扩展：(a) 题给模型蒸发率所对应的潜热等效温降 $L_v j_w / h$ 与模型实际的表面温差 $T_{\text{air}} - T_s$ （纵轴对数坐标，点线为 1 K）——前十余小时前者远大于后者，说明题给能量方程未支付蒸发潜热；(b) 题给模型、能量限制情形与三档平衡含水率驱动力下的中心水分浓度时程，点线为判据 0.15 kg/kg 。图 (a) 中模型温差在 2–4 h 间因烘房温度样本的波动（末 1 h 内相邻样本最大差 0.45 K ）而在零附近改变符号，负值在对数坐标下被截断，故呈现振荡带。

C 支撑材料文件清单

下表列出支撑材料压缩包内的全部文件（相对路径）及其用途。压缩包内另含 MANIFEST.json，记录每个文件的 SHA-256。

文件	说明
AI_USAGE.md	AI 工具使用记录（源文件）
AI 工具使用详情.pdf	AI 工具使用详情
MANIFEST.json	支撑材料成员清单与 SHA-256
Makefile	本地快捷命令（只提交云端任务）
README.md	仓库说明与复现入口
REPRODUCE.md	复现说明
UNLICENSE	公有领域声明（Unlicense）
docs/ARCHITECTURE.md	仓库架构与唯一计算面的设计
docs/CONTRIBUTING.md	协作与提交规范
docs/DATA_NOTES.md	数据口径与附件解析约定
docs/ENGINEERING_STANDARD.md	工程标准与门禁定义
docs/MODELING_STANDARD.md	建模与写作标准（审稿清单）
docs/RESULTS.md	本轮结果总览与逐项证据
docs/REVIEW_RESPONSE.md	评审意见的逐条处理说明
docs/RUNBOOK.md	运行手册（完整命令序列与实测用时）
docs/adr/0001-cloud-only-execution.md	工程规范、建模标准、决策记录
docs/adr/0002-stage-run-manifest-model.md	工程规范、建模标准、决策记录
docs/adr/0003-independent-repos-vendored-runtime.md	工程规范、建模标准、决策记录
docs/mdr/0000-template.md	建模决策记录（题意解释、假设、目标与方法的选择理由）
docs/mdr/0001-governing-equations-and-radial-reduction.md	建模决策记录（题意解释、假设、目标与方法的选择理由）
docs/mdr/0002-transfer-coefficients-for-q2-q4.md	建模决策记录（题意解释、假设、目标与方法的选择理由）
docs/mdr/0003-chamber-conditions-interpolation-and-plateau.md	建模决策记录（题意解释、假设、目标与方法的选择理由）

文件	说明
docs/mdr/0004-numerical-scheme.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0005-drying-end-criterion-and-output-ranges.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0006-moving-boundary-formulation-q4.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0007-verification-and-sensitivity-plan.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0008-latent-heat-and-equilibrium-moisture-extension.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0009-derived-paper-quantities.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0010-moisture-conservation-form.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0011-observed-order-and-four-decimal-claim.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0012-duhamel-benchmark-for-problem-one.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/README.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/problem_brief.md	题目要点与交付清单
forge/__init__.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/config.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/context.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/contracts.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/events.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/hashing.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/manifest.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/modal_app.py	云端应用定义(镜像、卷、四种规格的执行函数)
forge/packaging.py	云端运行时(阶段执行、清单、契约、排版、打包)

文件	说明
forge/pdfqa.py	云端运行时（阶段执行、清单、契约、排版、打包）
forge/plotting.py	绘图规范（矢量 PDF + 预览 PNG、色板、字体）
forge/runner.py	阶段注册与执行（依赖、清单、错误捕获）
forge/tex.py	云端运行时（阶段执行、清单、契约、排版、打包）
forge/xlsx.py	从组委会模板生成结果工作簿
manuscript/ai_usage.tex	论文 LaTeX 源码
manuscript/main.tex	论文 LaTeX 源码
manuscript/references.bib	论文 LaTeX 源码
manuscript/sections/00_abstract.tex	论文 LaTeX 源码
manuscript/sections/01_restatement.tex	论文 LaTeX 源码
manuscript/sections/02_analysis.tex	论文 LaTeX 源码
manuscript/sections/03_assumptions.tex	论文 LaTeX 源码
manuscript/sections/04_models.tex	论文 LaTeX 源码
manuscript/sections/085_conclusion.tex	论文 LaTeX 源码
manuscript/sections/08_validation.tex	论文 LaTeX 源码
manuscript/sections/09_evaluation.tex	论文 LaTeX 源码
manuscript/sections/10_ai_statement.tex	论文 LaTeX 源码
manuscript/sections/ai_usage_body.tex	论文 LaTeX 源码
manuscript/sections/appendix_repro.tex	论文 LaTeX 源码
pipelines/__init__.py	本题建模与求解程序
pipelines/a/__init__.py	本题建模与求解程序
pipelines/a/common.py	本题建模与求解程序
pipelines/a/contracts.py	本题建模与求解程序
pipelines/a/derived.py	论文派生量的集中计算
pipelines/a/extension.py	本题建模与求解程序
pipelines/a/figures.py	论文图与表的生成
pipelines/a/outputs.py	结果工作簿的生成
pipelines/a/physics.py	题给物性与经验公式（附录 2/3/4）
pipelines/a/science.py	四个问题的求解编排与输出采样
pipelines/a/solver.py	径向有限体积离散与隐式时间积分

文件	说明
pipelines/a/stages.py	本题各阶段的编排
pipelines/a/verify.py	与求解器分离的独立校验 (解析解、守恒、收敛阶)
pipelines/common/__init__.py	通用阶段：数据接入、门禁、 论文、质量检查、打包发布
pipelines/common/stages.py	通用阶段：数据接入、门禁、 论文、质量检查、打包发布
pipelines/common/validation.py	通用阶段：数据接入、门禁、 论文、质量检查、打包发布
pyproject.toml	项目元数据与静态检查配置
runs/20260912-153103-eb858bf/convergence/convergence.json	阶段数据产物
runs/20260912-153103-eb858bf/convergence/events.jsonl	结构化运行日志
runs/20260912-153103-eb858bf/convergence/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-153103-eb858bf/convergence/numbers.json	该阶段登记的论文数字宏
runs/20260912-153103-eb858bf/convergence/raw_t_dry.json	阶段数据产物
runs/20260912-153103-eb858bf/derived/derived.json	阶段数据产物
runs/20260912-153103-eb858bf/derived/events.jsonl	结构化运行日志
runs/20260912-153103-eb858bf/derived/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-153103-eb858bf/derived/numbers.json	该阶段登记的论文数字宏
runs/20260912-153103-eb858bf/extension/diagnostic.npz	
runs/20260912-153103-eb858bf/extension/events.jsonl	结构化运行日志
runs/20260912-153103-eb858bf/extension/extension.json	阶段数据产物
runs/20260912-153103-eb858bf/extension/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-153103-eb858bf/extension/numbers.json	该阶段登记的论文数字宏

文件	说明
runs/20260912-153103-eb858bf/extension/variants.npz	
runs/20260912-153103-eb858bf/extension/verification_extension.json	阶段数据产物
runs/20260912-153103-eb858bf/figures/events.jsonl	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figure_index.json	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_chamber.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_chamber.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_conservation.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_conservation.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_convergence.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_convergence.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_ndeffect.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_ndeffect.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_extension.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_extension.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_properties.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_properties.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_q1_history.pdf	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_q1_history.png	论文图 (矢量 PDF 与预览 PNG)
runs/20260912-153103-eb858bf/figures/figures/fig_q1_profiles.pdf	论文图 (矢量 PDF 与预览 PNG)

文件	说明
runs/20260912-153103-eb858bf/figures/figures/fig_q1_profiles.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/figures/figures/fig_q2_history.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/figures/figures/fig_q2_history.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/figures/figures/fig_q2_profiles.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/figures/figures/fig_q2_profiles.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/figures/figures/fig_q3.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/figures/figures/fig_q3.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/figures/figures/fig_q4.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/figures/figures/fig_q4.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/figures/figures/fig_sensitivity.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/figures/figures/fig_sensitivity.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/figures/manifest.json	论文图（矢量 PDF 与预览 PNG）
runs/20260912-153103-eb858bf/ingest/data/附件 1__Sheet1.parquet	原始数据接入结果(散列、题目文本、规范化数据)
runs/20260912-153103-eb858bf/ingest/data/附件 2__Sheet1.parquet	原始数据接入结果(散列、题目文本、规范化数据)
runs/20260912-153103-eb858bf/ingest/events.jsonl	原始数据接入结果(散列、题目文本、规范化数据)
runs/20260912-153103-eb858bf/ingest/inventory.json	原始数据接入结果(散列、题目文本、规范化数据)
runs/20260912-153103-eb858bf/ingest/manifest.json	原始数据接入结果(散列、题目文本、规范化数据)
runs/20260912-153103-eb858bf/ingest/problem_text.txt	原始数据接入结果(散列、题目文本、规范化数据)
runs/20260912-153103-eb858bf/lint/events.jsonl	静态检查报告
runs/20260912-153103-eb858bf/lint/lint_report.json	静态检查报告

文件	说明
runs/20260912-153103-eb858bf/lint/manifest.json	静态检查报告
runs/20260912-153103-eb858bf/lint/mypy.txt	静态检查报告
runs/20260912-153103-eb858bf/lint/ruff_check.txt	静态检查报告
runs/20260912-153103-eb858bf/lint/ruff_format.txt	静态检查报告
runs/20260912-153103-eb858bf/q1/events.jsonl	结构化运行日志
runs/20260912-153103-eb858bf/q1/history.json	阶段数据产物
runs/20260912-153103-eb858bf/q1/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-153103-eb858bf/q1/numbers.json	该阶段登记的论文数字宏
runs/20260912-153103-eb858bf/q1/profiles_moist.parquet	规范化后的输入数据
runs/20260912-153103-eb858bf/q1/profiles_temp.parquet	规范化后的输入数据
runs/20260912-153103-eb858bf/q1/snapshots.npz	
runs/20260912-153103-eb858bf/q1/summary.json	阶段数据产物
runs/20260912-153103-eb858bf/q1/tables.json	阶段数据产物
runs/20260912-153103-eb858bf/q1/verification_q1.json	阶段数据产物
runs/20260912-153103-eb858bf/q2/events.jsonl	结构化运行日志
runs/20260912-153103-eb858bf/q2/history.json	阶段数据产物
runs/20260912-153103-eb858bf/q2/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-153103-eb858bf/q2/numbers.json	该阶段登记的论文数字宏
runs/20260912-153103-eb858bf/q2/profiles_moist.parquet	规范化后的输入数据
runs/20260912-153103-eb858bf/q2/profiles_temp.parquet	规范化后的输入数据
runs/20260912-153103-eb858bf/q2/snapshots.npz	
runs/20260912-153103-eb858bf/q2/summary.json	阶段数据产物
runs/20260912-153103-eb858bf/q2/tables.json	阶段数据产物
runs/20260912-153103-eb858bf/q2/verification_q2.json	阶段数据产物
runs/20260912-153103-eb858bf/q3/dry_profile.npz	
runs/20260912-153103-eb858bf/q3/drying_q3.json	阶段数据产物

文件	说明
runs/20260912-153103-eb858bf/q3/events.jsonl	结构化运行日志
runs/20260912-153103-eb858bf/q3/history.json	阶段数据产物
runs/20260912-153103-eb858bf/q3/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-153103-eb858bf/q3/numbers.json	该阶段登记的论文数字宏
runs/20260912-153103-eb858bf/q3/profiles_moist.parquet	规范化后的输入数据
runs/20260912-153103-eb858bf/q3/profiles_temp.parquet	规范化后的输入数据
runs/20260912-153103-eb858bf/q3/snapshots.npz	
runs/20260912-153103-eb858bf/q3/summary.json	阶段数据产物
runs/20260912-153103-eb858bf/q3/tables.json	阶段数据产物
runs/20260912-153103-eb858bf/q3/verification_q3.json	阶段数据产物
runs/20260912-153103-eb858bf/q4/dry_profile.npz	
runs/20260912-153103-eb858bf/q4/drying_q4.json	阶段数据产物
runs/20260912-153103-eb858bf/q4/drying_q4_alt.json	阶段数据产物
runs/20260912-153103-eb858bf/q4/drying_q4_fixed.json	阶段数据产物
runs/20260912-153103-eb858bf/q4/events.jsonl	结构化运行日志
runs/20260912-153103-eb858bf/q4/history.json	阶段数据产物
runs/20260912-153103-eb858bf/q4/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-153103-eb858bf/q4/numbers.json	该阶段登记的论文数字宏
runs/20260912-153103-eb858bf/q4/profiles_moist.parquet	规范化后的输入数据
runs/20260912-153103-eb858bf/q4/profiles_moist_alt.parquet	规范化后的输入数据
runs/20260912-153103-eb858bf/q4/profiles_temp.parquet	规范化后的输入数据
runs/20260912-153103-eb858bf/q4/snapshots.npz	
runs/20260912-153103-eb858bf/q4/summary.json	阶段数据产物
runs/20260912-153103-eb858bf/q4/tables.json	阶段数据产物
runs/20260912-153103-eb858bf/q4/tables_alt.json	阶段数据产物

文件	说明
runs/20260912-153103-eb858bf/q4/verification_q4.json	阶段数据产物
runs/20260912-153103-eb858bf/qa/events.jsonl	论文与结果文件质量门禁报告
runs/20260912-153103-eb858bf/qa/manifest.json	论文与结果文件质量门禁报告
runs/20260912-153103-eb858bf/qa/qa_report.json	论文与结果文件质量门禁报告
runs/20260912-153103-eb858bf/results/events.jsonl	结果工作簿与求解输出
runs/20260912-153103-eb858bf/results/manifest.json	结果工作簿与求解输出
runs/20260912-153103-eb858bf/results/result1.xlsx	结果工作簿与求解输出
runs/20260912-153103-eb858bf/results/result2.xlsx	结果工作簿与求解输出
runs/20260912-153103-eb858bf/results/result2_全过程.xlsx	结果工作簿与求解输出
runs/20260912-153103-eb858bf/results/result3.xlsx	结果工作簿与求解输出
runs/20260912-153103-eb858bf/results/result4.xlsx	结果工作簿与求解输出
runs/20260912-153103-eb858bf/results/results_report.json	结果工作簿与求解输出
runs/20260912-153103-eb858bf/sensitivity/events.jsonl	结构化运行日志
runs/20260912-153103-eb858bf/sensitivity/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-153103-eb858bf/sensitivity/numbers.json	该阶段登记的论文数字宏
runs/20260912-153103-eb858bf/sensitivity/sensitivity.json	阶段数据产物
runs/20260912-153103-eb858bf/tables/events.jsonl	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/manifest.json	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_alternatives.csv	论文表(LaTeX 片段与 CSV)

文件	说明
runs/20260912-153103-eb858bf/tables/tables/tab _alternatives.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _analytic.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _analytic.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _convergence.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _convergence.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _convergence_drytime.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _convergence_drytime.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _extension.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _extension.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _outputgrid.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _outputgrid.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _q1_moist.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _q1_moist.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _q1_temp.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _q1_temp.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _q2_moist.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _q2_moist.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _q2_temp.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab _q2_temp.tex	论文表(LaTeX 片段与 CSV)

文件	说明
runs/20260912-153103-eb858bf/tables/tables/tab_q3_moist.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_q3_moist.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_q4_moist.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_q4_moist.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_q4_moist_alt.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_q4_moist_alt.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_sensitivity.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_sensitivity.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_time_independence.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_time_independence.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_verification.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/tables/tables/tab_verification.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-153103-eb858bf/test/coverage.json	测试报告与覆盖率
runs/20260912-153103-eb858bf/test/events.jsonl	测试报告与覆盖率
runs/20260912-153103-eb858bf/test/junit.xml	测试报告与覆盖率
runs/20260912-153103-eb858bf/test/manifest.json	测试报告与覆盖率
runs/20260912-153103-eb858bf/test/pytest.txt	测试报告与覆盖率
runs/20260912-153103-eb858bf/test/test_report.json	测试报告与覆盖率
runs/20260912-153103-eb858bf/validate/data_profile.json	数据契约校验报告与数据画像
runs/20260912-153103-eb858bf/validate/events.jsonl	数据契约校验报告与数据画像
runs/20260912-153103-eb858bf/validate/manifest.json	数据契约校验报告与数据画像

文件	说明
runs/20260912-153103-eb858bf/validate/validation_report.json	数据契约校验报告与数据画像
runs/20260912-153103-eb858bf/verification/analytic.json	阶段数据产物
runs/20260912-153103-eb858bf/verification/axism_snapshots.npz	
runs/20260912-153103-eb858bf/verification/events.jsonl	结构化运行日志
runs/20260912-153103-eb858bf/verification/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-153103-eb858bf/verification/numbers.json	该阶段登记的论文数字宏
runs/20260912-153103-eb858bf/verification/verification_report.json	阶段数据产物
tests/__init__.py	单元 / 性质 / 集成测试
tests/conftest.py	单元 / 性质 / 集成测试
tests/integration/__init__.py	单元 / 性质 / 集成测试
tests/integration/test_ingest_validate.py	单元 / 性质 / 集成测试
tests/unit/__init__.py	单元 / 性质 / 集成测试
tests/unit/test_config.py	单元 / 性质 / 集成测试
tests/unit/test_contracts.py	单元 / 性质 / 集成测试
tests/unit/test_derived.py	单元 / 性质 / 集成测试
tests/unit/test_drying.py	单元 / 性质 / 集成测试
tests/unit/test_extension.py	单元 / 性质 / 集成测试
tests/unit/test_hashing.py	单元 / 性质 / 集成测试
tests/unit/test_manifest.py	单元 / 性质 / 集成测试
tests/unit/test_packaging_text.py	单元 / 性质 / 集成测试
tests/unit/test_runner.py	单元 / 性质 / 集成测试
tests/unit/test_xlsx.py	单元 / 性质 / 集成测试
tools/__init__.py	本地控制面: 提交云端任务、下载、校验散列
tools/cli.py	本地控制面: 提交云端任务、下载、校验散列
tools/run_local.py	不依赖云端的本地驱动脚本

D 复现记录

论文中的全部数值、表格与图形均由同一次云端运行 (run 20260912-153103-eb858bf, 代码版本 47ec9ff3f166a857f30b1749bdc644e110b898a8, 构建日期 2026-09-12) 生成。各阶段状态与输出摘要如下。

阶段	状态	用时/s	输出摘要 (SHA-256 前 12 位)
convergence	completed	44.0	8744f13277a4
derived	completed	0.0	9e38afc4ea7d
extension	completed	14.0	517768db57ea
figures	completed	24.0	f4528c37f90a
fmt	completed	0.0	fa9c2aec2c2a
ingest	completed	0.0	8a1cfe99b7db
lint	completed	18.0	cd5e1ca31d0d
package	completed	10.0	b0ac49e0b5e5
paper	completed*	0.0	
q1	completed	1.0	2719c0545358
q2	completed	8.0	80eca2cfc51b
q3	completed	11.0	184ae69c83a3
q4	completed	86.0	d796b82cf9f0
qa	completed	6.0	effb2641de0d
release	completed	0.0	f976f1c58cd2
results	completed	14.0	410f44c2a0ad
sensitivity	completed	64.0	665ab84af17d
tables	completed	0.0	e7715c476599
test	completed	16.0	f03582d4efb4
validate	completed	0.0	7386c362f110
verification	completed	43.0	7ff7dd0f031c

复现步骤

本文的全部结果由一次云端运行产生。安装 Modal 客户端并完成登录后, 在仓库根目录依次执行:

1. `python3 tools/cli.py provision`——验证云端工具链 (T_EX Live、中文字体、科学计算栈)。
2. `python3 tools/cli.py run ingest,validate --new-run`——读入题目 PDF 与附件、记录其 SHA-256, 并执行输入数据契约 (时间列步长与单调性、半径单调不减、数值范围、无缺失)。

3. `python3 tools/cli.py run q1,q2,q3,q4 --size medium`——四个问题的求解与各自的独立校验。
4. `python3 tools/cli.py run convergence,verification,sensitivity,extension,derived --size medium`——收敛研究、解析解与二维交叉检验、灵敏度与备选解释、模型评价扩展、论文引用的派生量。
5. `python3 tools/cli.py run results,figures,tables`——由附件 3 的模板生成 `result1.xlsx-result4.xlsx` 与补充交付文件，生成矢量图与三线表。
6. `python3 tools/cli.py run lint,test,paper,qa,package,release`——静态检查、单元与性质测试、论文排版、质量门禁、支撑材料打包与发布；最后 `python3 tools/cli.py release --version v1.0.0` 下载交付物并逐文件核对 SHA-256。

可追溯性

每个阶段目录下的 `manifest.json` 记录：阶段状态与用时、代码树摘要与 Git 提交、配置摘要、依赖阶段的输出摘要、参数、全部输入与输出文件的 SHA-256、度量指标、运行环境（Python 版本、平台、CPU 数、依赖包版本）。论文中的每一个数字都由某个阶段以数值宏的形式登记（`numbers.json`），或来自 `tables` 阶段生成的三线表；`.tex` 源文件中不存在手工键入的结果数字。随机性（若有）只经统一的随机种子接口设置。因此在相同的代码版本与输入下，重复运行得到逐位相同的结果与相同的支撑材料散列。

质量门禁

发布前必须全部通过：`lint`（`ruff check`、`ruff format --check`、`mypy`）；`test`（全部单元与性质测试通过，附覆盖率报告）；`qa`（摘要 1 页、正文含参考文献不超过 30 页、无未定义引用、无 $\text{T}_\text{E}\text{X}$ 错误、字体全部嵌入、A4 页面、无过宽行、数值宏无冲突、含《AI 工具使用详情》、无身份信息、四个结果工作簿全部通过模板契约）；`package`（支撑材料成员与论文附录清单完全一致）。上表列出本次运行各阶段的状态与输出摘要；标注“completed*”的一行是生成该表时正在执行的阶段本身（表快照取自该阶段运行中，其输出散列在其完成后写入清单）。

E 完整程序

以下按文件收录建模、求解、检验、绘图、排版与打包的全部源程序，文件名与附录 C 一致。运行方式见附录 D。

Makefile

```

1 # Thin wrappers over tools/cli.py. Nothing here computes locally.
2 PY ?= python3
3 CLI := $(PY) tools/cli.py
4 STAGE ?= ingest
5 SIZE ?= small
6 RUN ?=
7 RUNFLAG := $(if $(RUN),--run-id $(RUN),)
8 FORCEFLAG := $(if $(FORCE),--force,)
```

```

9
10 .PHONY: fmt help provision run pipeline exec status logs runs download release lint test paper qa package smoke
11
12 help:
13     @echo "make provision          # verify cloud toolchain"
14     @echo "make run STAGE=q1 SIZE=medium # run one stage (RUN=<id> to continue a run, FORCE=1 to rerun)"
15     @echo "make pipeline          # ingest,validate,lint,test in one container"
16     @echo "make status | logs STAGE=q1 | runs | download STAGE=results"
17     @echo "make paper | qa | package | release VERSION=v1.0.0"
18
19 provision:
20     $(CLI) provision
21
22 run:
23     $(CLI) run $(STAGE) $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
24
25 pipeline:
26     $(CLI) run ingest,validate,lint,test $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
27
28 exec:
29     $(CLI) exec --code '$(CODE)' $(RUNFLAG)
30
31 status:
32     $(CLI) status $(RUNFLAG)
33
34 logs:
35     $(CLI) logs $(STAGE) $(RUNFLAG)
36
37 runs:
38     $(CLI) runs
39
40 download:
41     $(CLI) download $(RUNFLAG) $(if $(STAGE),--stage $(STAGE),)
42
43 lint:
44     $(CLI) run lint $(RUNFLAG) $(FORCEFLAG)
45
46 test:
47     $(CLI) run test $(RUNFLAG) $(FORCEFLAG)
48
49 paper:
50     $(CLI) run paper $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
51
52 qa:
53     $(CLI) run qa $(RUNFLAG) $(FORCEFLAG)
54
55 package:
56     $(CLI) run package $(RUNFLAG) $(FORCEFLAG)
57
58 release:
59     $(CLI) run release $(RUNFLAG) $(FORCEFLAG) && $(CLI) release --version $(VERSION) $(RUNFLAG) --force
60
61 smoke:
62     $(CLI) run ingest,validate,lint,test,paper,qa,package,release --new-run --param require_results=false --param
        ↪ allow_missing_results=true
63
64 fmt:
65     $(CLI) fmt $(RUNFLAG)

```

configs/default.toml

```

1 # Forge 运行时配置（默认档）。Stage 代码通过 ctx.cfg("a.b.c") 读取；
2 # `--profile <name>` 会叠加 configs/<name>.toml; `--param config_overrides='{...}'` 可临时覆盖。
3
4 [run]
5 seed = 20260912
6
7 [gates]

```

```

8 lint_required = ["ruff_check", "ruff_format", "mypy"]
9 mypy_paths = ["forge"]
10
11 [paper]
12 main = "main.tex"
13 # 需在同一 run 中已完成的 stage; 其 tables/*.tex 与 figures/*.pdf/png 会被并入排版
14 sources = ["figures", "tables"]
15 max_body_pages = 30
16 max_overfull_pt = 20.0
17 code_globs = [
18     "forge/**/*.py", "pipelines/**/*.py", "tools/**/*.py", "tests/**/*.py",
19     "configs/*.toml", "configs/*.json", "Makefile", "pyproject.toml",
20 ]
21
22 [qa]
23 required_stages = ["lint", "test"]
24 results_stage = "results"
25 require_results = true
26 identity_denylist = [] # 硬门禁（正则）：出现即失败
27 identity_watchlist = ["大学", "学院", "赛区", "队号", "指导教师", "[\\w.-]+@[\\w.-]+\\.\\w+"] # 软提示：写入报告供复核
28
29 [package]
30 stages = ["ingest", "validate", "q1", "q2", "q3", "q4", "convergence", "verification", "sensitivity", "extension", "
    ↳ derived", "results", "figures", "tables", "lint", "test", "qa"]
31 code_globs = [
32     "forge/**/*.py", "pipelines/**/*.py", "tools/**/*.py", "tests/**/*.py", "configs/**",
33     "manuscript/**/*.tex", "manuscript/**/*.bib", "docs/**/*.md", "Makefile", "pyproject.toml",
34     "README.md", "UNLICENSE", "AI_USAGE.md",
35 ]
36 max_bytes = 20971520
37
38 [release]
39 paper_name = "论文.pdf"
40
41 # ---- Problem A: herb drying (see docs/mdr/) ----
42 [a.grid]
43 refine = 5 # production grid: dr = 0.1 / 2refine cm (nodes coincide with the 0.1 cm output grid)
44 convergence_levels = 7 # k = 0 ... levels-1 in the convergence study
45
46 [a.time]
47 method = "BDF"
48 rtol = 1e-9
49 atol = 1e-11
50
51 [a.chamber]
52 plateau = "mean_last_hour" # constant-temperature stage after 14400 s (MDR-0003): mean_last_hour | last_sample |
    ↳ nominal
53
54 [a.scheme]
55 face = "midpoint" # interface coefficient: midpoint | arithmetic | harmonic (MDR-0004)
56 formulation = "lagrangian" # moving boundary: lagrangian (dry matter conserved) | eulerian (MDR-0006)
57 mass_form = "drybasis" # moisture conservation form: drybasis (rho_s constant) | conservative (MDR-0010)
58
59 [a.drying]
60 horizon_s = 432000 # first integration horizon for the drying-end search (5 days), doubled if needed
61 dt_out_s = 60
62
63 [a.verification]
64 axisym_nz = 25 # axial intervals of the independent 2-D model (dz = 0.5 cm)
65 axisym_refine = 0 # radial refinement of the 2-D model (dr = 0.1 cm)
66
67 [a.sensitivity]
68 factors = [0.8, 0.9, 1.1, 1.2]
69 workers = 8
70
71 [a.extension] # model-evaluation extension (MDR-0008), problem-3 setting
72 cap_factors = [0.5, 1.0, 2.0] # multiples of the wet-bulb energy limit h (Tair - Twb)/Lv on the evaporation flux
73 c_eq = [0.075, 0.10, 0.125] # equilibrium moisture replacing Cair in the driving force Cs - Ceq

```

configs/project.json

```
1 {
2   "letter": "A",
3   "package": "a",
4   "slug": "herb-drying",
5   "title": "药材的烘干问题",
6   "app": "cumcm2026-a-herb-drying",
7   "volume": "cumcm2026-a-artifacts",
8   "problem_pdf": "A题.pdf",
9   "attachments_dir": "附件",
10  "templates_dir": "附件/附件3",
11  "result_files": [
12    "result1.xlsx",
13    "result2.xlsx",
14    "result3.xlsx",
15    "result4.xlsx",
16    "result2_全过程.xlsx"
17  ],
18  "github_repo": "cumcm2026-a-herb-drying"
19 }
```

forge/__init__.py

```
1 """Forge — cloud-only research engineering runtime (CUMCM 2026).
2
3 Every computation (data parsing, modelling, solving, tests, figures, LaTeX, PDF QA,
4 packaging) runs inside Modal. The local machine only edits text, drives git,
5 submits jobs, downloads artifacts and verifies transport hashes.
6 """
7
8 __version__ = "1.0.0"
```

forge/config.py

```
1 """TOML configuration with profile overlays and dotted access."""
2
3 from __future__ import annotations
4
5 import json
6 import tomllib
7 from pathlib import Path
8 from typing import Any
9
10 from .hashing import sha256_json
11
12 _MISSING = object()
13
14
15 def deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
16     out = dict(base)
17     for key, value in override.items():
18         if isinstance(value, dict) and isinstance(out.get(key), dict):
19             out[key] = deep_merge(out[key], value)
20         else:
21             out[key] = value
22     return out
23
24
25 class Config:
26     """Read-only dotted-access view over a merged configuration mapping."""
27
28     def __init__(self, data: dict[str, Any]) -> None:
29         self._data = json.loads(json.dumps(data))
```

```

30
31 def get(self, key: str, default: Any = None) -> Any:
32     cursor: Any = self._data
33     for part in key.split("."):
34         if not isinstance(cursor, dict) or part not in cursor:
35             return default
36         cursor = cursor[part]
37     return cursor
38
39 def __getitem__(self, key: str) -> Any:
40     value = self.get(key, _MISSING)
41     if value is _MISSING:
42         raise KeyError(key)
43     return value
44
45 def section(self, key: str) -> dict[str, Any]:
46     value = self.get(key, {})
47     return dict(value) if isinstance(value, dict) else {}
48
49 def as_dict(self) -> dict[str, Any]:
50     return json.loads(json.dumps(self._data))
51
52 @property
53 def digest(self) -> str:
54     return sha256_json(self._data)
55
56
57 def load_project(repo: Path) -> dict[str, Any]:
58     return json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))
59
60
61 def load_config(repo: Path, profile: str | None = None, overrides: dict[str, Any] | None = None) -> Config:
62     data = tomlib.loads((repo / "configs" / "default.toml").read_text(encoding="utf-8"))
63     if profile:
64         data = deep_merge(data, tomlib.loads((repo / "configs" / f"{profile}.toml").read_text(encoding="utf-8")))
65     if overrides:
66         data = deep_merge(data, overrides)
67     return Config(data)

```

forge/context.py

```

1 """Stage execution context: paths, config, params, logging, seeds and paper numbers."""
2
3 from __future__ import annotations
4
5 import json
6 import random
7 import re
8 import zlib
9 from pathlib import Path
10 from typing import Any
11
12 from .config import Config, load_config, load_project
13 from .events import EventLogger
14
15 _NUMBER_KEY = re.compile(r"^[A-Za-z]+$")
16
17
18 class StageContext:
19     """Everything a stage needs. Stages must write only under ``stage_dir``."""
20
21     def __init__(self, run_id: str, stage: str, params: dict[str, Any], repo: Path, vol: Path) -> None:
22         self.run_id = run_id
23         self.stage = stage
24         self.params = dict(params)
25         self.repo = repo
26         self.vol = vol
27         self.project = load_project(repo)

```

```

28     overrides = self.params.get("config_overrides") or None
29     self.config: Config = load_config(repo, self.params.get("profile"), overrides)
30     self.run_dir = vol / "runs" / run_id
31     self.stage_dir = self.run_dir / stage
32     self.stage_dir.mkdir(parents=True, exist_ok=True)
33     self.log = EventLogger(self.stage_dir / "events.jsonl", run_id, stage)
34     self._numbers: dict[str, str] = {}
35
36 # ---- raw inputs (immutable, at repo root, exactly as delivered by the organisers) ----
37 @property
38 def problem_pdf(self) -> Path:
39     return self.repo / self.project["problem_pdf"]
40
41 @property
42 def attachments(self) -> Path:
43     return self.repo / self.project["attachments_dir"]
44
45 @property
46 def templates(self) -> Path:
47     return self.repo / self.project["templates_dir"]
48
49 # ---- outputs ----
50 def out(self, *parts: str) -> Path:
51     path = self.stage_dir.joinpath(*parts)
52     path.parent.mkdir(parents=True, exist_ok=True)
53     return path
54
55 def dep(self, stage: str) -> Path:
56     """Directory of a completed dependency stage in the same run (raises otherwise)."""
57     directory = self.run_dir / stage
58     manifest = directory / "manifest.json"
59     if not manifest.exists():
60         raise RuntimeError(f"dependency stage '{stage}' has not run in run {self.run_id}")
61     status = json.loads(manifest.read_text(encoding="utf-8")).get("status")
62     if status != "completed":
63         raise RuntimeError(f"dependency stage '{stage}' has status {status!r} in run {self.run_id}")
64     return directory
65
66 def has_stage(self, stage: str) -> bool:
67     manifest = self.run_dir / stage / "manifest.json"
68     return manifest.exists() and json.loads(manifest.read_text(encoding="utf-8")).get("status") == "completed"
69
70 def dep_data(self, stage: str, name: str) -> Any:
71     return json.loads((self.dep(stage) / name).read_text(encoding="utf-8"))
72
73 # ---- parameters / configuration ----
74 def param(self, key: str, default: Any = None) -> Any:
75     return self.params.get(key, default)
76
77 def cfg(self, key: str, default: Any = None) -> Any:
78     return self.config.get(key, default)
79
80 # ---- determinism ----
81 def seed(self, salt: str = "") -> int:
82     base = int(self.cfg("run.seed", 0))
83     return (base + zlib.crc32(salt.encode("utf-8"))) % (2**31 - 1)
84
85 def seed_everything(self, salt: str = "") -> int:
86     value = self.seed(salt)
87     random.seed(value)
88     try:
89         import numpy as np
90
91         np.random.seed(value) # noqa: NPY002 - legacy global RNG used by third-party libraries
92     except ImportError: # pragma: no cover
93         pass
94     self.log.info("seed", salt=salt, value=value)
95     return value
96
97 # ---- numbers that flow into the paper as \val<Key> macros ----
98 def number(self, key: str, value: Any, fmt: str | None = None) -> str:

```

```

99         if not _NUMBER_KEY.match(key):
100             raise ValueError(f"paper number keys must be letters only (TeX macro names); got {key!r}")
101         if fmt is not None:
102             text = format(value, fmt)
103         elif isinstance(value, bool):
104             text = str(value)
105         elif isinstance(value, int):
106             text = str(value)
107         elif isinstance(value, float):
108             text = f"{value:.4f}"
109         else:
110             text = str(value)
111         self._numbers[key] = text
112         return text
113
114     def flush_numbers(self) -> None:
115         if not self._numbers:
116             return
117         path = self.stage_dir / "numbers.json"
118         merged: dict[str, str] = {}
119         if path.exists():
120             merged.update(json.loads(path.read_text(encoding="utf-8")))
121         merged.update(self._numbers)
122         path.write_text(json.dumps(merged, ensure_ascii=False, indent=2, sort_keys=True), encoding="utf-8")
123
124     # ---- json helpers ----
125     def write_json(self, name: str, obj: Any) -> Path:
126         path = self.out(name)
127         path.write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=str), encoding="utf-8")
128         return path
129
130     def read_json(self, path: Path) -> Any:
131         return json.loads(path.read_text(encoding="utf-8"))

```

forge/contracts.py

```

1  """Lightweight DataFrame contracts (schema + range + monotonicity + custom checks)."""
2
3  from __future__ import annotations
4
5  import re
6  from collections.abc import Callable
7  from dataclasses import dataclass
8  from typing import Any
9
10 import numpy as np
11 import pandas as pd
12
13
14 @dataclass(frozen=True)
15 class Column:
16     name: str
17     kind: str = "any" # int | float | number | str | datetime | any
18     nullable: bool = False
19     min: float | None = None
20     max: float | None = None
21     regex: str | None = None
22     unique: bool = False
23     monotonic: str | None = None # increasing | non_decreasing | decreasing | non_increasing
24     allowed: tuple[Any, ...] | None = None
25     step: float | None = None # constant difference between consecutive values
26
27
28 @dataclass(frozen=True)
29 class FrameContract:
30     name: str
31     columns: tuple[Column, ...]
32     min_rows: int = 1

```

```

33     max_rows: int | None = None
34     extra_columns: str = "forbid" # forbid | allow
35     checks: tuple[Callable[[pd.DataFrame], list[str]], ...] = ()
36
37
38 def normalise_columns(df: pd.DataFrame) -> pd.DataFrame:
39     out = df.copy()
40     out.columns = pd.Index([str(c).strip() for c in out.columns])
41     return out
42
43
44 def validate_frame(df: pd.DataFrame, contract: FrameContract) -> dict[str, Any]:
45     df = normalise_columns(df)
46     errors: list[str] = []
47     warnings: list[str] = []
48     expected = [c.name for c in contract.columns]
49     missing = [c for c in expected if c not in df.columns]
50     extra = [c for c in df.columns if c not in expected]
51     if missing:
52         errors.append(f"missing columns: {missing}")
53     if extra and contract.extra_columns == "forbid":
54         errors.append(f"unexpected columns: {extra}")
55     n = len(df)
56     if n < contract.min_rows:
57         errors.append(f"rows {n} < min_rows {contract.min_rows}")
58     if contract.max_rows is not None and n > contract.max_rows:
59         errors.append(f"rows {n} > max_rows {contract.max_rows}")
60
61     for col in contract.columns:
62         if col.name not in df.columns:
63             continue
64         series = df[col.name]
65         nulls = int(series.isna().sum())
66         if nulls and not col.nullable:
67             errors.append(f"{col.name}: {nulls} null values")
68         values = series.dropna()
69         if col.kind in {"int", "float", "number"}:
70             numeric = pd.to_numeric(values, errors="coerce")
71             bad = int(numeric.isna().sum())
72             if bad:
73                 errors.append(f"{col.name}: {bad} non-numeric values")
74             numeric = numeric.dropna().astype(float)
75             if col.kind == "int" and len(numeric) and not np.all(np.mod(numeric.to_numpy(), 1) == 0):
76                 errors.append(f"{col.name}: non-integer values present")
77             if col.min is not None and len(numeric) and float(numeric.min()) < col.min:
78                 errors.append(f"{col.name}: min {numeric.min()} < {col.min}")
79             if col.max is not None and len(numeric) and float(numeric.max()) > col.max:
80                 errors.append(f"{col.name}: max {numeric.max()} > {col.max}")
81             if col.monotonic and len(numeric) > 1:
82                 diff = np.diff(numeric.to_numpy())
83                 ok = {
84                     "increasing": bool((diff > 0).all()),
85                     "non_decreasing": bool((diff >= 0).all()),
86                     "decreasing": bool((diff < 0).all()),
87                     "non_increasing": bool((diff <= 0).all()),
88                 }[col.monotonic]
89                 if not ok:
90                     errors.append(f"{col.name}: not {col.monotonic}")
91             if col.step is not None and len(numeric) > 1:
92                 diff = np.diff(numeric.to_numpy())
93                 if not np.allclose(diff, col.step):
94                     errors.append(f"{col.name}: step is not constant {col.step} (min {diff.min()}, max {diff.max()})")
95         elif col.kind == "str":
96             if col.regex:
97                 pattern = re.compile(col.regex)
98                 bad_values = [str(v) for v in values.astype(str) if not pattern.fullmatch(str(v))]
99                 if bad_values:
100                     errors.append(f"{col.name}: {len(bad_values)} values violate {col.regex}/, e.g. {bad_values[:3]}")
101         elif col.kind == "datetime":
102             parsed = pd.to_datetime(values, errors="coerce")
103             bad = int(parsed.isna().sum())

```

```

104         if bad:
105             errors.append(f"{col.name}: {bad} unparsable datetimes")
106         if col.unique and values.duplicated().any():
107             errors.append(f"{col.name}: {int(values.duplicated().sum())} duplicated values")
108         if col.allowed is not None:
109             unexpected = sorted(set(values.unique()) - set(col.allowed), key=str)
110             if unexpected:
111                 errors.append(f"{col.name}: values outside allowed set: {unexpected[:5]}")
112
113     for check in contract.checks:
114         try:
115             errors.extend(check(df))
116         except Exception as exc:
117             errors.append(f"check {getattr(check, '__name__', check)!r} raised {exc!r}")
118
119     return {
120         "contract": contract.name,
121         "ok": not errors,
122         "rows": n,
123         "columns": list(df.columns),
124         "errors": errors,
125         "warnings": warnings,
126     }

```

forge/events.py

```

1  """Structured JSONL event logging with a human-readable mirror on stdout."""
2
3  from __future__ import annotations
4
5  import json
6  import time
7  from datetime import UTC, datetime
8  from pathlib import Path
9  from typing import Any
10
11
12  def _short(value: Any, limit: int = 200) -> str:
13      text = value if isinstance(value, str) else json.dumps(value, ensure_ascii=False, default=str)
14      return text if len(text) <= limit else text[: limit - 1] + "..."
15
16
17  class EventLogger:
18      """Append-only event stream: one JSON object per line, mirrored to stdout."""
19
20      def __init__(self, path: Path, run_id: str, stage: str) -> None:
21          path.parent.mkdir(parents=True, exist_ok=True)
22          self._fh = path.open("a", encoding="utf-8")
23          self.path = path
24          self.run_id = run_id
25          self.stage = stage
26          self._t0 = time.monotonic()
27
28      def emit(self, event: str, level: str = "info", **fields: Any) -> None:
29          record = {
30              "ts": datetime.now(UTC).isoformat(timespec="milliseconds"),
31              "elapsed_s": round(time.monotonic() - self._t0, 3),
32              "run_id": self.run_id,
33              "stage": self.stage,
34              "level": level,
35              "event": event,
36              **fields,
37          }
38          self._fh.write(json.dumps(record, ensure_ascii=False, default=str) + "\n")
39          self._fh.flush()
40          extra = " ".join(f"{k}={_short(v)}" for k, v in fields.items())
41          print(f"[{record['elapsed_s']:9.3f}s] {level.upper():5s} {self.stage}:{event} {extra}", flush=True)
42

```

```

43     def info(self, event: str, **fields: Any) -> None:
44         self.emit(event, "info", **fields)
45
46     def warn(self, event: str, **fields: Any) -> None:
47         self.emit(event, "warn", **fields)
48
49     def error(self, event: str, **fields: Any) -> None:
50         self.emit(event, "error", **fields)
51
52     def metric(self, name: str, value: Any, **fields: Any) -> None:
53         self.emit("metric", "info", name=name, value=value, **fields)
54
55     def close(self) -> None:
56         if not self._fh.closed:
57             self._fh.close()

```

forge/hashing.py

```

1  """Content hashing helpers (SHA-256 everywhere; JSON hashed in canonical form)."""
2
3  from __future__ import annotations
4
5  import hashlib
6  import json
7  from collections.abc import Iterable
8  from pathlib import Path
9  from typing import Any
10
11  CHUNK = 1 << 20
12
13
14  def sha256_file(path: Path) -> str:
15      h = hashlib.sha256()
16      with path.open("rb") as fh:
17          for chunk in iter(lambda: fh.read(CHUNK), b""):
18              h.update(chunk)
19      return h.hexdigest()
20
21
22  def sha256_bytes(data: bytes) -> str:
23      return hashlib.sha256(data).hexdigest()
24
25
26  def sha256_json(obj: Any) -> str:
27      payload = json.dumps(obj, sort_keys=True, ensure_ascii=False, separators=(",", ":"), default=str)
28      return sha256_bytes(payload.encode("utf-8"))
29
30
31  def hash_tree(root: Path, ignore: Iterable[str] = ()) -> dict[str, dict[str, Any]]:
32      """Return ``{relative_posix_path: {"sha256": ..., "bytes": ...}}`` for every file under ``root``.
33
34      Any path component contained in ``ignore`` excludes the file (directory names or file names).
35      The result is sorted by path so that its canonical JSON digest is stable.
36      """
37      ignored = set(ignore)
38      out: dict[str, dict[str, Any]] = {}
39      if not root.exists():
40          return out
41      for path in sorted(root.rglob("*")):
42          if not path.is_file():
43              continue
44          rel = path.relative_to(root)
45          if ignored & set(rel.parts):
46              continue
47          out[rel.as_posix()] = {"sha256": sha256_file(path), "bytes": path.stat().st_size}
48      return out
49
50

```

```

51 def tree_digest(tree: dict[str, dict[str, Any]]) -> str:
52     return sha256_json({k: v["sha256"] for k, v in sorted(tree.items())})

```

forge/manifest.py

```

1  """Run/stage manifests: the unit of lineage, reproducibility and audit."""
2
3  from __future__ import annotations
4
5  import importlib.metadata
6  import json
7  import os
8  import platform
9  import sys
10 import traceback
11 from dataclasses import asdict, dataclass, field, fields
12 from datetime import UTC, datetime
13 from pathlib import Path
14 from typing import Any
15
16 from .hashing import hash_tree, tree_digest
17
18 TRACKED_PACKAGES = (
19     "numpy",
20     "scipy",
21     "pandas",
22     "pyarrow",
23     "openpyxl",
24     "matplotlib",
25     "scikit-learn",
26     "statsmodels",
27     "ortools",
28     "highspy",
29     "numba",
30     "cvxpy",
31     "networkx",
32     "pymupdf",
33     "pytest",
34     "hypothesis",
35     "ruff",
36     "mypy",
37 )
38 RECORD_FILES = ("manifest.json", "events.jsonl") # run records, excluded from the outputs digest
39
40
41 def utc_now() -> str:
42     return datetime.now(UTC).isoformat(timespec="seconds")
43
44
45 def runtime_info() -> dict[str, Any]:
46     packages: dict[str, str] = {}
47     for name in TRACKED_PACKAGES:
48         try:
49             packages[name] = importlib.metadata.version(name)
50         except importlib.metadata.PackageNotFoundError:
51             continue
52     return {
53         "python": sys.version.split()[0],
54         "platform": platform.platform(),
55         "cpu_count": os.cpu_count(),
56         "modal_task_id": os.getenv("MODAL_TASK_ID"),
57         "modal_is_remote": os.getenv("MODAL_IS_REMOTE") == "1" or os.getenv("MODAL_TASK_ID") is not None,
58         "packages": packages,
59         "env": {k: os.environ.get(k) for k in ("OMP_NUM_THREADS", "OPENBLAS_NUM_THREADS", "PYTHONHASHSEED")},
60     }
61
62
63 @dataclass

```

```

64 class Manifest:
65     schema: str = "forge.manifest/1"
66     run_id: str = ""
67     stage: str = ""
68     status: str = "running" # running | completed | failed
69     started_at: str = field(default_factory=utc_now)
70     finished_at: str | None = None
71     duration_s: float | None = None
72     code_ref: dict[str, Any] = field(default_factory=dict)
73     params: dict[str, Any] = field(default_factory=dict)
74     config_digest: str = ""
75     deps: dict[str, str] = field(default_factory=dict)
76     inputs: dict[str, dict[str, Any]] = field(default_factory=dict)
77     outputs: dict[str, dict[str, Any]] = field(default_factory=dict)
78     outputs_digest: str = ""
79     metrics: dict[str, Any] = field(default_factory=dict)
80     runtime: dict[str, Any] = field(default_factory=dict)
81     error: dict[str, Any] | None = None
82
83     @classmethod
84     def start(
85         cls,
86         run_id: str,
87         stage: str,
88         params: dict[str, Any],
89         code_ref: dict[str, Any],
90         config_digest: str,
91         deps: dict[str, str],
92         inputs: dict[str, dict[str, Any]],
93     ) -> Manifest:
94         manifest = cls(
95             run_id=run_id,
96             stage=stage,
97             params=params,
98             code_ref=code_ref,
99             config_digest=config_digest,
100             deps=deps,
101             inputs=inputs,
102         )
103         manifest.runtime = runtime_info()
104         return manifest
105
106     def complete(self, stage_dir: Path, metrics: dict[str, Any]) -> None:
107         self.outputs = hash_tree(stage_dir, ignore=RECORD_FILES)
108         self.outputs_digest = tree_digest(self.outputs)
109         self.metrics = json.loads(json.dumps(metrics, default=str))
110         self.status = "completed"
111         self._finish()
112
113     def fail(self, exc: BaseException) -> None:
114         self.error = {"type": type(exc).__name__, "message": str(exc), "traceback": traceback.format_exc()}
115         self.status = "failed"
116         self._finish()
117
118     def _finish(self) -> None:
119         self.finished_at = utc_now()
120         started = datetime.fromisoformat(self.started_at)
121         self.duration_s = round((datetime.fromisoformat(self.finished_at) - started).total_seconds(), 3)
122
123     def write(self, path: Path) -> None:
124         path.parent.mkdir(parents=True, exist_ok=True)
125         path.write_text(json.dumps(asdict(self), ensure_ascii=False, indent=2, default=str), encoding="utf-8")
126
127     @classmethod
128     def read(cls, path: Path) -> Manifest:
129         raw = json.loads(path.read_text(encoding="utf-8"))
130         known = {f.name for f in fields(cls)}
131         return cls(**{k: v for k, v in raw.items() if k in known})
132
133     def verify_outputs(self, stage_dir: Path) -> list[str]:
134         """Return a list of mismatches between the manifest and the files on disk (empty == verified)."""

```

```

135     problems: list[str] = []
136     current = hash_tree(stage_dir, ignore=RECORD_FILES)
137     for rel, meta in self.outputs.items():
138         if rel not in current:
139             problems.append(f"missing: {rel}")
140         elif current[rel]["sha256"] != meta["sha256"]:
141             problems.append(f"sha256 mismatch: {rel}")
142     for rel in current:
143         if rel not in self.outputs:
144             problems.append(f"unexpected file: {rel}")
145     return problems

```

forge/modal_app.py

```

1  """Modal application — the only compute surface of this repository.
2
3  Local side (tools/cli.py) only submits:
4      modal run forge/modal_app.py --stage ingest --run-id <id> --params '{...}' --size small
5  Inside the container the repository is mounted read-only at /repo and the persistent
6  volume at /vol; stages write exclusively to /vol/runs/<run_id>/<stage>/.
7  """
8
9  from __future__ import annotations
10
11  import contextlib
12  import io
13  import json
14  import os
15  import sys
16  import traceback
17  from pathlib import Path
18
19  import modal
20
21  _HERE = Path(__file__).resolve()
22  REPO = _HERE.parents[1]
23
24
25  def _load_project() -> dict:
26      for candidate in (REPO / "configs" / "project.json", Path("/repo/configs/project.json")):
27          if candidate.exists():
28              return json.loads(candidate.read_text(encoding="utf-8"))
29      raise FileNotFoundError("configs/project.json not found next to forge/modal_app.py")
30
31
32  PROJECT = _load_project()
33  app = modal.App(PROJECT["app"])
34  volume = modal.Volume.from_name(PROJECT["volume"], create_if_missing=True)
35
36  APT_PACKAGES = [
37      "texlive-xetex",
38      "texlive-lang-chinese",
39      "texlive-latex-base",
40      "texlive-latex-recommended",
41      "texlive-latex-extra",
42      "texlive-science",
43      "texlive-pictures",
44      "texlive-bibtex-extra",
45      "texlive-fonts-recommended",
46      "fonts-noto-cjk",
47      "fonts-noto-core",
48      "fonts-droid-fallback",
49      "fonts-wqy-microhei",
50      "fontconfig",
51      "poppler-utils",
52      "zip",
53      "unzip",
54  ]

```

```

55 PIP_PACKAGES = [
56     "numpy==2.2.6",
57     "scipy==1.15.3",
58     "pandas==2.2.3",
59     "pyarrow==20.0.0",
60     "openpyxl==3.1.5",
61     "matplotlib==3.10.3",
62     "scikit-learn==1.6.1",
63     "statsmodels==0.14.4",
64     "ortools==9.12.4544",
65     "highspy==1.10.0",
66     "numba==0.61.2",
67     "cvxpy==1.6.5",
68     "networkx==3.4.2",
69     "pydantic==2.11.4",
70     "pymupdf==1.26.7",
71     "pytest==8.3.5",
72     "pytest-cov==6.1.1",
73     "hypothesis==6.131.9",
74     "ruff==0.11.10",
75     "mypy==1.15.0",
76     "pandas-stubs==2.2.3.250308",
77     "types-openpyxl==3.1.5.20250602",
78 ]
79 SKIP_PARTS = {
80     ".git",
81     "artifacts",
82     "releases",
83     ".forge",
84     "__pycache__",
85     ".mypy_cache",
86     ".ruff_cache",
87     ".pytest_cache",
88     ".hypothesis",
89     ".DS_Store",
90 }
91
92
93 def _ignore(path: Path) -> bool:
94     return bool(SKIP_PARTS & set(path.parts)) or path.suffix in {".pyc", ".pyo"}
95
96
97 image = (
98     modal.Image.debian_slim(python_version="3.11")
99     .apt_install(*APT_PACKAGES)
100     .pip_install(*PIP_PACKAGES)
101     .env(
102         {
103             "OMP_NUM_THREADS": "1",
104             "OPENBLAS_NUM_THREADS": "1",
105             "MKL_NUM_THREADS": "1",
106             "MPLBACKEND": "Agg",
107             "MPLCONFIGDIR": "/tmp/mpl",
108             "PYTHONHASHSEED": "0",
109             "PYTHONUNBUFFERED": "1",
110             "PYTHONDONTWRITEBYTECODE": "1",
111             "PYTHONPATH": "/repo",
112             "FORGE_REPO": "/repo",
113             "FORGE_VOL": "/vol",
114             "MODAL_IS_REMOTE": "1",
115             "TEXMFVAR": "/tmp/texmf-var",
116             "TEXMFCONFIG": "/tmp/texmf-config",
117             "HOME": "/root",
118         }
119     )
120     .add_local_dir(REPO, "/repo", ignore=_ignore)
121 )
122
123 VOLUME_MOUNT = {"/vol": volume}
124
125

```

```

126 def _bootstrap() -> None:
127     if "/repo" not in sys.path:
128         sys.path.insert(0, "/repo")
129     os.chdir("/repo")
130     Path("/tmp/mpl").mkdir(exist_ok=True)
131
132
133 def _run(stage: str, run_id: str, params_json: str, force: bool) -> dict:
134     _bootstrap()
135     volume.reload()
136     from forge.runner import execute
137
138     params = json.loads(params_json or "{}")
139     return execute(
140         stage_name=stage,
141         run_id=run_id,
142         params=params,
143         force=force,
144         repo=Path("/repo"),
145         vol=Path("/vol"),
146         commit=volume.commit,
147     )
148
149
150 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=2, memory=4096, timeout=2 * 3600)
151 def run_small(stage: str, run_id: str, params: str, force: bool) -> dict:
152     return _run(stage, run_id, params, force)
153
154
155 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=8, memory=16384, timeout=6 * 3600)
156 def run_medium(stage: str, run_id: str, params: str, force: bool) -> dict:
157     return _run(stage, run_id, params, force)
158
159
160 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=32, memory=65536, timeout=12 * 3600)
161 def run_large(stage: str, run_id: str, params: str, force: bool) -> dict:
162     return _run(stage, run_id, params, force)
163
164
165 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=4, memory=8192, timeout=3600)
166 def exec_python(code: str, run_id: str) -> str:
167     """Run an ad-hoc Python snippet in the cloud (exploration, status, debugging). Never local."""
168     _bootstrap()
169     volume.reload()
170     buffer = io.StringIO()
171     scope = {
172         "__name__": "__forge_exec__",
173         "RUN_DIR": Path("/vol/runs") / run_id,
174         "VOL": Path("/vol"),
175         "REPO": Path("/repo"),
176         "RUNS": Path("/vol/runs"),
177     }
178     with contextlib.redirect_stdout(buffer), contextlib.redirect_stderr(buffer):
179         try:
180             exec(compile(code, "<forge-exec>", "exec"), scope) # noqa: S102 - explicit cloud REPL
181         except BaseException:
182             traceback.print_exc(file=buffer)
183     volume.commit()
184     return buffer.getvalue()
185
186
187 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=2, memory=4096, timeout=600)
188 def provision() -> dict:
189     """Prove the toolchain: interpreter, packages, TeX engine, fonts, solver availability."""
190     import importlib.metadata
191     import subprocess
192
193     _bootstrap()
194     info: dict = {"modal_task_id": os.getenv("MODAL_TASK_ID"), "cpu_count": os.cpu_count()}
195     info["python"] = sys.version.split()[0]
196     info["packages"] = {p.split("==")[0]: importlib.metadata.version(p.split("==")[0]) for p in PIP_PACKAGES}

```

```

197 info["xelatex"] = subprocess.run(["xelatex", "--version"], capture_output=True, text=True).stdout.splitlines()[0]
198 info["bibtex"] = subprocess.run(["bibtex", "--version"], capture_output=True, text=True).stdout.splitlines()[0]
199 info["fc_cjk"] = subprocess.run(["fc-match", "Noto Serif CJK SC"], capture_output=True, text=True).stdout.strip()
200 info["kpse"] = {
201     name: bool(subprocess.run(["kpsewhich", name], capture_output=True, text=True).stdout.strip())
202     for name in (
203         "ctexart.cls",
204         "gbt7714.sty",
205         "siunitx.sty",
206         "algorithm2e.sty",
207         "booktabs.sty",
208         "tikz.sty",
209         "listings.sty",
210         "cleveref.sty",
211         "tcolorbox.sty",
212         "pgfplots.sty",
213     )
214 }
215 from ortools.sat.python import cp_model
216
217 model = cp_model.CpModel()
218 x = model.NewIntVar(0, 10, "x")
219 model.Add(x >= 3)
220 model.Minimize(x)
221 solver = cp_model.CpSolver()
222 info["cpsat"] = {"status": solver.StatusName(solver.Solve(model)), "x": solver.Value(x)}
223 import highspy
224
225 info["highs"] = highspy.Highs().version()
226 from forge.plotting import cjk_font_family
227
228 info["figure_cjk_font"] = cjk_font_family()
229 return info
230
231 @app.local_entrypoint()
232 def main(
233     stage: str = "ingest",
234     run_id: str = "",
235     params: str = "{}",
236     force: bool = False,
237     size: str = "small",
238     spawn: bool = False,
239 ) -> None:
240     """Submit one stage (or ``all`` with params.stages) to the cloud and print its result."""
241     if stage == "provision":
242         print("PROVISION " + json.dumps(provision.remote(), ensure_ascii=False, default=str))
243         return
244     if stage == "exec":
245         payload = json.loads(params or "{}")
246         print(exec_python.remote(payload.get("code", ""), run_id or "adhoc"))
247         return
248     fn = {"small": run_small, "medium": run_medium, "large": run_large}[size]
249     if spawn:
250         call = fn.spawn(stage, run_id, params, force)
251         print(f"FUNCTION_CALL_ID {call.object_id}")
252         return
253     result = fn.remote(stage, run_id, params, force)
254     print("STAGE_RESULT " + json.dumps(result, ensure_ascii=False, default=str))

```

forge/packaging.py

```

1 """Deterministic packaging of support materials (stable member order and timestamps)."""
2
3 from __future__ import annotations
4
5 import zipfile
6 from collections.abc import Iterable

```

```

7 from pathlib import Path
8 from typing import Any
9
10 from .hashing import sha256_file
11
12 FIXED_TIME = (2026, 1, 1, 0, 0, 0)
13 SKIP_PARTS = {
14     ".git",
15     "__pycache__",
16     ".DS_Store",
17     ".mypy_cache",
18     ".ruff_cache",
19     ".pytest_cache",
20     ".hypothesis",
21     "artifacts",
22     "releases",
23     ".forge",
24     "build",
25 }
26
27
28 def code_members(repo: Path, globs: Iterable[str]) -> list[tuple[str, Path]]:
29     found: dict[str, Path] = {}
30     for pattern in globs:
31         for path in sorted(repo.glob(pattern)):
32             if not path.is_file():
33                 continue
34             rel = path.relative_to(repo)
35             if SKIP_PARTS & set(rel.parts) or path.suffix == ".pyc":
36                 continue
37             found[rel.as_posix()] = path
38     return sorted(found.items())
39
40
41 def run_members(run_dir: Path, stages: Iterable[str], *, prefix: str = "runs") -> list[tuple[str, Path]]:
42     members: list[tuple[str, Path]] = []
43     for stage in stages:
44         directory = run_dir / stage
45         if not directory.is_dir():
46             continue
47         for path in sorted(directory.rglob("*")):
48             rel = path.relative_to(directory)
49             if path.is_file() and not (SKIP_PARTS & set(rel.parts)):
50                 members.append((f"{prefix}/{run_dir.name}/{stage}/{rel.as_posix()}", path))
51     return members
52
53
54 def build_zip(members: Iterable[tuple[str, Path]], out: Path, *, comment: str = "") -> dict[str, Any]:
55     ordered = sorted(dict(members).items())
56     out.parent.mkdir(parents=True, exist_ok=True)
57     total = 0
58     with zipfile.ZipFile(out, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9) as zf:
59         if comment:
60             zf.comment = comment.encode("utf-8")
61         for name, source in ordered:
62             info = zipfile.ZipInfo(name, date_time=FIXED_TIME)
63             info.compress_type = zipfile.ZIP_DEFLATED
64             info.external_attr = 0o644 << 16
65             data = source.read_bytes()
66             total += len(data)
67             zf.writestr(info, data)
68     with zipfile.ZipFile(out) as zf:
69         bad = zf.testzip()
70         count = len(zf.namelist())
71     if bad:
72         raise RuntimeError(f"zip member failed CRC check: {bad}")
73     return {
74         "path": str(out),
75         "members": count,
76         "uncompressed_bytes": total,
77         "bytes": out.stat().st_size,

```

```

78     "sha256": sha256_file(out),
79 }

```

forge/pdfqa.py

```

1  """PDF inspection for quality gates (PyMuPDF)."""
2
3  from __future__ import annotations
4
5  import re
6  from collections.abc import Iterable
7  from pathlib import Path
8  from typing import Any
9
10 A4_PT = (595.276, 841.89)
11
12
13 def inspect_pdf(path: Path, *, with_text: bool = True) -> dict[str, Any]:
14     import fitz # PyMuPDF
15
16     doc = fitz.open(path)
17     pages: list[dict[str, Any]] = []
18     texts: list[str] = []
19     fonts: dict[int, dict[str, Any]] = {}
20     unembedded: list[str] = []
21     for index, page in enumerate(doc, start=1):
22         rect = page.rect
23         pages.append(
24             {
25                 "page": index,
26                 "width_pt": round(rect.width, 2),
27                 "height_pt": round(rect.height, 2),
28                 "a4": abs(rect.width - A4_PT[0]) < 3 and abs(rect.height - A4_PT[1]) < 3,
29             }
30         )
31         if with_text:
32             texts.append(page.get_text("text"))
33         for font in page.get_fonts(full=True):
34             xref, ftype, basefont = font[0], font[2], font[3]
35             if xref in fonts:
36                 continue
37             embedded = ftype == "Type3"
38             if not embedded and xref:
39                 try:
40                     embedded = len(doc.extract_font(xref)[3]) > 0
41                 except Exception:
42                     embedded = False
43             fonts[xref] = {"name": basefont, "type": ftype, "embedded": bool(embedded)}
44             if not embedded:
45                 unembedded.append(basefont)
46     return {
47         "pages": len(doc),
48         "page_boxes": pages,
49         "all_a4": all(p["a4"] for p in pages),
50         "fonts": list(fonts.values()),
51         "unembedded_fonts": sorted(set(unembedded)),
52         "texts": texts,
53         "metadata": doc.metadata,
54     }
55
56
57 def scan_strings(texts: Iterable[str], patterns: Iterable[str], *, context: int = 40) -> list[dict[str, Any]]:
58     compiled = [(p, re.compile(p)) for p in patterns]
59     hits: list[dict[str, Any]] = []
60     for page_no, text in enumerate(texts, start=1):
61         for raw, pattern in compiled:
62             for match in pattern.finditer(text):
63                 start, end = max(0, match.start() - context), min(len(text), match.end() + context)

```

```

64         hits.append({"page": page_no, "pattern": raw, "context": text[start:end].replace("\n", " ")})
65     return hits

```

forge/plotting.py

```

1  """House style for paper figures: vector PDF output, CJK-capable fonts, accessible palette."""
2
3  from __future__ import annotations
4
5  import logging
6  import warnings
7  from pathlib import Path
8  from typing import Any
9
10 OKABE_ITO = ["#0072B2", "#D55E00", "#009E73", "#E69F00", "#CC79A7", "#56B4E9", "#F0E442", "#000000"]
11 # (family name as matplotlib registers it, font file). TTC files expose only their first face,
12 # which is why the Noto CJK collection is addressed by its JP face name.
13 _CJK_CANDIDATES = (
14     ("WenQuanYi Micro Hei", "/usr/share/fonts/truetype/wqy/wqy-microhei.ttc"),
15     ("Noto Sans CJK JP", "/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc"),
16     ("Droid Sans Fallback", "/usr/share/fonts/truetype/droid/DroidSansFallbackFull.ttf"),
17 )
18 _PROBE_TEXT = "温度 T 水分 C 药材 频段 关键词 abc 0.25"
19 _STATE: dict[str, Any] = {}
20
21
22 class _Catcher(logging.Handler):
23     def __init__(self) -> None:
24         super().__init__()
25         self.records: list[str] = []
26
27     def emit(self, record: logging.LogRecord) -> None:
28         self.records.append(record.getMessage())
29
30
31 def _probe(family: str) -> bool:
32     """True when ``family`` (with DejaVu Sans as Latin fallback) renders the probe text into PDF cleanly."""
33     from matplotlib import pyplot as plt
34
35     catcher = _Catcher()
36     loggers = [
37         logging.getLogger(n) for n in ("matplotlib.font_manager", "matplotlib.backends.backend_pdf", "matplotlib")
38     ]
39     for lg in loggers:
40         lg.addHandler(catcher)
41     try:
42         with warnings.catch_warnings(record=True) as caught:
43             warnings.simplefilter("always")
44             fig, ax = plt.subplots(figsize=(2.5, 1))
45             ax.text(0.05, 0.5, _PROBE_TEXT, fontfamily=[family, "DejaVu Sans"])
46             ax.set_axis_off()
47             probe = Path("/tmp") / f"forge_font_probe_{family.replace(' ', '_')}.pdf"
48             fig.savefig(probe, format="pdf")
49             plt.close(fig)
50             messages = [str(w.message) for w in caught] + catcher.records
51             bad = [m for m in messages if "missing from font" in m or "not found" in m.lower()]
52             return not bad
53     except Exception:
54         return False
55     finally:
56         for lg in loggers:
57             lg.removeHandler(catcher)
58
59
60 def cjk_font_family() -> str:
61     """Pick a CJK font that renders into PDF without missing glyphs (cached per process)."""
62     if "family" in _STATE:
63         return str(_STATE["family"])

```

```

64     import matplotlib
65
66     matplotlib.use("Agg")
67     from matplotlib import font_manager
68
69     chosen = "DejaVu Sans"
70     for family, path in _CJK_CANDIDATES:
71         if not Path(path).exists():
72             continue
73         try:
74             font_manager.fontManager.addfont(path)
75         except Exception:
76             continue
77         if _probe(family):
78             chosen = family
79             break
80     _STATE["family"] = chosen
81     return chosen
82
83
84 def setup(font_size: float = 9.0) -> str:
85     import matplotlib
86
87     matplotlib.use("Agg")
88     from cycler import cycler
89     from matplotlib import pyplot as plt
90
91     family = cjk_font_family()
92     plt.rcParams.update(
93         {
94             "font.family": [family, "DejaVu Sans"],
95             "font.size": font_size,
96             "axes.titlesize": font_size + 1,
97             "axes.labelsize": font_size,
98             "legend.fontsize": font_size - 1,
99             "xtick.labelsize": font_size - 1,
100            "ytick.labelsize": font_size - 1,
101            "axes.prop_cycle": cycler(color=OKABE_ITO),
102            "axes.spines.top": False,
103            "axes.spines.right": False,
104            "axes.grid": True,
105            "grid.alpha": 0.25,
106            "grid.linewidth": 0.6,
107            "lines.linewidth": 1.4,
108            "figure.dpi": 150,
109            "savefig.dpi": 300,
110            "pdf.fonttype": 42,
111            "ps.fonttype": 42,
112            "axes.unicode_minus": False,
113            "figure.constrained_layout.use": True,
114        }
115     )
116     return family
117
118
119 def new_figure(width_in: float = 6.3, height_in: float = 3.4, **kwargs: Any) -> tuple[Any, Any]:
120     from matplotlib import pyplot as plt
121
122     setup()
123     return plt.subplots(figsize=(width_in, height_in), **kwargs)
124
125
126 def save(fig: Any, pdf_path: Path, *, png_preview: bool = True, dpi: int = 200) -> dict[str, Any]:
127     from matplotlib import pyplot as plt
128
129     pdf_path.parent.mkdir(parents=True, exist_ok=True)
130     fig.savefig(pdf_path, format="pdf")
131     out = {"pdf": str(pdf_path), "bytes": pdf_path.stat().st_size}
132     if png_preview:
133         png = pdf_path.with_suffix(".png")
134         fig.savefig(png, format="png", dpi=dpi)

```

```

135         out["png"] = str(png)
136     plt.close(fig)
137     return out

```

forge/runner.py

```

1  """Stage registry and executor. A stage is a function ``fn(ctx) -> metrics``.
2
3  Guarantees: one manifest per (run, stage); dependencies must be completed in the same run
4  (or copied from ``from_run``); a completed stage is skipped unless ``force``; failures are
5  recorded with a traceback and re-raised; the volume is committed after every stage.
6  """
7
8  from __future__ import annotations
9
10 import importlib
11 import json
12 import shutil
13 import time
14 from collections.abc import Callable, Iterable
15 from dataclasses import dataclass
16 from pathlib import Path
17 from typing import Any
18
19 from .context import StageContext
20 from .hashing import hash_tree, tree_digest
21 from .manifest import Manifest
22
23 StageFn = Callable[[StageContext], dict[str, Any] | None]
24 CODE_IGNORE = (
25     ".git",
26     "artifacts",
27     "releases",
28     "__pycache__",
29     ".mypy_cache",
30     ".ruff_cache",
31     ".pytest_cache",
32     ".hypothesis",
33     ".DS_Store",
34     ".forge",
35 )
36
37
38 @dataclass(frozen=True)
39 class StageSpec:
40     name: str
41     fn: StageFn
42     deps: tuple[str, ...]
43     description: str
44
45
46 STAGES: dict[str, StageSpec] = {}
47
48
49 def stage(name: str, deps: Iterable[str] = (), description: str = "") -> Callable[[StageFn], StageFn]:
50     def decorator(fn: StageFn) -> StageFn:
51         if name in STAGES:
52             raise ValueError(f"duplicate stage registration: {name}")
53         doc = (fn.__doc__ or "").strip().splitlines()
54         STAGES[name] = StageSpec(name, fn, tuple(deps), description or (doc[0] if doc else ""))
55         return fn
56     return decorator
57
58
59
60 def discover(repo: Path) -> None:
61     project = json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))
62     importlib.import_module("pipelines.common.stages")

```

```

63     importlib.import_module(f"pipelines.{project['package']}.stages")
64
65
66 def list_stages(repo: Path) -> list[dict[str, Any]]:
67     discover(repo)
68     return [{"name": s.name, "deps": list(s.deps), "description": s.description} for s in STAGES.values()]
69
70
71 def _raw_input_prefixes(repo: Path) -> tuple[str, ...]:
72     project = json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))
73     return (project["problem_pdf"], project["attachments_dir"].rstrip("/") + "/")
74
75
76 def execute(
77     stage_name: str,
78     run_id: str,
79     params: dict[str, Any],
80     force: bool,
81     repo: Path,
82     vol: Path,
83     commit: Callable[[], None] | None = None,
84 ) -> dict[str, Any]:
85     discover(repo)
86     if stage_name == "all":
87         results = [execute(s, run_id, params, force, repo, vol, commit) for s in params.get("stages", [])]
88         return {
89             "stage": "all",
90             "run_id": run_id,
91             "results": results,
92             "status": "completed" if all(r.get("status") in {"completed", "skipped"} for r in results) else "failed",
93         }
94     if stage_name not in STAGES:
95         raise KeyError(f"unknown stage {stage_name!r}; known stages: {sorted(STAGES)}")
96     spec = STAGES[stage_name]
97     run_dir = vol / "runs" / run_id
98     stage_dir = run_dir / stage_name
99     manifest_path = stage_dir / "manifest.json"
100
101     if manifest_path.exists() and not force:
102         previous = Manifest.read(manifest_path)
103         if previous.status == "completed":
104             return {
105                 "stage": stage_name,
106                 "run_id": run_id,
107                 "status": "skipped",
108                 "reason": "already completed in this run (use --force to rerun)",
109                 "outputs_digest": previous.outputs_digest,
110             }
111
112     deps_digest: dict[str, str] = {}
113     for dep in spec.deps:
114         dep_manifest = run_dir / dep / "manifest.json"
115         if not dep_manifest.exists() and params.get("from_run"):
116             source = vol / "runs" / str(params["from_run"]) / dep
117             if (source / "manifest.json").exists():
118                 shutil.copytree(source, run_dir / dep, dirs_exist_ok=True)
119         if not dep_manifest.exists():
120             raise RuntimeError(
121                 f"stage '{stage_name}' requires '{dep}', which has not run in run {run_id} "
122                 f"f"(run it first, or pass from_run=<run_id> to reuse its outputs)"
123             )
124         dep_state = Manifest.read(dep_manifest)
125         if dep_state.status != "completed":
126             raise RuntimeError(f"dependency '{dep}' has status {dep_state.status!r} in run {run_id}")
127         deps_digest[dep] = dep_state.outputs_digest
128
129     if stage_dir.exists():
130         shutil.rmtree(stage_dir)
131     ctx = StageContext(run_id, stage_name, params, repo, vol)
132     code_tree = hash_tree(repo, ignore=CODE_IGNORE)
133     prefixes = _raw_input_prefixes(repo)

```

```

134     inputs = {k: v for k, v in code_tree.items() if k.startswith(prefixes)}
135     code_ref = {**dict(params.get("code_ref") or {}), "code_digest": tree_digest(code_tree), "files": len(code_tree)}
136     manifest = Manifest.start(run_id, stage_name, params, code_ref, ctx.config.digest, deps_digest, inputs)
137     manifest.write(manifest_path)
138     ctx.log.info("stage.start", deps=list(spec.deps), description=spec.description, size=params.get("size"))
139     t0 = time.monotonic()
140     try:
141         metrics = spec.fn(ctx) or {}
142         ctx.flush_numbers()
143         manifest.complete(stage_dir, metrics)
144         ctx.log.info("stage.completed", duration_s=round(time.monotonic() - t0, 3), metrics=metrics)
145     except BaseException as exc:
146         manifest.fail(exc)
147         ctx.log.error("stage.failed", error=repr(exc))
148         manifest.write(manifest_path)
149         ctx.log.close()
150         if commit:
151             commit()
152         raise
153     manifest.write(manifest_path)
154     ctx.log.close()
155     if commit:
156         commit()
157     return {
158         "stage": stage_name,
159         "run_id": run_id,
160         "status": "completed",
161         "duration_s": manifest.duration_s,
162         "metrics": manifest.metrics,
163         "outputs_digest": manifest.outputs_digest,
164         "modal_task_id": manifest.runtime.get("modal_task_id"),
165     }

```

forge/tex.py

```

1  """XeLaTeX compilation and log analysis (only ever executed inside Modal)."""
2
3  from __future__ import annotations
4
5  import os
6  import re
7  import subprocess
8  from pathlib import Path
9  from typing import Any
10
11  _UNDEF_REF = re.compile(r"LaTeX Warning: Reference `([~]+)' on page \d+ undefined")
12  _UNDEF_CIT = re.compile(r"LaTeX Warning: Citation `([~]+)' on page \d+ undefined")
13  _OVERFULL = re.compile(r"Overfull \hbox \((([~]+)pt too wide\)")
14  _MISSING = re.compile(r"Missing character: There is no (.*?) in font")
15  _ERROR = re.compile(r"^! (.*?)", re.M)
16  _PAGES = re.compile(r"Output written on .*? \((\d+) pages?")
17
18
19  def tex_env() -> dict[str, str]:
20     env = dict(os.environ)
21     env.setdefault("TEXMFVAR", "/tmp/texmf-var")
22     env.setdefault("TEXMFCONFIG", "/tmp/texmf-config")
23     env.setdefault("HOME", "/tmp")
24     env["max_print_line"] = "2000"
25     env["error_line"] = "254"
26     env["half_error_line"] = "238"
27     return env
28
29
30  def summarize_log(log: str) -> dict[str, Any]:
31     overfull = [float(x) for x in _OVERFULL.findall(log)]
32     missing = _MISSING.findall(log)
33     pages = _PAGES.search(log)

```

```

34     return {
35         "undefined_references": sorted(set(_UNDEF_REF.findall(log))),
36         "undefined_citations": sorted(set(_UNDEF_CIT.findall(log))),
37         "overfull_count": len(overfull),
38         "overfull_max_pt": max(overfull) if overfull else 0.0,
39         "missing_characters": len(missing),
40         "missing_character_samples": sorted(set(missing))[:10],
41         "errors": _ERROR.findall(log)[:10],
42         "pages_from_log": int(pages.group(1)) if pages else None,
43     }
44
45
46 def aux_page(aux: Path, label: str) -> int | None:
47     """Page number recorded for ``\label{label}`` in the .aux file (hyperref-compatible)."""
48     if not aux.exists():
49         return None
50     pattern = re.compile(r"\newlabel\{" + re.escape(label) + r"\}\{\{[^\}]*\}\{(\d+)\}"
51 match = pattern.search(aux.read_text(encoding="utf-8", errors="replace"))
52     return int(match.group(1)) if match else None
53
54
55 def compile_tex(
56     workdir: Path,
57     main: str,
58     jobname: str,
59     *,
60     passes: int = 3,
61     bibtex: bool = True,
62     timeout: int = 1200,
63 ) -> dict[str, Any]:
64     env = tex_env()
65     runs: list[dict[str, Any]] = []
66     log_path = workdir / f"{jobname}.log"
67
68     def xelatex(tag: str) -> None:
69         proc = subprocess.run(
70             ["xelatex", "-interaction=nonstopmode", "-halt-on-error", "-file-line-error", f"-jobname={jobname}", main],
71             cwd=workdir,
72             capture_output=True,
73             text=True,
74             env=env,
75             timeout=timeout,
76             check=False,
77         )
78         runs.append({"pass": tag, "rc": proc.returncode})
79         if proc.returncode != 0:
80             log = log_path.read_text(encoding="utf-8", errors="replace") if log_path.exists() else proc.stdout
81             errors = _ERROR.findall(log)
82             raise RuntimeError(
83                 f"xelatex failed on pass {tag} for {main}: "
84                 + " | ".join(errors[:5])
85                 + "\n--- log tail ---\n"
86                 + log[-4000:]
87             )
88
89     xelatex("1")
90     aux = workdir / f"{jobname}.aux"
91     if bibtex and aux.exists():
92         aux_text = aux.read_text(encoding="utf-8", errors="replace")
93         if "\\citation" in aux_text and "\\bibdata" in aux_text:
94             proc = subprocess.run(
95                 ["bibtex", jobname], cwd=workdir, capture_output=True, text=True, env=env, timeout=timeout, check=False
96             )
97             runs.append({"pass": "bibtex", "rc": proc.returncode, "tail": proc.stdout[-1500:]})
98     for i in range(2, passes + 1):
99         xelatex(str(i))
100     log_text = log_path.read_text(encoding="utf-8", errors="replace")
101     return {
102         "pdf": workdir / f"{jobname}.pdf",
103         "log": log_path,
104         "aux": aux,

```

```

105         "runs": runs,
106         "summary": summarize_log(log_text),
107     }
108
109
110 def tex_escape(text: str) -> str:
111     replacements = {
112         "\\": r"\textbackslash{}",
113         "&": r"\&",
114         "%": r"\%",
115         "$": r"\$",
116         "#": r"\#",
117         "_": r"\_",
118         "{": r"\{",
119         "}": r"\}",
120         "~": r"\textasciitilde{}",
121         "~": r"\textasciicircum{}",
122     }
123     return "".join(replacements.get(ch, ch) for ch in text)
124
125
126 def tex_path(path: str) -> str:
127     """Typeset a file path so that it can break across lines (hyperref/xurl); falls back to \texttt."""
128     if any(ch in path for ch in "{}%#\\") or not path.isascii():
129         return f"\\texttt{{{tex_escape(path)}}}" # CJK names need the CJK mono font, not the url font
130     return f"\\nolinkurl{{{path}}}"

```

forge/xlsx.py

```

1  """Workbook I/O: reading attachments, filling result templates, checking result contracts."""
2
3  from __future__ import annotations
4
5  import math
6  from collections.abc import Iterable, Sequence
7  from dataclasses import dataclass
8  from itertools import pairwise
9  from pathlib import Path
10 from typing import Any
11
12 import openpyxl
13 import pandas as pd
14
15 BIG_SHEET_CELLS = 300_000
16
17
18 def read_sheets(path: Path, header: int | None = 0) -> dict[str, pd.DataFrame]:
19     with pd.ExcelFile(path) as xf:
20         return {str(name): xf.parse(name, header=header) for name in xf.sheet_names}
21
22
23 def sheet_headers(path: Path) -> dict[str, list[Any]]:
24     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
25     out: dict[str, list[Any]] = {}
26     for ws in wb.worksheets:
27         row = list(next(ws.iter_rows(min_row=1, max_row=1, values_only=True), ()))
28         while row and row[-1] is None:
29             row.pop()
30         out[str(ws.title)] = row
31     wb.close()
32     return out
33
34
35 def _clean(value: Any, decimals: int | None) -> Any:
36     if hasattr(value, "item") and not isinstance(value, str | bytes):
37         try:
38             value = value.item()
39         except (ValueError, TypeError):

```

```

40         pass
41     if isinstance(value, float):
42         if math.isnan(value) or math.isinf(value):
43             return None
44         return round(value, decimals) if decimals is not None else value
45     return value
46
47
48 def write_result(
49     template: Path,
50     out: Path,
51     sheets: dict[str, tuple[Sequence[Any], Iterable[Sequence[Any]]]],
52     *,
53     decimals: int | None = 4,
54     float_format: str = "0.0000",
55 ) -> dict[str, Any]:
56     """Fill a competition result template.
57
58     ``sheets`` maps sheet name -> (header_row, rows). The template's A1 text is always kept;
59     header cells after A1 are taken from ``header_row[1:]``. Sheets not mentioned keep their
60     template content. Floats are rounded to ``decimals`` and formatted with ``float_format``.
61     Very large sheets fall back to openpyxl's write-only mode (no per-cell number format).
62     """
63     template_wb = openpyxl.load_workbook(template)
64     names = list(template_wb.sheetnames)
65     unknown = [n for n in sheets if n not in names]
66     if unknown:
67         raise KeyError(f"sheets {unknown} not in template {template.name} (has {names})")
68     a1 = {ws.title: ws.cell(row=1, column=1).value for ws in template_wb.worksheets}
69     materialised = {name: (list(header), [list(r) for r in rows]) for name, (header, rows) in sheets.items()}
70     cells = sum(max(len(h), 1) * (len(rows) + 1) for h, rows in materialised.values())
71     stats: dict[str, Any] = {}
72     out.parent.mkdir(parents=True, exist_ok=True)
73
74     if cells > BIG_SHEET_CELLS:
75         wb = openpyxl.Workbook(write_only=True)
76         for name in names:
77             ws = wb.create_sheet(name)
78             if name in materialised:
79                 header, rows = materialised[name]
80                 full_header = [a1[name], *header[1:]]
81                 ws.append(full_header)
82                 for row in rows:
83                     ws.append([_clean(v, decimals) for v in row])
84                 stats[name] = {"rows": len(rows), "cols": len(full_header)}
85             else:
86                 for tpl_row in template_wb[name].iter_rows(values_only=True):
87                     ws.append(list(tpl_row))
88         wb.save(out)
89     else:
90         wb = template_wb
91         for name, (header, rows) in materialised.items():
92             index = wb.sheetnames.index(name)
93             wb.remove(wb[name])
94             ws = wb.create_sheet(name, index) # fresh sheet: no stale template cells or dimensions
95             full_header = [a1[name], *header[1:]]
96             for j, value in enumerate(full_header, start=1):
97                 ws.cell(row=1, column=j, value=_clean(value, None))
98             for i, row in enumerate(rows, start=2):
99                 for j, value in enumerate(row, start=1):
100                     cleaned = _clean(value, decimals)
101                     cell = ws.cell(row=i, column=j, value=cleaned)
102                     if isinstance(cleaned, float) and j > 1:
103                         cell.number_format = float_format
104             stats[name] = {"rows": len(rows), "cols": len(full_header)}
105         wb.save(out)
106     return {"path": str(out), "sheets": stats, "write_only": cells > BIG_SHEET_CELLS, "cells": cells}
107
108
109 @dataclass(frozen=True)
110 class SheetContract:

```

```

111     name: str
112     min_rows: int = 1 # data rows (excluding the header)
113     max_rows: int | None = None
114     header_len: int | None = None # exact number of header cells, if known
115     numeric_from_col: int | None = None # 0-based; every data cell from this column must be numeric
116     allow_blank: bool = False # blank numeric cells permitted (e.g. moving-boundary tables)
117     decimals: int | None = 4 # numeric cells must carry at most this many decimals
118     text_columns: tuple[int, ...] = () # 0-based columns that must be non-empty text
119     first_col_step: float | None = None # required constant step of the first (time) column
120     header_values: tuple[float, ...] = () # expected header values at ``header_value_columns``
121     header_value_columns: tuple[int, ...] = () # 0-based header positions carrying ``header_values``
122
123
124 @dataclass(frozen=True)
125 class WorkbookContract:
126     file: str
127     template: str
128     sheets: tuple[SheetContract, ...]
129
130
131 def check_workbook(path: Path, contract: WorkbookContract, template: Path) -> dict[str, Any]:
132     errors: list[str] = []
133     if not path.exists():
134         return {"file": contract.file, "ok": False, "errors": ["file missing"], "sheets": {}}
135     try:
136         tpl_headers = sheet_headers(template)
137     except Exception as exc:
138         tpl_headers = {}
139         errors.append(f"template unreadable: {exc!r}")
140     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
141     report: dict[str, Any] = {}
142     names = wb.sheetnames
143     if tpl_headers and names != list(tpl_headers):
144         errors.append(f"sheet names {names} differ from template {list(tpl_headers)}")
145     for sc in contract.sheets:
146         if sc.name not in names:
147             errors.append(f"sheet {sc.name!r} missing")
148             continue
149         ws = wb[sc.name]
150         rows = ws.iter_rows(values_only=True)
151         first = next(rows, None)
152         header: list[Any] = list(first) if first is not None else []
153         while header and header[-1] is None:
154             header.pop()
155         tpl = tpl_headers.get(sc.name)
156         if tpl and header and header[0] != tpl[0]:
157             errors.append(f"{sc.name}: A1 {header[0]!r} differs from template {tpl[0]!r}")
158         if sc.header_len is not None and len(header) != sc.header_len:
159             errors.append(f"{sc.name}: header has {len(header)} cells, expected {sc.header_len}")
160         for want, col in zip(sc.header_values, sc.header_value_columns):
161             got = header[col] if col < len(header) else None
162             if not isinstance(got, int | float) or abs(float(got) - want) > 1e-9:
163                 errors.append(f"{sc.name}: header cell {col + 1} is {got!r}, expected {want}")
164
165         n = 0
166         bad_numeric = blank = bad_decimals = bad_text = 0
167         first_col: list[float] = []
168         for row in rows:
169             values = list(row)
170             if all(v is None or (isinstance(v, str) and not v.strip()) for v in values):
171                 continue
172             n += 1
173             if sc.first_col_step is not None and values and isinstance(values[0], int | float):
174                 first_col.append(float(values[0]))
175             if sc.numeric_from_col is not None:
176                 for v in values[sc.numeric_from_col : len(header) or None]:
177                     if v is None or (isinstance(v, str) and not v.strip()):
178                         blank += 1
179                     elif not isinstance(v, int | float) or isinstance(v, bool):
180                         bad_numeric += 1
181                     elif sc.decimals is not None and isinstance(v, float) and abs(round(v, sc.decimals) - v) > 1e-12:
182                         bad_decimals += 1

```

```

182         for j in sc.text_columns:
183             v = values[j] if j < len(values) else None
184             if v is None or not str(v).strip():
185                 bad_text += 1
186         if sc.first_col_step is not None:
187             if len(first_col) != n:
188                 errors.append(f"{sc.name}: {n - len(first_col)} rows without a numeric time value")
189             else:
190                 steps = [b - a for a, b in pairwise(first_col)]
191                 off = [s for s in steps if abs(s - sc.first_col_step) > 1e-9]
192                 if off:
193                     errors.append(f"{sc.name}: {len(off)} time steps differ from {sc.first_col_step} (e.g. {off[0]})")
194                 if first_col and abs(first_col[0] - sc.first_col_step) > 1e-9:
195                     errors.append(f"{sc.name}: first time value {first_col[0]}, expected {sc.first_col_step}")
196         if n < sc.min_rows:
197             errors.append(f"{sc.name}: {n} data rows < {sc.min_rows}")
198         if sc.max_rows is not None and n > sc.max_rows:
199             errors.append(f"{sc.name}: {n} data rows > {sc.max_rows}")
200         if bad_numeric:
201             errors.append(f"{sc.name}: {bad_numeric} non-numeric cells in numeric region")
202         if blank and not sc.allow_blank:
203             errors.append(f"{sc.name}: {blank} blank cells in numeric region")
204         if bad_decimals:
205             errors.append(f"{sc.name}: {bad_decimals} cells with more than {sc.decimals} decimals")
206         if bad_text:
207             errors.append(f"{sc.name}: {bad_text} empty cells in required text columns")
208         report[sc.name] = {
209             "rows": n,
210             "header": [str(h) for h in header[:6]] + (["..."] if len(header) > 6 else []),
211             "blank": blank,
212             "first_col": [first_col[0], first_col[-1]] if first_col else None,
213         }
214     wb.close()
215     return {"file": contract.file, "ok": not errors, "errors": errors, "sheets": report}

```

pipelines/__init__.py

pipelines/a/__init__.py

pipelines/a/common.py

```

1  """Shared helpers for the Problem-A science stages: loading inputs, building specs, storing solutions."""
2
3  from __future__ import annotations
4
5  from typing import Any
6
7  import numpy as np
8  import pandas as pd
9
10 from forge.context import StageContext
11 from pipelines.a.physics import (
12     APPENDIX2,
13     APPENDIX3,
14     APPENDIX4,
15     RO,
16     Chamber,
17     FluxCap,
18     Properties,

```

```

19     Radius,
20 )
21 from pipelines.a.solver import ProblemSpec, Solution, sample_physical, subgrid_indices
22
23 R_OUT_CM = np.round(np.arange(0, 21) * 0.1, 1) # 0.0 ... 2.0 cm
24 R_TABLE_CM = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
25 COLS = [f"{r:.1f}" for r in R_OUT_CM]
26 PROPS = {"q1": APPENDIX2, "q2": APPENDIX3, "q3": APPENDIX3, "q4": APPENDIX4}
27
28
29 def load_chamber(ctx: StageContext, plateau: str | None = None) -> Chamber:
30     df = pd.read_parquet(ctx.dep("ingest") / "data" / "附件1__Sheet1.parquet")
31     rule = plateau or str(ctx.cfg("a.chamber.plateau", "mean_last_hour"))
32     return Chamber.from_series(
33         df["时间"].to_numpy(float), df["温度"].to_numpy(float), df["水分浓度"].to_numpy(float), rule
34     )
35
36
37 def load_radius(ctx: StageContext, shrink_scale: float = 1.0) -> Radius:
38     df = pd.read_parquet(ctx.dep("ingest") / "data" / "附件2__Sheet1.parquet")
39     return Radius.from_series(df["时间"].to_numpy(float), df["半径"].to_numpy(float), shrink_scale)
40
41
42 def production_n(ctx: StageContext) -> int:
43     return 20 * 2 ** int(ctx.cfg("a.grid.refine", 3))
44
45
46 def time_options(ctx: StageContext) -> dict[str, Any]:
47     return {
48         "method": str(ctx.cfg("a.time.method", "BDF")),
49         "rtol": float(ctx.cfg("a.time.rtol", 1e-7)),
50         "atol": float(ctx.cfg("a.time.atol", 1e-9)),
51     }
52
53
54 def build_spec(
55     ctx: StageContext,
56     problem: str,
57     *,
58     props: Properties | None = None,
59     chamber: Chamber | None = None,
60     radius: Radius | None = None,
61     formulation: str | None = None,
62     face_scheme: str | None = None,
63     mass_form: str | None = None,
64 ) -> ProblemSpec:
65     """Default specification of problem q1...q4 with optional overrides (sensitivity / alternatives)."""
66     if radius is None:
67         radius = load_radius(ctx) if problem == "q4" else Radius.constant(R0)
68     return ProblemSpec(
69         props=props or PROPS[problem],
70         chamber=chamber or load_chamber(ctx),
71         radius=radius,
72         formulation=formulation or str(ctx.cfg("a.scheme.formulation", "lagrangian")),
73         face_scheme=face_scheme or str(ctx.cfg("a.scheme.face", "midpoint")),
74         mass_form=mass_form or str(ctx.cfg("a.scheme.mass_form", "drybasis")),
75     )
76
77
78 def profiles_frame(sol: Solution, field: str) -> pd.DataFrame:
79     """Field values on the 0.1 cm output grid (fixed radius: coincident nodes) as a DataFrame with a 't' column."""
80     values = getattr(sol, field)
81     idx = subgrid_indices(len(sol.xi) - 1)
82     frame = pd.DataFrame(values[:, idx], columns=COLS)
83     frame.insert(0, "t", sol.t)
84     return frame
85
86
87 def physical_frame(sol: Solution, field: str) -> pd.DataFrame:
88     """Field values at fixed physical radii 0...2 cm (NaN beyond R(t)) plus the surface value, shrinking domain."""
89     values = getattr(sol, field)

```

```

90     rows = np.full((len(sol.t), len(R_OUT_CM)), np.nan)
91     for i in range(len(sol.t)):
92         rows[i] = sample_physical(values[i], sol.xi, float(sol.radius[i]), R_OUT_CM / 100.0)
93     frame = pd.DataFrame(rows, columns=COLS)
94     frame.insert(0, "t", sol.t)
95     frame["surface"] = values[:, -1]
96     frame["radius_cm"] = sol.radius * 100.0
97     return frame
98
99
100 def table_cells(frame: pd.DataFrame, times: np.ndarray, radii_cm: np.ndarray = R_TABLE_CM) -> list[list[float]]:
101     """Rows [t, v(r1), ...] of a profiles frame at the requested instants (must exist exactly in 't')."""
102     out: list[list[float]] = []
103     lookup = {float(t): i for i, t in enumerate(frame["t"].to_numpy(float))}
104     for t in times:
105         i = lookup[float(t)]
106         out.append([float(t)] + [float(frame.at[i, f"{r:.1f}"]) for r in radii_cm])
107     return out
108
109
110 def snapshot_arrays(sol: Solution, times: np.ndarray) -> dict[str, np.ndarray]:
111     lookup = {float(t): i for i, t in enumerate(sol.t)}
112     idx = [lookup[float(t)] for t in times if float(t) in lookup]
113     return {
114         "t": sol.t[idx],
115         "xi": sol.xi,
116         "temp": sol.temp[idx],
117         "moist": sol.moist[idx],
118         "radius": sol.radius[idx],
119     }
120
121
122 def mean_moisture(sol: Solution) -> np.ndarray:
123     """Volume-averaged (dry-matter-weighted for the Lagrangian case) moisture history: 2 * sum voli Ci."""
124     from pipelines.a.verify import control_volumes
125
126     return 2.0 * (sol.moist @ control_volumes(sol.xi))
127
128
129 def spec_to_recipe(spec: ProblemSpec) -> dict[str, Any]:
130     """Pickleable description of a spec (workers rebuild the interpolants themselves)."""
131     from dataclasses import asdict
132
133     ch = spec.chamber
134     rd = spec.radius
135     return {
136         "props": asdict(spec.props),
137         "chamber": {
138             "t": ch.t.tolist(),
139             "temp": ch.temp.tolist(),
140             "hum": ch.hum.tolist(),
141             "temp_plateau": ch.temp_plateau,
142             "hum_plateau": ch.hum_plateau,
143             "temp_scale": ch.temp_scale,
144             "hum_scale": ch.hum_scale,
145         },
146         "radius": {"t": rd.t.tolist(), "r": rd.r.tolist(), "shrink_scale": rd.shrink_scale},
147         "formulation": spec.formulation,
148         "face_scheme": spec.face_scheme,
149         "mass_form": spec.mass_form,
150         "t_init": spec.t_init,
151         "c_init": spec.c_init,
152         "flux_cap": None
153         if spec.flux_cap is None
154         else {
155             "t": spec.flux_cap.t.tolist(),
156             "cap": spec.flux_cap.cap.tolist(),
157             "plateau": spec.flux_cap.plateau,
158             "rho_s": spec.flux_cap.rho_s,
159             "scale": spec.flux_cap.scale,
160             "local_rho_s": spec.flux_cap.local_rho_s,

```

```

161     },
162 }
163
164
165 def spec_from_recipe(recipe: dict[str, Any]) -> ProblemSpec:
166     ch = recipe["chamber"]
167     rd = recipe["radius"]
168     fc = recipe.get("flux_cap")
169     return ProblemSpec(
170         props=Properties(**recipe["props"]),
171         chamber=Chamber(
172             np.asarray(ch["t"], float),
173             np.asarray(ch["temp"], float),
174             np.asarray(ch["hum"], float),
175             float(ch["temp_plateau"]),
176             float(ch["hum_plateau"]),
177             float(ch["temp_scale"]),
178             float(ch["hum_scale"]),
179         ),
180         radius=Radius(np.asarray(rd["t"], float), np.asarray(rd["r"], float), float(rd["shrink_scale"])),
181         formulation=str(recipe["formulation"]),
182         face_scheme=str(recipe["face_scheme"]),
183         mass_form=str(recipe.get("mass_form", "drybasis")),
184         t_init=float(recipe["t_init"]),
185         c_init=float(recipe["c_init"]),
186         flux_cap=None
187         if fc is None
188         else FluxCap(
189             np.asarray(fc["t"], float),
190             np.asarray(fc["cap"], float),
191             float(fc["plateau"]),
192             float(fc["rho_s"]),
193             float(fc["scale"]),
194             bool(fc.get("local_rho_s", False)),
195         ),
196     )

```

pipelines/a/contracts.py

```

1  """Data contracts for Problem A (herb drying): inputs and result workbooks."""
2
3  from __future__ import annotations
4
5  from forge.contracts import Column, FrameContract
6  from forge.xlsx import SheetContract, WorkbookContract
7
8  INPUT_CONTRACTS: dict[str, FrameContract] = {
9      "附件1__Sheet1": FrameContract(
10         name="A.附件1.烘房温湿度",
11         columns=(
12             Column("时间", "int", min=0, unique=True, monotonic="increasing", step=60),
13             Column("温度", "float", min=0, max=120),
14             Column("水分浓度", "float", min=0, max=1),
15         ),
16         min_rows=200,
17     ),
18     "附件2__Sheet1": FrameContract(
19         name="A.附件2.药材半径",
20         columns=(
21             Column("时间", "int", min=0, unique=True, monotonic="increasing"),
22             Column("半径", "float", min=0.1, max=2.0, monotonic="non_increasing"),
23         ),
24         min_rows=100,
25     ),
26 }
27
28 _RADIAL = 22 # A1 label + 21 distances 0.0 ... 2.0 (step 0.1 cm)
29 _HEADER = [0.0, 0.5, 1.0, 1.5, 2.0] # first-row distances checked at columns B, G, L, Q, V (A1 is the label)

```

```

30 WHOLE_PROCESS_FILE = "result2_全过程.xlsx" # supplementary deliverable, whole process at 60 s (MDR-0005)
31
32
33 def _sheet(name: str, step: float, **kw: object) -> SheetContract:
34     """Result sheet contract: header positions, time-column step and the numeric region."""
35     return SheetContract(
36         name,
37         header_len=kw.pop("header_len", _RADIAL), # type: ignore[arg-type]
38         numeric_from_col=0,
39         first_col_step=step,
40         header_values=tuple(_HEADER),
41         header_value_columns=(1, 6, 11, 16, 21),
42         **kw, # type: ignore[arg-type]
43     )
44
45
46 RESULT_CONTRACTS: list[WorkbookContract] = [
47     WorkbookContract(
48         "result1.xlsx",
49         "result1.xlsx",
50         (
51             _sheet("温度", 1.0, min_rows=1800, max_rows=1801),
52             _sheet("水分浓度", 1.0, min_rows=1800, max_rows=1801),
53         ),
54     ),
55     WorkbookContract(
56         "result2.xlsx",
57         "result2.xlsx",
58         (
59             _sheet("温度", 1.0, min_rows=10800, max_rows=10801),
60             _sheet("水分浓度", 1.0, min_rows=10800, max_rows=10801),
61         ),
62     ),
63     WorkbookContract("result3.xlsx", "result3.xlsx", (_sheet("Sheet1", 60.0, min_rows=60),)),
64     WorkbookContract(
65         "result4.xlsx",
66         "result4.xlsx",
67         (_sheet("Sheet1", 60.0, min_rows=60, header_len=_RADIAL + 1, allow_blank=True),),
68     ),
69     WorkbookContract(
70         WHOLE_PROCESS_FILE,
71         "result2.xlsx",
72         (_sheet("温度", 60.0, min_rows=60), _sheet("水分浓度", 60.0, min_rows=60)),
73     ),
74 ]

```

pipelines/a/derived.py

```

1 """Derived quantities quoted in the paper: dimensionless groups, time scales, extrapolated errors, solver statistics.
2
3 Every number the manuscript quotes must come from a stage's ``ctx.number`` registration (docs/ENGINEERING_STANDARD.md,
4 MDR-0009). This stage computes the a-priori estimates used in the analysis and verification sections from the stored
5 outputs of the science stages only (no new simulations), so that the text of the paper carries no hand-typed arithmetic
6     ↪ .
7
8 """
9
10 from __future__ import annotations
11
12 from typing import Any
13
14 import numpy as np
15 import pandas as pd
16
17 from forge.context import StageContext
18 from forge.runner import stage
19 from pipelines.a.physics import (
20     APPENDIX3,
21     APPENDIX4,

```

```

20     C_INIT,
21     C_TARGET,
22     LENGTH,
23     PLATEAU_WINDOW,
24     R0,
25     dry_solid_density,
26     thermal_diffusivity,
27 )
28
29 HOUR = 3600.0
30 Q1_END_S = 1800.0
31
32
33 def fourier_number(diffusivity: float, t: float, length: float) -> float:
34     """Fo = D t / L^2."""
35     return float(diffusivity * t / length**2)
36
37
38 def richardson_error(values: list[float], order: float = 2.0) -> tuple[float, float]:
39     """Richardson extrapolation of a sequence computed on grids halved at every level (finest last).
40
41     Returns ``(limit, error)``: the limit estimated from the two finest levels and the absolute error of the
42     second-finest level (the production grid) with respect to that limit.
43     """
44     if len(values) < 2:
45         raise ValueError("need at least two levels")
46     fine, coarse = float(values[-1]), float(values[-2])
47     limit = fine + (fine - coarse) / (2.0**order - 1.0)
48     return limit, abs(coarse - limit)
49
50
51 def first_time_within(t: np.ndarray, a: np.ndarray, b: np.ndarray, tol: float) -> float | None:
52     """First instant after which |a - b| stays below ``tol`` for the rest of the record (None if it never does)."""
53     ok = np.abs(np.asarray(a, dtype=float) - np.asarray(b, dtype=float)) < tol
54     if not ok.any():
55         return None
56     bad = np.where(~ok)[0]
57     start = 0 if len(bad) == 0 else int(bad[-1]) + 1
58     return None if start >= len(t) else float(np.asarray(t, dtype=float)[start])
59
60
61 def value_at(t: np.ndarray, values: np.ndarray, instant: float) -> float:
62     """Value of a recorded history at the output instant nearest ``instant`` (the instants are exact samples)."""
63     tt = np.asarray(t, dtype=float)
64     return float(np.asarray(values, dtype=float)[int(np.argmin(np.abs(tt - float(instant))))])
65
66
67 @stage(
68     "derived",
69     deps=("ingest", "q1", "q2", "q3", "q4", "convergence"),
70     description="Dimensionless groups, time scales, extrapolated grid errors and solver statistics quoted in the paper"
71     ↪ ,
72 )
73 def derived(ctx: StageContext) -> dict[str, Any]:
74     q1 = ctx.dep_data("q1", "summary.json")
75     q2h = ctx.dep_data("q2", "history.json")
76     q3 = ctx.dep_data("q3", "summary.json")
77     q3h = ctx.dep_data("q3", "history.json")
78     q4 = ctx.dep_data("q4", "summary.json")
79     q4h = ctx.dep_data("q4", "history.json")
80     conv = ctx.dep_data("convergence", "convergence.json")
81
82     # problem 1 (appendix 2, initial state): time scales, Fourier numbers, diffusion length, stiffness ratio
83     alpha1 = float(q1["biot"]["alpha"])
84     d1 = float(q1["biot"]["D"])
85     t_heat = R0**2 / alpha1
86     t_mass = R0**2 / d1
87     fo_heat = fourier_number(alpha1, Q1_END_S, R0)
88     fo_mass = fourier_number(d1, Q1_END_S, R0)
89     diffusion_length = float(np.sqrt(d1 * Q1_END_S))

```

```

90 # problem 2: thermal equilibration of the centre with the chamber
91 t2 = np.asarray(q2h["t"], dtype=float)
92 gap_1k = first_time_within(t2, q2h["centre_temp"], q2h["air_temp"], 1.0)
93 gap_half = first_time_within(t2, q2h["centre_temp"], q2h["air_temp"], 0.5)
94
95 # problem 3 (appendix 3): axial Fourier numbers for the 1-D reduction, diffusivity range (stiffness)
96 t3 = float(q3["t_dry_s"])
97 t_plateau = float(q3["spec"]["chamber"]["temp_plateau"])
98 alpha3 = thermal_diffusivity(APPENDIX3, C_INIT)
99 d_max = float(APPENDIX3.diffusivity(np.array(C_INIT), np.array(t_plateau)))
100 d_target = float(APPENDIX3.diffusivity(np.array(C_TARGET), np.array(t_plateau)))
101 d_surface_dry = float(APPENDIX3.diffusivity(np.array(q3["surface_moist_at_dry"]), np.array(t_plateau)))
102 bi_mass_dry = APPENDIX3.hm * R0 / d_surface_dry
103 d4_max = float(APPENDIX4.diffusivity(np.array(C_INIT), np.array(t_plateau)))
104 d4_target = float(APPENDIX4.diffusivity(np.array(C_TARGET), np.array(t_plateau)))
105 half_length = LENGTH / 2.0
106 fo_axial_heat = fourier_number(alpha3, t3, half_length)
107 fo_axial_mass = fourier_number(d_max, t3, half_length)
108
109 # drying-time discretisation error of the production grid (Richardson, order 2, two finest levels)
110 rows3 = conv["grid"]["q3_t_dry"]
111 rows4 = conv["grid"]["q4_t_dry"]
112
113 def _observed(rows: list[dict[str, Any]]) -> float:
114     """Finest unbiased grid-triplet order, clipped to a sane range (falls back to the formal order 2)."""
115     vals = [r["observed_order_triplet"] for r in rows if r.get("observed_order_triplet") is not None]
116     return float(vals[-1]) if vals and 0.5 <= float(vals[-1]) <= 3.0 else 2.0
117
118 order3, order4 = _observed(rows3), _observed(rows4)
119 limit3, err3 = richardson_error([float(r["t_dry_h"]) for r in rows3], order3)
120 limit4, err4 = richardson_error([float(r["t_dry_h"]) for r in rows4], order4)
121
122 # problem 4: the given R(t) and the given rho(C) are not compatible under conservation of dry matter
123 rho_s_wet = float(dry_solid_density(APPENDIX4, np.array(C_INIT)))
124 rho_s_dry = float(dry_solid_density(APPENDIX4, np.array(C_TARGET)))
125 r4_end_cm = float(q4["spec"]["radius"]["r_end_cm"])
126 dry_matter_ratio = (rho_s_dry * r4_end_cm**2) / (rho_s_wet * R0**2 * 1e4)
127 implied_r_end_cm = 100.0 * R0 / np.sqrt(rho_s_dry / rho_s_wet)
128 shrink_delta_cm = 100.0 * R0 - r4_end_cm
129 shrink_scaling = -2.0 * shrink_delta_cm / r4_end_cm # dln t/dln s from t ~ R_inf^2, R_inf = R0 - s (R0 - R_data)
130
131 # chamber data: plateau-window statistics and the jump at the switch to the plateau
132 chamber = pd.read_parquet(ctx.dep("ingest") / "data" / "附件1_Sheet1.parquet")
133 ct = chamber["时间"].to_numpy(float)
134 c_temp = chamber["温度"].to_numpy(float)
135 c_hum = chamber["水分浓度"].to_numpy(float)
136 window = ct >= ct[-1] - PLATEAU_WINDOW
137 plateau_temp = float(q3["spec"]["chamber"]["temp_plateau"])
138 noise = {
139     "window_s": PLATEAU_WINDOW,
140     "temp_min": float(c_temp[window].min()),
141     "temp_max": float(c_temp[window].max()),
142     "temp_step_max": float(np.abs(np.diff(c_temp[window])).max()),
143     "temp_std": float(c_temp[window].std(ddof=1)),
144     "temp_step_max_all": float(np.abs(np.diff(c_temp)).max()),
145     "hum_step_max": float(np.abs(np.diff(c_hum[window])).max()),
146     "plateau_jump_K": float(abs(c_temp[-1] - plateau_temp)),
147     "plateau_jump_hum": float(abs(c_hum[-1] - float(q3["spec"]["chamber"]["hum_plateau"]))),
148 }
149
150 # problem 4: shrinkage geometry
151 r_end_cm = float(q4["spec"]["radius"]["r_end_cm"])
152 r0_cm = float(q4["spec"]["radius"]["r_start_cm"])
153 area_ratio = (r_end_cm / r0_cm) ** 2
154 fixed_ratio = float(q4["fixed_radius"]["t_dry_h"]) / float(q4["main"]["t_dry_h"])
155
156 # centre moisture on the last 60 s output row of result3/result4: exactly below C_TARGET but displayed as 0.1500
157 centre3 = value_at(q3h["t"], q3h["centre_moist"], float(q3["t_dry_grid_s"]))
158 centre4 = value_at(q4h["t"], q4h["centre_moist"], float(q4["main"]["t_dry_grid_s"]))
159
160 n = int(q1["n"])

```

```

161 report = {
162     "aspect_ratio": LENGTH / R0,
163     "system_size": 2 * (n + 1) + 2,
164     "q1": {
165         "alpha": alpha1,
166         "D0": d1,
167         "t_heat_s": t_heat,
168         "t_mass_h": t_mass / HOUR,
169         "stiffness_ratio": alpha1 / d1,
170         "Fo_heat_1800s": fo_heat,
171         "Fo_mass_1800s": fo_mass,
172         "diffusion_length_cm": diffusion_length * 100.0,
173     },
174     "q2": {"centre_within_1K_h": None if gap_1k is None else gap_1k / HOUR, "centre_within_0p5K_h": gap_half},
175     "q3": {
176         "alpha3_C0": alpha3,
177         "D_max": d_max,
178         "D_at_target": d_target,
179         "D_surface_at_dry": d_surface_dry,
180         "D_ratio_max_to_surface": d_max / d_surface_dry,
181         "Bi_mass_at_dry": bi_mass_dry,
182         "Fo_axial_heat": fo_axial_heat,
183         "Fo_axial_mass_upper": fo_axial_mass,
184         "richardson_limit_h": limit3,
185         "production_error_min": err3 * 60.0,
186         "centre_moist_grid_row": centre3,
187         "stats": q3["stats"],
188     },
189     "q4": {
190         "richardson_limit_h": limit4,
191         "production_error_min": err4 * 60.0,
192         "r_end_cm": r_end_cm,
193         "area_ratio": area_ratio,
194         "inverse_radius_ratio_squared": 1.0 / area_ratio,
195         "fixed_to_moving_ratio": fixed_ratio,
196         "D4_max": d4_max,
197         "D4_at_target": d4_target,
198         "D_ratio_three_to_four_init": d_max / d4_max,
199         "centre_moist_grid_row": centre4,
200         "stats": q4["stats"],
201     },
202 }
203 report["chamber_noise"] = noise
204 report["q4"]["dry_matter_ratio"] = dry_matter_ratio
205 report["q4"]["implied_r_end_cm"] = implied_r_end_cm
206 report["q4"]["rho_s_wet"] = rho_s_wet
207 report["q4"]["rho_s_dry"] = rho_s_dry
208 report["q4"]["shrink_scaling_prediction"] = shrink_scaling
209 report["q3"]["richardson_order"] = order3
210 report["q4"]["richardson_order"] = order4
211 ctx.write_json("derived.json", report)
212
213 ctx.number("AspectRatio", LENGTH / R0, ".1f")
214 ctx.number("SystemSize", 2 * (n + 1) + 2)
215 ctx.number("QoneHeatTimeScaleS", t_heat, ".0f")
216 ctx.number("QoneMassTimeScaleH", t_mass / HOUR, ".1f")
217 ctx.number("QoneStiffnessRatio", alpha1 / d1, ".0f")
218 ctx.number("QoneFourierHeat", fo_heat, ".2f")
219 ctx.number("QoneFourierMass", fo_mass, ".3f")
220 ctx.number("QoneDiffusionLengthCm", diffusion_length * 100.0, ".2f")
221 if gap_1k is not None:
222     ctx.number("QtwoCentreWithinOneKelvinHours", gap_1k / HOUR, ".2f")
223 if gap_half is not None:
224     ctx.number("QtwoCentreWithinHalfKelvinHours", gap_half / HOUR, ".2f")
225 ctx.number("FoAxialHeat", fo_axial_heat, ".2f")
226 ctx.number("FoAxialMassUpper", fo_axial_mass, ".2f")
227 ctx.number("DmaxAppendixThree", d_max, ".2e")
228 ctx.number("DtargetAppendixThree", d_target, ".1e")
229 ctx.number("DsurfaceAtDryQthree", d_surface_dry, ".1e")
230 ctx.number("DratioQthree", d_max / d_surface_dry, ".0f")
231 ctx.number("BiotMassAtDryQthree", bi_mass_dry, ".0f")

```

```

232 ctx.number("DmaxAppendixFour", d4_max, ".2e")
233 ctx.number("DtargetAppendixFour", d4_target, ".1e")
234 ctx.number("DratioThreeToFourInit", d_max / d4_max, ".1f")
235 for prob, tag in (("q1", "Qone"), ("q2", "Qtwo"), ("q3", "Qthree"), ("q4", "Qfour")):
236     sampled = ctx.dep_data(prob, f"verification_{prob}.json")["moisture_balance"].get("sampled")
237     if sampled is not None:
238         ctx.number(f"SampledResidual{tag}", sampled["max_relative_residual"], ".1e")
239 last3 = float(np.asarray(q3h["t"], dtype=float)[-1])
240 last4 = float(np.asarray(q4h["t"], dtype=float)[-1])
241 ctx.number("QthreeLastRowHours", last3 / HOUR, ".2f")
242 ctx.number("QfourLastRowHours", last4 / HOUR, ".2f")
243 ctx.number("QthreeLastRowExtraMin", (last3 - t3) / 60.0, ".1f")
244 ctx.number("QfourLastRowExtraMin", (last4 - float(q4["main"]["t_dry_s"])) / 60.0, ".1f")
245 ctx.number("QfourRadiusHalfHourCm", value_at(q4h["t"], q4h["radius_cm"], 1800.0), ".3f")
246 q1m = pd.read_parquet(ctx.dep("q1") / "profiles_moist.parquet")
247 one_cm_end = float(q1m["1.0"].to_numpy(float)[-1])
248 ctx.number("QoneMoistOneCmEnd", one_cm_end, ".2f")
249 ctx.number("QoneMoistOneCmDropPct", 100.0 * (1.0 - one_cm_end / C_INIT), ".1f")
250 ctx.number("QtwoRhoSWet", float(dry_solid_density(APPENDIX3, np.array(C_INIT))), ".0f")
251 ctx.number("QtwoRhoSDry", float(dry_solid_density(APPENDIX3, np.array(C_TARGET))), ".0f")
252 ctx.number("QthreeDryOrderUsed", order3, ".2f")
253 ctx.number("QfourDryOrderUsed", order4, ".2f")
254 ctx.number("QthreeDryHoursExtrapolated", limit3, ".4f")
255 ctx.number("QthreeDryHoursExtrapolatedTwo", limit3, ".2f")
256 ctx.number("QfourDryHoursExtrapolatedTwo", limit4, ".2f")
257 ctx.number(
258     "QtwoBiotMassInit", APPENDIX3.hm * R0 / float(APPENDIX3.diffusivity(np.array(C_INIT), np.array(28.0))), ".2f"
259 )
260 ctx.number("QfourRhoSWet", rho_s_wet, ".0f")
261 ctx.number("QfourRhoSDry", rho_s_dry, ".0f")
262 ctx.number("QfourDryMatterRatio", dry_matter_ratio, ".3f")
263 ctx.number("QfourDryMatterGapPct", 100.0 * (1.0 - dry_matter_ratio), ".1f")
264 ctx.number("QfourImpliedRadiusCm", implied_r_end_cm, ".3f")
265 ctx.number("QfourImpliedRadiusGapPct", 100.0 * (implied_r_end_cm / r4_end_cm - 1.0), ".1f")
266 ctx.number("ShrinkDeltaCm", shrink_delta_cm, ".3f")
267 ctx.number("ShrinkRadiusLowCm", 100.0 * R0 - 0.8 * shrink_delta_cm, ".3f")
268 ctx.number("ShrinkRadiusHighCm", 100.0 * R0 - 1.2 * shrink_delta_cm, ".3f")
269 ctx.number("ShrinkScalingPrediction", shrink_scaling, ".2f")
270 ctx.number("ChamberPlateauTempMin", noise["temp_min"], ".2f")
271 ctx.number("ChamberPlateauTempMax", noise["temp_max"], ".2f")
272 ctx.number("ChamberPlateauStepMax", noise["temp_step_max"], ".2f")
273 ctx.number("ChamberPlateauStd", noise["temp_std"], ".2f")
274 ctx.number("ChamberStepMaxAll", noise["temp_step_max_all"], ".2f")
275 ctx.number("ChamberPlateauJump", noise["plateau_jump_K"], ".2f")
276 ctx.number("QthreeGridErrorMin", err3 * 60.0, ".2f")
277 ctx.number("QthreeCentreMoistGridRow", centre3, ".6f")
278 ctx.number("QfourDryHoursExtrapolated", limit4, ".4f")
279 ctx.number("QfourGridErrorMin", err4 * 60.0, ".2f")
280 ctx.number("QfourCentreMoistGridRow", centre4, ".6f")
281 ctx.number("QthreeNfev", int(q3["stats"]["nfev"]))
282 ctx.number("QthreeNlu", int(q3["stats"]["nlu"]))
283 ctx.number("QthreeSegments", int(q3["stats"]["segments"]))
284 ctx.number("QfourNfev", int(q4["stats"]["nfev"]))
285 ctx.number("QfourRadiusFinalCm", r_end_cm, ".3f")
286 ctx.number("QfourAreaRatioPct", 100.0 * area_ratio, ".1f")
287 ctx.number("QfourRadiusRatioSquared", 1.0 / area_ratio, ".2f")
288 ctx.number("QfourFixedToMovingRatio", fixed_ratio, ".2f")
289 ctx.number("ChamberSamples", int(q3["spec"]["chamber"]["samples"]))
290 ctx.number("ChamberEndHours", float(q3["spec"]["chamber"]["t_end"]) / HOUR, ".0f")
291 ctx.number("RadiusSamples", int(q4["spec"]["radius"]["samples"]))
292 ctx.number("RadiusEndHours", float(q4["spec"]["radius"]["t_end"]) / HOUR, ".0f")
293 return {"numbers": 41, "fo_axial_heat": fo_axial_heat, "fo_axial_mass_upper": fo_axial_mass}

```

pipelines/a/extension.py

```
1 """Model-evaluation extension (MDR-0008): the physics the problem's stylised model leaves out, quantified.
2
3 * a-posteriori latent-heat diagnostic of the problem-3 solution: the latent-heat demand of the model's own
4   evaporation rate, expressed as the equivalent surface cooling  $L_v j_w / h$ , against the convective supply;
5 * psychrometric reading of the chamber state (relative humidity, wet-bulb temperature);
6 * energy-limited variant: the evaporation flux capped by the wet-bulb limit  $h (T_{\text{air}} - T_{\text{wb}}) / L_v$  (the
7   constant-rate period of classical drying theory) and the pure energy lower bound on the drying time;
8 * equilibrium-moisture driving force:  $C_{\text{air}}$  replaced by a constant  $C_{\text{eq}}$  in the mass boundary condition.
9
10 The main results of problems 1-4 are untouched; everything here feeds the paper's model-evaluation section.
11 """
12
13 from __future__ import annotations
14
15 from dataclasses import replace
16 from typing import Any
17
18 import numpy as np
19
20 from forge.context import StageContext
21 from forge.runner import stage
22 from pipelines.a.common import build_spec, production_n, spec_to_recipe, time_options
23 from pipelines.a.physics import (
24     C_INIT,
25     C_TARGET,
26     R0,
27     FluxCap,
28     dry_solid_density,
29     latent_heat,
30     relative_humidity,
31     wet_bulb_temperature,
32 )
33 from pipelines.a.science import HOUR, _parallel
34 from pipelines.a.solver import ProblemSpec
35
36 ORDINALS = ["One", "Two", "Three", "Four", "Five", "Six"]
37
38
39 def slug(label: str) -> str:
40     """File-system and npz friendly variant key: 'cap:1.0' -> 'cap_1p0'."""
41     return label.replace(":", "_").replace(".", "p")
42
43
44 def _first_time_below(t: np.ndarray, values: np.ndarray, threshold: float) -> float | None:
45     idx = np.where(values < threshold)[0]
46     return None if len(idx) == 0 else float(t[idx[0]])
47
48
49 def latent_diagnostic(hist: dict[str, Any], spec: ProblemSpec, rho_s: float) -> dict[str, Any]:
50     """Latent-heat demand implied by the model's own evaporation rate versus its convective heat supply.
51
52      $j_w = \rho_s h_m (C_s - C_{\text{air}})$  is the evaporation mass flux behind the moisture boundary condition; the energy
53     equation of the problem ignores its latent heat  $L_v j_w$ . The equivalent surface cooling  $L_v j_w / h$  is the
54     temperature depression a coupled model would need to carry that load by convection alone.
55     """
56     t = np.asarray(hist["t"], dtype=float)
57     ts = np.asarray(hist["surface_temp"], dtype=float)
58     ta = np.asarray(hist["air_temp"], dtype=float)
59     cs = np.asarray(hist["surface_moist"], dtype=float)
60     ca = np.asarray(hist["air_hum"], dtype=float)
61     mean_c = np.asarray(hist["mean_moist"], dtype=float)
62     flux = np.asarray(spec.surface_moisture_loss(t, cs, ca), dtype=float) # (kg/kg) m/s
63     j_w = rho_s * flux # kg/(m^2 s)
64     q_lat = latent_heat(ts) * j_w # W/m^2
65     q_conv = spec.props.h * (ta - ts) # W/m^2 delivered by the model's thermal boundary condition
66     dt_lat = q_lat / spec.props.h # K
67     circ = 2.0 * np.pi * R0
68     e_lat = np.concatenate([[0.0], np.cumsum(0.5 * (q_lat[1:] + q_lat[:-1]) * np.diff(t))]) * circ # J/m
```

```

69 e_conv = np.concatenate([[0.0], np.cumsum(0.5 * (q_conv[1:] + q_conv[:-1]) * np.diff(t))] * circ
70 water_removed = rho_s * np.pi * R0**2 * (mean_c[0] - mean_c) # kg per m length
71 at: dict[str, dict[str, float]] = {}
72 for label, hours in (("three_h", 3.0), ("twelve_h", 12.0), ("twenty_four_h", 24.0)):
73     i = int(np.argmax(np.abs(t - hours * HOUR)))
74     at[label] = {
75         "t_h": float(t[i] / HOUR),
76         "dt_lat_K": float(dt_lat[i]),
77         "q_lat_W_m2": float(q_lat[i]),
78         "q_conv_W_m2": float(q_conv[i]),
79         "e_lat_MJ_m": float(e_lat[i] / 1e6),
80         "e_conv_kJ_m": float(e_conv[i] / 1e3),
81     }
82 below_1k = _first_time_below(t, dt_lat, 1.0)
83 below_01k = _first_time_below(t, dt_lat, 0.1)
84 return {
85     "arrays": {
86         "t": t,
87         "dt_lat": dt_lat,
88         "model_dt": ta - ts,
89         "q_lat": q_lat,
90         "q_conv": q_conv,
91         "e_lat": e_lat,
92         "e_conv": e_conv,
93         "water_removed": water_removed,
94     },
95     "summary": {
96         "rho_s0": rho_s,
97         "j_w_initial": float(j_w[0]),
98         "dt_lat_peak_K": float(dt_lat.max()),
99         "dt_lat_peak_t_h": float(t[int(np.argmax(dt_lat))] / HOUR),
100         "dt_lat_below_1K_h": None if below_1k is None else below_1k / HOUR,
101         "dt_lat_below_0p1K_h": None if below_01k is None else below_01k / HOUR,
102         "at": at,
103         "e_lat_total_MJ_per_m": float(e_lat[-1] / 1e6),
104         "e_conv_total_kJ_per_m": float(e_conv[-1] / 1e3),
105         "latent_to_convective_ratio": float(e_lat[-1] / max(e_conv[-1], 1e-300)),
106         "water_removed_kg_per_m": float(water_removed[-1]),
107     },
108 }
109
110
111 def chamber_psychrometrics(spec: ProblemSpec, cap: FluxCap) -> dict[str, Any]:
112     ch = spec.chamber
113     t_plateau = ch.temp_plateau * ch.temp_scale
114     w_plateau = ch.hum_plateau * ch.hum_scale
115     wb_plateau = wet_bulb_temperature(t_plateau, w_plateau)
116     t0, w0 = float(ch.temp[0]), float(ch.hum_scale * ch.hum[0])
117     wb0 = wet_bulb_temperature(t0, w0)
118     return {
119         "assumption": "chamber moisture concentration read as the humidity ratio of the air (kg water per kg dry air)"
120         " at 101325 Pa; Magnus saturation pressure; ASHRAE adiabatic-saturation wet bulb",
121         "initial": {
122             "temp": t0,
123             "w": w0,
124             "rh": float(relative_humidity(t0, w0)),
125             "wet_bulb": wb0,
126             "wet_bulb_depression": t0 - wb0,
127             "flux_cap": float(cap.cap[0]),
128         },
129         "plateau": {
130             "temp": t_plateau,
131             "w": w_plateau,
132             "rh": float(relative_humidity(t_plateau, w_plateau)),
133             "wet_bulb": wb_plateau,
134             "wet_bulb_depression": t_plateau - wb_plateau,
135             "flux_cap": cap.plateau,
136             "latent_heat": float(latent_heat(wb_plateau)),
137         },
138     }
139

```

```

140
141 @stage(
142     "extension",
143     deps=("ingest", "q3"),
144     description="Model-evaluation extension: latent-heat consistency, wet-bulb energy limit, C_eq driving force",
145 )
146 def extension(ctx: StageContext) -> dict[str, Any]:
147     n = production_n(ctx)
148     opts = time_options(ctx)
149     horizon = float(ctx.cfg("a.drying.horizon_s", 432000))
150     cap_factors = [float(f) for f in ctx.cfg("a.extension.cap_factors", [0.5, 1.0, 2.0])]
151     c_eq_values = [float(c) for c in ctx.cfg("a.extension.c_eq", [0.075, 0.10, 0.125])]
152     base = build_spec(ctx, "q3")
153     chamber, props = base.chamber, base.props
154     rho_s0 = float(dry_solid_density(props, C_INIT))
155     q3 = ctx.dep_data("q3", "summary.json")
156     t3 = float(q3["t_dry_s"])
157     hist = ctx.dep_data("q3", "history.json")
158
159     # 1. a-posteriori latent-heat diagnostic of the problem-3 solution
160     diag = latent_diagnostic(hist, base, rho_s0)
161     np.savez_compressed(ctx.out("diagnostic.npz"), **diag["arrays"])
162     ctx.log.info("extension.diagnostic", **{k: v for k, v in diag["summary"].items() if not isinstance(v, dict)})
163
164     # 2. psychrometrics of the chamber and the wet-bulb energy limit on the evaporation flux
165     cap = FluxCap.wet_bulb(chamber, props.h, rho_s0)
166     psychro = chamber_psychrometrics(base, cap)
167     mean_end = float(q3["mean_moist_at_dry"])
168     t_bound = rho_s0 * R0 * (C_INIT - mean_end) / (2.0 * cap.plateau) # lateral surface only, plateau limit
169     ctx.log.info("extension.psychrometrics", **psychro["plateau"], energy_bound_h=t_bound / HOUR)
170
171     # 3. variants: energy-capped evaporation (x factors) and equilibrium-moisture driving force
172     jobs: list[dict[str, Any]] = []
173     for f in cap_factors:
174         spec = replace(base, flux_cap=replace(cap, scale=f))
175         jobs.append(
176             {
177                 "label": f"cap:{f}",
178                 "recipe": spec_to_recipe(spec),
179                 "n": n,
180                 "opts": opts,
181                 "mode": "dry",
182                 "horizon": horizon,
183                 "checks": True,
184             }
185         )
186     local_spec = replace(base, flux_cap=replace(cap, scale=1.0, local_rho_s=True))
187     jobs.append(
188         {
189             "label": "caplocal:1.0",
190             "recipe": spec_to_recipe(local_spec),
191             "n": n,
192             "opts": opts,
193             "mode": "dry",
194             "horizon": horizon,
195             "checks": True,
196         }
197     )
198     for c in c_eq_values:
199         spec = replace(base, chamber=replace(chamber, hum=np.full_like(chamber.hum, c), hum_plateau=c, hum_scale=1.0))
200         jobs.append(
201             {
202                 "label": f"ceq:{c}",
203                 "recipe": spec_to_recipe(spec),
204                 "n": n,
205                 "opts": opts,
206                 "mode": "dry",
207                 "horizon": horizon,
208                 "checks": True,
209             }
210         )

```

```

211 results = {r["label"]: r for r in _parallel(ctx, jobs)}
212
213 variants: list[dict[str, Any]] = []
214 arrays: dict[str, np.ndarray] = {}
215 checks: dict[str, Any] = {}
216 for label, res in results.items():
217     kind, raw = label.split(":")
218     value = float(raw)
219     t_dry = float(res["t_dry"])
220     row: dict[str, Any] = {
221         "case": kind,
222         "value": value,
223         "label": label,
224         "slug": slug(label),
225         "t_dry_h": t_dry / HOUR,
226         "t_dry_grid_h": float(res["t_dry_grid"]) / HOUR,
227         "diff_pct": 100.0 * (t_dry / t3 - 1.0),
228         "checks_passed": bool(res["checks"]["passed"]),
229         "moisture_residual": float(res["checks"]["moisture_balance"]["max_relative_residual"]),
230         "nfev": int(res["stats"]["nfev"]),
231     }
232     tt = np.asarray(res["mean_moist_t"], dtype=float)
233     if kind in ("cap", "caplocal"):
234         scaled = replace(cap, scale=value)
235         cs = np.asarray(res["surface_moist"], dtype=float)
236         ca = np.asarray(res["air_hum"], dtype=float)
237         rho_s = np.asarray(dry_solid_density(props, cs), dtype=float) if kind == "caplocal" else rho_s0
238         active = props.hm * (cs - ca) > np.asarray(scaled.value(tt), dtype=float) / rho_s * (1.0 + 1e-9)
239         row["cap_release_h"] = float(tt[np.where(active)[0][-1]] / HOUR) if active.any() else 0.0
240         row["cap_plateau_kg_m2_s"] = cap.plateau * value
241         row["cap_plateau_kg_m2_h"] = cap.plateau * value * HOUR
242     checks[label] = res["checks"]
243     variants.append(row)
244     for key in ("mean_moist", "centre_moist", "surface_moist"):
245         arrays[f"{slug(label)}_{key}"] = np.asarray(res[key], dtype=float)
246     arrays[f"{slug(label)}_t"] = tt
247 np.savez_compressed(ctx.out("variants.npz"), **arrays)
248 checks["passed"] = bool(all(v["checks_passed"] for v in variants))
249 ctx.write_json("verification_extension.json", checks)
250 report = {
251     "base_t_dry_h": t3 / HOUR,
252     "rho_s0": rho_s0,
253     "psychrometrics": psychro,
254     "diagnostic": diag["summary"],
255     "energy_bound_h": t_bound / HOUR,
256     "cap_factors": cap_factors,
257     "c_eq": c_eq_values,
258     "variants": variants,
259     "n": n,
260 }
261 ctx.write_json("extension.json", report)
262
263 # 4. paper numbers
264 d = diag["summary"]
265 ctx.number("ExtDrySolidDensity", rho_s0, ".0f")
266 ctx.number("ExtRhInitPct", 100.0 * psychro["initial"]["rh"], ".0f")
267 ctx.number("ExtRhPlateauPct", 100.0 * psychro["plateau"]["rh"], ".0f")
268 ctx.number("ExtWetBulbInit", psychro["initial"]["wet_bulb"], ".1f")
269 ctx.number("ExtWetBulbPlateau", psychro["plateau"]["wet_bulb"], ".1f")
270 ctx.number("ExtWetBulbDepression", psychro["plateau"]["wet_bulb_depression"], ".1f")
271 ctx.number("ExtLatentHeatMJ", psychro["plateau"]["latent_heat"] / 1e6, ".3f")
272 ctx.number("ExtFluxCapPlateau", cap.plateau, ".2e")
273 ctx.number("ExtFluxCapPlateauKgPerHour", cap.plateau * HOUR, ".3f")
274 ctx.number("ExtFluxCapInit", float(cap.cap[0]), ".2e")
275 ctx.number("ExtFluxCapInitKgPerHour", float(cap.cap[0]) * HOUR, ".3f")
276 ctx.number("ExtModelFluxInitKgPerHour", d["j_w_initial"] * HOUR, ".2f")
277 ctx.number("ExtFluxRatioPlateau", d["j_w_initial"] / cap.plateau, ".1f")
278 ctx.number("ExtDrySolidDensityTarget", float(dry_solid_density(props, np.array(C_TARGET))), ".0f")
279 ctx.number("ExtModelFluxInit", d["j_w_initial"], ".2e")
280 ctx.number("ExtFluxRatioInit", d["j_w_initial"] / float(cap.cap[0]), ".0f")
281 ctx.number("ExtLatentDeltaTPeak", d["dt_lat_peak_K"], ".0f")

```

```

282     ctx.number("ExtLatentDeltaTThreeH", d["at"]["three_h"]["dt_lat_K"], ".1f")
283     ctx.number("ExtLatentDeltaTTwelveH", d["at"]["twelve_h"]["dt_lat_K"], ".1f")
284     ctx.number("ExtLatentDeltaTTwentyFourH", d["at"]["twenty_four_h"]["dt_lat_K"], ".2f")
285     if d["dt_lat_below_1K_h"] is not None:
286         ctx.number("ExtLatentBelowOneKHours", d["dt_lat_below_1K_h"], ".1f")
287     if d["dt_lat_below_Op1K_h"] is not None:
288         ctx.number("ExtLatentBelowTenthKHours", d["dt_lat_below_Op1K_h"], ".1f")
289     ctx.number("ExtLatentEnergyMJ", d["e_lat_total_MJ_per_m"], ".2f")
290     ctx.number("ExtConvEnergyKJ", d["e_conv_total_kJ_per_m"], ".1f")
291     ctx.number("ExtLatentToConvRatio", d["latent_to_convective_ratio"], ".0f")
292     ctx.number("ExtWaterRemovedKg", d["water_removed_kg_per_m"], ".3f")
293     ctx.number("ExtEnergyBoundHours", t_bound / HOUR, ".1f")
294     ctx.number("ExtEnergyBoundPct", 100.0 * (t_bound / t3 - 1.0), ".1f")
295     for row in [v for v in variants if v["case"] == "caplocal"]:
296         ctx.number("ExtCapLocalHours", row["t_dry_h"], ".2f")
297         ctx.number("ExtCapLocalPct", row["diff_pct"], ".1f")
298         ctx.number("ExtCapLocalReleaseHours", row["cap_release_h"], ".1f")
299     cap_rows = [v for v in variants if v["case"] == "cap"]
300     ceq_rows = [v for v in variants if v["case"] == "ceq"]
301     for i, row in enumerate(cap_rows):
302         tag = ORDINALS[i]
303         ctx.number(f"ExtCap{tag}Factor", row["value"], ".1f")
304         ctx.number(f"ExtCap{tag}Hours", row["t_dry_h"], ".2f")
305         ctx.number(f"ExtCap{tag}Pct", row["diff_pct"], ".1f")
306         ctx.number(f"ExtCap{tag}ReleaseHours", row["cap_release_h"], ".1f")
307         if row["value"] == 1.0:
308             ctx.number("ExtCappedHours", row["t_dry_h"], ".2f")
309             ctx.number("ExtCappedPct", row["diff_pct"], ".1f")
310             ctx.number("ExtCappedReleaseHours", row["cap_release_h"], ".1f")
311     for i, row in enumerate(ceq_rows):
312         tag = ORDINALS[i]
313         ctx.number(f"ExtCeq{tag}Value", row["value"], ".3f")
314         ctx.number(f"ExtCeq{tag}Hours", row["t_dry_h"], ".2f")
315         ctx.number(f"ExtCeq{tag}Pct", row["diff_pct"], ".1f")
316     return {
317         "jobs": len(jobs),
318         "checks_passed": checks["passed"],
319         "energy_bound_h": t_bound / HOUR,
320         "variants": {v["label"]: v["t_dry_h"] for v in variants},
321     }

```

pipelines/a/figures.py

```

1  """Paper figures (vector PDF + PNG preview) built only from stage outputs.
2
3  Style: house palette (Okabe-Ito, fixed order for categorical series), single-hue sequential ramps for
4  time-ordered profiles (light = early, dark = late), one measure per axis, thin marks, recessive grid.
5  """
6
7  from __future__ import annotations
8
9  import json
10 from pathlib import Path
11 from typing import Any
12
13 import numpy as np
14 import pandas as pd
15
16 from forge import plotting
17 from forge.context import StageContext
18 from forge.runner import stage
19 from pipelines.a.physics import APPENDIX2, APPENDIX3, APPENDIX4, C_TARGET, Radius
20
21 HOUR = 3600.0
22 INK = "#1F2933"
23 MUTED = "#6B7280"
24 CAT = plotting.OKABE_ITO
25

```

```

26
27 def _ramp(name: str, count: int) -> list[Any]:
28     from matplotlib import colormaps
29
30     cmap = colormaps[name]
31     return [cmap(x) for x in np.linspace(0.35, 0.95, count)]
32
33
34 def _load(path: Path) -> Any:
35     return json.loads(path.read_text(encoding="utf-8"))
36
37
38 def _finish(ctx: StageContext, fig: Any, name: str, index: list[dict[str, str]], purpose: str) -> None:
39     info = plotting.save(fig, ctx.out("figures", f"{name}.pdf"))
40     index.append({"file": f"{name}.pdf", "purpose": purpose, "bytes": str(info["bytes"])})
41     ctx.log.info("figure", name=name, bytes=info["bytes"])
42
43
44 def _profiles(ax: Any, snaps: Any, field: str, unit: str, cmap: str, label_fmt: Any) -> None:
45     t = snaps["t"]
46     colors = _ramp(cmap, len(t))
47     for i, ti in enumerate(t):
48         r_cm = snaps["xi"] * snaps["radius"][i] * 100.0
49         ax.plot(r_cm, snaps[field][i], color=colors[i], label=label_fmt(ti), lw=1.4)
50     ax.set_xlabel("到药材中心的距离 $r$ / cm")
51     ax.set_ylabel(unit)
52     ax.legend(frameon=False, ncol=2, title="时间", title_fontsize=8)
53
54
55 def _history(ax: Any, hist: dict[str, Any], keys: list[tuple[str, str]], unit: str, scale: float = 1.0) -> None:
56     t = np.asarray(hist["t"]) / scale
57     for j, (key, label) in enumerate(keys):
58         style = "---" if key.startswith("air") else "-"
59         ax.plot(t, hist[key], style, color=CAT[j], label=label, lw=1.4)
60     ax.set_ylabel(unit)
61     ax.legend(frameon=False)
62
63
64 @stage(
65     "figures",
66     deps=("ingest", "q1", "q2", "q3", "q4", "convergence", "verification", "sensitivity", "extension"),
67     description="Paper figures: data, properties, profiles/histories of q1-q4, convergence, verification, sensitivity",
68 )
69 def figures(ctx: StageContext) -> dict[str, Any]:
70     plotting.setup()
71     index: list[dict[str, str]] = []
72     data = ctx.dep("ingest") / "data"
73     a1 = pd.read_parquet(data / "附件1__Sheet1.parquet")
74     a2 = pd.read_parquet(data / "附件2__Sheet1.parquet")
75
76     # 1. chamber conditions and radius data -----
77     q3sum = _load(ctx.dep("q3") / "summary.json")
78     fig, axes = plotting.new_figure(6.3, 5.4, nrows=3, sharex=False)
79     t1 = a1["时间"].to_numpy(float) / HOUR
80     axes[0].plot(t1, a1["温度"], color=CAT[1], lw=1.2, label="附件 1 样本")
81     axes[0].axhline(
82         q3sum["spec"]["chamber"]["temp_plateau"], color=MUTED, ls=":", lw=1, label="恒温阶段平台 (末 1 h 均值)"
83     )
84     axes[0].set_ylabel("烘房温度 / °C")
85     axes[0].legend(frameon=False, loc="lower right")
86     axes[1].plot(t1, a1["水分浓度"], color=CAT[0], lw=1.2, label="附件 1 样本")
87     axes[1].axhline(q3sum["spec"]["chamber"]["hum_plateau"], color=MUTED, ls=":", lw=1, label="平台 (末 1 h 均值)")
88     axes[1].set_ylabel("烘房水分浓度 / (kg/kg)")
89     axes[1].set_xlabel("时间 / h")
90     axes[1].legend(frameon=False, loc="lower right")
91     t2 = a2["时间"].to_numpy(float) / HOUR
92     hist4 = _load(ctx.dep("q4") / "history.json")
93     radius = Radius.from_series(a2["时间"].to_numpy(float), a2["半径"].to_numpy(float))
94     t_fine = np.linspace(0.0, float(a2["时间"].max()), 2000)
95     axes[2].plot(t_fine / HOUR, np.asarray(radius.value(t_fine)) * 100.0, color=CAT[2], lw=1.2, label="PCHIP 插值 R(t)"
96     ↪ )

```

```

96 axes[2].plot(
97     t2[:,4], a2["半径"].to_numpy(float)[::4], "o", ms=3, color=INK, mfc="white", label="附件 2 样本 (每 2 h) "
98 )
99 axes[2].set_ylabel("半径 / cm")
100 axes[2].set_xlabel("时间 / h")
101 axes[2].legend(frameon=False)
102 for ax, tag in zip(axes, "abc"):
103     ax.set_title(f"({tag})", loc="left", fontsize=9)
104 _finish(ctx, fig, "fig_chamber", index, "附件 1 烘房温湿度 (a,b) 与附件 2 半径及其 PCHIP 插值 (c) ; 边界条件数据")
105
106 # 2. property laws -----
107 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
108 c = np.linspace(0.02, 2.6, 400)
109 axes[0].plot(c, APPENDIX2.diffusivity(c, np.full_like(c, 28.0)), color=CAT[0], label="附录 2 (不含温度项) ")
110 for j, (props, name) in enumerate(((APPENDIX3, "附录 3"), (APPENDIX4, "附录 4"))):
111     for temp, ls in ((28.0, "--"), (50.0, "-")):
112         axes[0].plot(
113             c, props.diffusivity(c, np.full_like(c, temp)), ls, color=CAT[j + 1], label=f"{name}, {temp:g} °C"
114         )
115 axes[0].set_yscale("log")
116 axes[0].set_xlabel("水分浓度  $C$  / (kg/kg)")
117 axes[0].set_ylabel("扩散系数  $D$  / ( $m^2/s$ )")
118 axes[0].set_ylim(1e-13, 3e-8) # the range actually visited during drying ( $C \geq 0.05$ )
119 axes[0].axvline(C_TARGET, color=MUTED, ls=":", lw=1)
120 axes[0].annotate(
121     f" $C^*=C\_TARGET$ ", xy=(C_TARGET, 3e-13), xytext=(C_TARGET + 0.12, 3e-13), fontsize=7, color=MUTED
122 )
123 axes[0].legend(frameon=False, fontsize=7, loc="lower right")
124 axes[1].axhline(APPENDIX2.k0 / (APPENDIX2.rho0 * APPENDIX2.cp0) * 1e7, color=CAT[0], label="附录 2")
125 for j, (props, name) in enumerate(((APPENDIX3, "附录 3"), (APPENDIX4, "附录 4"))):
126     axes[1].plot(c, props.k(c) / (props.rho(c) * props.cp(c)) * 1e7, color=CAT[j + 1], label=name)
127 axes[1].set_xlabel("水分浓度  $C$  / (kg/kg)")
128 axes[1].set_ylabel(r"热扩散率  $\alpha$  / ( $10^{-7}$ )  $m^2/s$ ")
129 axes[1].legend(frameon=False, fontsize=7)
130 for ax, tag in zip(axes, "ab"):
131     ax.set_title(f"({tag})", loc="left", fontsize=9)
132 _finish(
133     ctx,
134     fig,
135     "fig_properties",
136     index,
137     "三组附录物性: 水分扩散系数随  $C$ 、 $T$  的变化 (a, 对数坐标) 与热扩散率 (b) ; 说明干燥后期表面干层的刚性",
138 )
139
140 # 3-4. problem 1 -----
141 snaps = np.load(ctx.dep("q1") / "snapshots.npz")
142 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
143 _profiles(axes[0], snaps, "temp", "温度 / °C", "Oranges", lambda t: f"{t:.0f} s")
144 _profiles(axes[1], snaps, "moist", "水分浓度 / (kg/kg)", "Blues", lambda t: f"{t:.0f} s")
145 for ax, tag in zip(axes, "ab"):
146     ax.set_title(f"({tag})", loc="left", fontsize=9)
147 _finish(ctx, fig, "fig_q1_profiles", index, "问题 1: 表 1/2 各时刻的温度 (a) 与水分浓度 (b) 径向剖面")
148 hist = _load(ctx.dep("q1") / "history.json")
149 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
150 _history(
151     axes[0], hist, [("centre_temp", "中心  $r=0$ "), ("surface_temp", "表面  $r=2$  cm"), ("air_temp", "烘房")], "温度 / °C"
152 )
153 _history(
154     axes[1],
155     hist,
156     [("centre_moist", "中心  $r=0$ "), ("surface_moist", "表面  $r=2$  cm"), ("mean_moist", "体积平均")],
157     "水分浓度 / (kg/kg)",
158 )
159 for ax, tag in zip(axes, "ab"):
160     ax.set_xlabel("时间 / s")
161     ax.set_title(f"({tag})", loc="left", fontsize=9)
162 _finish(
163     ctx,
164     fig,
165     "fig_q1_history",
166     index,

```

```

167     "问题 1: 中心、表面与烘房温度 (a) 及中心、表面、平均水分浓度 (b) 随时间的变化",
168 )
169
170 # 5-6. problem 2 -----
171 snaps = np.load(ctx.dep("q2") / "snapshots.npz")
172 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
173 _profiles(axes[0], snaps, "temp", "温度 / °C", "Oranges", lambda t: f"{t / HOUR:.1f} h")
174 _profiles(axes[1], snaps, "moist", "水分浓度 / (kg/kg)", "Blues", lambda t: f"{t / HOUR:.1f} h")
175 for ax, tag in zip(axes, "ab"):
176     ax.set_title(f"({tag})", loc="left", fontsize=9)
177 _finish(ctx, fig, "fig_q2_profiles", index, "问题 2: 表 3/4 各时刻的温度 (a) 与水分浓度 (b) 径向剖面")
178 hist = _load(ctx.dep("q2") / "history.json")
179 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
180 _history(
181     axes[0],
182     hist,
183     [("centre_temp", "中心 r=0"), ("surface_temp", "表面 r=2 cm"), ("air_temp", "烘房")],
184     "温度 / °C",
185     HOUR,
186 )
187 _history(
188     axes[1],
189     hist,
190     [("centre_moist", "中心 r=0"), ("surface_moist", "表面 r=2 cm"), ("mean_moist", "体积平均")],
191     "水分浓度 / (kg/kg)",
192     HOUR,
193 )
194 for ax, tag in zip(axes, "ab"):
195     ax.set_xlabel("时间 / h")
196     ax.set_title(f"({tag})", loc="left", fontsize=9)
197 _finish(
198     ctx, fig, "fig_q2_history", index, "问题 2: 3 h 内温度 (a) 与水分浓度 (b) 的时程; 表面在预热阶段即接近烘房温度"
199 )
200
201 # 7. problem 3 -----
202 snaps = np.load(ctx.dep("q3") / "snapshots.npz")
203 hist = _load(ctx.dep("q3") / "history.json")
204 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
205 _profiles(axes[0], snaps, "moist", "水分浓度 / (kg/kg)", "Blues", lambda t: f"{t / HOUR:.0f} h")
206 axes[0].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
207 axes[0].annotate("0.15", xy=(0.02, C_TARGET), xytext=(0.02, C_TARGET + 0.08), color=MUTED, fontsize=7)
208 _history(
209     axes[1],
210     hist,
211     [("centre_moist", "中心 r=0"), ("surface_moist", "表面"), ("mean_moist", "体积平均")],
212     "水分浓度 / (kg/kg)",
213     HOUR,
214 )
215 axes[1].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
216 axes[1].axvline(q3sum["t_dry_h"], color=MUTED, ls="--", lw=1)
217 axes[1].annotate(
218     f"烘干结束 {q3sum['t_dry_h']:.2f} h",
219     xy=(q3sum["t_dry_h"], 1.2),
220     xytext=(q3sum["t_dry_h"] * 0.55, 1.6),
221     color=INK,
222     fontsize=7,
223     arrowprops={"arrowstyle": "-", "color": MUTED, "lw": 0.8},
224 )
225 axes[1].set_xlabel("时间 / h")
226 for ax, tag in zip(axes, "ab"):
227     ax.set_title(f"({tag})", loc="left", fontsize=9)
228 _finish(
229     ctx,
230     fig,
231     "fig_q3",
232     index,
233     "问题 3: 每 6 h 与烘干结束时刻的水分剖面 (a) 及中心、表面、平均水分浓度时程 (b), 虚线为 0.15 kg/kg 判据",
234 )
235
236 # 8. problem 4 -----
237 snaps = np.load(ctx.dep("q4") / "snapshots.npz")

```

```

238 q4sum = _load(ctx.dep("q4") / "summary.json")
239 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
240 _profiles(axes[0], snaps, "moist", "水分浓度 / (kg/kg)", "Blues", lambda t: f"{t / HOUR:.0f} h")
241 axes[0].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
242 surf_r = snaps["radius"] * 100.0
243 axes[0].plot(surf_r, snaps["moist"][:, -1], "o", ms=3, color=INK, mfc="white", label="药材表面", zorder=5)
244 axes[0].legend(frameon=False, ncol=2, fontsize=7, title="时间", title_fontsize=8)
245 t4 = np.asarray(hist4["t"]) / HOUR
246 axes[1].plot(
247     t4, hist4["centre_moist"], color=CAT[0], label=f"收缩, 拉格朗日 (主): {q4sum['main']['t_dry_h']:.1f} h"
248 )
249 alt_hist = pd.read_parquet(ctx.dep("q4") / "profiles_moist_alt.parquet")
250 axes[1].plot(
251     alt_hist["t"].to_numpy(float) / HOUR,
252     alt_hist["0.0"].to_numpy(float),
253     "--",
254     color=CAT[1],
255     label=f"收缩, 欧拉 (备选): {q4sum['alternative']['t_dry_h']:.1f} h",
256 )
257 fixed_t = q4sum["fixed_radius"]["t_dry_h"]
258 axes[1].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
259 axes[1].axvline(fixed_t, color=CAT[2], ls="-.", lw=1, label=f"半径固定 2 cm: {fixed_t:.1f} h")
260 axes[1].set_xlabel("时间 / h")
261 axes[1].set_ylabel("中心水分浓度 / (kg/kg)")
262 axes[1].legend(frameon=False, fontsize=7)
263 for ax, tag in zip(axes, "ab"):
264     ax.set_title(f"({tag})", loc="left", fontsize=9)
265 _finish(
266     ctx,
267     fig,
268     "fig_q4",
269     index,
270     "问题 4: 物理坐标下每 6 h 的水分剖面与表面位置 (a); 中心水分时程: 主模型、欧拉备选、固定半径 (b)",
271 )
272
273 # 9. convergence and analytic errors -----
274 conv = _load(ctx.dep("convergence") / "convergence.json")
275 ver = _load(ctx.dep("verification") / "verification_report.json")
276 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
277 keys = [
278     ("q1_temp", "问题 1 温度"),
279     ("q1_moist", "问题 1 水分"),
280     ("q2_temp", "问题 2 温度"),
281     ("q2_moist", "问题 2 水分"),
282     ("q3_moist", "问题 3 水分"),
283     ("q4_moist", "问题 4 水分"),
284 ]
285 markers = "osD~v<"
286 for j, (key, label) in enumerate(keys):
287     rows = conv["grid"][key]
288     axes[0].loglog(
289         [r["dr_cm"] for r in rows],
290         [r["max_abs_error"] for r in rows],
291         marker=markers[j],
292         ms=4,
293         color=CAT[j],
294         label=label,
295         lw=1.2,
296     )
297 dr = np.array([0.1 / 2**k for k in range(conv["levels"] - 1)])
298 e0 = conv["grid"]["q1_temp"][0]["max_abs_error"]
299 axes[0].loglog(dr, e0 * (dr / dr[0]) ** 2, color=MUTED, ls=":", lw=1, label="二阶参考")
300 axes[0].set_xlabel(r"$\Delta r$ / cm")
301 axes[0].set_ylabel("相对最细网格的最大误差")
302 axes[0].legend(frameon=False, fontsize=6.5)
303 for j, (field, label) in enumerate(((("temp", "温度 (贝塞尔级数)"), ("moist", "水分 (贝塞尔级数)"))):
304     rows = ver["analytic"][field]["levels"]
305     axes[1].loglog(
306         [r["dr_cm"] for r in rows],
307         [r["max_abs_error"] for r in rows],
308         marker=markers[j],

```

```

309         ms=4,
310         color=CAT[j],
311         label=label,
312         lw=1.2,
313     )
314     drv = np.array([r["dr_cm"] for r in ver["analytic"]["temp"]["levels"]])
315     e0 = ver["analytic"]["temp"]["levels"][0]["max_abs_error"]
316     axes[1].loglog(drv, e0 * (drv / drv[0]) ** 2, color=MUTED, ls=":", lw=1, label="二阶参考")
317     axes[1].set_xlabel(r"$\Delta r$ / cm")
318     axes[1].set_ylabel("与解析解的最大误差")
319     axes[1].legend(frameon=False, fontsize=6.5)
320     for ax, tag in zip(axes, "ab"):
321         ax.set_title(f"({tag})", loc="left", fontsize=9)
322     _finish(
323         ctx,
324         fig,
325         "fig_convergence",
326         index,
327         "网格收敛：各问题相对最细网格的误差（a）与常物性算例相对贝塞尔级数解的误差（b），虚线为二阶斜率",
328     )
329
330 # 10. conservation residual histories -----
331 fig, ax = plotting.new_figure(6.3, 2.6)
332 for j, prob in enumerate(("q1", "q2", "q3", "q4")):
333     h = _load(ctx.dep(prob) / "history.json")
334     t = np.asarray(h["t"])
335     w = np.asarray(h["mean_moist"]) / 2.0
336     rad = np.asarray(h["radius_cm"]) / 100.0
337     cum = np.asarray(h["cum_moist_loss"]) # int hm (C_s - C_air)/R dt accumulated inside the integrator
338     if np.allclose(rad, rad[0]): # d/dt (R^2 W) = -R hm (C_s - C_air)
339         lhs = rad[0] ** 2 * (w - w[0])
340         cum = -(rad[0] ** 2) * cum
341     else: # Lagrangian: dW/dt = -hm (C_s - C_air)/R
342         lhs = w - w[0]
343         cum = -cum
344     res = np.abs(lhs - cum) / max(abs(lhs[-1]), 1e-300)
345     ax.semilogy(t / HOUR, np.maximum(res, 1e-17), color=CAT[j], label=f"问题 {prob[1]}", lw=1.2)
346 ax.set_xlabel("时间 / h")
347 ax.set_ylabel("水分守恒相对残差")
348 ax.legend(frameon=False)
349 _finish(
350     ctx,
351     fig,
352     "fig_conservation",
353     index,
354     "离散水分总量变化与积分器内累积的表面通量之差（相对最终失水量）：四个问题均在机器精度量级",
355 )
356
357 # 11. sensitivity -----
358 sens = _load(ctx.dep("sensitivity") / "sensitivity.json")
359 names = {
360     "h": "$h$",
361     "hm": "$h_m$",
362     "D0": "$D_0$（问题 3）",
363     "Cair": "$C_{\mathrm{air}}$",
364     "Tplateau": "$T_{\mathrm{air}}$ 平台",
365     "shrink": "收缩幅度 $s$（问题 4）",
366     "D0q4": "$D_0$（问题 4）",
367 }
368 rows = [r for r in sens["table"] if r.get("elasticity") is not None]
369 fig, ax = plotting.new_figure(6.3, 2.6)
370 labels = [names.get(r["parameter"], r["parameter"]) for r in rows]
371 vals = [r["elasticity"] for r in rows]
372 colors = [CAT[1] if v > 0 else CAT[0] for v in vals]
373 ax.barh(labels, vals, color=colors, height=0.55)
374 for y, v in enumerate(vals):
375     ax.text(
376         v + (0.02 if v >= 0 else -0.02),
377         y,
378         f"{v:+.2f}",
379         va="center",

```

```

380         ha="left" if v >= 0 else "right",
381         fontsize=7,
382         color=INK,
383     )
384     ax.axvline(0, color=INK, lw=0.8)
385     ax.set_xlabel("烘干时间的弹性 $E$" )
386     span = max(abs(v) for v in vals)
387     ax.set_xlim(min(vals) - 0.22 * span - 0.05, max(max(vals), 0.0) + 0.22 * span + 0.05)
388     ax.invert_yaxis()
389     _finish(ctx, fig, "fig_sensitivity", index, "烘干时间对各参数的弹性 ( $\pm 10\%$  中心差分)：正值表示参数增大使烘干变慢")
390
391 # 12. end effects (2-D axisymmetric) -----
392 ax2 = np.load(ctx.dep("verification") / "axisym_snapshots.npz")
393 ee = ver["end_effect_2d"]
394 fig, axes = plotting.new_figure(6.3, 2.7, ncols=2)
395 axes[0].plot(
396     ee["axial_profile_at_1d_dry"]["z_cm"],
397     ee["axial_profile_at_1d_dry"]["centre_line_moist"],
398     color=CAT[0],
399     label="二维轴对称, 中心线 r=0",
400 )
401 axes[0].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
402 axes[0].set_xlabel("到中截面的轴向距离 $z$ / cm")
403 axes[0].set_ylabel("水分浓度 / (kg/kg)")
404 axes[0].legend(frameon=False, fontsize=7)
405 t = ax2["t"] / HOUR
406 axes[1].plot(t, ax2["moist_1d"][:, 0], color=CAT[0], label="一维模型, 中心")
407 axes[1].plot(t, ax2["moist_mid"][:, 0], "--", color=CAT[1], label="二维模型, 中截面中心")
408 axes[1].plot(t, ax2["moist_end"][:, 0], "-.", color=CAT[2], label="二维模型, 端面中心")
409 axes[1].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
410 axes[1].set_xlabel("时间 / h")
411 axes[1].set_ylabel("水分浓度 / (kg/kg)")
412 axes[1].legend(frameon=False, fontsize=7)
413 for ax, tag in zip(axes, "ab"):
414     ax.set_title(f"({tag})", loc="left", fontsize=9)
415 _finish(
416     ctx,
417     fig,
418     "fig_endeffect",
419     index,
420     "端部效应：一维烘干结束时刻二维模型中心线沿轴向的水分 (a)；一维与二维 (中截面、端面) 中心水分时程 (b)",
421 )
422
423 # 13. model-evaluation extension (MDR-0008) -----
424 ext = _load(ctx.dep("extension") / "extension.json")
425 dg = np.load(ctx.dep("extension") / "diagnostic.npz")
426 va = np.load(ctx.dep("extension") / "variants.npz")
427 hist3 = _load(ctx.dep("q3") / "history.json")
428 fig, axes = plotting.new_figure(6.3, 2.9, ncols=2)
429 t = dg["t"] / HOUR
430 axes[0].semilogy(t, np.maximum(dg["dt_lat"], 1e-3), color=CAT[1], lw=1.4, label="潜热等效温降 $L_v j_w/h$")
431 axes[0].semilogy(
432     t, np.maximum(dg["model_dt"], 1e-3), color=CAT[0], lw=1.4, label="题给模型 $T_{\\mathrm{air}}-T_s$"
433 )
434 axes[0].axhline(1.0, color=MUTED, ls=":", lw=1)
435 axes[0].set_ylim(1e-3, 2e2)
436 axes[0].set_xlabel("时间 / h")
437 axes[0].set_ylabel("温差 / K")
438 axes[0].legend(frameon=False, fontsize=7)
439 axes[1].plot(
440     np.asarray(hist3["t"]) / HOUR,
441     hist3["centre_moist"],
442     color=CAT[0],
443     lw=1.4,
444     label=f"题给模型: {ext['base_t_dry_h']:.1f} h",
445 )
446 styles = {"cap": ("--", CAT[1]), "ceq": ("-.", CAT[2])}
447 shown = [v for v in ext["variants"] if (v["case"] == "cap" and v["value"] == 1.0) or v["case"] == "ceq"]
448 ceq_colors = [CAT[2], CAT[3], CAT[4], CAT[5]]
449 k = 0
450 for v in shown:

```

```

451         ls, color = styles[v["case"]]
452     if v["case"] == "ceq":
453         color = ceq_colors[k % len(ceq_colors)]
454         k += 1
455         label = f"$C_{{\\mathrm{{eq}}}}={v['value']:g}$: {v['t_dry_h']:.1f} h"
456     else:
457         label = f"能量限制（湿球极限）: {v['t_dry_h']:.1f} h"
458     axes[1].plot(
459         va[f"{v['slug']}_t"] / HOUR, va[f"{v['slug']}_centre_moist"], ls, color=color, lw=1.3, label=label
460     )
461     axes[1].axhline(C_TARGET, color=MUTED, ls=":", lw=1)
462     axes[1].set_xlabel("时间 / h")
463     axes[1].set_ylabel("中心水分浓度 / (kg/kg)")
464     axes[1].legend(frameon=False, fontsize=7)
465     for ax, tag in zip(axes, "ab"):
466         ax.set_title(f"({tag})", loc="left", fontsize=9)
467     _finish(
468         ctx,
469         fig,
470         "fig_extension",
471         index,
472         "模型评价扩展：题给模型蒸发率的潜热等效温降与模型实际温差（a）；能量限制与平衡含水率驱动力下的中心水分时程（b）",
473     )
474
475     ctx.write_json("figure_index.json", index)
476     return {"figures": len(index)}

```

pipelines/a/outputs.py

```

1  """Deliverable stages: result workbooks (organisers' templates) and booktabs tables (+ CSV twins)."""
2
3  from __future__ import annotations
4
5  import csv
6  import json
7  from pathlib import Path
8  from typing import Any
9
10 import numpy as np
11 import pandas as pd
12
13 from forge import xlsx
14 from forge.context import StageContext
15 from forge.runner import stage
16 from pipelines.a.common import COLS, R_OUT_CM, R_TABLE_CM
17 from pipelines.a.contracts import WHOLE_PROCESS_FILE
18
19 HOUR = 3600.0
20
21
22 def _rows(frame: pd.DataFrame, t_min: float, columns: list[str]) -> list[list[Any]]:
23     sel = frame[frame["t"] >= t_min]
24     times = sel["t"].to_numpy(float)
25     values = sel[columns].to_numpy(float)
26     out: list[list[Any]] = []
27     for t, row in zip(times, values):
28         out.append([int(t + 0.5), *[None if np.isnan(v) else float(v) for v in row]])
29     return out
30
31
32 @stage("results", deps=("q1", "q2", "q3", "q4"), description="Fill result1-4.xlsx from the organisers' templates")
33 def results(ctx: StageContext) -> dict[str, Any]:
34     header = [""], *[float(r) for r in R_OUT_CM]
35     report: dict[str, Any] = {}
36     for prob, name in (("q1", "result1.xlsx"), ("q2", "result2.xlsx")):
37         d = ctx.dep(prob)
38         temp = pd.read_parquet(d / "profiles_temp.parquet")
39         moist = pd.read_parquet(d / "profiles_moist.parquet")

```

```

40     sheets = {"温度": (header, _rows(temp, 1.0, COLS)), "水分浓度": (header, _rows(moist, 1.0, COLS))}
41     report[name] = xlsx.write_result(ctx.templates / name, ctx.out(name), sheets)
42     ctx.log.info("results.written", file=name, **report[name]["sheets"])
43 moist3 = pd.read_parquet(ctx.dep("q3") / "profiles_moist.parquet")
44 report["result3.xlsx"] = xlsx.write_result(
45     ctx.templates / "result3.xlsx", ctx.out("result3.xlsx"), {"Sheet1": (header, _rows(moist3, 60.0, COLS))}
46 )
47 moist4 = pd.read_parquet(ctx.dep("q4") / "profiles_moist.parquet")
48 header4 = [*header, "药材表面"]
49 report["result4.xlsx"] = xlsx.write_result(
50     ctx.templates / "result4.xlsx",
51     ctx.out("result4.xlsx"),
52     {"Sheet1": (header4, _rows(moist4, 60.0, [*COLS, "surface"]))},
53 )
54 # supplementary deliverable (MDR-0005): the whole drying process, both fields, at the 60 s resolution the
55 # problem itself uses for the whole process. result2.xlsx keeps the 1 s resolution over the 3 h of tables 3-4;
56 # the same file at 1 s over the whole process would be 2.1e5 rows and ~57 MB (measured), beyond the
57 # submission size budget, so the whole-process fields are delivered at 60 s instead.
58 temp3 = pd.read_parquet(ctx.dep("q3") / "profiles_temp.parquet")
59 sheets_full = {
60     "温度": (header, _rows(temp3, 60.0, COLS)),
61     "水分浓度": (header, _rows(moist3, 60.0, COLS)),
62 }
63 report[WHOLE_PROCESS_FILE] = xlsx.write_result(
64     ctx.templates / "result2.xlsx", ctx.out(WHOLE_PROCESS_FILE), sheets_full
65 )
66 ctx.write_json("results_report.json", report)
67 return {name: info["sheets"] for name, info in report.items()}
68
69
70 # -----
71 # tables
72 # -----
73
74
75 def _fmt(v: Any, digits: int = 4) -> str:
76     if v is None or (isinstance(v, float) and np.isnan(v)):
77         return "--"
78     if isinstance(v, str):
79         return v
80     return f"{float(v):.{digits}f}"
81
82
83 def _write_table(
84     ctx: StageContext,
85     name: str,
86     caption: str,
87     label: str,
88     header: list[str],
89     rows: list[list[Any]],
90     *,
91     digits: int | list[int] = 4,
92     align: str | None = None,
93     note: str | None = None,
94 ) -> None:
95     ncol = len(header)
96     digs = [digits] * ncol if isinstance(digits, int) else digits
97     align = align or ("l" + "r" * (ncol - 1))
98     lines = [
99         "% generated by forge (tables stage) — do not edit",
100         "\\begin{table}[htbp]",
101         "\\centering",
102         f"\\caption{{{caption}}}",
103         f"\\label{{{label}}}",
104         f"\\begin{tabular}{{{align}}@{{{}}}}",
105         "\\toprule",
106         " & ".join(header) + " \\\\",
107         "\\midrule",
108     ]
109     for row in rows:
110         lines.append(" & ".join(_fmt(v, digs[j]) for j, v in enumerate(row)) + " \\\\")

```

```

111     lines += ["\\bottomrule", "\\end{tabular}"]
112     if note:
113         lines.append(f"\\par\\smallskip\\footnotesize {note}")
114     lines.append("\\end{table}")
115     ctx.out("tables", f"{name}.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")
116     with ctx.out("tables", f"{name}.csv").open("w", encoding="utf-8", newline="") as fh:
117         writer = csv.writer(fh)
118         writer.writerow(header)
119         for row in rows:
120             writer.writerow([_fmt(v, digs[j]) for j, v in enumerate(row)])
121
122
123 def _load(path: Path) -> Any:
124     return json.loads(path.read_text(encoding="utf-8"))
125
126
127 @stage(
128     "tables",
129     deps=("q1", "q2", "q3", "q4", "convergence", "verification", "sensitivity", "extension"),
130     description="LaTeX (booktabs) tables of the paper with CSV twins",
131 )
132 def tables(ctx: StageContext) -> dict[str, Any]:
133     r_head = [f"{r:g}" for r in R_TABLE_CM]
134     q1 = _load(ctx.dep("q1") / "tables.json")
135     _write_table(
136         ctx,
137         "tab_q1_temp",
138         "30 分钟内药材的温度 (\\si{\\celsius}; 列为到药材中心的距离/cm) ",
139         "tab:q1-temp",
140         ["时间/s", *r_head],
141         [[int(r[0]), *r[1:]] for r in q1["temp"]],
142         digits=[0, 4, 4, 4, 4, 4],
143     )
144     _write_table(
145         ctx,
146         "tab_q1_moist",
147         "30 分钟内药材的水分浓度 (kg/kg; 列为到药材中心的距离/cm) ",
148         "tab:q1-moist",
149         ["时间/s", *r_head],
150         [[int(r[0]), *r[1:]] for r in q1["moist"]],
151         digits=[0, 4, 4, 4, 4, 4],
152     )
153     q2 = _load(ctx.dep("q2") / "tables.json")
154     _write_table(
155         ctx,
156         "tab_q2_temp",
157         "3 小时内药材的温度 (\\si{\\celsius}; 列为到药材中心的距离/cm) ",
158         "tab:q2-temp",
159         ["时间/h", *r_head],
160         [[r[0] / HOUR, *r[1:]] for r in q2["temp"]],
161         digits=[1, 4, 4, 4, 4, 4],
162     )
163     _write_table(
164         ctx,
165         "tab_q2_moist",
166         "3 小时内药材的水分浓度 (kg/kg; 列为到药材中心的距离/cm) ",
167         "tab:q2-moist",
168         ["时间/h", *r_head],
169         [[r[0] / HOUR, *r[1:]] for r in q2["moist"]],
170         digits=[1, 4, 4, 4, 4, 4],
171     )
172     q3 = _load(ctx.dep("q3") / "tables.json")["moist"]
173     rows3 = [[f"{r[0]:g}", *r[1:]] for r in q3[:-1]] + [[f"烘干结束时间 ({q3[-1][0]:.2f} h) ", *q3[-1][1:]]]
174     _write_table(
175         ctx,
176         "tab_q3_moist",
177         "药材烘干过程的水分浓度 (kg/kg; 固定半径 2 cm, 附录 3 物性; 列为到药材中心的距离/cm) ",
178         "tab:q3-moist",
179         ["时间/h", *r_head],
180         rows3,
181         note="烘干结束时间为 $\\max_r C(r,t)<0.15$ 首次成立的精确时刻 (事件求根), 结果文件按 60 s 分辨率给出。",

```

```

182 )
183 q4 = _load(ctx.dep("q4") / "tables.json")["moist"]
184 rows4 = [[f"{r[0]:g}", *r[1:]] for r in q4[:-1]] + [[f"烘干结束时间 ({q4[-1][0]:.2f} h) ", *q4[-1][1:]]]
185 _write_table(
186     ctx,
187     "tab_q4_moist",
188     "考虑尺寸变化的药材烘干过程水分浓度 (kg/kg; 附录 4 物性; 列为到药材中心的距离/cm, 超出当前半径处为 "--) ",
189     "tab:q4-moist",
190     ["时间/h", *r_head, "药材表面"],
191     rows4,
192     note="固定物理位置的值由  $\xi=r/R(t)$  网格上的三次样条取得; “药材表面”为  $\xi=1$  的节点值。",
193 )
194 q4_alt = _load(ctx.dep("q4") / "tables_alt.json")
195 rows4a = [[f"{r[0]:g}", *r[1:]] for r in q4_alt["moist"][:-1]] + [
196     [f"烘干结束时间 ({q4_alt['moist'][-1][0]:.2f} h) ", *q4_alt["moist"][-1][1:]]
197 ]
198 _write_table(
199     ctx,
200     "tab_q4_moist_alt",
201     f"备选写法 ({q4_alt['formulation']}, MDR-0006) 下的问题 4 水分浓度 (kg/kg) ",
202     "tab:q4-moist-alt",
203     ["时间/h", *r_head, "药材表面"],
204     rows4a,
205 )
206
207 # convergence
208 conv = _load(ctx.dep("convergence") / "convergence.json")
209 rows_c: list[list[Any]] = []
210 keys = [
211     ("q1_temp", "问题 1 温度"),
212     ("q1_moist", "问题 1 水分"),
213     ("q2_temp", "问题 2 温度"),
214     ("q2_moist", "问题 2 水分"),
215     ("q3_moist", "问题 3 水分"),
216     ("q4_moist", "问题 4 水分"),
217 ]
218 for key, label in keys:
219     for row in conv["grid"][key]:
220         rows_c.append(
221             [
222                 label,
223                 row["k"],
224                 row["dr_cm"],
225                 row["max_abs_error"],
226                 row.get("observed_order"),
227                 row.get("observed_order_triplet"),
228             ]
229         )
230 _write_table(
231     ctx,
232     "tab_convergence",
233     f"网格收敛: 以  $\Delta r = 0.1/2^{\{(conv['levels'] - 1)\}}$  cm 为参照的最大绝对误差与两种观测阶",
234     "tab:convergence",
235     ["量", "$k$", " $\Delta r$ /cm", "最大误差", "自收敛阶", "三网格阶"],
236     [
237         [a, b, c, f"{d:.2e}", "--" if e is None else f"{e:.2f}", "--" if f is None else f"{f:.2f}"]
238         for a, b, c, d, e, f in rows_c
239     ],
240     digits=[0, 0, 4, 0, 0, 0],
241     align="llrrrr",
242     note="“自收敛阶”  $p_k = \log_2(e_k/e_{k+1})$  以最细网格为参照, 最后一对网格被系统性抬高 "
243     " $\log_2[(1-4^{-(m-k)})/(1-4^{-(m-k-1)})]$  (二阶格式在  $m-k=2$  时为 2.32); “三网格阶” "
244     " $p_k = \log_2(\lim_{k \rightarrow \infty} u_k - u_{k+1} / \lim_{k \rightarrow \infty} u_{k+1} - u_{k+2})$  "
245     "不需要参照解, 无此偏置 (MDR-0011) 。",
246 )
247 rows_d = []
248 for prob, label in (("q3", "问题 3"), ("q4", "问题 4")):
249     for row in conv["grid"][f"{prob}_t_dry"]:
250         rows_d.append(
251             [
252                 label,

```

```

253         row["k"],
254         row["dr_cm"],
255         row["t_dry_h"],
256         row.get("diff_to_finetest_min"),
257         row.get("observed_order_triplet"),
258     ]
259 )
260 _write_table(
261     ctx,
262     "tab_convergence_drytime",
263     "烘干时间随网格加密的收敛 (与最细网格之差, 单位 min) 与三网格观测阶",
264     "tab:convergence-drytime",
265     ["问题", "$k$", "$\\Delta r$/cm", "$t_{\\mathrm{end}}$/h", "与最细网格之差/min", "三网格阶"],
266     [
267         [a, b, c, f"{d:.4f}", "--" if e is None else f"{e:.3f}", "---" if f is None else f"{f:.2f}"]
268         for a, b, c, d, e, f in rows_d
269     ],
270     digits=[0, 0, 4, 0, 0, 0],
271     align="llrrrr",
272     note="$t_{\\mathrm{end}}$ 的收敛阶低于场量本身: 判据时刻随网格移动, 误差同时包含解误差与穿零时刻的移动。"
273     "Richardson 外推按该观测阶进行 (正文第~\\ref{subsec:gridconv} 小节)。",
274 )
275 rows_t = []
276 for row in conv["time"].values():
277     opt = row["options"]
278     label = f"{opt['method']}, rtol={opt['rtol']:.0e}, atol={opt['atol']:.0e}"
279     if opt.get("max_step"):
280         label += f", $\\Delta t\\le{opt['max_step']:g}$ s"
281     rows_t.append([label, f"{row['max_abs_diff_moist']:.2e}", f"{row['t_dry_diff_s']:.2f}", row["nfev"]])
282 _write_table(
283     ctx,
284     "tab_time_independence",
285     "时间积分无关性 (问题 3, 生产网格; 与生产设置 "
286     f"{ctx.cfg('a.time.method', 'BDF')}, rtol={float(ctx.cfg('a.time.rtol', 1e-9)):.0e}, "
287     f"atol={float(ctx.cfg('a.time.atol', 1e-11)):.0e}、步长不限的差异)",
288     "tab:time-independence",
289     ["积分器、容差与最大步长", "水分最大差", "$t_{\\mathrm{end}}$ 差/s", "右端项调用次数"],
290     rows_t,
291     digits=[0, 0, 0, 0],
292     align="lrrrr",
293     note="$\\Delta t\\le$ 行为限制最大步长的自适应 BDF (其余设置同生产); "
294     "步长限制改变的只是步序列, 结果差异应在容差量级。",
295 )
296
297 # verification summary
298 ver = _load(ctx.dep("verification") / "verification_report.json")
299 checks: list[list[Any]] = []
300 for prob in ("q1", "q2", "q3", "q4"):
301     v = _load(ctx.dep(prob) / f"verification_{prob}.json")
302     mb = v["moisture_balance"]
303     ex = v["extremum"]
304     sampled = mb.get("sampled", {}).get("max_relative_residual")
305     checks.append(
306         [
307             f"问题 {prob[1]}",
308             f"{mb['max_relative_residual']:.2e}",
309             "--" if sampled is None else f"{sampled:.2e}",
310             "是" if ex["moist_within_bounds"] and ex["temp_within_bounds"] else "否",
311             "是" if ex["moist_monotone_in_r"] else "否",
312             "是" if v["passed"] else "否",
313         ]
314     )
315 _write_table(
316     ctx,
317     "tab_verification",
318     "各问题的独立校验: 水分守恒的两种相对残差、极值原理、沿径向单调性、综合结论",
319     "tab:verification",
320     ["问题", "不变量残差", "重采样残差", "极值原理", "单调性", "通过"],
321     checks,
322     digits=[0, 0, 0, 0, 0, 0],
323     align="lrrccc",

```

```

324     note="“不变量残差”比较离散存量与积分器内累积的边界通量（半离散系统的精确线性不变量，检验格式本身）；"
325     "“重采样残差”由输出样本按梯形法独立重算通量积分，受输出间隔的梯形误差限制，检验的是实际通量。",
326 )
327 rows_a = []
328 analytic_keys = [
329     ("temp", "常环境温度 (Bi=%.3f) "),
330     ("moist", "常环境水分 (Bi$_m$=%.3f) "),
331     ("duhamel", "问题 1 温度、Duhamel (Bi=%.3f) "),
332 ]
333 for field, label in analytic_keys:
334     a = ver["analytic"][field]
335     for row in a["levels"]:
336         rows_a.append(
337             [
338                 label % a["biot"],
339                 row["k"],
340                 row["dr_cm"],
341                 f"{row['max_abs_error']:.2e}",
342                 "--" if row.get("observed_order") is None else f"{row['observed_order']:.2f}",
343             ]
344         )
345 _write_table(
346     ctx,
347     "tab_analytic",
348     "与 Robin 圆柱级数解的最大绝对误差及观测阶数（前两组为人为构造的常环境算例，第三组为问题 1 本身）",
349     "tab:analytic",
350     ["量", "$k$", "$\\Delta r$/cm", "最大误差", "观测阶"],
351     rows_a,
352     digits=[0, 0, 4, 0, 0],
353     align="llrrr",
354     note="第三组把附件 1 的分段线性烘房温度用 Duhamel 叠加代入级数解，因而是问题 1 温度场的\\textbf{真解}对照，"
355     "而非常环境算例。",
356 )
357
358 # output-grid (result-file resolution) refinement study
359 og = conv.get("output_grid", {})
360 og_names = {
361     "q1_temp": "问题 1 温度",
362     "q1_moist": "问题 1 水分",
363     "q2_temp": "问题 2 温度",
364     "q2_moist": "问题 2 水分",
365 }
366 rows_og = [
367     [
368         og_names.get(key, key),
369         f"{row['cells_total']:d}",
370         f"{row['production_error']:.2e}",
371         f"{row['max_at_t_s']:g} s / {row['max_at_r_cm']:g} cm",
372         f"{row['observed_order']:.2f}",
373         f"{row['cells_above_half_ulp']:d}",
374         f"{row['last_above_half_ulp_t_s']:g}",
375         f"{row['error_after_half_ulp']:.1e}",
376     ]
377     for key, row in og.items()
378 ]
379 if rows_og:
380     _write_table(
381         ctx,
382         "tab_outputgrid",
383         "结果文件完整输出网格（每 1 s  $\times$  每 0.1 cm）上的离散误差与四位小数的可靠性",
384         "tab:outputgrid",
385         [
386             "量",
387             "单元数",
388             "最大误差",
389             "误差最大处",
390             "三网格阶",
391             "误差  $>5 \times 10^{-5}$  的单元",
392             "最后一处/s",
393             "此后最大误差",
394         ],

```

```

395         rows_og,
396         digits=[0, 0, 0, 0, 0, 0, 0, 0],
397         align="lrrlrrrr",
398         note="误差相对由 $k=6,7$ 两级网格作 Richardson 外推得到的极限。"
399         "$5\\times 10^{-5}$ 是四位小数的半个末位：误差低于它的单元，第四位小数由离散误差决定的可能性已被排除；"
400         "“最后一处”之后的全部输出时刻上误差见未列。",
401     )
402
403 # sensitivity
404 sens = _load(ctx.dep("sensitivity") / "sensitivity.json")
405 names = {
406     "h": "$h$",
407     "hm": "$h_m$",
408     "D0": "$D_0$ (问题 3) ",
409     "Cair": "$C_{\\mathrm{air}}$",
410     "Tplateau": "$T_{\\mathrm{air}}$ 平台",
411     "shrink": "收缩幅度 $s$ (问题 4) ",
412     "D0q4": "$D_0$ (问题 4) ",
413 }
414 rows_s = []
415 for row in sens["table"]:
416     rows_s.append(
417         [names.get(row["parameter"], row["parameter"]), row["base_h"]]
418         + [row.get(f"x{f}") for f in sens["factors"]]
419         + [row.get("elasticity")]
420     )
421 _write_table(
422     ctx,
423     "tab_sensitivity",
424     "参数  $\pm 10\%$ 、 $\pm 20\%$  时的烘干时间 (h) 与弹性  $E=(\Delta t_{\\mathrm{end}}/t_{\\mathrm{end}})/(\Delta p/p)$ ",
425     "tab:sensitivity",
426     ["参数", "基准"] + [f"$\\times{f:g}$" for f in sens["factors"]] + ["弹性"],
427     rows_s,
428     digits=[0, 2, 2, 2, 2, 2, 3],
429 )
430 alt_names = {
431     "plateau_last_sample": "平台取末样本 (MDR-0003 方案 2) ",
432     "plateau_nominal": "平台取  $50 \\si{\\celsius}$ 、 $0.05$  (方案 3) ",
433     "face_arithmetic": "界面系数算术平均",
434     "face_harmonic": "界面系数调和平均",
435     "q4_eulerian": "问题 4 欧拉写法 (含对流项) ",
436     "q4_lagrangian": "问题 4 拉格朗日写法",
437     "q4_fixed_radius": "问题 4 物性、半径固定  $2 \\text{cm}$ ",
438     "q3_mean_criterion": "以平均含水率  $<0.15$  为判据",
439     "mass_conservative": "守恒型水分方程  $\\partial_t(\\rho_s C)=\\nabla\\cdot(\\rho_s D\\nabla C)$ ",
440 }
441 rows_alt = [[alt_names.get(r["case"], r["case"]), r["t_dry_h"], r["diff_pct"]] for r in sens["alternatives"]]
442 _write_table(
443     ctx,
444     "tab_alternatives",
445     "备选解释与离散化选择下的烘干时间 (相对主结果的变化) ",
446     "tab:alternatives",
447     ["备选", "$t_{\\mathrm{end}}$/h", "变化/\\%"],
448     rows_alt,
449     digits=[0, 2, 2],
450     align="lrr",
451 )
452 # model-evaluation extension (MDR-0008)
453 ext = _load(ctx.dep("extension") / "extension.json")
454 rows_e: list[list[Any]] = [{"题给模型 (主结果) ", "--", ext["base_t_dry_h"], 0.0}]
455 for row in ext["variants"]:
456     if row["case"] == "caplocal":
457         name = "蒸发通量受能量限制 (换算用局部  $\\rho_s(C_s)$ ) "
458         param = f"$j_{\\{\\max\\}}={row['cap_plateau_kg_m2_h']:.3f}$ kg/(m2·h)"
459     elif row["case"] == "cap":
460         name = "蒸发通量受能量限制 (湿球极限  $\\times{row['value']:g}$ ) "
461         param = f"$j_{\\{\\max\\}}={row['cap_plateau_kg_m2_h']:.3f}$ kg/(m2·h)"
462     else:
463         name = "驱动力取  $C_s-C_{\\mathrm{eq}}$  (平衡含水率) "
464         param = f"$C_{\\mathrm{eq}}={row['value']:g}$ kg/kg"
465     rows_e.append([name, param, row["t_dry_h"], row["diff_pct"]])

```

```

466     rows_e.append(
467         [
468             "纯能量下界（全程按湿球极限蒸发）",
469             f"$j_{\\max}={ext['psychrometrics']['plateau']['flux_cap'] * HOUR:.3f}$ kg/(m$^2\\cdot$h)",
470             ext["energy_bound_h"],
471             100.0 * (ext["energy_bound_h"] / ext["base_t_dry_h"] - 1.0),
472         ]
473     )
474     _write_table(
475         ctx,
476         "tab_extension",
477         "模型评价扩展（MDR-0008，问题 3 设定）：能量限制与平衡含水率驱动力下的烘干时间及相对主结果的变化",
478         "tab:extension",
479         ["情形", "参数", "$t_{\\mathrm{end}}$/h", "变化/\\%"],
480         rows_e,
481         digits=[0, 0, 2, 1],
482         align="llrr",
483         note="湿球极限 $j_{\\max}=h\\,(T_{\\mathrm{air}}-T_{\\mathrm{wb}})/L_v$ 随烘房状态变化，表中给出恒温阶段的值。",
484     )
485     written = sorted(p.name for p in (ctx.stage_dir / "tables").glob("*.tex"))
486     return {"tables": len(written), "files": written}

```

pipelines/a/physics.py

```

1  """Physical laws and boundary data of the herb-drying problem (SI units; temperatures in degC).
2
3  Property laws follow the problem's appendices (MDR-0001/0002), the drying-chamber history and its
4  constant-temperature plateau follow MDR-0003, the shrinking radius follows MDR-0006.
5  """
6
7  from __future__ import annotations
8
9  from dataclasses import dataclass, replace
10 from typing import Any
11
12 import numpy as np
13 from scipy.interpolate import PchipInterpolator
14
15 R0 = 0.02 # initial radius, m
16 LENGTH = 0.25 # cylinder length, m
17 T_INIT = 28.0 # degC
18 C_INIT = 2.55 # kg/kg (dry basis)
19 C_TARGET = 0.15 # kg/kg, drying finished when every point is below this value
20 KELVIN = 273.15
21 C_FLOOR = 1e-6 # only used inside D(C, T) to avoid division by zero
22 T_DATA_END = 14400.0 # last chamber sample, s
23 PLATEAU_WINDOW = 3600.0 # averaging window for the constant-temperature plateau, s
24
25
26 @dataclass(frozen=True)
27 class Properties:
28     """rho = rho0 + rho1*C; cp = cp0 + cp1*C/(C+1); k = k0 + k1*C/(C+1); D = d0*exp(-da/C)*exp(-de/T_K)."""
29
30     name: str
31     rho0: float
32     rho1: float
33     cp0: float
34     cp1: float
35     k0: float
36     k1: float
37     d0: float
38     da: float
39     de: float
40     h: float = 25.0 # W/(m^2 K), appendix 2 (carried over to problems 2-4, MDR-0002)
41     hm: float = 8e-7 # m/s, appendix 2
42     d_const: float | None = None # constant-D override (analytic verification only)
43
44     def rho(self, c: np.ndarray) -> np.ndarray:

```

```

45     return self.rho0 + self.rho1 * c
46
47 def cp(self, c: np.ndarray) -> np.ndarray:
48     return self.cp0 + self.cp1 * c / (c + 1.0)
49
50 def k(self, c: np.ndarray) -> np.ndarray:
51     return self.k0 + self.k1 * c / (c + 1.0)
52
53 def diffusivity(self, c: np.ndarray, t_degc: np.ndarray) -> np.ndarray:
54     if self.d_const is not None:
55         return np.full_like(np.asarray(c, dtype=float), self.d_const)
56     cs = np.maximum(c, C_FLOOR)
57     out = self.d0 * np.exp(-self.da / cs)
58     if self.de:
59         out = out * np.exp(-self.de / (np.asarray(t_degc, dtype=float) + KELVIN))
60     return out
61
62 def scaled(self, **factors: float) -> Properties:
63     """Return a copy with the named fields multiplied by the given factors (sensitivity runs)."""
64     return replace(self, **{key: getattr(self, key) * value for key, value in factors.items()})
65
66 def describe(self) -> dict[str, Any]:
67     return {
68         "name": self.name,
69         "rho": f"{self.rho0} + {self.rho1} C",
70         "cp": f"{self.cp0} + {self.cp1} C/(C+1)",
71         "k": f"{self.k0} + {self.k1} C/(C+1)",
72         "D": f"{self.d0} exp(-{self.da}/C) exp(-{self.de}/T_K)" if self.d_const is None else f"{self.d_const}",
73         "h": self.h,
74         "hm": self.hm,
75     }
76
77
78 APPENDIX2 = Properties("appendix2", 820.0, 0.0, 2600.0, 0.0, 0.36, 0.0, 7e-9, 0.89, 0.0)
79 APPENDIX3 = Properties("appendix3", 650.0, 128.0, 1450.0, 2736.0, 0.21, 0.38, 2.4e-3, 0.45, 3850.0)
80 APPENDIX4 = Properties("appendix4", 760.0, 90.0, 1850.0, 2150.0, 0.12, 0.20, 4.2e-4, 0.30, 3850.0)
81 PROPERTIES = {"appendix2": APPENDIX2, "appendix3": APPENDIX3, "appendix4": APPENDIX4}
82
83
84 @dataclass(frozen=True)
85 class Chamber:
86     """Drying-chamber temperature and humidity: linear interpolation of attachment 1, constant plateau after it."""
87
88     t: np.ndarray
89     temp: np.ndarray
90     hum: np.ndarray
91     temp_plateau: float
92     hum_plateau: float
93     temp_scale: float = 1.0 # multiplies the plateau only (sensitivity)
94     hum_scale: float = 1.0 # multiplies the whole humidity history (sensitivity)
95
96     @classmethod
97     def from_series(cls, t: np.ndarray, temp: np.ndarray, hum: np.ndarray, plateau: str = "mean_last_hour") -> Chamber:
98         t = np.asarray(t, dtype=float)
99         temp = np.asarray(temp, dtype=float)
100         hum = np.asarray(hum, dtype=float)
101         if plateau == "mean_last_hour":
102             window = t >= t[-1] - PLATEAU_WINDOW
103             tp, hp = float(temp[window].mean()), float(hum[window].mean())
104         elif plateau == "last_sample":
105             tp, hp = float(temp[-1]), float(hum[-1])
106         elif plateau == "nominal":
107             tp, hp = 50.0, 0.05
108         else:
109             raise ValueError(f"unknown plateau rule {plateau!r}")
110         return cls(t, temp, hum, tp, hp)
111
112     @classmethod
113     def constant(cls, temp: float, hum: float) -> Chamber:
114         return cls(np.array([0.0, 1.0]), np.array([temp, temp]), np.array([hum, hum]), temp, hum)
115

```

```

116 def air_temperature(self, t: Any) -> Any:
117     return np.interp(t, self.t, self.temp, right=self.temp_plateau * self.temp_scale)
118
119 def air_humidity(self, t: Any) -> Any:
120     return self.hum_scale * np.interp(t, self.t, self.hum, right=self.hum_plateau)
121
122 def knots(self) -> np.ndarray:
123     """Instants where the forcing is not smooth (sample times incl. the switch to the plateau)."""
124     return self.t[1:] if len(self.t) > 2 else np.array([])
125
126 def describe(self) -> dict[str, Any]:
127     return {
128         "samples": len(self.t),
129         "t_end": float(self.t[-1]),
130         "temp_plateau": self.temp_plateau * self.temp_scale,
131         "hum_plateau": self.hum_plateau * self.hum_scale,
132         "temp_scale": self.temp_scale,
133         "hum_scale": self.hum_scale,
134     }
135
136
137 class Radius:
138     """Radius history R(t): monotone PCHIP through attachment 2 (m), held at its last value afterwards."""
139
140     def __init__(
141         self, t: np.ndarray, r: np.ndarray, shrink_scale: float = 1.0, *, force_interpolant: bool = False
142     ) -> None:
143         self.t = np.asarray(t, dtype=float)
144         self.r = np.asarray(r, dtype=float)
145         self.shrink_scale = shrink_scale
146         self.is_constant = bool(np.all(self.r == self.r[0])) and not force_interpolant
147         self._t0, self._t1 = float(self.t[0]), float(self.t[-1])
148         if not self.is_constant:
149             self._pchip = PchipInterpolator(self.t, self.r, extrapolate=False)
150             self._deriv = self._pchip.derivative()
151
152     @classmethod
153     def from_series(cls, t: np.ndarray, r_cm: np.ndarray, shrink_scale: float = 1.0) -> Radius:
154         r = R0 - shrink_scale * (R0 - np.asarray(r_cm, dtype=float) / 100.0)
155         return cls(np.asarray(t, dtype=float), r, shrink_scale)
156
157     @classmethod
158     def constant(cls, r: float = R0) -> Radius:
159         return cls(np.array([0.0, 1.0]), np.array([r, r]), 0.0)
160
161     def value(self, t: Any) -> Any:
162         if self.is_constant:
163             return np.full_like(np.asarray(t, dtype=float), self.r[0]) if np.ndim(t) else float(self.r[0])
164         return self._pchip(np.clip(t, self._t0, self._t1))
165
166     def knots(self) -> np.ndarray:
167         """PCHIP knots (curvature jumps) including the switch to the constant tail."""
168         return np.array([]) if self.is_constant else self.t[1:]
169
170     def rate(self, t: Any) -> Any:
171         if self.is_constant:
172             return np.zeros_like(np.asarray(t, dtype=float)) if np.ndim(t) else 0.0
173         tt = np.asarray(t, dtype=float)
174         inside = (tt > self._t0) & (tt < self._t1)
175         return np.where(inside, self._deriv(np.clip(tt, self._t0, self._t1)), 0.0)
176
177     def describe(self) -> dict[str, Any]:
178         return {
179             "samples": len(self.t),
180             "t_end": float(self.t[-1]),
181             "r_start_cm": float(self.r[0] * 100),
182             "r_end_cm": float(self.r[-1] * 100),
183             "shrink_scale": self.shrink_scale,
184         }
185
186

```

```

187 def thermal_diffusivity(props: Properties, c: float) -> float:
188     cc = np.asarray(c, dtype=float)
189     return float(props.k(cc) / (props.rho(cc) * props.cp(cc)))
190
191
192 def biot_numbers(props: Properties, c: float, t_degc: float, r: float = R0) -> dict[str, float]:
193     cc = np.asarray(c, dtype=float)
194     return {
195         "Bi_heat": float(props.h * r / props.k(cc)),
196         "Bi_mass": float(props.hm * r / props.diffusivity(cc, np.asarray(t_degc))),
197         "alpha": thermal_diffusivity(props, c),
198         "D": float(props.diffusivity(cc, np.asarray(t_degc))),
199     }
200
201
202 # -----
203 # Psychrometrics and the energy-limited evaporation cap (model-evaluation extension, MDR-0008)
204 # -----
205
206 P_ATM = 101325.0 # Pa
207 L_REF = 2.501e6 # J/kg, latent heat of vaporisation of water at 0 degC
208 L_SLOPE = 2361.0 # J/(kg K), linear decrease of the latent heat with temperature (0-100 degC)
209
210
211 def latent_heat(t_degc: Any) -> Any:
212     """Latent heat of vaporisation of water, J/kg (linear fit to the steam tables, 0-100 degC)."""
213     return L_REF - L_SLOPE * np.asarray(t_degc, dtype=float)
214
215
216 def saturation_pressure(t_degc: Any) -> Any:
217     """Saturation vapour pressure over liquid water, Pa (Magnus form, Alduchov and Eskridge 1996)."""
218     t = np.asarray(t_degc, dtype=float)
219     return 610.94 * np.exp(17.625 * t / (t + 243.04))
220
221
222 def humidity_ratio(p_vapour: Any, p_total: float = P_ATM) -> Any:
223     """Humidity ratio (kg water per kg dry air) of moist air with vapour partial pressure ``p_vapour``."""
224     pv = np.asarray(p_vapour, dtype=float)
225     return 0.622 * pv / (p_total - pv)
226
227
228 def vapour_pressure(w: Any, p_total: float = P_ATM) -> Any:
229     """Vapour partial pressure, Pa, of moist air with humidity ratio ``w``."""
230     ww = np.asarray(w, dtype=float)
231     return ww * p_total / (0.622 + ww)
232
233
234 def relative_humidity(t_degc: Any, w: Any) -> Any:
235     """Relative humidity (0-1) of air at dry-bulb temperature ``t_degc`` with humidity ratio ``w``."""
236     return vapour_pressure(w) / saturation_pressure(t_degc)
237
238
239 def wet_bulb_temperature(t_degc: float, w: float) -> float:
240     """Thermodynamic wet-bulb (adiabatic-saturation) temperature, degC.
241
242     ASHRAE Handbook - Fundamentals (2017), chapter 1, eq. (33), temperatures in degC and energies in kJ/kg:
243      $w = ((2501 - 2.326 t^*) w_s(t^*) - 1.006 (t - t^*)) / (2501 + 1.86 t - 4.186 t^*)$ . Saturated air returns t.
244     """
245     from scipy.optimize import brentq
246
247     t = float(t_degc)
248     target = float(w)
249
250     def residual(tw: float) -> float:
251         ws = float(humidity_ratio(saturation_pressure(tw)))
252         return ((2501.0 - 2.326 * tw) * ws - 1.006 * (t - tw)) / (2501.0 + 1.86 * t - 4.186 * tw) - target
253
254     if residual(t) <= 0.0:
255         return t
256     return float(brentq(residual, -40.0, t, xtol=1e-10, maxiter=200))
257

```

```

258
259 def dry_solid_density(props: Properties, c: Any) -> Any:
260     """Dry-matter density  $\rho_s = \rho(C)/(1 + C)$ , kg/m3 (bulk density per unit of dry-basis moisture)."""
261     cc = np.asarray(c, dtype=float)
262     return props.rho(cc) / (1.0 + cc)
263
264
265 def storage_coefficient(props: Properties, c: Any) -> Any:
266     """ $\sigma(C) = d[\rho_s(C) C]/dC = (\rho_0 + 2 \rho_1 C + \rho_1 C^2)/(1 + C)^2$ , kg/m3 (MDR-0010).
267
268     Storage coefficient of the conservative moisture form  $d(\rho_s C)/dt = \text{div}(\rho_s D \text{ grad } C)$ ; it reduces to
269     the constant  $\rho_s$  only when  $\rho_1 = 0$  (appendix 2), which is what the dry-basis Fick form assumes.
270     """
271     cc = np.asarray(c, dtype=float)
272     return (props.rho0 + 2.0 * props.rho1 * cc + props.rho1 * cc**2) / (1.0 + cc) ** 2
273
274
275 @dataclass(frozen=True)
276 class FluxCap:
277     """Upper bound on the surface evaporation mass flux, kg/(m2 s), sampled on the chamber history.
278
279     ``value(t)`` interpolates linearly between the samples and holds ``plateau`` afterwards; ``rho_s`` converts
280     the bound into the (kg/kg) m/s units of the moisture boundary condition; ``scale`` multiplies the bound.
281     With ``local_rho_s`` the conversion uses the local  $\rho_s(C_s)$  instead of the constant ``rho_s`` (MDR-0008
282     amendment):  $\rho_s$  nearly doubles over the process, so the constant- $\rho_s$  conversion relaxes the cap late on.
283     """
284
285     t: np.ndarray
286     cap: np.ndarray
287     plateau: float
288     rho_s: float
289     scale: float = 1.0
290     local_rho_s: bool = False
291
292     @classmethod
293     def wet_bulb(
294         cls, chamber: Chamber, h: float, rho_s: float, scale: float = 1.0, local_rho_s: bool = False
295     ) -> FluxCap:
296         """Energy limit  $h (T_{\text{air}} - T_{\text{wb}})/L_v(T_{\text{wb}})$  of a surface held at the wet-bulb temperature."""
297
298         def bound(temp: float, hum: float) -> float:
299             tw = wet_bulb_temperature(temp, hum)
300             return float(h * (temp - tw) / latent_heat(tw))
301
302         caps = np.array([bound(float(a), float(b)) for a, b in zip(chamber.temp, chamber.hum_scale * chamber.hum)])
303         plateau = bound(chamber.temp_plateau * chamber.temp_scale, chamber.hum_plateau * chamber.hum_scale)
304         return cls(np.asarray(chamber.t, dtype=float), caps, plateau, rho_s, scale, local_rho_s)
305
306     def value(self, t: Any) -> Any:
307         return self.scale * np.interp(t, self.t, self.cap, right=self.plateau)
308
309     def describe(self) -> dict[str, Any]:
310         return {
311             "cap_initial": float(self.cap[0]) * self.scale,
312             "cap_plateau": self.plateau * self.scale,
313             "rho_s": self.rho_s,
314             "scale": self.scale,
315             "local_rho_s": self.local_rho_s,
316         }

```

pipelines/a/science.py

```

1 """Science stages of Problem A: q1-q4 (solutions + independent checks), convergence, verification, sensitivity."""
2
3 from __future__ import annotations
4
5 import math
6 import re

```

```

7 from concurrent.futures import ProcessPoolExecutor
8 from dataclasses import replace
9 from typing import Any
10
11 import numpy as np
12 import pandas as pd
13
14 from forge.context import StageContext
15 from forge.runner import stage
16 from pipelines.a import verify
17 from pipelines.a.common import (
18     COLS,
19     PROPS,
20     R_OUT_CM,
21     R_TABLE_CM,
22     build_spec,
23     load_chamber,
24     load_radius,
25     mean_moisture,
26     physical_frame,
27     production_n,
28     profiles_frame,
29     snapshot_arrays,
30     spec_from_recipe,
31     spec_to_recipe,
32     table_cells,
33     time_options,
34 )
35 from pipelines.a.physics import (
36     APPENDIX2,
37     C_INIT,
38     C_TARGET,
39     R0,
40     T_INIT,
41     Chamber,
42     Radius,
43     biot_numbers,
44 )
45 from pipelines.a.solver import ProblemSpec, Solution, sample_physical, solve, solve_until_dry, subgrid_indices
46
47 Q1_TABLE_T = np.array([100, 300, 600, 900, 1200, 1500, 1800], dtype=float)
48 Q2_TABLE_T = np.arange(1, 7) * 1800.0
49 HOUR = 3600.0
50
51
52 # -----
53 # helpers
54 # -----
55
56
57 def _store_solution(ctx: StageContext, sol: Solution, spec: ProblemSpec, snapshot_times: np.ndarray) -> None:
58     moving = not spec.radius.is_constant
59     for field in ("temp", "moist"):
60         frame = physical_frame(sol, field) if moving else profiles_frame(sol, field)
61         frame.to_parquet(ctx.out(f"profiles_{field}.parquet"), index=False)
62     np.savez_compressed(ctx.out("snapshots.npz"), **snapshot_arrays(sol, snapshot_times))
63     history = {
64         "t": sol.t.tolist(),
65         "air_temp": sol.air_temp.tolist(),
66         "air_hum": sol.air_hum.tolist(),
67         "radius_cm": (sol.radius * 100.0).tolist(),
68         "centre_temp": sol.temp[:, 0].tolist(),
69         "surface_temp": sol.temp[:, -1].tolist(),
70         "centre_moist": sol.moist[:, 0].tolist(),
71         "surface_moist": sol.moist[:, -1].tolist(),
72         "mean_moist": mean_moisture(sol).tolist(),
73         "surface_moist_flux": sol.surface_moisture_flux(spec.props).tolist(),
74         "cum_moist_loss": None if sol.cum_moist_loss is None else sol.cum_moist_loss.tolist(),
75     }
76     ctx.write_json("history.json", history)
77

```

```

78
79 def _checks(spec: ProblemSpec, sol: Solution) -> dict[str, Any]:
80     if spec.mass_form == "conservative":
81         assert sol.cum_moist_loss is not None
82         balance = verify.conservative_moisture_balance(
83             sol.t, sol.xi, sol.moist, sol.radius, spec.props, sol.cum_moist_loss
84         )
85     else:
86         balance = verify.moisture_balance(
87             sol.t,
88             sol.xi,
89             sol.moist,
90             sol.radius,
91             sol.air_hum,
92             spec.props.hm,
93             cum_loss=sol.cum_moist_loss,
94             flux=spec.surface_moisture_loss(sol.t, sol.moist[:, -1], sol.air_hum),
95         )
96     extremum = verify.extremum_checks(sol.temp, sol.moist, sol.air_temp, sol.air_hum, spec.t_init, spec.c_init)
97     report: dict[str, Any] = {"moisture_balance": balance, "extremum": extremum}
98     if spec.props.rho1 == 0 and spec.props.cpl == 0 and spec.props.k1 == 0 and spec.radius.is_constant:
99         report["energy_balance"] = verify.energy_balance(
100             sol.t, sol.xi, sol.temp, float(sol.radius[0]), sol.air_temp, spec.props, cum_loss=sol.cum_heat_loss
101         )
102     if sol.t_dry is not None and sol.dry_profile is not None:
103         report["drying"] = verify.drying_checks(sol.t, sol.moist, sol.t_dry, sol.dry_profile)
104     passed = [
105         balance["max_relative_residual"] < 1e-4,
106         extremum["temp_within_bounds"],
107         extremum["moist_within_bounds"],
108         extremum["moist_monotone_in_r"],
109         extremum["moist_argmax_is_centre"],
110     ]
111     if "energy_balance" in report:
112         passed.append(report["energy_balance"]["max_relative_residual"] < 1e-4)
113     if "drying" in report:
114         d = report["drying"]
115         passed += [d["rows_before_all_wet"], d["rows_after_all_dry"], d["profile_max_error_at_t_dry"] < 1e-6]
116     report["passed"] = bool(all(passed))
117     return report
118
119
120 def _run_recipe(job: dict[str, Any]) -> dict[str, Any]:
121     """Worker: solve one recipe and return compact arrays (used by the parallel studies)."""
122     spec = spec_from_recipe(job["recipe"])
123     n = int(job["n"])
124     opts = dict(job["opts"])
125     out: dict[str, Any] = {"label": job["label"], "n": n, "opts": opts}
126     if job["mode"] == "dry":
127         sol = solve_until_dry(spec, n, float(job.get("dt_out", 60.0)), horizon=float(job["horizon"]), **opts)
128         idx = subgrid_indices(n)
129         snap_t = np.asarray(job.get("snapshot_times", []), dtype=float)
130         lookup = {float(t): i for i, t in enumerate(sol.t)}
131         rows = [lookup[float(t)] for t in snap_t if float(t) in lookup]
132         out.update(
133             {
134                 "t_dry": sol.t_dry,
135                 "t_dry_grid": sol.stats.get("t_dry_grid"),
136                 "snapshot_t": sol.t[rows].tolist(),
137                 "snapshot_moist": sol.moist[rows][:, idx].tolist(),
138                 "dry_profile_sub": None if sol.dry_profile is None else sol.dry_profile[idx].tolist(),
139                 "mean_moist_t": sol.t.tolist(),
140                 "mean_moist": mean_moisture(sol).tolist(),
141                 "centre_moist": sol.moist[:, 0].tolist(),
142                 "surface_moist": sol.moist[:, -1].tolist(),
143                 "air_hum": sol.air_hum.tolist(),
144                 "stats": sol.stats,
145             }
146         )
147     if job.get("checks"):
148         out["checks"] = _checks(spec, sol)

```

```

149     else:
150         t_out = np.asarray(job["t_out"], dtype=float)
151         sol = solve(spec, n, t_out, **opts)
152         idx = subgrid_indices(n)
153         out.update({"t": sol.t.tolist(), "temp": sol.temp[:, idx].tolist(), "moist": sol.moist[:, idx].tolist()})
154         out["stats"] = sol.stats
155     return out
156
157
158 def _parallel(ctx: StageContext, jobs: list[dict[str, Any]], workers: int | None = None) -> list[dict[str, Any]]:
159     limit = int(ctx.cfg("a.sensitivity.workers", 8)) if workers is None else int(workers)
160     workers = max(1, min(limit, len(jobs)))
161     ctx.log.info("parallel.start", jobs=len(jobs), workers=workers)
162     results: list[dict[str, Any]] = []
163     with ProcessPoolExecutor(max_workers=workers) as pool:
164         for i, res in enumerate(pool.map(_run_recipe, jobs)):
165             results.append(res)
166             ctx.log.info("parallel.progress", done=i + 1, total=len(jobs), label=res["label"])
167     return results
168
169
170 def _tag(text: str) -> str:
171     """Letters-only CamelCase key fragment for ctx.number (digits are spelled out)."""
172     out = text
173     for old, new in (("q1", "Qone"), ("q2", "Qtwo"), ("q3", "Qthree"), ("q4", "Qfour"), ("D0", "Dzero")):
174         out = out.replace(old, new)
175     parts = re.split(r"[^A-Za-z]+", out)
176     return "".join(p[:1].upper() + p[1:] for p in parts if p)
177
178
179 def _table_times_hours(t_dry: float, step_h: float = 6.0) -> np.ndarray:
180     last = math.floor(t_dry / HOUR / step_h)
181     return np.arange(1, last + 1) * step_h * HOUR
182
183
184 # -----
185 # q1: preheating stage, constant properties (appendix 2)
186 # -----
187
188
189 @stage("q1", deps=("ingest",), description="Problem 1: preheating stage (0-1800 s), constant properties + D(C)")
190 def q1(ctx: StageContext) -> dict[str, Any]:
191     spec = build_spec(ctx, "q1")
192     n = production_n(ctx)
193     opts = time_options(ctx)
194     t_out = np.arange(0.0, 1800.0 + 0.5, 1.0)
195     sol = solve(spec, n, t_out, **opts)
196     ctx.log.info("q1.solved", **sol.stats)
197     _store_solution(ctx, sol, spec, Q1_TABLE_T)
198     temp = profiles_frame(sol, "temp")
199     moist = profiles_frame(sol, "moist")
200     tables = {"temp": table_cells(temp, Q1_TABLE_T), "moist": table_cells(moist, Q1_TABLE_T)}
201     ctx.write_json("tables.json", tables)
202     checks = _checks(spec, sol)
203     ctx.write_json("verification_q1.json", checks)
204     bi = biot_numbers(APPENDIX2, C_INIT, T_INIT)
205     ctx.write_json("summary.json", {"spec": spec.describe(), "n": n, "stats": sol.stats, "biot": bi})
206     mean_c = mean_moisture(sol)
207     ctx.number("QoneCentreTempEnd", float(sol.temp[-1, 0]))
208     ctx.number("QoneSurfaceTempEnd", float(sol.temp[-1, -1]))
209     ctx.number("QoneCentreMoistEnd", float(sol.moist[-1, 0]))
210     ctx.number("QoneSurfaceMoistEnd", float(sol.moist[-1, -1]))
211     ctx.number("QoneMeanMoistEnd", float(mean_c[-1]))
212     ctx.number("QoneMoistLossPct", float(100.0 * (1.0 - mean_c[-1] / mean_c[0])), ".2f")
213     ctx.number("QoneBiotHeat", bi["Bi_heat"], ".3f")
214     ctx.number("QoneBiotMass", bi["Bi_mass"], ".3f")
215     ctx.number("QoneAlpha", bi["alpha"], ".3e")
216     ctx.number("QoneDinit", bi["D"], ".3e")
217     ctx.number("QoneAirTempEnd", float(sol.air_temp[-1]), ".3f")
218     ctx.number("QoneSurfaceTempLag", float(sol.air_temp[-1] - sol.temp[-1, -1]), ".3f")
219     ctx.number("GridNodes", n + 1)

```

```

220     ctx.number("GridDrCm", 2.0 / n, ".4f")
221     ctx.number("TimeRtol", opts["rtol"], ".0e")
222     ctx.number("TimeAtol", opts["atol"], ".0e")
223     return {"n": n, "checks_passed": checks["passed"], **{k: v for k, v in sol.stats.items() if k != "message"}}
224
225
226 # -----
227 # q2: whole process, variable properties (appendix 3), first 3 h at 1 s
228 # -----
229
230
231 @stage("q2", deps=("ingest",), description="Problem 2: whole-process model with appendix-3 properties, 0-3 h at 1 s")
232 def q2(ctx: StageContext) -> dict[str, Any]:
233     spec = build_spec(ctx, "q2")
234     n = production_n(ctx)
235     opts = time_options(ctx)
236     t_out = np.arange(0.0, 3.0 * HOUR + 0.5, 1.0)
237     sol = solve(spec, n, t_out, **opts)
238     ctx.log.info("q2.solved", **sol.stats)
239     _store_solution(ctx, sol, spec, Q2_TABLE_T)
240     temp = profiles_frame(sol, "temp")
241     moist = profiles_frame(sol, "moist")
242     tables = {"temp": table_cells(temp, Q2_TABLE_T), "moist": table_cells(moist, Q2_TABLE_T)}
243     ctx.write_json("tables.json", tables)
244     checks = _checks(spec, sol)
245     ctx.write_json("verification-q2.json", checks)
246     bi0 = biot_numbers(spec.props, C_INIT, T_INIT)
247     bi_end = biot_numbers(spec.props, float(sol.moist[-1, -1]), float(sol.temp[-1, -1]))
248     ctx.write_json(
249         "summary.json", {"spec": spec.describe(), "n": n, "stats": sol.stats, "biot_init": bi0, "biot_end": bi_end}
250     )
251     mean_c = mean_moisture(sol)
252     ctx.number("QtwoCentreTempThreeH", float(sol.temp[-1, 0]))
253     ctx.number("QtwoSurfaceTempThreeH", float(sol.temp[-1, -1]))
254     ctx.number("QtwoCentreMoistThreeH", float(sol.moist[-1, 0]))
255     ctx.number("QtwoSurfaceMoistThreeH", float(sol.moist[-1, -1]))
256     ctx.number("QtwoMeanMoistThreeH", float(mean_c[-1]))
257     ctx.number("QtwoMoistLossPct", float(100.0 * (1.0 - mean_c[-1] / mean_c[0])), ".2f")
258     ctx.number("QtwoDinit", bi0["D"], ".3e")
259     ctx.number("QtwoDsurfaceThreeH", bi_end["D"], ".3e")
260     ctx.number("QtwoBiotMassThreeH", bi_end["Bi_mass"], ".2f")
261     ctx.number("QtwoAlphaInit", bi0["alpha"], ".3e")
262     return {"n": n, "checks_passed": checks["passed"], **{k: v for k, v in sol.stats.items() if k != "message"}}
263
264
265 # -----
266 # q3: drying time (appendix 3, fixed radius)
267 # -----
268
269
270 def _drying_outputs(
271     ctx: StageContext, sol: Solution, spec: ProblemSpec, key: str, moving: bool
272 ) -> tuple[dict[str, Any], list[list[Any]]]:
273     assert sol.t_dry is not None and sol.dry_profile is not None
274     t_dry = sol.t_dry
275     rows_t = _table_times_hours(t_dry)
276     frame = physical_frame(sol, "moist") if moving else profiles_frame(sol, "moist")
277     table: list[list[Any]] = []
278     if moving:
279         lookup = {float(t): i for i, t in enumerate(sol.t)}
280         for t, row in zip(rows_t, table_cells(frame, rows_t)):
281             surface = float(frame.at[lookup[float(t)], "surface"])
282             table.append([t / HOUR, *(None if np.isnan(v) else float(v) for v in row[1:]), surface])
283             radius_dry = float(spec.radius.value(t_dry))
284             end_vals = sample_physical(sol.dry_profile, sol.xi, radius_dry, R_TABLE_CM / 100.0)
285             end_row = [t_dry / HOUR, *(None if np.isnan(v) else float(v) for v in end_vals), float(sol.dry_profile[-1])]
286     else:
287         table = [[t / HOUR, *row[1:]] for t, row in zip(rows_t, table_cells(frame, rows_t))]
288         idx = subgrid_indices(len(sol.xi) - 1)
289         prof = sol.dry_profile[idx]
290         end_row = [t_dry / HOUR, *[float(prof[round(r / 0.1)]) for r in R_TABLE_CM]]

```

```

291     table.append(end_row)
292     mean_c = mean_moisture(sol)
293     below = np.where(mean_c < C_TARGET)[0]
294     half = np.where(mean_c <= 0.5 * mean_c[0])[0]
295     summary = {
296         "t_dry_s": t_dry,
297         "t_dry_h": t_dry / HOUR,
298         "t_dry_grid_s": sol.stats.get("t_dry_grid"),
299         "t_dry_grid_h": float(sol.stats.get("t_dry_grid", 0.0)) / HOUR,
300         "t_mean_below_h": None if len(below) == 0 else float(sol.t[below[0]] / HOUR),
301         "t_half_loss_h": None if len(half) == 0 else float(sol.t[half[0]] / HOUR),
302         "surface_moist_at_dry": float(sol.dry_profile[-1]),
303         "mean_moist_at_dry": float(2.0 * (sol.dry_profile @ verify.control_volumes(sol.xi))),
304         "rows": int(len(sol.t) - 1),
305         "table_times_h": (rows_t / HOUR).tolist(),
306     }
307     ctx.write_json(f"drying_{key}.json", summary)
308     return summary, table
309
310
311 @stage("q3", deps=("ingest",), description="Problem 3: drying time until every point is below 0.15 kg/kg")
312 def q3(ctx: StageContext) -> dict[str, Any]:
313     spec = build_spec(ctx, "q3")
314     n = production_n(ctx)
315     opts = time_options(ctx)
316     sol = solve_until_dry(
317         spec, n, float(ctx.cfg("a.drying.dt_out_s", 60.0)), horizon=float(ctx.cfg("a.drying.horizon_s", 432000)), **
318         ↪ opts
319     )
320     assert sol.t_dry is not None
321     ctx.log.info("q3.solved", t_dry_h=sol.t_dry / HOUR, **sol.stats)
322     snaps = np.concatenate([table_times_hours(sol.t_dry), [sol.t[-1]]])
323     _store_solution(ctx, sol, spec, snaps)
324     np.savez_compressed(ctx.out("dry_profile.npz"), xi=sol.xi, moist=sol.dry_profile, temp=sol.dry_temp, t=sol.t_dry)
325     summary, table = _drying_outputs(ctx, sol, spec, "q3", moving=False)
326     ctx.write_json("tables.json", {"moist": table})
327     checks = _checks(spec, sol)
328     ctx.write_json("verification_q3.json", checks)
329     ctx.write_json("summary.json", {"spec": spec.describe(), "n": n, "stats": sol.stats, **summary})
330     ctx.number("QthreeDryHours", summary["t_dry_h"], ".2f")
331     ctx.number("QthreeDryHoursFour", summary["t_dry_h"], ".4f")
332     ctx.number("QthreeDryHoursGrid", summary["t_dry_grid_h"], ".4f")
333     ctx.number("QthreeDrySeconds", summary["t_dry_s"], ".1f")
334     ctx.number("QthreeDryDays", summary["t_dry_h"] / 24.0, ".2f")
335     ctx.number("QthreeSurfaceMoistAtDry", summary["surface_moist_at_dry"])
336     ctx.number("QthreeMeanMoistAtDry", summary["mean_moist_at_dry"])
337     if summary["t_mean_below_h"] is not None:
338         ctx.number("QthreeMeanCriterionHours", summary["t_mean_below_h"], ".2f")
339     if summary["t_half_loss_h"] is not None:
340         ctx.number("QthreeHalfLossHours", summary["t_half_loss_h"], ".2f")
341     ctx.number("QthreeResultRows", summary["rows"])
342     ctx.number("QthreeAirTempPlateau", spec.chamber.temp_plateau, ".3f")
343     ctx.number("QthreeAirHumPlateau", spec.chamber.hum_plateau, ".5f")
344     return {"n": n, "t_dry_h": summary["t_dry_h"], "checks_passed": checks["passed"], "nfev": sol.stats["nfev"]}
345
346 # -----
347 # q4: shrinking radius (attachment 2), appendix-4 properties, Lagrangian main + Eulerian alternative
348 # -----
349
350
351 @stage("q4", deps=("ingest",), description="Problem 4: shrinking cylinder (attachment 2) with appendix-4 properties")
352 def q4(ctx: StageContext) -> dict[str, Any]:
353     n = production_n(ctx)
354     opts = time_options(ctx)
355     dt_out = float(ctx.cfg("a.drying.dt_out_s", 60.0))
356     horizon = float(ctx.cfg("a.drying.horizon_s", 432000))
357     main_form = str(ctx.cfg("a.scheme.formulation", "lagrangian"))
358     alt_form = "eulerian" if main_form == "lagrangian" else "lagrangian"
359     spec = build_spec(ctx, "q4", formulation=main_form)
360     sol = solve_until_dry(spec, n, dt_out, horizon=horizon, **opts)

```

```

361 assert sol.t_dry is not None and sol.dry_profile is not None
362 ctx.log.info("q4.solved", formulation=main_form, t_dry_h=sol.t_dry / HOUR, **sol.stats)
363 snaps = np.concatenate([_table_times_hours(sol.t_dry), [sol.t[-1]]])
364 _store_solution(ctx, sol, spec, snaps)
365 np.savez_compressed(
366     ctx.out("dry_profile.npz"),
367     xi=sol.xi,
368     moist=sol.dry_profile,
369     temp=sol.dry_temp,
370     t=sol.t_dry,
371     radius=float(spec.radius.value(sol.t_dry)),
372 )
373 summary, table = _drying_outputs(ctx, sol, spec, "q4", moving=True)
374 ctx.write_json("tables.json", {"moist": table})
375 checks = _checks(spec, sol)
376
377 # alternative formulation (MDR-0006) and the fixed-radius reference with the same properties
378 alt_spec = replace(spec, formulation=alt_form)
379 alt = solve_until_dry(alt_spec, n, dt_out, horizon=horizon, **opts)
380 assert alt.t_dry is not None
381 ctx.log.info("q4.alternative", formulation=alt_form, t_dry_h=alt.t_dry / HOUR, **alt.stats)
382 alt_summary, alt_table = _drying_outputs(ctx, alt, alt_spec, "q4_alt", moving=True)
383 physical_frame(alt, "moist").to_parquet(ctx.out("profiles_moist_alt.parquet"), index=False)
384 ctx.write_json("tables_alt.json", {"moist": alt_table, "formulation": alt_form})
385 alt_checks = _checks(alt_spec, alt)
386 fixed_spec = replace(spec, radius=Radius.constant(R0))
387 fixed = solve_until_dry(fixed_spec, n, dt_out, horizon=horizon, **opts)
388 assert fixed.t_dry is not None
389 fixed_summary, _ = _drying_outputs(ctx, fixed, fixed_spec, "q4_fixed", moving=False)
390
391 # degenerate check: both moving-boundary code paths with a (formally non-constant) constant radius
392 pseudo = Radius(np.array([0.0, 1.0, 2.0]), np.array([R0, R0, R0]), force_interpolant=True)
393 t_short = np.arange(0.0, 6.0 * HOUR + 0.5, 600.0)
394 ref = solve(fixed_spec, n, t_short, **opts)
395 degenerate: dict[str, Any] = {}
396 for form in ("lagrangian", "eulerian"):
397     test = solve(replace(spec, radius=pseudo, formulation=form), n, t_short, **opts)
398     degenerate[form] = {
399         "max_abs_diff_moist": float(np.max(np.abs(test.moist - ref.moist))),
400         "max_abs_diff_temp": float(np.max(np.abs(test.temp - ref.temp))),
401     }
402 degenerate["passed"] = bool(
403     all(v["max_abs_diff_moist"] < 1e-6 and v["max_abs_diff_temp"] < 1e-4 for v in degenerate.values())
404 )
405 checks["degenerate_constant_radius"] = degenerate
406 checks["passed"] = bool(checks["passed"] and degenerate["passed"])
407 checks["alternative_formulation"] = alt_checks
408 ctx.write_json("verification_q4.json", checks)
409
410 radius_dry = float(spec.radius.value(sol.t_dry)) * 100.0
411 ctx.write_json(
412     "summary.json",
413     {
414         "spec": spec.describe(),
415         "n": n,
416         "stats": sol.stats,
417         "main": summary,
418         "alternative": {"formulation": alt_form, **alt_summary},
419         "fixed_radius": fixed_summary,
420         "radius_at_dry_cm": radius_dry,
421     },
422 )
423 ctx.number("QfourDryHours", summary["t_dry_h"], ".2f")
424 ctx.number("QfourDryHoursFour", summary["t_dry_h"], ".4f")
425 ctx.number("QfourDryHoursGrid", summary["t_dry_grid_h"], ".4f")
426 ctx.number("QfourDrySeconds", summary["t_dry_s"], ".1f")
427 ctx.number("QfourDryDays", summary["t_dry_h"] / 24.0, ".2f")
428 ctx.number("QfourDryHoursAlt", alt_summary["t_dry_h"], ".2f")
429 ctx.number("QfourDryHoursFixed", fixed_summary["t_dry_h"], ".2f")
430 ctx.number("QfourAltDiffPct", 100.0 * (alt_summary["t_dry_h"] / summary["t_dry_h"] - 1.0), ".2f")
431 ctx.number("QfourFixedDiffPct", 100.0 * (fixed_summary["t_dry_h"] / summary["t_dry_h"] - 1.0), ".2f")

```

```

432 ctx.number("QfourRadiusAtDryCm", radius_dry, ".3f")
433 ctx.number("QfourSurfaceMoistAtDry", summary["surface_moist_at_dry"])
434 ctx.number("QfourMeanMoistAtDry", summary["mean_moist_at_dry"])
435 ctx.number("QfourResultRows", summary["rows"])
436 worst = max(v["max_abs_diff_moist"] for v in degenerate.values() if isinstance(v, dict))
437 ctx.number("QfourDegenerateMaxDiff", worst, ".1e")
438 return {
439     "n": n,
440     "t_dry_h": summary["t_dry_h"],
441     "t_dry_alt_h": alt_summary["t_dry_h"],
442     "t_dry_fixed_h": fixed_summary["t_dry_h"],
443     "checks_passed": checks["passed"],
444 }
445
446 # -----
447 # convergence: grid refinement, time tolerance, integrator cross-check
448 # -----
449
450
451
452 def _output_grid_study(ctx: StageContext, refine: int, opts: dict[str, Any]) -> dict[str, Any]:
453     """Refinement study on the *full* result-file grid (every 1 s x 0.1 cm), not only the table instants.
454
455     The table instants start at 100 s (problem 1) and 0.5 h (problem 2), but result1/result2.xlsx start at
456     t = 1 s, where the surface boundary layer sqrt(D t) is thinner than a few production cells. This study
457     therefore compares the *delivered* arrays against a Richardson limit built from two finer grids on every
458     one of the 1800 x 21 (resp. 10800 x 21) output cells, and counts the cells whose fourth decimal differs.
459     """
460     levels = [refine + 1, refine + 2]
461     grids = {"q1": np.arange(0.0, 1800.0 + 0.5, 1.0), "q2": np.arange(0.0, 3.0 * HOUR + 0.5, 1.0)}
462     jobs = [
463         {
464             "label": f"{prob}:out:k{k}",
465             "recipe": spec_to_recipe(build_spec(ctx, prob)),
466             "n": 20 * 2**k,
467             "opts": opts,
468             "mode": "fixed",
469             "t_out": t_out.tolist(),
470         }
471         for prob, t_out in grids.items()
472         for k in levels
473     ]
474     results = {r["label"]: r for r in _parallel(ctx, jobs, workers=2)}
475     out: dict[str, Any] = {}
476     for prob, t_out in grids.items():
477         for field in ("temp", "moist"):
478             delivered = pd.read_parquet(ctx.dep(prob) / f"profiles_{field}.parquet")[COLS].to_numpy(float)
479             fine = {k: np.asarray(results[f"{prob}:out:k{k}"][field], dtype=float) for k in levels}
480             d1 = float(np.max(np.abs(delivered - fine[levels[0]])))
481             d2 = float(np.max(np.abs(fine[levels[0]] - fine[levels[1]])))
482             order = float(np.log2(d1 / d2)) if d2 > 0.0 else float("nan")
483             used = order if 1.0 <= order <= 3.0 else 2.0
484             limit = fine[levels[1]] + (fine[levels[1]] - fine[levels[0]]) / (2.0**used - 1.0)
485             err = np.abs(delivered - limit)[1:] # t = 0 is the exact initial condition
486             times = t_out[1:]
487             i, j = np.unravel_index(int(np.argmax(err)), err.shape)
488             differing = np.round(delivered[1:], 4) != np.round(limit[1:], 4)
489             bad_rows = np.where(differing.any(axis=1))[0]
490             last_t = float(times[bad_rows[-1]]) if len(bad_rows) else 0.0
491             after = err[times > last_t]
492             # the digit-flip count is dominated by values that happen to sit on a rounding boundary; the
493             # decisive statistic is how many cells carry a discretisation error above half an ulp of the
494             # fourth decimal (5e-5), i.e. cells whose fourth decimal is genuinely not resolved
495             coarse = err > 5e-5
496             coarse_rows = np.where(coarse.any(axis=1))[0]
497             last_coarse_t = float(times[coarse_rows[-1]]) if len(coarse_rows) else 0.0
498             beyond = err[times > last_coarse_t]
499             out[f"{prob}_{field}"] = {
500                 "cells_above_half_ulp": int(coarse.sum()),
501                 "last_above_half_ulp_t_s": last_coarse_t,
502                 "error_after_half_ulp": float(beyond.max()) if beyond.size else 0.0,

```

```

503         "grid_levels": levels,
504         "observed_order": order,
505         "order_used": used,
506         "production_error": float(err.max()),
507         "max_at_t_s": float(times[i]),
508         "max_at_r_cm": float(R_OUT_CM[j]),
509         "cells_total": int(err.size),
510         "cells_differing": int(differing.sum()),
511         "last_differing_t_s": last_t,
512         "error_after_last_differing": float(after.max()) if after.size else 0.0,
513     }
514     ctx.log.info("convergence.output_grid", quantity=f"{prob}_{field}", **out[f"{prob}_{field}"])
515     return out
516
517 @stage(
518     "convergence",
519     deps=("ingest", "q1", "q2", "q3", "q4"),
520     description="Grid/time convergence of all four problems (observed orders, drying-time convergence)",
521 )
522 def convergence(ctx: StageContext) -> dict[str, Any]:
523     levels = int(ctx.cfg("a.grid.convergence_levels", 5))
524     refine = int(ctx.cfg("a.grid.refine", 3))
525     opts = time_options(ctx)
526     horizon = float(ctx.cfg("a.drying.horizon_s", 432000))
527     t3 = ctx.dep_data("q3", "summary.json")["t_dry_s"]
528     t4 = ctx.dep_data("q4", "summary.json")["main"]["t_dry_s"]
529     jobs: list[dict[str, Any]] = []
530     for k in range(levels):
531         n = 20 * 2**k
532         for prob, t_out in (("q1", Q1_TABLE_T), ("q2", Q2_TABLE_T)):
533             jobs.append(
534                 {
535                     "label": f"{prob}:k{k}",
536                     "recipe": spec_to_recipe(build_spec(ctx, prob)),
537                     "n": n,
538                     "opts": opts,
539                     "mode": "fixed",
540                     "t_out": np.concatenate([[0.0], t_out]).tolist(),
541                 }
542             )
543         for prob, t_dry in (("q3", t3), ("q4", t4)):
544             jobs.append(
545                 {
546                     "label": f"{prob}:k{k}",
547                     "recipe": spec_to_recipe(build_spec(ctx, prob)),
548                     "n": n,
549                     "opts": opts,
550                     "mode": "dry",
551                     "horizon": horizon,
552                     "snapshot_times": _table_times_hours(t_dry).tolist(),
553                 }
554             )
555     # time-tolerance, integrator and maximum-step cross-checks on the production grid (problem 3, the longest run)
556     n_prod = 20 * 2**refine
557     time_cases = (
558         ("tol:strict", {"rtol": 1e-10, "atol": 1e-12}),
559         ("tol:radau", {"method": "Radau"}),
560         ("tol:loose", {"rtol": 1e-7, "atol": 1e-9}),
561         ("dt:3600", {"max_step": 3600.0}),
562         ("dt:900", {"max_step": 900.0}),
563         ("dt:300", {"max_step": 300.0}),
564     )
565     for label, extra in time_cases:
566         jobs.append(
567             {
568                 "label": label,
569                 "recipe": spec_to_recipe(build_spec(ctx, "q3")),
570                 "n": n_prod,
571                 "opts": {**opts, **extra},
572                 "mode": "dry",

```

```

574         "horizon": horizon,
575         "snapshot_times": _table_times_hours(t3).tolist(),
576     }
577 )
578 results = {r["label"]: r for r in _parallel(ctx, jobs)}
579 report: dict[str, Any] = {"levels": levels, "production_refine": refine, "grid": {}, "time": {}}
580 for prob in ("q1", "q2"):
581     for field in ("temp", "moist"):
582         lv = [
583             {"k": k, "dr_cm": 0.1 / 2**k, "v": np.asarray(results[f"{prob}:k{k}"][field])[1:, :5]}
584             for k in range(levels)
585         ]
586         report["grid"][f"{prob}_{field}"] = verify.convergence_table(lv, "v")
587 for prob in ("q3", "q4"):
588     common = set.intersection(*(set(results[f"{prob}:k{k}"]["snapshot_t"]) for k in range(levels)))
589     lv = []
590     for k in range(levels):
591         res = results[f"{prob}:k{k}"]
592         keep = [i for i, t in enumerate(res["snapshot_t"]) if t in common]
593         lv.append({"k": k, "dr_cm": 0.1 / 2**k, "v": np.asarray(res["snapshot_moist"])[keep[:, :5])})
594     report["grid"][f"{prob}_moist"] = verify.convergence_table(lv, "v")
595     report["grid"][f"{prob}_moist_times_h"] = sorted(t / HOUR for t in common)
596     dry = [results[f"{prob}:k{k}"]["t_dry"] for k in range(levels)]
597     rows = [{"k": k, "dr_cm": 0.1 / 2**k, "t_dry_h": d / HOUR} for k, d in enumerate(dry)]
598     orders_t = verify.triplet_orders([d / HOUR for d in dry])
599     for i in range(levels - 1):
600         rows[i]["diff_to_fineest_min"] = (dry[i] - dry[-1]) / 60.0
601     for i, row in enumerate(rows):
602         row["observed_order_triplet"] = orders_t[i]
603     report["grid"][f"{prob}_t_dry"] = rows
604 base = results[f"q3:k{refine}"]
605 for label, _extra in time_cases:
606     r = results[label]
607     common = set(r["snapshot_t"]) & set(base["snapshot_t"])
608     a = np.asarray([row for t, row in zip(r["snapshot_t"], r["snapshot_moist"]) if t in common])
609     b = np.asarray([row for t, row in zip(base["snapshot_t"], base["snapshot_moist"]) if t in common])
610     report["time"][label] = {
611         "t_dry_diff_s": float(r["t_dry"] - base["t_dry"]),
612         "max_abs_diff_moist": float(np.max(np.abs(a - b))),
613         "nfev": r["stats"]["nfev"],
614         "options": r["opts"],
615     }
616 report["production"] = {
617     "q1_temp_err": report["grid"]["q1_temp"][refine]["max_abs_error"] if refine < levels - 1 else None,
618     "q1_moist_err": report["grid"]["q1_moist"][refine]["max_abs_error"] if refine < levels - 1 else None,
619     "q2_temp_err": report["grid"]["q2_temp"][refine]["max_abs_error"] if refine < levels - 1 else None,
620     "q2_moist_err": report["grid"]["q2_moist"][refine]["max_abs_error"] if refine < levels - 1 else None,
621     "q3_moist_err": report["grid"]["q3_moist"][refine]["max_abs_error"] if refine < levels - 1 else None,
622     "q4_moist_err": report["grid"]["q4_moist"][refine]["max_abs_error"] if refine < levels - 1 else None,
623 }
624 report["output_grid"] = _output_grid_study(ctx, refine, opts)
625 ctx.write_json("convergence.json", report)
626 ctx.write_json("raw_t_dry.json", {k: v.get("t_dry") for k, v in results.items() if "t_dry" in v})
627 for key, row in report["output_grid"].items():
628     tag = _tag(key)
629     ctx.number(f"OutGridErr{tag}", row["production_error"], ".1e")
630     ctx.number(f"OutGridOrder{tag}", row["observed_order"], ".2f")
631     ctx.number(f"OutGridCells{tag}", row["cells_differing"])
632     ctx.number(f"OutGridCellPct{tag}", 100.0 * row["cells_differing"] / row["cells_total"], ".2f")
633     ctx.number(f"OutGridLastDiffSec{tag}", row["last_differing_t_s"], ".0f")
634     ctx.number(f"OutGridErrAfter{tag}", row["error_after_last_differing"], ".1e")
635     ctx.number(f"OutGridCoarse{tag}", row["cells_above_half_ulp"])
636     ctx.number(f"OutGridCoarsePct{tag}", 100.0 * row["cells_above_half_ulp"] / row["cells_total"], ".2f")
637     ctx.number(f"OutGridCoarseSec{tag}", row["last_above_half_ulp_t_s"], ".0f")
638     ctx.number(f"OutGridErrCoarse{tag}", row["error_after_half_ulp"], ".1e")
639 orders = {}
640 for key, rows in report["grid"].items():
641     if key.endswith(("_t_dry", "_times_h")):
642         continue
643     vals = [r["observed_order"] for r in rows if r.get("observed_order") is not None]
644     orders[key] = vals[-1] if vals else None

```

```

645     tag = _tag(key)
646     if vals:
647         ctx.number(f"ConvOrder{tag}", vals[-1], ".2f")
648     triplet = [r["observed_order_triplet"] for r in rows if r.get("observed_order_triplet") is not None]
649     if triplet:
650         ctx.number(f"ConvTripletOrder{tag}", triplet[-1], ".2f")
651     if refine < levels - 1:
652         ctx.number(f"ConvErr{tag}", rows[refine]["max_abs_error"], ".1e")
653 for prob in ("q3", "q4"):
654     rows = report["grid"][f"{prob}_t_dry"]
655     tag = "Qthree" if prob == "q3" else "Qfour"
656     ctx.number(f"ConvDryDiff{tag}Min", abs(rows[refine]["diff_to_fineest_min"]), ".2f")
657     triplet = [r["observed_order_triplet"] for r in rows if r.get("observed_order_triplet") is not None]
658     if triplet:
659         ctx.number(f"ConvDryOrder{tag}", triplet[-1], ".2f")
660     ctx.number("TimeStrictDiffMoist", report["time"]["tol:strict"]["max_abs_diff_moist"], ".1e")
661     ctx.number("TimeRadauDiffMoist", report["time"]["tol:radau"]["max_abs_diff_moist"], ".1e")
662     ctx.number("TimeStrictDryDiffSec", abs(report["time"]["tol:strict"]["t_dry_diff_s"]), ".2f")
663     ctx.number("TimeRadauDryDiffSec", abs(report["time"]["tol:radau"]["t_dry_diff_s"]), ".2f")
664     ctx.number("TimeStepCoarseExtraNfev", int(report["time"]["dt:3600"]["nfev"]) - int(base["stats"]["nfev"]))
665     ctx.number("TimeStepCoarseDiffMoist", report["time"]["dt:3600"]["max_abs_diff_moist"], ".1e")
666     ctx.number("TimeStepCoarseDryDiffSec", abs(report["time"]["dt:3600"]["t_dry_diff_s"]), ".2f")
667     ctx.number("TimeStepFineDiffMoist", report["time"]["dt:300"]["max_abs_diff_moist"], ".1e")
668     ctx.number("TimeStepFineDryDiffSec", abs(report["time"]["dt:300"]["t_dry_diff_s"]), ".2f")
669     return {"orders": orders, "jobs": len(jobs)}
670
671 # -----
672 # verification: analytic solution, independent 2-D model (cross-check and end effects)
673 # -----
674
675
676
677 def _analytic_case(ctx: StageContext, field: str, levels: int, opts: dict[str, Any]) -> dict[str, Any]:
678     """Constant-property Robin cylinder with constant ambient; solver vs Bessel series on several grids."""
679     t_air, c_air = 50.0, 0.05
680     d_const = float(APPENDIX2.diffusivity(np.array(C_INIT), np.array(T_INIT)))
681     props = replace(APPENDIX2, d_const=d_const)
682     chamber = Chamber.constant(t_air, c_air)
683     spec = ProblemSpec(props, chamber, Radius.constant(R0))
684     if field == "temp":
685         t_out = np.concatenate([[0.0], Q1_TABLE_T])
686         diffusivity, transfer, coeff, u0, u_inf = props.k0 / (props.rho0 * props.cp0), props.h, props.k0, T_INIT, t_air
687     else:
688         t_out = np.array([0.0, 300.0, 600.0, 1800.0, 3600.0, 7200.0, 14400.0, 28800.0, 43200.0, 86400.0])
689         diffusivity, transfer, coeff, u0, u_inf = d_const, props.hm, d_const, C_INIT, c_air
690     rows = []
691     for k in range(levels):
692         n = 20 * 2**k
693         sol = solve(spec, n, t_out, **opts)
694         num = getattr(sol, field)[1:, ::n // 20]
695         exact = verify.analytic_field(np.linspace(0, 1, 21), t_out[1:], diffusivity, transfer, coeff, u0, u_inf)
696         rows.append({"k": k, "dr_cm": 0.1 / 2**k, "max_abs_error": float(np.max(np.abs(num - exact)))})
697     for i in range(len(rows) - 1):
698         rows[i]["observed_order"] = float(np.log2(rows[i]["max_abs_error"] / rows[i + 1]["max_abs_error"]))
699     return {
700         "field": field,
701         "biot": transfer * R0 / coeff,
702         "diffusivity": diffusivity,
703         "times_s": t_out[1:].tolist(),
704         "levels": rows,
705     }
706
707
708 def _duhamel_case(ctx: StageContext, levels: int, opts: dict[str, Any]) -> dict[str, Any]:
709     """Problem 1's *actual* temperature field against the exact Duhamel series (time-dependent ambient).
710
711     The ambient of problem 1 is not constant (28 -> 41.5 degC over 1800 s), so the separation-of-variables
712     series is not its solution; Duhamel superposition over the piecewise-linear chamber history is. This gives
713     a true-solution benchmark for the delivered temperature sheet of result1.xlsx, not only for a constructed
714     constant-ambient case.
715     """

```

```

716 chamber = load_chamber(ctx)
717 props = APPENDIX2
718 alpha = props.k0 / (props.rho0 * props.cp0)
719 spec = ProblemSpec(props, chamber, Radius.constant(R0))
720 xi_table = R_TABLE_CM / (100.0 * R0)
721 t_table = np.concatenate([[0.0], Q1_TABLE_T])
722 exact_table = verify.duhamel_field(
723     xi_table, t_table, alpha, props.h, props.k0, T_INIT, chamber.t, chamber.temp, chamber.temp_plateau
724 )
725 rows = []
726 for k in range(levels):
727     n = 20 * 2**k
728     sol = solve(spec, n, t_table, **opts)
729     num = sol.temp[1:, :: n // 4]
730     rows.append({"k": k, "dr_cm": 0.1 / 2**k, "max_abs_error": float(np.max(np.abs(num - exact_table[1:]))))}
731 for i in range(len(rows) - 1):
732     rows[i]["observed_order"] = float(np.log2(rows[i]["max_abs_error"] / rows[i + 1]["max_abs_error"]))
733 delivered = pd.read_parquet(ctx.dep("q1") / "profiles_temp.parquet")
734 t_full = delivered["t"].to_numpy(float)
735 exact_full = verify.duhamel_field(
736     R_OUT_CM / (100.0 * R0),
737     t_full,
738     alpha,
739     props.h,
740     props.k0,
741     T_INIT,
742     chamber.t,
743     chamber.temp,
744     chamber.temp_plateau,
745 )
746 err_full = np.abs(delivered[COLS].to_numpy(float) - exact_full)[1:]
747 times = t_full[1:]
748 i, j = np.unravel_index(int(np.argmax(err_full)), err_full.shape)
749 return {
750     "field": "temp",
751     "biot": props.h * R0 / props.k0,
752     "diffusivity": alpha,
753     "ambient": "piecewise linear, attachment 1 (28.0 -> 41.5 degC)",
754     "times_s": Q1_TABLE_T.tolist(),
755     "levels": rows,
756     "full_grid": {
757         "cells": int(err_full.size),
758         "max_abs_error": float(err_full.max()),
759         "max_at_t_s": float(times[i]),
760         "max_at_r_cm": float(R_OUT_CM[j]),
761         "cells_differing_fourth_decimal": int(
762             (np.round(delivered[COLS].to_numpy(float)[1:], 4) != np.round(exact_full[1:], 4)).sum()
763         ),
764     },
765 }
766
767
768 @stage(
769     "verification",
770     deps=("ingest", "q1", "q2", "q3"),
771     description="Analytic Bessel-series comparison and independent 2-D axisymmetric cross-check / end effects",
772 )
773 def verification(ctx: StageContext) -> dict[str, Any]:
774     opts = time_options(ctx)
775     refine = int(ctx.cfg("a.grid.refine", 3))
776     levels = min(int(ctx.cfg("a.grid.convergence_levels", 5)), refine + 2)
777     analytic = {f: _analytic_case(ctx, f, levels, opts) for f in ("temp", "moist")}
778     analytic["duhamel"] = _duhamel_case(ctx, levels, opts)
779     ctx.log.info("verification.analytic", **{f: a["levels"][refine]["max_abs_error"] for f, a in analytic.items()})
780     ctx.log.info("verification.duhamel", **analytic["duhamel"]["full_grid"])
781     ctx.write_json("analytic.json", analytic)
782
783     # problems 2 and 3 are the same initial-boundary-value problem (appendix 3, fixed radius) integrated over
784     # different horizons; result2.xlsx must therefore be the first 3 h of the whole-process solution (MDR-0005)
785     q2m = pd.read_parquet(ctx.dep("q2") / "profiles_moist.parquet")
786     q2t = pd.read_parquet(ctx.dep("q2") / "profiles_temp.parquet")

```

```

787 q3m = pd.read_parquet(ctx.dep("q3") / "profiles_moist.parquet")
788 q3t = pd.read_parquet(ctx.dep("q3") / "profiles_temp.parquet")
789 shared = np.intersect1d(q2m["t"].to_numpy(float), q3m["t"].to_numpy(float))
790 sel2, sel3 = np.isin(q2m["t"].to_numpy(float), shared), np.isin(q3m["t"].to_numpy(float), shared)
791 horizon_check = {
792     "shared_instants": len(shared),
793     "max_abs_diff_moist": float(np.max(np.abs(q2m[COLS].to_numpy(float)[sel2] - q3m[COLS].to_numpy(float)[sel3]))),
794     "max_abs_diff_temp": float(np.max(np.abs(q2t[COLS].to_numpy(float)[sel2] - q3t[COLS].to_numpy(float)[sel3]))),
795 }
796 horizon_check["passed"] = bool(
797     horizon_check["max_abs_diff_moist"] < 1e-6 and horizon_check["max_abs_diff_temp"] < 1e-4
798 )
799 ctx.log.info("verification.horizon_consistency", **horizon_check)
800
801 # independent 2-D axisymmetric model, coarse grid; insulated ends must reproduce the 1-D scheme exactly
802 chamber = load_chamber(ctx)
803 props = PROPS["q3"]
804 nz = int(ctx.cfg("a.verification.axisym_nz", 25))
805 k2 = int(ctx.cfg("a.verification.axisym_refine", 0))
806 nr = 20 * 2**k2
807 t3 = float(ctx.dep_data("q3", "summary.json")["t_dry_s"])
808 t_out = np.arange(0.0, 12.0 * HOUR + 0.5, 600.0)
809 spec = ProblemSpec(props, chamber, Radius.constant(R0))
810 one_d = solve(spec, nr, t_out, **opts)
811 ins = verify.solve_axisymmetric(props, chamber, t_out, nr=nr, nz=nz, end_bc="insulated", **opts)
812 cross = {
813     "max_abs_diff_moist": float(np.max(np.abs(ins["moist"][:, :, 0] - one_d.moist))),
814     "max_abs_diff_temp": float(np.max(np.abs(ins["temp"][:, :, 0] - one_d.temp))),
815     "z_uniformity_moist": float(np.max(np.abs(ins["moist"] - ins["moist"][:, :, 1]))),
816 }
817 cross["passed"] = bool(cross["max_abs_diff_moist"] < 1e-6 and cross["max_abs_diff_temp"] < 1e-4)
818 ctx.log.info("verification.cross_check", **cross)
819
820 # end effects: Robin conditions on the end faces, whole drying period, coarse grid
821 t_end = np.arange(0.0, math.ceil(1.05 * t3 / HOUR) * HOUR + 0.5, 1800.0)
822 one_d_long = solve(spec, nr, t_end, detect_dry=True, **opts)
823 two_d = verify.solve_axisymmetric(props, chamber, t_end, nr=nr, nz=nz, end_bc="robin", detect_dry=True, **opts)
824 mid = two_d["moist"][:, :, 0]
825 end_face = two_d["moist"][:, :, -1]
826 diff_mid = np.abs(mid - one_d_long.moist)
827 end_effect = {
828     "nz": nz,
829     "dz_cm": 12.5 / nz,
830     "nr": nr,
831     "t_dry_1d_h": None if one_d_long.t_dry is None else one_d_long.t_dry / HOUR,
832     "t_dry_2d_h": two_d.get("t_dry", np.nan) / HOUR,
833     "max_abs_diff_midplane_moist": float(diff_mid.max()),
834     "max_abs_diff_midplane_temp": float(np.max(np.abs(two_d["temp"][:, :, 0] - one_d_long.temp))),
835     "centre_moist_diff_at_1d_dry": float(
836         abs(
837             np.interp(one_d_long.t_dry or t3, t_end, mid[:, 0])
838             - np.interp(one_d_long.t_dry or t3, t_end, one_d_long.moist[:, 0])
839         )
840     ),
841     "end_face_centre_moist_at_1d_dry": float(np.interp(one_d_long.t_dry or t3, t_end, end_face[:, 0])),
842     "axial_profile_at_1d_dry": {
843         "z_cm": (two_d["z"] * 100).tolist(),
844         "centre_line_moist": [
845             float(np.interp(one_d_long.t_dry or t3, t_end, two_d["moist"][:, 0, j])) for j in range(nz + 1)
846         ],
847     },
848 }
849 if one_d_long.t_dry is not None and "t_dry" in two_d:
850     end_effect["t_dry_rel_diff_pct"] = 100.0 * (two_d["t_dry"] / one_d_long.t_dry - 1.0)
851 ctx.log.info("verification.end_effect", **{k: v for k, v in end_effect.items() if not isinstance(v, dict)})
852 np.savez_compressed(
853     ctx.out("axisym_snapshots.npz"),
854     t=t_end,
855     r=two_d["r"],
856     z=two_d["z"],
857     moist_mid=mid,

```

```

858     moist_end=end_face,
859     moist_id=one_d_long.moist,
860     centre_line=two_d["moist"][:, 0, :],
861 )
862 report = {
863     "analytic": analytic,
864     "cross_check_2d": cross,
865     "end_effect_2d": end_effect,
866     "horizon_consistency": horizon_check,
867 }
868 report["passed"] = bool(
869     cross["passed"]
870     and horizon_check["passed"]
871     and all(a["levels"][refine]["max_abs_error"] < 1e-4 for a in analytic.values())
872 )
873 ctx.write_json("verification_report.json", report)
874 ctx.number("AnalyticErrTemp", analytic["temp"]["levels"][refine]["max_abs_error"], ".1e")
875 ctx.number("AnalyticErrMoist", analytic["moist"]["levels"][refine]["max_abs_error"], ".1e")
876 ctx.number("AnalyticOrderTemp", analytic["temp"]["levels"][refine - 1]["observed_order"], ".2f")
877 ctx.number("AnalyticOrderMoist", analytic["moist"]["levels"][refine - 1]["observed_order"], ".2f")
878 ctx.number("AnalyticBiotHeat", analytic["temp"]["biot"], ".3f")
879 ctx.number("AnalyticBiotMass", analytic["moist"]["biot"], ".3f")
880 duh = analytic["duhamel"]
881 ctx.number("DuhamelErrTable", duh["levels"][refine]["max_abs_error"], ".1e")
882 ctx.number("DuhamelOrder", duh["levels"][refine - 1]["observed_order"], ".2f")
883 ctx.number("DuhamelErrFull", duh["full_grid"]["max_abs_error"], ".1e")
884 ctx.number("DuhamelCellsDiffer", duh["full_grid"]["cells_differing_fourth_decimal"])
885 ctx.number("DuhamelCells", duh["full_grid"]["cells"])
886 ctx.number("HorizonDiffMoist", horizon_check["max_abs_diff_moist"], ".1e")
887 ctx.number("HorizonDiffTemp", horizon_check["max_abs_diff_temp"], ".1e")
888 ctx.number("HorizonSharedInstants", horizon_check["shared_instants"])
889 ctx.number("CrossCheckDiffMoist", cross["max_abs_diff_moist"], ".1e")
890 ctx.number("CrossCheckDiffTemp", cross["max_abs_diff_temp"], ".1e")
891 ctx.number("EndEffectMidplaneDiff", end_effect["max_abs_diff_midplane_moist"], ".1e")
892 ctx.number("EndEffectCentreDiffAtDry", end_effect["centre_moist_diff_at_1d_dry"], ".1e")
893 if "t_dry_rel_diff_pct" in end_effect:
894     ctx.number("EndEffectDryDiffPct", end_effect["t_dry_rel_diff_pct"], ".3f")
895     ctx.number("EndEffectDryHoursTwoD", end_effect["t_dry_2d_h"], ".2f")
896     ctx.number("EndEffectDryHoursOneD", end_effect["t_dry_1d_h"], ".2f")
897 ctx.number("EndEffectDzCm", end_effect["dz_cm"], ".2f")
898 return {
899     "passed": report["passed"],
900     "cross_check": cross["passed"],
901     "duhamel_full_grid_error": duh["full_grid"]["max_abs_error"],
902     "horizon_consistency": horizon_check["passed"],
903     "end_effect_pct": end_effect.get("t_dry_rel_diff_pct"),
904 }
905
906
907 # -----
908 # sensitivity: parameter elasticities of the drying time and alternative interpretations
909 # -----
910
911
912 @stage(
913     "sensitivity",
914     deps=("ingest", "q3", "q4"),
915     description="Drying-time elasticities (h, h_m, D0, C_air, T plateau, shrinkage) and alternative interpretations",
916 )
917 def sensitivity(ctx: StageContext) -> dict[str, Any]:
918     factors = [float(f) for f in ctx.cfg("a.sensitivity.factors", [0.8, 0.9, 1.1, 1.2])]
919     n = production_n(ctx)
920     opts = time_options(ctx)
921     horizon = float(ctx.cfg("a.drying.horizon_s", 432000))
922     base3 = build_spec(ctx, "q3")
923     base4 = build_spec(ctx, "q4")
924     t3 = float(ctx.dep_data("q3", "summary.json")["t_dry_s"])
925     t4 = float(ctx.dep_data("q4", "summary.json")["main"]["t_dry_s"])
926
927     def job(label: str, spec: ProblemSpec) -> dict[str, Any]:
928         return {"label": label, "recipe": spec_to_recipe(spec), "n": n, "opts": opts, "mode": "dry", "horizon": horizon

```

↔ }

```
929
930 jobs: list[dict[str, Any]] = []
931 params = {
932     "h": lambda f: replace(base3, props=base3.props.scaled(h=f)),
933     "hm": lambda f: replace(base3, props=base3.props.scaled(hm=f)),
934     "D0": lambda f: replace(base3, props=base3.props.scaled(d0=f)),
935     "Cair": lambda f: replace(base3, chamber=replace(base3.chamber, hum_scale=f)),
936     "Tplateau": lambda f: replace(base3, chamber=replace(base3.chamber, temp_scale=f)),
937     "shrink": lambda f: replace(base4, radius=load_radius(ctx, f)),
938     "D0q4": lambda f: replace(base4, props=base4.props.scaled(d0=f)),
939 }
940 for name, make in params.items():
941     for f in factors:
942         jobs.append(job(f"{name}:{f}", make(f)))
943 alternatives = {
944     "plateau_last_sample": replace(base3, chamber=load_chamber(ctx, "last_sample")),
945     "plateau_nominal": replace(base3, chamber=load_chamber(ctx, "nominal")),
946     "face_arithmetic": replace(base3, face_scheme="arithmetic"),
947     "face_harmonic": replace(base3, face_scheme="harmonic"),
948     "mass_conservative": replace(base3, mass_form="conservative"),
949 }
950 for name, spec in alternatives.items():
951     jobs.append(job(f"alt:{name}", spec))
952 results = {r["label"]: r for r in _parallel(ctx, jobs)}
953
954 table: list[dict[str, Any]] = []
955 elasticities: dict[str, float] = {}
956 for name in params:
957     base_t = t4 if name in ("shrink", "D0q4") else t3
958     row: dict[str, Any] = {"parameter": name, "base_h": base_t / HOUR}
959     for f in factors:
960         row[f"x{f}"] = results[f"{name}:{f}"]["t_dry"] / HOUR
961     if 0.9 in factors and 1.1 in factors:
962         e = ((results[f"{name}:1.1"]["t_dry"] - results[f"{name}:0.9"]["t_dry"]) / base_t) / 0.2
963         row["elasticity"] = e
964         elasticities[name] = e
965     table.append(row)
966 alt_rows = [
967     {
968         "case": name,
969         "t_dry_h": results[f"alt:{name}"]["t_dry"] / HOUR,
970         "diff_pct": 100.0 * (results[f"alt:{name}"]["t_dry"] / t3 - 1.0),
971     }
972     for name in alternatives
973 ]
974 q4s = ctx.dep_data("q4", "summary.json")
975 q3s = ctx.dep_data("q3", "summary.json")
976 alt_rows.append(
977     {
978         "case": "q4_" + q4s["alternative"]["formulation"],
979         "t_dry_h": q4s["alternative"]["t_dry_h"],
980         "diff_pct": 100.0 * (q4s["alternative"]["t_dry_h"] / q4s["main"]["t_dry_h"] - 1.0),
981     }
982 )
983 alt_rows.append(
984     {
985         "case": "q4_fixed_radius",
986         "t_dry_h": q4s["fixed_radius"]["t_dry_h"],
987         "diff_pct": 100.0 * (q4s["fixed_radius"]["t_dry_h"] / q4s["main"]["t_dry_h"] - 1.0),
988     }
989 )
990 if q3s.get("t_mean_below_h") is not None:
991     alt_rows.append(
992         {
993             "case": "q3_mean_criterion",
994             "t_dry_h": q3s["t_mean_below_h"],
995             "diff_pct": 100.0 * (q3s["t_mean_below_h"] / q3s["t_dry_h"] - 1.0),
996         }
997     )
998 ctx.write_json(
```

```

999     "sensitivity.json", {"factors": factors, "table": table, "elasticities": elasticities, "alternatives": alt_rows
    ↵ }
1000 )
1001 for name, e in elasticities.items():
1002     ctx.number("Elas" + _tag(name), e, ".3f")
1003 for row in alt_rows:
1004     key = "Alt" + _tag(row["case"])
1005     ctx.number(key + "Hours", row["t_dry_h"], ".2f")
1006     ctx.number(key + "Pct", row["diff_pct"], ".2f")
1007 return {"jobs": len(jobs), "elasticities": elasticities}

```

pipelines/a/solver.py

```

1  """Radial finite-volume solver for the coupled heat/moisture equations on a (possibly shrinking) cylinder.
2
3  Discretisation (MDR-0004): vertex-centred finite volumes on the Landau coordinate  $\xi = r/R(t)$  in  $[0, 1]$ ,
4  nodes  $\xi_i = i/n$ , control volumes  $[\xi_{i-1/2}, \xi_{i+1/2}]$  with the radial measure  $(\xi^2 \text{ differences})/2$ ,
5  Robin conditions applied as face fluxes on the outer half-cell, symmetry at the axis. Time integration
6  by scipy's adaptive BDF (or Radau) with a block-tridiagonal Jacobian sparsity pattern; every requested
7  output instant is sampled from the integrator's dense interpolant.
8
9  Two formulations for a moving boundary (MDR-0006):
10 * ``lagrangian`` - material coordinate, dry matter conserved, no convective term,  $1/R(t)^2$  scaling;
11 * ``eulerian`` - laboratory-frame PDE on  $0 < r < R(t)$ ; the transformation adds  $+\xi (R\dot{r}/R) d/d\xi$ .
12 For a constant radius both coincide with the fixed-domain scheme.
13 """
14
15 from __future__ import annotations
16
17 import math
18 from dataclasses import dataclass, field
19 from typing import Any
20
21 import numpy as np
22 from scipy import sparse
23 from scipy.integrate import solve_ivp
24
25 from pipelines.a.physics import (
26     C_INIT,
27     C_TARGET,
28     T_INIT,
29     Chamber,
30     FluxCap,
31     Properties,
32     Radius,
33     dry_solid_density,
34     storage_coefficient,
35 )
36
37 FACE_SCHEMES = ("midpoint", "arithmetic", "harmonic")
38 FORMULATIONS = ("lagrangian", "eulerian")
39 MASS_FORMS = ("drybasis", "conservative")
40
41
42 @dataclass(frozen=True)
43 class Grid:
44     n: int
45     xi: np.ndarray # nodes, n+1
46     dxi: float
47     xi_face: np.ndarray # interior faces  $\xi_{i+1/2}$ , n values
48     vol: np.ndarray # control-volume measure  $(\xi_{i+1/2}^2 - \xi_{i-1/2}^2)/2$ , n+1 values
49
50
51 def make_grid(n: int) -> Grid:
52     xi = np.linspace(0.0, 1.0, n + 1)
53     faces = 0.5 * (xi[:-1] + xi[1:])
54     lower = np.concatenate([[0.0], faces])
55     upper = np.concatenate([faces, [1.0]])

```

```

56     return Grid(n, xi, 1.0 / n, faces, 0.5 * (upper**2 - lower**2))
57
58
59 @dataclass(frozen=True)
60 class ProblemSpec:
61     """Everything that defines one initial-boundary-value problem."""
62
63     props: Properties
64     chamber: Chamber
65     radius: Radius
66     formulation: str = "lagrangian"
67     face_scheme: str = "midpoint"
68     t_init: float = T_INIT
69     c_init: float = C_INIT
70     flux_cap: FluxCap | None = None # energy-limited evaporation (model-evaluation extension, MDR-0008)
71     mass_form: str = "drybasis" # moisture conservation form (MDR-0010): drybasis | conservative
72
73     def describe(self) -> dict[str, Any]:
74         return {
75             "properties": self.props.describe(),
76             "chamber": self.chamber.describe(),
77             "radius": self.radius.describe(),
78             "formulation": self.formulation,
79             "face_scheme": self.face_scheme,
80             "mass_form": self.mass_form,
81             "t_init": self.t_init,
82             "c_init": self.c_init,
83             "flux_cap": None if self.flux_cap is None else self.flux_cap.describe(),
84         }
85
86     def surface_moisture_loss(self, t: Any, c_surface: Any, c_air: Any) -> Any:
87         """Boundary loss law  $h_m (C_s - C_{air})$  in (kg/kg) m/s, bounded by the energy cap when one is set."""
88         cs = np.asarray(c_surface, dtype=float)
89         loss = self.props.hm * (cs - np.asarray(c_air, dtype=float))
90         if self.flux_cap is None:
91             return loss
92         rho_s = dry_solid_density(self.props, cs) if self.flux_cap.local_rho_s else self.flux_cap.rho_s
93         return np.minimum(loss, np.asarray(self.flux_cap.value(t), dtype=float) / rho_s)
94
95
96 @dataclass
97 class Solution:
98     t: np.ndarray # output instants, s
99     xi: np.ndarray # nodes
100     temp: np.ndarray # (len(t), n+1) degC
101     moist: np.ndarray # (len(t), n+1) kg/kg
102     radius: np.ndarray # R(t) at output instants, m
103     air_temp: np.ndarray
104     air_hum: np.ndarray
105     stats: dict[str, Any] = field(default_factory=dict)
106     cum_moist_loss: np.ndarray | None = None # integral of  $h_m (C_s - C_{air})/R$  dt, integrated as an ODE state
107     cum_heat_loss: np.ndarray | None = None # integral of  $h (T_s - T_{air})/R$  dt
108     t_dry: float | None = None # exact instant at which max_i C_i first drops below C_TARGET
109     dry_profile: np.ndarray | None = None # moisture profile at t_dry
110     dry_temp: np.ndarray | None = None
111
112     @property
113     def r(self) -> np.ndarray:
114         """Physical node positions (len(t), n+1) in m."""
115         return np.outer(self.radius, self.xi)
116
117     def surface_moisture_flux(self, props: Properties) -> np.ndarray:
118         """ $h_m (C_s - C_{air})$  at the output instants, (kg/kg) m/s."""
119         return props.hm * (self.moist[:, -1] - self.air_hum)
120
121     def surface_heat_flux(self, props: Properties) -> np.ndarray:
122         return props.h * (self.temp[:, -1] - self.air_temp)
123
124
125 class RadialSystem:
126     """Right-hand side  $f(t, y)$  of the semi-discrete system,  $y = [T_0..T_n, C_0..C_n, Q_m, Q_h]$ ."

```

```

127
128 The two trailing states accumulate the boundary losses  $hm (C_s - C_{air})/R$  and  $h (T_s - T_{air})/R$  so that
129 the discrete balances  $\sum_i vol_i C_i + Q_m = \text{const}$  (and its thermal analogue for constant properties)
130 are linear invariants of the ODE system, which BDF/Radau preserve to integrator accuracy. In the
131 ``conservative`` moisture form (MDR-0010) the accumulated loss carries the surface dry-matter density,
132 so that the invariant is the  $\rho_s$ -weighted inventory  $\sum_i vol_i \rho_s(C_i) C_i + Q_m$ .
133 """
134
135 def __init__(self, spec: ProblemSpec, grid: Grid) -> None:
136     if spec.face_scheme not in FACE_SCHEMES:
137         raise ValueError(f"face_scheme must be one of {FACE_SCHEMES}")
138     if spec.formulation not in FORMULATIONS:
139         raise ValueError(f"formulation must be one of {FORMULATIONS}")
140     if spec.mass_form not in MASS_FORMS:
141         raise ValueError(f"mass_form must be one of {MASS_FORMS}")
142     self.spec = spec
143     self.grid = grid
144     self.n1 = grid.n + 1
145     self.convective = spec.formulation == "eulerian" and not spec.radius.is_constant
146     self.conservative = spec.mass_form == "conservative"
147
148 def face_coefficients(self, temp: np.ndarray, moist: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
149     props = self.spec.props
150     scheme = self.spec.face_scheme
151     if scheme == "midpoint":
152         cf = 0.5 * (moist[:-1] + moist[1:])
153         tf = 0.5 * (temp[:-1] + temp[1:])
154         return props.diffusivity(cf, tf), props.k(cf)
155     dn = props.diffusivity(moist, temp)
156     kn = props.k(moist)
157     if scheme == "arithmetic":
158         return 0.5 * (dn[:-1] + dn[1:]), 0.5 * (kn[:-1] + kn[1:])
159     return 2.0 * dn[:-1] * dn[1:] / (dn[:-1] + dn[1:]), 2.0 * kn[:-1] * kn[1:] / (kn[:-1] + kn[1:])
160
161 def __call__(self, t: float, y: np.ndarray) -> np.ndarray:
162     g = self.grid
163     n1 = self.n1
164     temp = y[:n1]
165     moist = y[n1 : 2 * n1]
166     props = self.spec.props
167     radius = float(self.spec.radius.value(t))
168     r2 = radius * radius
169     t_air = float(self.spec.chamber.air_temperature(t))
170     c_air = float(self.spec.chamber.air_humidity(t))
171     d_face, k_face = self.face_coefficients(temp, moist)
172     if self.conservative: # rho_s-weighted flux and storage (MDR-0010)
173         d_face = d_face * dry_solid_density(props, 0.5 * (moist[:-1] + moist[1:]))
174     flux_c = g.xi_face * d_face * (moist[1:] - moist[:-1]) / g.dxi # xi D dC/dxi at interior faces
175     flux_t = g.xi_face * k_face * (temp[1:] - temp[:-1]) / g.dxi
176     div_c = np.empty(n1)
177     div_t = np.empty(n1)
178     div_c[:-1] = flux_c
179     div_t[:-1] = flux_t
180     loss_c = float(self.spec.surface_moisture_loss(t, moist[-1], c_air)) # hm (C_s - C_air), capped if set
181     rho_s_surface = float(dry_solid_density(props, moist[-1])) if self.conservative else 1.0
182     div_c[-1] = -radius * rho_s_surface * loss_c # outer face: xi D dC/dxi = R (-hm (C_s - C_air))
183     div_t[-1] = -radius * props.h * (temp[-1] - t_air)
184     div_c[1:] -= flux_c
185     div_t[1:] -= flux_t
186     storage = storage_coefficient(props, moist) if self.conservative else 1.0
187     dc = div_c / (r2 * g.vol * storage)
188     dt = div_t / (r2 * g.vol * props.rho(moist) * props.cp(moist))
189     if self.convective:
190         rate = float(self.spec.radius.rate(t)) / radius
191         grad_c = np.empty(n1)
192         grad_t = np.empty(n1)
193         grad_c[0] = 0.0
194         grad_t[0] = 0.0
195         grad_c[1:-1] = (moist[2:] - moist[:-2]) / (2.0 * g.dxi)
196         grad_t[1:-1] = (temp[2:] - temp[:-2]) / (2.0 * g.dxi)
197         grad_c[-1] = (3.0 * moist[-1] - 4.0 * moist[-2] + moist[-3]) / (2.0 * g.dxi)

```

```

198         grad_t[-1] = (3.0 * temp[-1] - 4.0 * temp[-2] + temp[-3]) / (2.0 * g.dxi)
199         dc = dc + g.xi * rate * grad_c
200         dt = dt + g.xi * rate * grad_t
201         losses = [rho_s_surface * loss_c / radius, props.h * (temp[-1] - t_air) / radius]
202         return np.concatenate([dt, dc, losses])
203
204     def jacobian_sparsity(self) -> sparse.csr_matrix:
205         n1 = self.n1
206         rows: list[int] = []
207         cols: list[int] = []
208         for i in range(n1):
209             neighbours = [j for j in (i - 1, i, i + 1) if 0 <= j < n1]
210             if self.convective and i == n1 - 1:
211                 neighbours.append(n1 - 3)
212             for j in neighbours:
213                 for row_off in (0, n1):
214                     for col_off in (0, n1):
215                         rows.append(i + row_off)
216                         cols.append(j + col_off)
217         rows += [2 * n1, 2 * n1 + 1]
218         cols += [2 * n1 - 1, n1 - 1] # Q_m depends on C_n, Q_h on T_n
219         data = np.ones(len(rows))
220         return sparse.csr_matrix((data, (rows, cols)), shape=(2 * n1 + 2, 2 * n1 + 2))
221
222
223     def _dry_event(n1: int, target: float) -> Any:
224         def event(t: float, y: np.ndarray) -> float:
225             return float(np.max(y[n1 : 2 * n1]) - target)
226
227         event.terminal = False # type: ignore[attr-defined]
228         event.direction = -1 # type: ignore[attr-defined]
229         return event
230
231
232     def integrate_piecewise(
233         fun: Any,
234         y0: np.ndarray,
235         t_out: np.ndarray,
236         breakpoints: np.ndarray,
237         *,
238         method: str,
239         rtol: float,
240         atol: float,
241         jac_sparsity: Any,
242         event: Any = None,
243         max_step: float = np.inf,
244     ) -> tuple[np.ndarray, dict[str, Any], float | None, np.ndarray | None]:
245         """Integrate y' = fun(t, y) from t = 0 to t_out[-1], restarting the integrator at every breakpoint.
246
247         The chamber history is piecewise linear (slope jumps every 60 s) and the radius is a PCHIP spline
248         (curvature jumps every 1800 s); a multistep method that steps across such kinks loses accuracy in a
249         way that is not controlled by the tolerance. Restarting at the kinks keeps the forcing smooth inside
250         every segment. Returns the states at ``t_out`` (which must start at 0), integrator statistics, and
251         the first root of ``event`` with the state there (if an event is given and fires).
252         """
253         t_out = np.asarray(t_out, dtype=float)
254         t_end = float(t_out[-1])
255         stops = np.unique(np.concatenate([np.asarray(breakpoints, dtype=float), [t_end]]))
256         stops = stops[(stops > 0.0) & (stops <= t_end)]
257         y = np.asarray(y0, dtype=float)
258         states = np.empty((len(t_out), len(y)))
259         states[0] = y
260         filled = 1
261         t_prev = 0.0
262         stats: dict[str, Any] = {"nfev": 0, "njev": 0, "nlu": 0, "segments": len(stops)}
263         event_t: float | None = None
264         event_y: np.ndarray | None = None
265         for t_next in stops:
266             pts = t_out[(t_out > t_prev) & (t_out <= t_next)]
267             t_eval = pts if len(pts) and pts[-1] == t_next else np.concatenate([pts, [t_next]])
268             watching = event is not None and event_t is None

```

```

269         result = solve_ivp(
270             fun,
271             (t_prev, float(t_next)),
272             y,
273             method=method,
274             t_eval=t_eval,
275             rtol=rtol,
276             atol=atol,
277             jac_sparsity=jac_sparsity,
278             dense_output=watching,
279             events=[event] if watching else None,
280             max_step=max_step,
281         )
282         if not result.success:
283             raise RuntimeError(f"solve_ivp failed on [{t_prev}, {t_next}]: {result.message}")
284         k = len(pts)
285         states[filled : filled + k] = result.y[:, :k].T
286         filled += k
287         y = result.y[:, -1]
288         for key in ("nfev", "njev", "nlu"):
289             stats[key] += int(getattr(result, key))
290         if watching and result.t_events and len(result.t_events[0]):
291             event_t = float(result.t_events[0][0])
292             event_y = np.asarray(result.sol(event_t), dtype=float)
293             t_prev = float(t_next)
294         if filled != len(t_out):
295             raise RuntimeError("output instants were not all visited (t_out must be strictly increasing from 0)")
296         return states, stats, event_t, event_y
297
298
299 def solve(
300     spec: ProblemSpec,
301     n: int,
302     t_out: np.ndarray,
303     *,
304     method: str = "BDF",
305     rtol: float = 1e-7,
306     atol: float = 1e-9,
307     detect_dry: bool = False,
308     target: float = C_TARGET,
309     max_step: float = np.inf,
310 ) -> Solution:
311     """Integrate the semi-discrete system and sample the dense solution at ``t_out`` (must start at 0)."""
312     grid = make_grid(n)
313     system = RadialSystem(spec, grid)
314     t_out = np.asarray(t_out, dtype=float)
315     if t_out[0] != 0.0 or np.any(np.diff(t_out) <= 0):
316         raise ValueError("t_out must start at 0 and be strictly increasing")
317     y0 = np.concatenate([np.full(grid.n + 1, spec.t_init), np.full(grid.n + 1, spec.c_init), [0.0, 0.0]])
318     breakpoints = np.concatenate([spec.chamber.knots(), spec.radius.knots()])
319     states, stats, t_dry, state = integrate_pieewise(
320         system,
321         y0,
322         t_out,
323         breakpoints,
324         method=method,
325         rtol=rtol,
326         atol=atol,
327         jac_sparsity=system.jacobian_sparsity(),
328         event=_dry_event(grid.n + 1, target) if detect_dry else None,
329         max_step=max_step,
330     )
331     n1 = grid.n + 1
332     radius = np.asarray(spec.radius.value(t_out), dtype=float)
333     sol = Solution(
334         t=t_out,
335         xi=grid.xi,
336         temp=states[:, :n1].copy(),
337         moist=states[:, n1 : 2 * n1].copy(),
338         radius=radius,
339         air_temp=np.asarray(spec.chamber.air_temperature(t_out), dtype=float),

```

```

340     air_hum=np.asarray(spec.chamber.air_humidity(t_out), dtype=float),
341     stats={"method": method, "rtol": rtol, "atol": atol, "n_intervals": n, **stats},
342     cum_moist_loss=states[:, 2 * n1].copy(),
343     cum_heat_loss=states[:, 2 * n1 + 1].copy(),
344 )
345 if t_dry is not None and state is not None:
346     sol.t_dry = t_dry
347     sol.dry_temp = state[n1].copy()
348     sol.dry_profile = state[n1 : 2 * n1].copy()
349 return sol
350
351
352 def solve_until_dry(
353     spec: ProblemSpec,
354     n: int,
355     dt_out: float = 60.0,
356     *,
357     horizon: float = 5 * 86400.0,
358     max_horizon: float = 30 * 86400.0,
359     display_decimals: int | None = 4,
360     **kwargs: Any,
361 ) -> Solution:
362     """Integrate until max_i C_i < C_TARGET, keeping output rows a little past the drying instant.
363
364     The last kept row is the first ``dt_out`` multiple at or after ``t_dry`` whose values, rounded to
365     ``display_decimals`` places as the problem requires, are all strictly below the target; with
366     ``display_decimals=None`` it is simply the first multiple at or after ``t_dry``. Because the criterion
367     holds with equality at ``t_dry`` itself, the unrounded value on the first multiple after it still rounds
368     to 0.1500, which would read as "criterion not met" in the delivered workbook (MDR-0005).
369     """
370     while True:
371         t_out = np.arange(0.0, horizon + 0.5 * dt_out, dt_out)
372         sol = solve(spec, n, t_out, detect_dry=True, **kwargs)
373         if sol.t_dry is not None:
374             break
375         horizon *= 2.0
376         if horizon > max_horizon:
377             raise RuntimeError("drying criterion not met within the maximum horizon")
378     last = math.ceil(sol.t_dry / dt_out - 1e-9)
379     if display_decimals is not None:
380         shown = np.round(sol.moist.max(axis=1), display_decimals)
381         while last + 1 < len(sol.t) and shown[last] >= C_TARGET:
382             last += 1
383     keep = slice(0, last + 1)
384     sol.t = sol.t[keep]
385     sol.temp = sol.temp[keep]
386     sol.moist = sol.moist[keep]
387     sol.radius = sol.radius[keep]
388     sol.air_temp = sol.air_temp[keep]
389     sol.air_hum = sol.air_hum[keep]
390     if sol.cum_moist_loss is not None and sol.cum_heat_loss is not None:
391         sol.cum_moist_loss = sol.cum_moist_loss[keep]
392         sol.cum_heat_loss = sol.cum_heat_loss[keep]
393     sol.stats["t_dry_grid"] = float(math.ceil(sol.t_dry / dt_out - 1e-9) * dt_out)
394     sol.stats["t_last_row"] = float(sol.t[-1])
395     return sol
396
397
398 def subgrid_indices(n: int, n_out: int = 20) -> np.ndarray:
399     """Indices of the nodes coinciding with the coarse output grid (n must be an integer multiple of n_out)."""
400     if n % n_out:
401         raise ValueError(f"n={n} is not a multiple of the output grid {n_out}")
402     return np.arange(0, n + 1, n // n_out)
403
404
405 def sample_physical(profile: np.ndarray, xi: np.ndarray, radius: float, r_targets: np.ndarray) -> np.ndarray:
406     """Cubic-spline sample of a nodal profile at physical radii; NaN where r > R(t)."""
407     from scipy.interpolate import CubicSpline
408
409     spline = CubicSpline(xi, profile, bc_type=((1, 0.0), "not-a-knot"))
410     xi_t = np.asarray(r_targets, dtype=float) / radius

```

```

411     out = np.full(xi_t.shape, np.nan)
412     inside = xi_t <= 1.0 + 1e-12
413     out[inside] = spline(np.minimum(xi_t[inside], 1.0))
414     return out

```

pipelines/a/stages.py

```

1  """Problem A stages: validate (here) plus the science and deliverable stages of the submodules.
2
3  Conventions (see docs/ENGINEERING_STANDARD.md):
4  * every stage is ``@stage(name, deps=(...))`` and returns a JSON-serialisable metrics dict;
5  * outputs go only to ``ctx.out(...)``; paper numbers via ``ctx.number("Key", value)``;
6  * ``results`` writes result*.xlsx with ``forge.xlsx.write_result`` from the organisers' templates;
7  * ``figures`` writes figures/*.pdf (+png preview) and ``tables`` writes tables/*.tex.
8
9  Stage map: q1, q2, q3, q4, convergence, verification, sensitivity (pipelines/a/science.py); derived (derived.py);
10 results, tables (pipelines/a/outputs.py); figures (pipelines/a/figures.py).
11 """
12
13 from __future__ import annotations
14
15 from typing import Any
16
17 from forge.context import StageContext
18 from forge.runner import stage
19 from pipelines.a import derived, extension, figures, outputs, science # noqa: F401 - registers the stages on import
20 from pipelines.a.contracts import INPUT_CONTRACTS
21 from pipelines.common.validation import run_validation
22
23
24 @stage(
25     "validate", deps=("ingest",), description="Validate the drying-chamber and radius series against their contracts"
26 )
27 def validate(ctx: StageContext) -> dict[str, Any]:
28     return run_validation(ctx, INPUT_CONTRACTS)

```

pipelines/a/verify.py

```

1  """Independent checks of the drying solutions (MDR-0007). Nothing here calls the radial solver's internals.
2
3  * analytic Bessel-series solution of the constant-property Robin cylinder;
4  * discrete balances (moisture, energy) recomputed from the solution arrays only;
5  * extremum-principle, monotonicity and drying-criterion checks;
6  * grid-convergence bookkeeping (errors against the finest grid, observed orders);
7  * an independently written two-dimensional axisymmetric finite-volume model (physical coordinates)
8    used both as a code cross-check (insulated ends) and to quantify end effects (Robin ends).
9  """
10
11 from __future__ import annotations
12
13 from itertools import pairwise
14 from typing import Any
15
16 import numpy as np
17 from scipy import sparse
18 from scipy.optimize import brentq
19 from scipy.special import j0, j1, jn_zeros
20
21 from pipelines.a.physics import C_TARGET, LENGTH, R0, Chamber, Properties, dry_solid_density
22
23 # -----
24 # 1. Analytic solution: infinite cylinder, uniform initial value, Robin boundary, constant ambient
25 # -----
26

```

```

27
28 def robin_eigenvalues(bi: float, count: int) -> np.ndarray:
29     """Roots of  $\lambda J_1(\lambda) = Bi J_0(\lambda)$ ; the n-th root lies between consecutive zeros of  $J_0$ ."""
30     zeros = np.concatenate([[0.0], jn_zeros(0, count)])
31     roots = []
32     for lo, hi in pairwise(zeros):
33         f = lambda x: x * j1(x) - bi * j0(x) # noqa: E731
34         roots.append(brentq(f, lo + 1e-12, hi - 1e-12, xtol=1e-14, rtol=1e-14, maxiter=500))
35     return np.asarray(roots)
36
37
38 def analytic_robin_cylinder(xi: np.ndarray, fourier: np.ndarray, bi: float, n_terms: int = 400) -> np.ndarray:
39     """Dimensionless excess  $\theta(x_i, Fo) = (u - u_{\infty}) / (u_0 - u_{\infty})$ ; returns shape  $(\text{len}(\text{fourier}), \text{len}(xi))$ ."""
40     lam = robin_eigenvalues(bi, n_terms)
41     coeff = 2.0 * bi / ((lam**2 + bi**2) * j0(lam))
42     basis = j0(np.outer(lam, xi)) # (n_terms, nxi)
43     decay = np.exp(-np.outer(np.asarray(fourier, dtype=float), lam**2)) # (nt, n_terms)
44     return (decay * coeff) @ basis
45
46
47 def analytic_field(
48     xi: np.ndarray, t: np.ndarray, diffusivity: float, transfer: float, coeff: float, u0: float, u_inf: float
49 ) -> np.ndarray:
50     """Physical field  $u(x_i, t)$  for a constant-property cylinder of radius  $R_0$  with Robin coefficient ``transfer``."""
51     bi = transfer * R0 / coeff
52     fo = diffusivity * np.asarray(t, dtype=float) / R0**2
53     theta = analytic_robin_cylinder(xi, fo, bi)
54     theta[np.asarray(t) == 0.0] = 1.0 # the series converges slowly at  $Fo = 0$ ; the initial value is exact
55     return u_inf + (u0 - u_inf) * theta
56
57
58 def duhamel_field(
59     xi: np.ndarray,
60     t: np.ndarray,
61     diffusivity: float,
62     transfer: float,
63     coeff: float,
64     u0: float,
65     knots: np.ndarray,
66     values: np.ndarray,
67     tail: float | None = None,
68     n_terms: int = 400,
69 ) -> np.ndarray:
70     """Exact series solution for a *time-dependent, piecewise-linear* ambient history (Duhamel superposition).
71
72     The constant-property Robin cylinder with ambient  $u_{\text{air}}(t)$  and uniform initial value  $u_0$  has
73      $u = u_{\text{air}}(t) + \sum_m c_m B_m(t) J_0(\lambda_m x_i)$ ,  $c_m = 2Bi / ((\lambda_m^2 + Bi^2) J_0(\lambda_m))$ ,
74      $B_m(t) = (u_0 - u_{\text{air}}(0)) e^{-\mu_m t} - \int_0^t e^{-\mu_m(t-s)} u_{\text{air}}'(s) ds$ ,  $\mu_m = \lambda_m^2 R_0^2$ .
75     For a piecewise-linear  $u_{\text{air}}$  the integral is elementary on every segment, so the result is exact up to the
76     series truncation. ``tail`` is the constant value held after the last knot (a jump there is handled as a
77     Dirac in  $u_{\text{air}}'$ ). This is the analytic benchmark for problem 1, whose ambient is *not* constant.
78     """
79     bi = transfer * R0 / coeff
80     lam = robin_eigenvalues(bi, n_terms)
81     cm = 2.0 * bi / ((lam**2 + bi**2) * j0(lam))
82     mu = diffusivity * lam**2 / R0**2
83     tt = np.asarray(t, dtype=float)
84     kn = np.asarray(knots, dtype=float)
85     va = np.asarray(values, dtype=float)
86     amp = np.empty((len(tt), len(lam)))
87     amp[:, 0] = (u0 - va[0]) * np.exp(-np.outer(tt, mu))
88     for j in range(len(kn) - 1):
89         a, b = float(kn[j]), float(kn[j + 1])
90         slope = float((va[j + 1] - va[j]) / (b - a))
91         if slope == 0.0 or a >= tt.max():
92             continue
93         active = tt > a
94         upper = np.minimum(tt, b)
95         e_b = np.exp(-np.outer(tt - upper, mu))
96         e_a = np.exp(-np.outer(np.maximum(tt - a, 0.0), mu))
97         amp[active] -= (slope / mu)[None, :] * (e_b[active] - e_a[active])

```

```

98     if tail is not None:
99         jump = float(tail - va[-1])
100         if jump != 0.0:
101             after = tt > kn[-1]
102             if after.any():
103                 amp[after] -= jump * np.exp(-np.outer(tt[after] - kn[-1], mu))
104     air = np.interp(tt, kn, va, right=va[-1] if tail is None else tail)
105     field = air[:, None] + (amp * cm[None, :]) @ j0(np.outer(lam, np.asarray(xi, dtype=float)))
106     field[tt == 0.0] = u0 # the series for the constant 1 converges slowly at t = 0; the initial value is exact
107     return field
108
109
110 # -----
111 # 2. Balances and qualitative properties computed from output arrays
112 # -----
113
114
115 def control_volumes(xi: np.ndarray) -> np.ndarray:
116     faces = 0.5 * (xi[:-1] + xi[1:])
117     lower = np.concatenate([[0.0], faces])
118     upper = np.concatenate([faces, [1.0]])
119     return 0.5 * (upper**2 - lower**2)
120
121
122 def _balance(lhs: np.ndarray, cumulative: np.ndarray) -> dict[str, float]:
123     residual = lhs - cumulative
124     scale = max(abs(lhs[-1]), 1e-300)
125     return {
126         "max_abs_residual": float(np.max(np.abs(residual))),
127         "final_relative_residual": float(abs(residual[-1]) / scale),
128         "max_relative_residual": float(np.max(np.abs(residual)) / scale),
129     }
130
131
132 def moisture_balance(
133     t: np.ndarray,
134     xi: np.ndarray,
135     moist: np.ndarray,
136     radius: np.ndarray,
137     air_hum: np.ndarray,
138     hm: float,
139     cum_loss: np.ndarray | None = None,
140     flux: np.ndarray | None = None,
141 ) -> dict[str, Any]:
142     """Discrete water inventory W = sum_i vol_i C_i versus the time-integrated surface flux.
143
144     Fixed radius: d/dt (R^2 W) = -R hm (C_s - C_air); shrinking (Lagrangian): d/dt W = -hm (C_s - C_air)/R(t).
145     ``sampled``: flux from the output samples (trapezoid rule, limited by the output spacing);
146     ``exact``: flux accumulated inside the integrator (``cum_loss`` = int hm (C_s - C_air)/R dt), which turns the
147     balance into a linear invariant of the ODE system and therefore tests the scheme itself.
148     ``flux`` overrides the boundary loss law hm (C_s - C_air) (e.g. the energy-capped law of MDR-0008).
149     """
150     vol = control_volumes(xi)
151     inventory = moist @ vol
152     flux = hm * (moist[:, -1] - air_hum) if flux is None else np.asarray(flux, dtype=float)
153     if np.allclose(radius, radius[0]):
154         lhs = radius[0] ** 2 * (inventory - inventory[0])
155         integrand = -radius[0] * flux
156         exact_scale = radius[0] ** 2
157     else:
158         lhs = inventory - inventory[0]
159         integrand = -flux / radius
160         exact_scale = 1.0
161     cumulative = np.concatenate([[0.0], np.cumsum(0.5 * (integrand[1:] + integrand[:-1]) * np.diff(t))])
162     report: dict[str, Any] = {
163         "inventory_initial": float(inventory[0]),
164         "inventory_final": float(inventory[-1]),
165         "loss_fraction": float(1.0 - inventory[-1] / inventory[0]),
166         "sampled": _balance(lhs, cumulative),
167     }
168     if cum_loss is not None:

```

```

169     report["exact"] = _balance(lhs, -exact_scale * cum_loss)
170     best = report.get("exact", report["sampled"])
171     report["max_relative_residual"] = best["max_relative_residual"]
172     report["final_relative_residual"] = best["final_relative_residual"]
173     return report
174
175
176 def energy_balance(
177     t: np.ndarray,
178     xi: np.ndarray,
179     temp: np.ndarray,
180     radius: float,
181     air_temp: np.ndarray,
182     props: Properties,
183     cum_loss: np.ndarray | None = None,
184 ) -> dict[str, Any]:
185     """Constant-property energy balance:  $\rho \text{ cp } R^2 \frac{d}{dt} \sum \text{vol}_i T_i = -R h (T_s - T_{\text{air}})$ ."""
186     vol = control_volumes(xi)
187     rho_cp = float(props.rho(np.array(0.0)) * props.cp(np.array(0.0)))
188     inventory = rho_cp * radius**2 * (temp @ vol)
189     integrand = -radius * props.h * (temp[:, -1] - air_temp)
190     cumulative = np.concatenate([[0.0], np.cumsum(0.5 * (integrand[1:] + integrand[:-1]) * np.diff(t))])
191     report: dict[str, Any] = {"sampled": _balance(inventory - inventory[0], cumulative)}
192     if cum_loss is not None: #  $\rho \text{ cp } R^2 \frac{dW}{dt} = -R h (T_s - T_{\text{air}})$  and  $Q_h = \int h (T_s - T_{\text{air}}) / R \, dt$ 
193         report["exact"] = _balance(inventory - inventory[0], -(radius**2) * cum_loss)
194     best = report.get("exact", report["sampled"])
195     report["max_relative_residual"] = best["max_relative_residual"]
196     report["final_relative_residual"] = best["final_relative_residual"]
197     return report
198
199
200 def conservative_moisture_balance(
201     t: np.ndarray,
202     xi: np.ndarray,
203     moist: np.ndarray,
204     radius: np.ndarray,
205     props: Properties,
206     cum_loss: np.ndarray,
207 ) -> dict[str, Any]:
208     """Balance of the  $\rho_s$ -weighted inventory of the conservative moisture form (MDR-0010).
209
210     For  $d(\rho_s C)/dt = \text{div}(\rho_s D \text{ grad } C)$  on a fixed domain the discrete invariant is
211      $\sum_i V_i \rho_s(C_i) C_i + Q_m / R^2$  with  $Q_m = \int \rho_s(C_s) h_m (C_s - C_{\text{air}}) / R \, dt$ .
212     """
213     vol = control_volumes(xi)
214     inventory = (dry_solid_density(props, moist) * moist) @ vol
215     lhs = radius[0] ** 2 * (inventory - inventory[0])
216     report: dict[str, Any] = {"exact": _balance(lhs, -(radius[0] ** 2) * np.asarray(cum_loss, dtype=float))}
217     report["max_relative_residual"] = report["exact"]["max_relative_residual"]
218     report["final_relative_residual"] = report["exact"]["final_relative_residual"]
219     report["inventory_initial"] = float(inventory[0])
220     report["inventory_final"] = float(inventory[-1])
221     return report
222
223
224 def extremum_checks(
225     temp: np.ndarray, moist: np.ndarray, air_temp: np.ndarray, air_hum: np.ndarray, t0: float, c0: float
226 ) -> dict[str, Any]:
227     t_lo, t_hi = min(t0, float(air_temp.min())), max(t0, float(air_temp.max()))
228     c_lo, c_hi = min(c0, float(air_hum.min())), max(c0, float(air_hum.max()))
229     tol = 1e-9
230     dc = np.diff(moist, axis=1) # along r: should be  $\leq 0$  (drying from the surface)
231     return {
232         "temp_min": float(temp.min()),
233         "temp_max": float(temp.max()),
234         "temp_bounds": [t_lo, t_hi],
235         "temp_within_bounds": bool(temp.min()  $\geq$  t_lo - tol and temp.max()  $\leq$  t_hi + tol),
236         "moist_min": float(moist.min()),
237         "moist_max": float(moist.max()),
238         "moist_bounds": [c_lo, c_hi],
239         "moist_within_bounds": bool(moist.min()  $\geq$  c_lo - tol and moist.max()  $\leq$  c_hi + tol),

```

```

240     "moist_monotone_in_r": bool(np.all(dc <= 1e-8)),
241     "moist_max_positive_slope": float(max(dc.max(), 0.0)),
242     "moist_argmax_is_centre": bool(np.all(moist.max(axis=1) - moist[:, 0] <= 1e-9)), # ties up to roundoff
243 }
244
245
246 def drying_checks(
247     t: np.ndarray, moist: np.ndarray, t_dry: float, dry_profile: np.ndarray, target: float = C_TARGET
248 ) -> dict[str, Any]:
249     """The exact drying instant must separate 'not yet dry' rows from 'dry' rows on the output grid."""
250     max_c = moist.max(axis=1)
251     before = t < t_dry
252     return {
253         "t_dry": float(t_dry),
254         "t_dry_hours": float(t_dry / 3600.0),
255         "profile_max_at_t_dry": float(dry_profile.max()),
256         "profile_max_error_at_t_dry": float(abs(dry_profile.max() - target)),
257         "rows_before_all_wet": bool(np.all(max_c[before] >= target)),
258         "rows_after_all_dry": bool(np.all(max_c[-before] < target)),
259         "last_row_time": float(t[-1]),
260         "last_row_max": float(max_c[-1]),
261     }
262
263
264 # -----
265 # 3. Convergence bookkeeping
266 # -----
267
268
269 def convergence_table(levels: list[dict[str, Any]], key: str) -> list[dict[str, Any]]:
270     """levels: [{"k": k, "dr_cm": ..., key: array}] with arrays sampled on common points; finest is last.
271
272     Two observed orders are reported. ``observed_order`` uses errors against the finest level as the reference,
273     which is what a self-convergence study can measure directly but which is biased upward on the last pair by
274     the factor  $\log_2[(1-4^{-(m-k)})/(1-4^{-(m-k-1)})]$  (2.32 for the last usable pair of a second-order scheme).
275     ``observed_order_triplet`` is the unbiased grid-triplet estimate  $p = \log_2(|u_k - u_{k+1}| / |u_{k+1} - u_{k+2}|)$ ,
276     which needs no reference solution at all (MDR-0011).
277
278     """
279     reference = levels[-1][key]
280     rows: list[dict[str, Any]] = []
281     errors: list[float] = []
282     for level in levels[:-1]:
283         err = float(np.max(np.abs(level[key] - reference)))
284         errors.append(err)
285         rows.append({"k": level["k"], "dr_cm": level["dr_cm"], "max_abs_error": err})
286     diffs = [float(np.max(np.abs(levels[i][key] - levels[i + 1][key]))) for i in range(len(levels) - 1)]
287     for i, row in enumerate(rows):
288         if i + 1 < len(rows) and errors[i + 1] > 0:
289             row["observed_order"] = float(np.log2(errors[i] / errors[i + 1]))
290         else:
291             row["observed_order"] = None
292         if i + 1 < len(diffs) and diffs[i + 1] > 0:
293             row["observed_order_triplet"] = float(np.log2(diffs[i] / diffs[i + 1]))
294         else:
295             row["observed_order_triplet"] = None
296     return rows
297
298
299 def triplet_orders(values: list[float]) -> list[float | None]:
300     """Grid-triplet observed orders  $p_k = \log_2(|u_k - u_{k+1}| / |u_{k+1} - u_{k+2}|)$  of a scalar sequence."""
301     out: list[float | None] = []
302     for i in range(len(values)):
303         if i + 2 < len(values):
304             d1 = float(values[i] - values[i + 1])
305             d2 = float(values[i + 1] - values[i + 2])
306             out.append(float(np.log2(abs(d1 / d2))) if d2 != 0.0 and d1 != 0.0 else None)
307         else:
308             out.append(None)
309     return out
310

```

```

311 def richardson_estimate(coarse_err: float, fine_err: float, order: float = 2.0) -> float:
312     """Error estimate of the fine level from two successive errors against a common finer reference."""
313     return float(abs(fine_err) / (2**order - 1))
314
315
316 # -----
317 # 4. Independent two-dimensional axisymmetric finite-volume model (fixed radius, physical coordinates)
318 # -----
319
320
321 class AxisymmetricSystem:
322     """Vertex-centred FV on (r, z) in [0, R0] x [0, L/2]; z = 0 is the mid-plane (symmetry)."""
323
324     def __init__(self, props: Properties, chamber: Chamber, nr: int, nz: int, end_bc: str = "robin") -> None:
325         self.props = props
326         self.chamber = chamber
327         self.nr, self.nz = nr, nz
328         self.end_bc = end_bc
329         self.dr = R0 / nr
330         self.dz = (LENGTH / 2.0) / nz
331         r = np.linspace(0.0, R0, nr + 1)
332         faces = 0.5 * (r[:-1] + r[1:])
333         self.r_face = faces
334         lower = np.concatenate([[0.0], faces])
335         upper = np.concatenate([faces, [R0]])
336         self.m_r = 0.5 * (upper**2 - lower**2) # radial measure per node
337         self.w_z = np.full(nz + 1, self.dz)
338         self.w_z[0] = self.w_z[-1] = 0.5 * self.dz # half cells at the mid-plane and the end face
339         self.shape = (nr + 1, nz + 1)
340         self.m = (nr + 1) * (nz + 1)
341
342     def __call__(self, t: float, y: np.ndarray) -> np.ndarray:
343         p = self.props
344         temp = y[:, self.m].reshape(self.shape)
345         moist = y[self.m:].reshape(self.shape)
346         t_air = float(self.chamber.air_temperature(t))
347         c_air = float(self.chamber.air_humidity(t))
348         # radial faces (between i and i+1), shape (nr, nz+1)
349         cf = 0.5 * (moist[:-1, :] + moist[1:, :])
350         tf = 0.5 * (temp[:-1, :] + temp[1:, :])
351         fc_r = self.r_face[:, None] * p.diffusivity(cf, tf) * (moist[1:, :] - moist[:-1, :]) / self.dr
352         ft_r = self.r_face[:, None] * p.k(cf) * (temp[1:, :] - temp[:-1, :]) / self.dr
353         # axial faces (between j and j+1), shape (nr+1, nz)
354         cz = 0.5 * (moist[:, :-1] + moist[:, 1:])
355         tz = 0.5 * (temp[:, :-1] + temp[:, 1:])
356         fc_z = p.diffusivity(cz, tz) * (moist[:, 1:] - moist[:, :-1]) / self.dz
357         ft_z = p.k(cz) * (temp[:, 1:] - temp[:, :-1]) / self.dz
358         div_c = np.zeros(self.shape)
359         div_t = np.zeros(self.shape)
360         div_c[:, :-1, :] += fc_r
361         div_c[1:, :] -= fc_r
362         div_t[:, :-1, :] += ft_r
363         div_t[1:, :] -= ft_r
364         div_c[:, -1, :] += -R0 * p.hm * (moist[-1, :] - c_air)
365         div_t[:, -1, :] += -R0 * p.h * (temp[-1, :] - t_air)
366         div_c *= self.w_z[None, :]
367         div_t *= self.w_z[None, :]
368         ax_c = np.zeros(self.shape)
369         ax_t = np.zeros(self.shape)
370         ax_c[:, :-1] += fc_z
371         ax_c[:, 1:] -= fc_z
372         ax_t[:, :-1] += ft_z
373         ax_t[:, 1:] -= ft_z
374         if self.end_bc == "robin":
375             ax_c[:, -1] += -p.hm * (moist[:, -1] - c_air)
376             ax_t[:, -1] += -p.h * (temp[:, -1] - t_air)
377         div_c += ax_c * self.m_r[:, None]
378         div_t += ax_t * self.m_r[:, None]
379         measure = self.m_r[:, None] * self.w_z[None, :]
380         dc = div_c / measure
381         dt = div_t / (measure * p.rho(moist) * p.cp(moist))

```

```

382         return np.concatenate([dt.ravel(), dc.ravel()])
383
384     def jacobian_sparsity(self) -> sparse.csr_matrix:
385         nr1, nz1 = self.shape
386         idx = np.arange(self.m).reshape(self.shape)
387         pairs = [(idx, idx)]
388         pairs.append((idx[:-1, :], idx[1:, :]))
389         pairs.append((idx[1:, :], idx[:-1, :]))
390         pairs.append((idx[:, :-1], idx[:, 1:]))
391         pairs.append((idx[:, 1:], idx[:, :-1]))
392         rows = np.concatenate([a.ravel() for a, _ in pairs])
393         cols = np.concatenate([b.ravel() for _, b in pairs])
394         blocks = []
395         for ro in (0, self.m):
396             for co in (0, self.m):
397                 blocks.append((rows + ro, cols + co))
398         r_all = np.concatenate([b[0] for b in blocks])
399         c_all = np.concatenate([b[1] for b in blocks])
400         return sparse.csr_matrix((np.ones(len(r_all)), (r_all, c_all)), shape=(2 * self.m, 2 * self.m))
401
402
403     def solve_axisymmetric(
404         props: Properties,
405         chamber: Chamber,
406         t_out: np.ndarray,
407         nr: int = 20,
408         nz: int = 25,
409         end_bc: str = "robin",
410         t_init: float = 28.0,
411         c_init: float = 2.55,
412         rtol: float = 1e-7,
413         atol: float = 1e-9,
414         detect_dry: bool = False,
415         method: str = "BDF",
416     ) -> dict[str, Any]:
417         system = AxisymmetricSystem(props, chamber, nr, nz, end_bc)
418         y0 = np.concatenate([np.full(system.m, t_init), np.full(system.m, c_init)])
419
420         def event(t: float, y: np.ndarray) -> float:
421             return float(np.max(y[system.m :]) - C_TARGET)
422
423         event.terminal = False # type: ignore[attr-defined]
424         event.direction = -1 # type: ignore[attr-defined]
425         from pipelines.a.solver import integrate_piecewise # generic ODE driver only (not the radial scheme)
426
427         states, stats, t_dry, _ = integrate_piecewise(
428             system,
429             y0,
430             np.asarray(t_out, dtype=float),
431             chamber.knots(),
432             method=method,
433             rtol=rtol,
434             atol=atol,
435             jac_sparsity=system.jacobian_sparsity(),
436             event=event if detect_dry else None,
437         )
438         temp = states[:, : system.m].reshape(len(t_out), *system.shape)
439         moist = states[:, system.m :].reshape(len(t_out), *system.shape)
440         out: dict[str, Any] = {
441             "t": np.asarray(t_out, dtype=float),
442             "r": np.linspace(0.0, R0, nr + 1),
443             "z": np.linspace(0.0, LENGTH / 2.0, nz + 1),
444             "temp": temp,
445             "moist": moist,
446             "nfev": stats["nfev"],
447             "nlu": stats["nlu"],
448         }
449         if t_dry is not None:
450             out["t_dry"] = t_dry
451         return out

```

pipelines/common/__init__.py

pipelines/common/stages.py

```
1 """Problem-agnostic stages: ingest, lint, test, paper, qa, package, release."""
2
3 from __future__ import annotations
4
5 import importlib
6 import json
7 import os
8 import re
9 import shlex
10 import shutil
11 import subprocess
12 import xml.etree.ElementTree as ET
13 from datetime import UTC, datetime
14 from pathlib import Path
15 from typing import Any
16
17 from forge import packaging, pdfqa, xlsx
18 from forge.context import StageContext
19 from forge.hashing import hash_tree, sha256_file
20 from forge.runner import stage
21 from forge.tex import aux_page, compile_tex, tex_escape, tex_path
22
23 PLANNED_STAGE_FILES = {
24     "lint": ["ruff_check.txt", "ruff_format.txt", "mypy.txt", "lint_report.json", "manifest.json", "events.jsonl"],
25     "test": ["junit.xml", "coverage.json", "pytest.txt", "test_report.json", "manifest.json", "events.jsonl"],
26     "qa": ["qa_report.json", "manifest.json", "events.jsonl"],
27 }
28 _BOOT_STAGES = ("ingest", "validate", "fmt")
29 MEMBER_DESCRIPTIONS = (
30     (r"~forge/plotting\.py$", "绘图规范 (矢量 PDF + 预览 PNG、色板、字体)"),
31     (r"~forge/xlsx\.py$", "从组委会模板生成结果工作簿"),
32     (r"~forge/runner\.py$", "阶段注册与执行 (依赖、清单、错误捕获)"),
33     (r"~forge/modal_app\.py$", "云端应用定义 (镜像、卷、四种规格的执行函数)"),
34     (r"~forge/", "云端运行时 (阶段执行、清单、契约、排版、打包)"),
35     (r"~pipelines/common/", "通用阶段: 数据接入、门禁、论文、质量检查、打包发布"),
36     (r"~pipelines/a/physics\.py$", "题给物性与经验公式 (附录 2/3/4)"),
37     (r"~pipelines/a/solver\.py$", "径向有限体积分散与隐式时间积分"),
38     (r"~pipelines/a/science\.py$", "四个问题的求解编排与输出采样"),
39     (r"~pipelines/a/verify\.py$", "与求解器分离的独立校验 (解析解、守恒、收敛阶)"),
40     (r"~pipelines/a/derived\.py$", "论文派生量的集中计算"),
41     (r"~pipelines/a/outputs\.py$", "结果工作簿的生成"),
42     (r"~pipelines/a/figures\.py$", "论文图与表的生成"),
43     (r"~pipelines/a/stages\.py$", "本题各阶段的编排"),
44     (r"~pipelines/", "本题建模与求解程序"),
45     (r"~tools/run_local\.py$", "不依赖云端的本地驱动脚本"),
46     (r"~tools/", "本地控制面: 提交云端任务、下载、校验散列"),
47     (r"~tests/", "单元 / 性质 / 集成测试"),
48     (r"~configs/", "运行配置与项目元数据"),
49     (r"~manuscript/", "论文 LaTeX 源码"),
50     (r"~docs/mdr/", "建模决策记录 (题意解释、假设、目标与方法的选择理由)"),
51     (r"~docs/ARCHITECTURE\.md$", "仓库架构与唯一计算面的设计"),
52     (r"~docs/CONTRIBUTING\.md$", "协作与提交规范"),
53     (r"~docs/DATA_NOTES\.md$", "数据口径与附件解析约定"),
54     (r"~docs/ENGINEERING_STANDARD\.md$", "工程标准与门禁定义"),
55     (r"~docs/MODELING_STANDARD\.md$", "建模与写作标准 (审稿清单)"),
56     (r"~docs/RESULTS\.md$", "本轮结果总览与逐项证据"),
57     (r"~docs/RUNBOOK\.md$", "运行手册 (完整命令序列与实测用时)"),
58     (r"~docs/REVIEW_RESPONSE\.md$", "评审意见的逐条处理说明"),
59     (r"~docs/problem_brief\.md$", "题目要点与交付清单"),
60     (r"~docs/", "工程规范、建模标准、决策记录"),
61     (r"/ingest/", "原始数据接入结果 (散列、题目文本、规范化数据)"),
62     (r"/validate/", "数据契约校验报告与数据画像"),
```

```

63     (r"/results/", "结果工作簿与求解输出"),
64     (r"/figures/", "论文图 (矢量 PDF 与预览 PNG) "),
65     (r"/tables/", "论文表 (LaTeX 片段与 CSV) "),
66     (r"/lint/", "静态检查报告"),
67     (r"/test/", "测试报告与覆盖率"),
68     (r"/qa/", "论文与结果文件质量门禁报告"),
69     (r"manifest\.json$", "阶段清单 (输入输出散列、环境、参数) "),
70     (r"events\.jsonl$", "结构化运行日志"),
71     (r"^AI工具使用详情\.pdf$", "AI 工具使用详情"),
72     (r"^AI_USAGE\.md$", "AI 工具使用记录 (源文件) "),
73     (r"^Makefile$", "本地快捷命令 (只提交云端任务) "),
74     (r"^pyproject\.toml$", "项目元数据与静态检查配置"),
75     (r"^README\.md$", "仓库说明与复现入口"),
76     (r"^CHANGELOG\.md$", "变更记录"),
77     (r"^UNLICENSE$", "公有领域声明 (Unlicense) "),
78     (r"^REPRODUCE\.md$", "复现说明"),
79     (r"^MANIFEST\.json$", "支撑材料成员清单与 SHA-256"),
80     (r"numbers\.json$", "该阶段登记的论文数字宏"),
81     (r"validation_report\.json$", "该阶段结果的独立校验报告"),
82     (r"\.parquet$", "规范化后的输入数据"),
83     (r"\.json$", "阶段数据产物"),
84     (r"\.csv$", "表格数据 (与 LaTeX 表同源) "),
85 )
86
87
88 def _describe(name: str) -> str:
89     for pattern, text in MEMBER_DESCRIPTIONS:
90         if re.search(pattern, name):
91             return text
92     return ""
93
94
95 def _to_parquet(df: Any, target: Path) -> None:
96     frame = df.copy()
97     frame.columns = [str(c).strip() for c in frame.columns]
98     try:
99         frame.to_parquet(target, index=False)
100     except Exception:
101         for col in frame.columns:
102             if frame[col].dtype == object:
103                 frame[col] = frame[col].map(
104                     lambda v: None if v is None or (isinstance(v, float) and v != v) else str(v)
105                 )
106         frame.to_parquet(target, index=False)
107
108
109 @stage("ingest", description="Hash raw inputs, extract the problem text, convert every attachment sheet to parquet")
110 def ingest(ctx: StageContext) -> dict[str, Any]:
111     import fitz
112
113     document = fitz.open(ctx.problem_pdf)
114     text = "\n\n".join(page.get_text("text", sort=True) for page in document)
115     ctx.out("problem_text.txt").write_text(text, encoding="utf-8")
116     inventory: dict[str, Any] = {
117         "problem_pdf": {
118             "path": ctx.project["problem_pdf"],
119             "sha256": sha256_file(ctx.problem_pdf),
120             "pages": len(document),
121         },
122         "files": [],
123     }
124     for path in sorted(ctx.attachments.rglob("*.xlsx")):
125         if path.name.startswith("~$"):
126             continue
127         rel = path.relative_to(ctx.attachments).as_posix()
128         is_template = path.parent == ctx.templates or ctx.templates in path.parents
129         entry: dict[str, Any] = {
130             "path": rel,
131             "sha256": sha256_file(path),
132             "bytes": path.stat().st_size,
133             "template": is_template,

```

```

134         "sheets": [],
135     }
136     for name, df in xlsx.read_sheets(path).items():
137         sheet: dict[str, Any] = {"name": name, "rows": len(df), "columns": [str(c) for c in df.columns]}
138         if not is_template:
139             safe = re.sub(r"[^\\w.-]+", "_", f"{path.stem}_{name}")
140             target = ctx.out("data", f"{safe}.parquet")
141             _to_parquet(df, target)
142             sheet["parquet"] = target.relative_to(ctx.stage_dir).as_posix()
143             sheet["dtypes"] = {str(c): str(t) for c, t in df.dtypes.items()}
144             sheet["head"] = df.head(5).astype(str).values.tolist()
145             entry["sheets"].append(sheet)
146             inventory["files"].append(entry)
147             ctx.log.info("ingest.file", path=rel, template=is_template, sheets=[s["name"] for s in entry["sheets"]])
148         ctx.write_json("inventory.json", inventory)
149     return {"files": len(inventory["files"]), "problem_chars": len(text)}
150
151
152 @stage("lint", description="Static gates: ruff check, ruff format --check, mypy")
153 def lint(ctx: StageContext) -> dict[str, Any]:
154     returncodes: dict[str, int] = {}
155
156     def run(name: str, cmd: list[str]) -> None:
157         proc = subprocess.run(cmd, cwd=str(ctx.repo), capture_output=True, text=True, check=False)
158         ctx.out(f"{name}.txt").write_text(proc.stdout + ("\n" + proc.stderr if proc.stderr else ""), encoding="utf-8")
159         returncodes[name] = proc.returncode
160         ctx.log.info("lint.tool", tool=name, rc=proc.returncode, tail=(proc.stdout + proc.stderr)[-400:])
161
162     run("ruff_check", ["ruff", "check", "--no-cache", "--output-format", "concise", "."])
163     run("ruff_format", ["ruff", "format", "--check", "--no-cache", "."])
164     run("mypy", ["mypy", "--cache-dir", "/tmp/mypy", *ctx.cfg("gates.mypy_paths", ["forge"])]])
165     required = list(ctx.cfg("gates.lint_required", ["ruff_check", "ruff_format", "mypy"]))
166     passed = all(returncodes.get(name, 1) == 0 for name in required)
167     ctx.write_json("lint_report.json", {"passed": passed, "required": required, "returncodes": returncodes})
168     if not passed:
169         raise RuntimeError(f"lint gate failed: {returncodes}")
170     return {"passed": passed, **returncodes}
171
172
173 @stage("test", description="pytest (unit, property, integration) with JUnit and coverage reports")
174 def test(ctx: StageContext) -> dict[str, Any]:
175     env = {
176         **os.environ,
177         "COVERAGE_FILE": "/tmp/.coverage",
178         "HYPOTHESIS_STORAGE_DIRECTORY": "/tmp/hypothesis",
179         "FORGE_RUN_DIR": str(ctx.run_dir),
180     }
181     cmd = [
182         "python",
183         "-m",
184         "pytest",
185         "tests",
186         "-q",
187         "-p",
188         "no:cacheprovider",
189         "--rootdir",
190         str(ctx.repo),
191         f"--junitxml={ctx.out('junit.xml')}",
192         "--cov=forge",
193         "--cov=pipelines",
194         f"--cov-report=json:{ctx.out('coverage.json')}",
195         "--cov-report=term",
196     ]
197     extra = ctx.param("pytest_args")
198     if extra:
199         cmd += shlex.split(str(extra))
200     proc = subprocess.run(cmd, cwd=str(ctx.repo), capture_output=True, text=True, env=env, check=False)
201     ctx.out("pytest.txt").write_text(proc.stdout + "\n" + proc.stderr, encoding="utf-8")
202     print(proc.stdout[-5000:], flush=True)
203     summary: dict[str, Any] = {"returncode": proc.returncode}
204     junit = ctx.stage_dir / "junit.xml"

```

```

205     if junit.exists():
206         root = ET.parse(junit).getroot()
207         suite = root if root.tag == "testsuite" else root.find("testsuite")
208         if suite is not None:
209             summary.update({k: int(suite.get(k, 0)) for k in ("tests", "failures", "errors", "skipped")})
210     coverage = ctx.stage_dir / "coverage.json"
211     if coverage.exists():
212         summary["coverage_percent"] = round(json.loads(coverage.read_text())["totals"]["percent_covered"], 2)
213     ctx.write_json("test_report.json", summary)
214     if proc.returncode != 0:
215         raise RuntimeError(f"tests failed: {summary}\n{proc.stdout[-2500:]}")
216     return summary
217
218
219 def _support_members(ctx: StageContext, *, planned: bool) -> list[tuple[str, Path]]:
220     members = packaging.code_members(ctx.repo, ctx.cfg("package.code_globs", []))
221     for name in ctx.cfg("package.stages", []):
222         directory = ctx.run_dir / name
223         if (directory / "manifest.json").exists():
224             members += packaging.run_members(ctx.run_dir, [name])
225         elif planned and name in PLANNED_STAGE_FILES:
226             members += [(f"runs/{ctx.run_id}/{name}/{f}", directory / f) for f in PLANNED_STAGE_FILES[name]]
227     members.append(("AI工具使用详情.pdf", ctx.run_dir / "paper" / "AI工具使用详情.pdf"))
228     members.append(("REPRODUCE.md", ctx.run_dir / "package" / "REPRODUCE.md"))
229     members.append(("MANIFEST.json", ctx.run_dir / "package" / "MANIFEST.json"))
230     return sorted(dict(members).items())
231
232
233 def _load_numbers(run_dir: Path) -> tuple[dict[str, str], list[str]]:
234     numbers: dict[str, str] = {}
235     conflicts: list[str] = []
236     for path in sorted(run_dir.glob("*/numbers.json")):
237         for key, value in json.loads(path.read_text(encoding="utf-8")).items():
238             if key in numbers and numbers[key] != value:
239                 conflicts.append(f"{key}: {numbers[key]!r} vs {value!r} ({path.parent.name})")
240             numbers[key] = value
241     return numbers, conflicts
242
243
244 def _stage_rows(ctx: StageContext) -> list[dict[str, Any]]:
245     """Status of every stage of this run. The stage that is *writing* this table is necessarily still running;
246     report it as completed-in-progress so the record does not read as an unfinished pipeline."""
247     rows = []
248     for manifest in sorted(ctx.run_dir.glob("*/manifest.json")):
249         data = json.loads(manifest.read_text(encoding="utf-8"))
250         status = data["status"]
251         if status == "running" and data["stage"] == ctx.stage:
252             status = "completed*"
253         rows.append(
254             {
255                 "stage": data["stage"],
256                 "status": status,
257                 "duration_s": data.get("duration_s"),
258                 "outputs_digest": data.get("outputs_digest", ""),
259                 "modal_task_id": data.get("runtime", {}).get("modal_task_id"),
260             }
261         )
262     return rows
263
264
265 @stage(
266     "paper",
267     description="Assemble generated inputs (numbers, tables, figures, code, support list) and compile with XeLaTeX",
268 )
269 def paper(ctx: StageContext) -> dict[str, Any]:
270     sources = list(ctx.cfg("paper.sources", []))
271     for source in sources:
272         ctx.dep(source)
273     build = Path("/tmp/build")
274     shutil.rmtree(build, ignore_errors=True)
275     shutil.copytree(

```

```

276     ctx.repo / "manuscript", build, ignore=shutil.ignore_patterns("generated", "*.aux", "*.log", "*.pdf")
277 )
278 generated = build / "generated"
279 (generated / "tables").mkdir(parents=True, exist_ok=True)
280 (build / "figures").mkdir(exist_ok=True)
281
282 numbers, conflicts = _load_numbers(ctx.run_dir)
283 for conflict in conflicts:
284     ctx.log.warn("paper.number_conflict", detail=conflict)
285 lines = ["% generated by forge — do not edit"] + [
286     f"\newcommand{{\val{key}}}{{\tex_escape(value)}}}" for key, value in sorted(numbers.items())
287 ]
288 (generated / "numbers.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")
289
290 n_tables = n_figures = 0
291 for source in sources:
292     directory = ctx.run_dir / source
293     for table in sorted((directory / "tables").glob("*.tex")) if (directory / "tables").is_dir() else []:
294         shutil.copy2(table, generated / "tables" / table.name)
295         n_tables += 1
296     for figure in sorted((directory / "figures").glob("*.png")) if (directory / "figures").is_dir() else []:
297         if figure.suffix.lower() in {".pdf", ".png"}:
298             shutil.copy2(figure, build / "figures" / figure.name)
299             n_figures += 1
300
301 code_ref = ctx.params.get("code_ref") or {}
302 meta = [
303     f"\newcommand{{\RunId}}{{\tex_escape(ctx.run_id)}}",
304     f"\newcommand{{\CodeSha}}{{\tex_escape(str(code_ref.get('git_sha', ''))}}",
305     f"\newcommand{{\BuildDate}}{{\datetime.now(UTC).strftime('%Y-%m-%d')}}",
306 ]
307 (generated / "meta.tex").write_text("\n".join(meta) + "\n", encoding="utf-8")
308
309 files = packaging.code_members(ctx.repo, ctx.cfg("paper.code_globs", []))
310 listing = ["% generated by forge — complete source listing"]
311 for rel, src in files:
312     target = build / "code" / rel
313     target.parent.mkdir(parents=True, exist_ok=True)
314     shutil.copy2(src, target)
315     language = {"py": "Python", ".tex": "[LaTeX]TeX"}.get(src.suffix, "{}")
316     listing.append(f"\needspace{{6\baselineskip}}\subsection*{{\tex_path(rel)}}")
317     listing.append(f"\lstinputlisting[language={language}][{code}/{rel}]")
318 (generated / "code_listing.tex").write_text("\n".join(listing) + "\n", encoding="utf-8")
319
320 members = _support_members(ctx, planned=True)
321 rows = [
322     "% generated by forge — support material file list",
323     "\begin{longtable}{{@{}p{0.64\linewidth}p{0.32\linewidth}@{}}",
324     "\toprule 文件 & 说明 \\\midrule \\\endhead \\\bottomrule \\\endfoot",
325 ]
326 rows += [f"{{\tex_path(name)}} & {{\tex_escape(_describe(name))}} \\\\" for name, _ in members]
327 rows.append("\end{longtable}")
328 (generated / "support_files.tex").write_text("\n".join(rows) + "\n", encoding="utf-8")
329 ctx.write_json("support_members.json", [name for name, _ in members])
330
331 stage_rows = _stage_rows(ctx)
332 repro = [
333     "% generated by forge — reproducibility record",
334     "\begin{longtable}{{@{}llrl@{}}",
335     "\toprule 阶段 & 状态 & 用时/s & 输出摘要 (SHA-256 前 12 位) \\\midrule \\\endhead \\\bottomrule \\\endfoot",
336 ]
337 repro += [
338     f"\texttt{{\tex_escape(r['stage'])}} & {{r['status']}} & {{r['duration_s']}} or 0:.1f & "
339     f"\texttt{{r['outputs_digest'][:12]}} \\\\"
340     for r in stage_rows
341 ]
342 repro.append("\end{longtable}")
343 (generated / "repro_table.tex").write_text("\n".join(repro) + "\n", encoding="utf-8")
344
345 result = compile_tex(build, str(ctx.cfg("paper.main", "main.tex")), "main", passes=3, bibtex=True)
346 shutil.copy2(result["pdf"], ctx.out("main.pdf"))

```

```

347 shutil.copy2(result["log"], ctx.out("main.log"))
348 if result["aux"].exists():
349     shutil.copy2(result["aux"], ctx.out("main.aux"))
350 body_end = aux_page(result["aux"], "page:body-end")
351 abstract_end = aux_page(result["aux"], "page:abstract-end")
352 ai_summary = None
353 if (build / "ai_usage.tex").exists():
354     ai = compile_tex(build, "ai_usage.tex", "ai_usage", passes=2, bibtex=False)
355     shutil.copy2(ai["pdf"], ctx.out("AI工具使用详情.pdf"))
356     ai_summary = ai["summary"]
357 info = pdfqa.inspect_pdf(ctx.out("main.pdf"), with_text=False)
358 body_pages = (body_end - abstract_end) if body_end and abstract_end else None
359 report = {
360     "pages": info["pages"],
361     "all_a4": info["all_a4"],
362     "abstract_end_page": abstract_end,
363     "body_end_page": body_end,
364     "body_pages": body_pages,
365     "log": result["summary"],
366     "unembedded_fonts": info["unembedded_fonts"],
367     "fonts": info["fonts"],
368     "numbers": len(numbers),
369     "number_conflicts": conflicts,
370     "tables": n_tables,
371     "figures": n_figures,
372     "code_files": len(files),
373     "support_members": len(members),
374     "ai_usage_log": ai_summary,
375     "stages": stage_rows,
376 }
377 ctx.write_json("paper_report.json", report)
378 summary = result["summary"]
379 return {
380     "pages": info["pages"],
381     "body_pages": body_pages,
382     "undefined_refs": len(summary["undefined_references"]) + len(summary["undefined_citations"]),
383     "overfull_max_pt": summary["overfull_max_pt"],
384     "missing_chars": summary["missing_characters"],
385     "figures": n_figures,
386     "tables": n_tables,
387 }
388
389
390 @stage("qa", deps=("paper",), description="Quality gates over the compiled paper and the result workbooks")
391 def qa(ctx: StageContext) -> dict[str, Any]:
392     for required in ctx.cfg("qa.required_stages", ["lint", "test"]):
393         ctx.dep(required)
394     paper_dir = ctx.dep("paper")
395     report = json.loads((paper_dir / "paper_report.json").read_text(encoding="utf-8"))
396     checks: list[dict[str, Any]] = []
397
398     def check(name: str, ok: bool, detail: Any = "") -> None:
399         checks.append({"name": name, "ok": bool(ok), "detail": detail})
400         ctx.log.info("qa.check", name=name, ok=bool(ok), detail=detail)
401
402     max_body = int(ctx.cfg("paper.max_body_pages", 30))
403     check("abstract_single_page", report["abstract_end_page"] == 1, report["abstract_end_page"])
404     check(
405         "body_pages_within_limit",
406         report["body_pages"] is not None and report["body_pages"] <= max_body,
407         f"{report['body_pages']} <= {max_body}",
408     )
409     log = report["log"]
410     check(
411         "no_undefined_references",
412         not log["undefined_references"] and not log["undefined_citations"],
413         {"refs": log["undefined_references"][:10], "cites": log["undefined_citations"][:10]},
414     )
415     check("no_tex_errors", not log["errors"], log["errors"][:5])
416     check("fonts_embedded", not report["unembedded_fonts"], report["unembedded_fonts"])
417     check("a4_pages", report["all_a4"])

```

```

418     check(
419         "overfull_within_limit",
420         log["overfull_max_pt"] <= float(ctx.cfg("paper.max_overfull_pt", 20.0)),
421         f"max {log['overfull_max_pt']} pt, count {log['overfull_count']}",
422     )
423     check("no_missing_characters", log["missing_characters"] == 0, log["missing_character_samples"])
424     check("no_number_conflicts", not report["number_conflicts"], report["number_conflicts"][:5])
425     check("ai_usage_pdf_present", (paper_dir / "AI工具使用详情.pdf").exists())
426
427     info = pdfqa.inspect_pdf(paper_dir / "main.pdf", with_text=True)
428     deny_hits = pdfqa.scan_strings(info["texts"], ctx.cfg("qa.identity_denylist", []))
429     watch_hits = pdfqa.scan_strings(info["texts"], ctx.cfg("qa.identity_watchlist", []))
430     check("identity_denylist_clean", not deny_hits, deny_hits[:5])
431     check("paper_size_under_20mb", (paper_dir / "main.pdf").stat().st_size <= 20 * 1024 * 1024)
432
433     results_reports: list[dict[str, Any]] = []
434     results_stage = str(ctx.cfg("qa.results_stage", "results"))
435     require_results = bool(ctx.param("require_results", ctx.cfg("qa.require_results", True)))
436     if ctx.has_stage(results_stage):
437         contracts = importlib.import_module(f"pipelines.{ctx.project['package']}.contracts")
438         results_dir = ctx.run_dir / results_stage
439         for contract in contracts.RESULT_CONTRACTS:
440             rep = xlsx.check_workbook(results_dir / contract.file, contract, ctx.templates / contract.template)
441             results_reports.append(rep)
442             check(f"result:{contract.file}", rep["ok"], rep["errors"][:5])
443     else:
444         check("results_stage_present", not require_results, f"stage '{results_stage}' not completed in this run")
445
446     passed = all(c["ok"] for c in checks)
447     ctx.write_json(
448         "qa_report.json",
449         {
450             "passed": passed,
451             "checks": checks,
452             "identity_watchlist_hits": watch_hits,
453             "results": results_reports,
454             "paper": {k: v for k, v in report.items() if k != "fonts"},
455         },
456     )
457     if not passed:
458         raise RuntimeError("QA gate failed: " + ", ".join(c["name"] for c in checks if not c["ok"]))
459     return {"passed": passed, "checks": len(checks), "watchlist_hits": len(watch_hits)}
460
461
462 def _reproduce_md(ctx: StageContext) -> str:
463     rows = _stage_rows(ctx)
464     code_ref = ctx.params.get("code_ref") or {}
465     manifest = json.loads((ctx.run_dir / "ingest" / "manifest.json").read_text(encoding="utf-8"))
466     packages = manifest.get("runtime", {}).get("packages", {})
467     lines = [
468         f"# 复现说明 (run `{ctx.run_id}`) ",
469         "",
470         "本目录内所有结果均由 Modal 云端按下列阶段生成; 本地只提交任务、下载和核对 SHA-256。",
471         "",
472         f"- 代码版本: git `{code_ref.get('git_sha', '')}` (分支 `{code_ref.get('branch', '')}`) ",
473         f"- 运行环境: Python {manifest.get('runtime', {}).get('python', '')}, "
474         f"{manifest.get('runtime', {}).get('platform', '')}",
475         f"- 依赖版本: " + ", ".join(f"{k}=={v}" for k, v in sorted(packages.items())),
476         "",
477         "## 阶段",
478         "",
479         "| 阶段 | 状态 | 用时 (s) | 输出摘要 | Modal 任务 |",
480         "|---|---|---:|---|---|",
481     ]
482     lines += [
483         f"| {r['stage']} | {r['status']} | {r['duration_s'] or 0:.1f} | {r['outputs_digest'][:16]} | "
484         f"{r['modal_task_id'] or ''} |"
485         for r in rows
486     ]
487     lines += [
488         "",

```

```

489     """ 复现命令",
490     "",
491     "### A. 云端 (本文使用的方式, 需要 Modal 账号) ",
492     "",
493     "```bash",
494     "python3 tools/cli.py provision",
495     "python3 tools/cli.py run ingest,validate --new-run",
496     *[f"python3 tools/cli.py run {r['stage']} --size medium" for r in rows if r["stage"] not in _BOOT_STAGES],
497     "python3 tools/cli.py release --version <tag>",
498     "```",
499     "",
500     "各阶段的确切参数 (线程数、逐级时限) 见每个阶段目录下 `manifest.json` 的 `params` 字段, ",
501     "以及 `docs/RUNBOOK.md` 第 7 节。",
502     "",
503     "### B. 本地 (不需要任何云端账号) ",
504     "",
505     "```bash",
506     "pip install pandas numpy openpyxl pyarrow networkx matplotlib ortools highspy",
507     "python3 tools/run_local.py --quick          # 数分钟, 得到 result1-4.xlsx、图与表",
508     "python3 tools/run_local.py                  # 使用默认 (较大) 预算",
509     "```",
510     "",
511     "本地驱动脚本调用的是同一批阶段函数, 只是把云端卷换成本地目录; ",
512     "附件需按原样放在 `附件/` 下。论文排版阶段 (paper/qa/package/release) 需要 TeX Live 与中文字体, ",
513     "不在本地默认序列中。",
514     "",
515     "每个阶段目录下的 `manifest.json` 记录输入、输出散列、参数与环境; `events.jsonl` 为结构化日志。",
516     "",
517 ]
518 return "\n".join(lines)
519
520
521 @stage("package", deps=("qa",), description="Build the deterministic support-materials zip and its member manifest")
522 def package(ctx: StageContext) -> dict[str, Any]:
523     planned = set(json.loads((ctx.dep("paper") / "support_members.json").read_text(encoding="utf-8")))
524     ctx.out("REPRODUCE.md").write_text(_reproduce_md(ctx), encoding="utf-8")
525     members = [(name, path) for name, path in _support_members(ctx, planned=False) if name != "MANIFEST.json"]
526     missing = [name for name, path in members if not path.exists()]
527     if missing:
528         raise RuntimeError(f"support members missing on disk: {missing[:10]}")
529     actual = {name for name, _ in members} | {"MANIFEST.json"}
530     drift = sorted(actual ^ planned)
531     if drift and not ctx.param("allow_member_drift"):
532         raise RuntimeError(f"support material members differ from the list printed in the paper: {drift[:20]}")
533     manifest = {
534         "run_id": ctx.run_id,
535         "created_at": datetime.now(UTC).isoformat(timespec="seconds"),
536         "members": [
537             {"path": name, "sha256": sha256_file(path), "bytes": path.stat().st_size} for name, path in members
538         ],
539     }
540     manifest_path = ctx.out("MANIFEST.json")
541     manifest_path.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
542     members.append(("MANIFEST.json", manifest_path))
543     zip_info = packaging.build_zip(members, ctx.out("支撑材料.zip"), comment=f"forge run {ctx.run_id}")
544     limit = int(ctx.cfg("package.max_bytes", 20 * 1024 * 1024))
545     ctx.write_json("package_report.json", {**zip_info, "limit_bytes": limit, "drift": drift})
546     if zip_info["bytes"] > limit:
547         raise RuntimeError(f"support zip {zip_info['bytes']} bytes exceeds limit {limit}")
548     return {"members": zip_info["members"], "bytes": zip_info["bytes"], "drift": len(drift)}
549
550
551 @stage(
552     "release",
553     deps=("package",),
554     description="Assemble deliverables (paper, result workbooks, support zip) with SHA-256 manifest",
555 )
556 def release(ctx: StageContext) -> dict[str, Any]:
557     qa_report = json.loads((ctx.dep("qa") / "qa_report.json").read_text(encoding="utf-8"))
558     if not qa_report["passed"] and not ctx.param("allow_failed_qa"):
559         raise RuntimeError("release refused: QA gate did not pass")

```

```

560 paper_dir = ctx.dep("paper")
561 files: dict[str, Path] = {str(ctx.cfg("release.paper_name", "论文.pdf")): paper_dir / "main.pdf"}
562 results_stage = str(ctx.cfg("qa.results_stage", "results"))
563 if ctx.has_stage(results_stage):
564     for name in ctx.project.get("result_files", []):
565         files[name] = ctx.run_dir / results_stage / name
566 elif not ctx.param("allow_missing_results"):
567     raise RuntimeError("release refused: results stage missing")
568 files["支撑材料.zip"] = ctx.dep("package") / "支撑材料.zip"
569 files["AI工具使用详情.pdf"] = paper_dir / "AI工具使用详情.pdf"
570 manifest: dict[str, Any] = {
571     "run_id": ctx.run_id,
572     "code_ref": ctx.params.get("code_ref"),
573     "files": {},
574     "qa_passed": qa_report["passed"],
575     "created_at": datetime.now(UTC).isoformat(timespec="seconds"),
576 }
577 for name, source in files.items():
578     if not source.exists():
579         raise RuntimeError(f"release file missing: {name} ({source})")
580     target = ctx.out(name)
581     shutil.copy2(source, target)
582     manifest["files"][name] = {"sha256": sha256_file(target), "bytes": target.stat().st_size}
583 notes = [
584     f"# 发布说明 — run {ctx.run_id}",
585     "",
586     f"- 代码: `{(ctx.params.get('code_ref') or {}).get('git_sha', '')}`",
587     f"- QA: {'通过' if qa_report['passed'] else '未通过 (强制发布)'}",
588     "",
589     "| 文件 | 字节 | SHA-256 |",
590     "|---|---:|---|",
591 ]
592 notes += [f"| {n} | {m['bytes']} | `{m['sha256']}`" for n, m in manifest["files"].items()]
593 ctx.out("RELEASE_NOTES.md").write_text("\n".join(notes) + "\n", encoding="utf-8")
594 ctx.write_json("release_manifest.json", manifest)
595 return {
596     "files": len(files),
597     "paper_bytes": manifest["files"][str(ctx.cfg("release.paper_name", "论文.pdf"))]["bytes"],
598 }
599
600
601 FMT_IGNORE = (".git", "artifacts", "releases", "__pycache__", ".forge", ".DS_Store", ".mypy_cache", ".ruff_cache")
602
603
604 @stage("fmt", description="ruff --fix and ruff format in the cloud; changed files are exported for a local copy-back")
605 def fmt(ctx: StageContext) -> dict[str, Any]:
606     work = Path("/tmp/fmt")
607     shutil.rmtree(work, ignore_errors=True)
608     shutil.copytree(ctx.repo, work, ignore=shutil.ignore_patterns(*FMT_IGNORE, "*.pyc"))
609     before = hash_tree(work, ignore=FMT_IGNORE)
610     fix = subprocess.run(
611         ["ruff", "check", "--fix", "--no-cache", "--output-format", "concise", "."],
612         cwd=work,
613         capture_output=True,
614         text=True,
615         check=False,
616     )
617     form = subprocess.run(["ruff", "format", "--no-cache", "."], cwd=work, capture_output=True, text=True, check=False)
618     after = hash_tree(work, ignore=FMT_IGNORE)
619     changed = sorted(rel for rel in after if rel not in before or after[rel]["sha256"] != before[rel]["sha256"])
620     for rel in changed:
621         shutil.copy2(work / rel, ctx.out("files", rel))
622     ctx.out("ruff_fix.txt").write_text(fix.stdout + fix.stderr, encoding="utf-8")
623     ctx.out("ruff_format.txt").write_text(form.stdout + form.stderr, encoding="utf-8")
624     ctx.write_json("fmt_report.json", {"changed": changed, "remaining": fix.stdout.strip().splitlines()[-1:]})
625     ctx.log.info("fmt.done", changed=len(changed), remaining=fix.stdout.strip().splitlines()[-1:])
626     return {"changed": len(changed)}

```

pipelines/common/validation.py

```
1 """Shared implementation of the ``validate`` stage (input data contracts + profiling)."""
2
3 from __future__ import annotations
4
5 from typing import Any
6
7 import pandas as pd
8
9 from forge.context import StageContext
10 from forge.contracts import FrameContract, normalise_columns, validate_frame
11
12
13 def run_validation(ctx: StageContext, contracts: dict[str, FrameContract]) -> dict[str, Any]:
14     """Validate every ingested parquet named in ``contracts`` (key = parquet stem)."""
15     data_dir = ctx.dep("ingest") / "data"
16     reports: list[dict[str, Any]] = []
17     profile: dict[str, Any] = {}
18     for stem, contract in contracts.items():
19         path = data_dir / f"{stem}.parquet"
20         if not path.exists():
21             reports.append({"contract": contract.name, "ok": False, "errors": [f"parquet missing: {stem}"], "rows": 0})
22             continue
23         df = normalise_columns(pd.read_parquet(path))
24         report = validate_frame(df, contract)
25         report["parquet"] = path.name
26         reports.append(report)
27         desc = df.describe(include="all").astype(str).to_dict()
28         profile[stem] = {
29             "rows": len(df),
30             "columns": list(df.columns),
31             "describe": desc,
32             "nulls": {c: int(df[c].isna().sum()) for c in df.columns},
33         }
34         ctx.log.info(
35             "validate.frame", contract=contract.name, ok=report["ok"], rows=report["rows"], errors=report["errors"][:5]
36         )
37     ok = all(r["ok"] for r in reports)
38     ctx.write_json("validation_report.json", {"ok": ok, "frames": reports})
39     ctx.write_json("data_profile.json", profile)
40     if not ok:
41         failed = [r["contract"] for r in reports if not r["ok"]]
42         raise RuntimeError(f"input contracts violated: {failed}")
43     return {"ok": ok, "frames": len(reports)}
```

pyproject.toml

```
1 [project]
2 name = "cumcm2026-a"
3 version = "0.1.0"
4 description = "药材的烘干问题 — cloud-only reproducible modelling study (CUMCM 2026 Problem A)"
5 requires-python = ">=3.11"
6 license = "Unlicense"
7
8 [tool.ruff]
9 line-length = 120
10 target-version = "py311"
11 extend-exclude = ["artifacts", "releases", "manuscript"]
12
13 [tool.ruff.lint]
14 select = ["E", "F", "W", "I", "B", "UP", "NPY", "RUF", "S102"]
15 ignore = ["RUF001", "RUF002", "RUF003", "E741", "B905"]
16
17 [tool.ruff.lint.per-file-ignores]
18 "tests/**" = ["S101", "B011"]
19
```

```

20 [tool.mypy]
21 python_version = "3.11"
22 ignore_missing_imports = true
23 check_untyped_defs = true
24 warn_unused_ignores = false
25 no_implicit_optional = true
26 exclude = "(artifacts|releases|manuscript)/"
27
28 [tool.pytest.ini_options]
29 testpaths = ["tests"]
30 addopts = "-p no:cacheprovider -q"
31 markers = ["integration: needs the mounted repository inputs (runs in Modal)"]
32
33 [tool.coverage.run]
34 source = ["forge", "pipelines"]

```

tests/__init__.py

tests/conftest.py

```

1  """Shared test configuration: deterministic hypothesis profile, no on-disk example database."""
2
3  from __future__ import annotations
4
5  import os
6  from pathlib import Path
7
8  import pytest
9  from hypothesis import HealthCheck, settings
10
11  settings.register_profile(
12      "forge", database=None, deadline=None, max_examples=60, suppress_health_check=[HealthCheck.too_slow]
13  )
14  settings.load_profile("forge")
15
16  REPO = Path(os.environ.get("FORGE_REPO", Path(__file__).resolve().parents[1]))
17
18
19  @pytest.fixture(scope="session")
20  def repo() -> Path:
21      return REPO
22
23
24  @pytest.fixture
25  def tmp_repo(tmp_path: Path) -> Path:
26      """A minimal repository layout (configs only) for runtime tests that must not touch real data."""
27      (tmp_path / "configs").mkdir()
28      (tmp_path / "configs" / "default.toml").write_text("[run]\nseed = 7\n[paper]\nsources = []\n", encoding="utf-8")
29      (tmp_path / "configs" / "project.json").write_text(
30          '{ "letter": "T", "package": "t", "problem_pdf": "T.pdf", "attachments_dir": "att", '
31          '"templates_dir": "att/tpl", "result_files": []}',
32          encoding="utf-8",
33      )
34      (tmp_path / "att" / "tpl").mkdir(parents=True)
35      return tmp_path

```

tests/integration/__init__.py

tests/integration/test_ingest_validate.py

```
1 """End-to-end ingest + validate against the real attachments (runs inside Modal where /repo exists)."""
2
3 from __future__ import annotations
4
5 import json
6 import os
7 from pathlib import Path
8
9 import pytest
10
11 from forge import runner
12
13 REPO = Path(os.environ.get("FORGE_REPO", "/repo"))
14 pytestmark = pytest.mark.integration
15
16
17 @pytest.mark.skipif(not (REPO / "configs" / "project.json").exists(), reason="repository inputs not mounted")
18 def test_ingest_then_validate_on_real_inputs(tmp_path: Path) -> None:
19     vol = tmp_path / "vol"
20     ingest = runner.execute("ingest", "itest", {"code_ref": {"git_sha": "test"}}, False, REPO, vol)
21     assert ingest["status"] == "completed" and ingest["metrics"]["files"] >= 1
22     inventory = json.loads((vol / "runs" / "itest" / "ingest" / "inventory.json").read_text(encoding="utf-8"))
23     assert any(not f["template"] for f in inventory["files"])
24     validate = runner.execute("validate", "itest", {}, False, REPO, vol)
25     assert validate["status"] == "completed" and validate["metrics"]["ok"]
```

tests/unit/__init__.py

tests/unit/test_config.py

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import pytest
6
7 from forge.config import Config, deep_merge, load_config
8
9
10 def test_deep_merge_is_recursive_and_non_destructive() -> None:
11     base = {"a": {"x": 1, "y": 2}, "b": 1}
12     merged = deep_merge(base, {"a": {"y": 3}, "c": 4})
13     assert merged == {"a": {"x": 1, "y": 3}, "b": 1, "c": 4}
14     assert base == {"a": {"x": 1, "y": 2}, "b": 1}
15
16
17 def test_config_dotted_access_and_digest() -> None:
18     cfg = Config({"paper": {"sources": ["q1"], "max_body_pages": 30}})
19     assert cfg.get("paper.max_body_pages") == 30
20     assert cfg.get("paper.missing", "d") == "d"
21     assert cfg["paper.sources"] == ["q1"]
22     with pytest.raises(KeyError):
23         cfg["nope.nope"]
24     assert cfg.digest == Config({"paper": {"max_body_pages": 30, "sources": ["q1"]}}).digest
25
```

```

26
27 def test_load_config_with_profile_and_overrides(tmp_repo: Path) -> None:
28     (tmp_repo / "configs" / "fast.toml").write_text("[run]\nseed = 99\n", encoding="utf-8")
29     cfg = load_config(tmp_repo, profile="fast", overrides={"paper": {"sources": ["x"]}})
30     assert cfg.get("run.seed") == 99
31     assert cfg.get("paper.sources") == ["x"]

```

tests/unit/test_contracts.py

```

1 from __future__ import annotations
2
3 import numpy as np
4 import pandas as pd
5 from hypothesis import given
6 from hypothesis import strategies as st
7
8 from forge.contracts import Column, FrameContract, validate_frame
9
10 CONTRACT = FrameContract(
11     name="t",
12     columns=(
13         Column("t", "int", min=0, unique=True, monotonic="increasing", step=60),
14         Column("v", "float", min=0, max=1),
15     ),
16     min_rows=2,
17 )
18
19
20 def test_valid_frame_passes() -> None:
21     df = pd.DataFrame({"t": [0, 60, 120], "v": [0.1, 0.2, 0.3]})
22     report = validate_frame(df, CONTRACT)
23     assert report["ok"], report["errors"]
24
25
26 def test_violations_are_reported_precisely() -> None:
27     df = pd.DataFrame({"t": [0, 60, 60], "v": [0.1, 1.5, np.nan], "extra": [1, 2, 3]})
28     report = validate_frame(df, CONTRACT)
29     joined = " | ".join(report["errors"])
30     assert not report["ok"]
31     assert "unexpected columns" in joined
32     assert "duplicated" in joined
33     assert "max 1.5 > 1" in joined
34     assert "null" in joined
35     assert "not increasing" in joined
36
37
38 def test_column_names_are_stripped() -> None:
39     df = pd.DataFrame({"t ": [0, 60], " v": [0.5, 0.6]})
40     assert validate_frame(df, CONTRACT)["ok"]
41
42
43 @given(st.lists(st.floats(min_value=0, max_value=1, allow_nan=False), min_size=2, max_size=40))
44 def test_property_ranges_within_bounds_always_pass(values: list[float]) -> None:
45     df = pd.DataFrame({"t": np.arange(len(values)) * 60, "v": values})
46     assert validate_frame(df, CONTRACT)["ok"]
47
48
49 def test_regex_and_custom_checks() -> None:
50     contract = FrameContract(
51         "ids", (Column("id", "str", regex=r"[ABC]\d{3}"),), checks=(lambda df: ["custom"] if len(df) > 1 else [],)
52     )
53     assert validate_frame(pd.DataFrame({"id": ["A001"]}), contract)["ok"]
54     report = validate_frame(pd.DataFrame({"id": ["A001", "Z9"]}), contract)
55     assert any("violate" in e for e in report["errors"]) and "custom" in report["errors"]

```

tests/unit/test_derived.py

```
1 """Unit tests of the helpers behind the derived paper quantities (MDR-0009)."""
2
3 from __future__ import annotations
4
5 import numpy as np
6
7 from pipelines.a.derived import first_time_within, fourier_number, richardson_error
8
9
10 def test_fourier_number() -> None:
11     assert fourier_number(2.0, 3.0, 2.0) == 1.5
12
13
14 def test_richardson_error_recovers_the_limit_of_a_second_order_sequence() -> None:
15     limit = 10.0
16     values = [limit + 4.0 / 4**k for k in range(4)] # error /4 per level: grids halved, order 2
17     est, err = richardson_error(values, order=2.0)
18     assert abs(est - limit) < 1e-12
19     assert abs(err - 4.0 / 4**2) < 1e-12
20
21
22 def test_first_time_within_requires_the_gap_to_stay_small() -> None:
23     t = np.arange(6.0)
24     a = np.array([0.0, 0.9, 0.2, 0.7, 0.1, 0.05])
25     b = np.zeros(6)
26     assert first_time_within(t, a, b, 0.5) == 4.0
27     assert first_time_within(t, a + 10.0, b, 0.5) is None
28     assert first_time_within(t, b, b, 0.5) == 0.0
29
30
31 def test_value_at_picks_the_nearest_recorded_instant() -> None:
32     from pipelines.a.derived import value_at
33
34     t = np.array([0.0, 60.0, 120.0, 180.0])
35     v = np.array([1.0, 2.0, 3.0, 4.0])
36     assert value_at(t, v, 120.0) == 3.0
37     assert value_at(t, v, 119.0) == 3.0
38     assert value_at(t, v, 1000.0) == 4.0
```

tests/unit/test_drying.py

```
1 """Tests of the Problem-A physics, solver and independent verifiers (small grids, short horizons)."""
2
3 from __future__ import annotations
4
5 import math
6 from dataclasses import replace
7
8 import numpy as np
9 import pytest
10 from hypothesis import given
11 from hypothesis import strategies as st
12 from scipy.special import j0, j1
13
14 from pipelines.a import verify
15 from pipelines.a.physics import (
16     APPENDIX2,
17     APPENDIX3,
18     APPENDIX4,
19     C_INIT,
20     C_TARGET,
21     R0,
22     T_INIT,
23     Chamber,
24     Radius,
```

```

25 )
26 from pipelines.a.solver import (
27     ProblemSpec,
28     RadialSystem,
29     make_grid,
30     sample_physical,
31     solve,
32     solve_until_dry,
33     subgrid_indices,
34 )
35
36 # ---- physics -----
37
38
39 def test_property_laws_match_the_appendices() -> None:
40     c = np.array(2.55)
41     assert APPENDIX2.diffusivity(c, np.array(28.0)) == pytest.approx(7e-9 * math.exp(-0.89 / 2.55))
42     assert APPENDIX3.rho(c) == pytest.approx(650 + 128 * 2.55)
43     assert APPENDIX3.cp(c) == pytest.approx(1450 + 2736 * 2.55 / 3.55)
44     assert APPENDIX3.k(c) == pytest.approx(0.21 + 0.38 * 2.55 / 3.55)
45     d3 = 2.4e-3 * math.exp(-0.45 / 2.55) * math.exp(-3850 / (28 + 273.15))
46     assert APPENDIX3.diffusivity(c, np.array(28.0)) == pytest.approx(d3, rel=1e-12)
47     d4 = 4.2e-4 * math.exp(-0.30 / 0.2) * math.exp(-3850 / (50 + 273.15))
48     assert APPENDIX4.diffusivity(np.array(0.2), np.array(50.0)) == pytest.approx(d4, rel=1e-12)
49     assert APPENDIX4.h == 25.0 and APPENDIX4.hm == 8e-7
50     assert APPENDIX2.diffusivity(np.array(0.0), np.array(28.0)) == 0.0 # floor: no division by zero
51
52
53 def test_chamber_interpolates_linearly_and_holds_the_plateau() -> None:
54     t = np.arange(0, 14401, 60, dtype=float)
55     temp = 28 + 22 * np.minimum(t / 7200, 1.0)
56     hum = np.full_like(t, 0.05)
57     ch = Chamber.from_series(t, temp, hum, "mean_last_hour")
58     assert ch.air_temperature(30.0) == pytest.approx(28 + 22 * 30 / 7200)
59     assert ch.air_temperature(1e6) == pytest.approx(50.0)
60     assert Chamber.from_series(t, temp, hum, "nominal").hum_plateau == 0.05
61     assert Chamber.from_series(t, temp, hum, "last_sample").temp_plateau == pytest.approx(temp[-1])
62
63
64 def test_radius_is_monotone_clamped_and_scaled() -> None:
65     t = np.arange(0, 7201, 1800, dtype=float)
66     r_cm = np.array([2.0, 1.8, 1.7, 1.7, 1.65])
67     rad = Radius.from_series(t, r_cm)
68     fine = np.linspace(0, 9000, 500)
69     values = rad.value(fine)
70     assert np.all(np.diff(values) <= 1e-15)
71     assert rad.value(0.0) == pytest.approx(0.02) and rad.value(1e5) == pytest.approx(0.0165)
72     assert rad.rate(1e5) == 0.0 and rad.rate(900.0) < 0
73     half = Radius.from_series(t, r_cm, shrink_scale=0.5)
74     assert half.value(7200.0) == pytest.approx(0.02 - 0.5 * (0.02 - 0.0165))
75     assert Radius.constant().is_constant and not rad.is_constant
76
77
78 # ---- solver -----
79
80
81 def test_grid_measures_sum_to_half() -> None:
82     g = make_grid(20)
83     assert g.vol.sum() == pytest.approx(0.5)
84     assert g.vol[0] == pytest.approx(g.dxi**2 / 8)
85     np.testing.assert_allclose(verify.control_volumes(g.xi), g.vol)
86
87
88 def test_equilibrium_is_a_steady_state() -> None:
89     spec = ProblemSpec(APPENDIX2, Chamber.constant(T_INIT, C_INIT), Radius.constant())
90     system = RadialSystem(spec, make_grid(10))
91     y = np.concatenate([np.full(11, T_INIT), np.full(11, C_INIT)])
92     np.testing.assert_allclose(system(0.0, y), 0.0, atol=1e-14)
93
94
95 def test_solver_matches_the_bessel_series_for_constant_properties() -> None:

```

```

96     d_const = 5e-9
97     props = replace(APPENDIX2, d_const=d_const)
98     spec = ProblemSpec(props, Chamber.constant(50.0, 0.05), Radius.constant())
99     t_out = np.array([0.0, 300.0, 900.0, 1800.0])
100     sol = solve(spec, 40, t_out, rtol=1e-8, atol=1e-10)
101     alpha = props.k0 / (props.rho0 * props.cp0)
102     exact_t = verify.analytic_field(sol.xi, t_out, alpha, props.h, props.k0, T_INIT, 50.0)
103     exact_c = verify.analytic_field(sol.xi, t_out, d_const, props.hm, d_const, C_INIT, 0.05)
104     assert np.max(np.abs(sol.temp - exact_t)) < 2e-3
105     assert np.max(np.abs(sol.moist - exact_c)) < 5e-3 # thin early boundary layer on the coarse test grid
106
107
108 def test_discrete_moisture_and_energy_balances_hold() -> None:
109     spec = ProblemSpec(APPENDIX2, Chamber.constant(50.0, 0.05), Radius.constant())
110     sol = solve(spec, 20, np.arange(0.0, 1801.0, 2.0))
111     mb = verify.moisture_balance(
112         sol.t, sol.xi, sol.moist, sol.radius, sol.air_hum, spec.props.hm, cum_loss=sol.cum_moist_loss
113     )
114     eb = verify.energy_balance(sol.t, sol.xi, sol.temp, R0, sol.air_temp, spec.props, cum_loss=sol.cum_heat_loss)
115     assert mb["sampled"]["max_relative_residual"] < 1e-4 and eb["sampled"]["max_relative_residual"] < 1e-4
116     assert mb["exact"]["max_relative_residual"] < 1e-8 and eb["exact"]["max_relative_residual"] < 1e-8
117     ex = verify.extremum_checks(sol.temp, sol.moist, sol.air_temp, sol.air_hum, T_INIT, C_INIT)
118     assert ex["temp_within_bounds"] and ex["moist_within_bounds"] and ex["moist_monotone_in_r"]
119
120
121 def test_moving_boundary_reduces_to_the_fixed_domain_for_constant_radius() -> None:
122     chamber = Chamber.constant(50.0, 0.05)
123     t_out = np.arange(0.0, 3601.0, 600.0)
124     ref = solve(ProblemSpec(APPENDIX4, chamber, Radius.constant()), 20, t_out, rtol=1e-10, atol=1e-12)
125     pseudo = Radius(np.array([0.0, 1.0, 2.0]), np.array([R0, R0, R0]), force_interpolant=True)
126     for form in ("lagrangian", "eulerian"):
127         test = solve(ProblemSpec(APPENDIX4, chamber, pseudo, formulation=form), 20, t_out, rtol=1e-10, atol=1e-12)
128         assert np.max(np.abs(test.moist - ref.moist)) < 1e-8
129         assert np.max(np.abs(test.temp - ref.temp)) < 1e-6
130
131
132 def test_lagrangian_shrinkage_conserves_dry_basis_water() -> None:
133     t = np.array([0.0, 1800.0, 3600.0, 7200.0])
134     rad = Radius.from_series(t, np.array([2.0, 1.8, 1.7, 1.6]))
135     spec = ProblemSpec(APPENDIX4, Chamber.constant(50.0, 0.05), rad, formulation="lagrangian")
136     sol = solve(spec, 20, np.arange(0.0, 7201.0, 5.0))
137     mb = verify.moisture_balance(
138         sol.t, sol.xi, sol.moist, sol.radius, sol.air_hum, spec.props.hm, cum_loss=sol.cum_moist_loss
139     )
140     assert mb["sampled"]["max_relative_residual"] < 2e-4
141     assert mb["exact"]["max_relative_residual"] < 1e-8
142     # the Eulerian alternative loses water through the receding boundary: its balance residual is not small
143     alt = solve(replace(spec, formulation="eulerian"), 20, np.arange(0.0, 7201.0, 5.0))
144     mb_alt = verify.moisture_balance(
145         alt.t, alt.xi, alt.moist, alt.radius, alt.air_hum, spec.props.hm, cum_loss=alt.cum_moist_loss
146     )
147     assert mb_alt["exact"]["max_relative_residual"] > 1e-3
148
149
150 def test_solve_until_dry_detects_the_crossing_consistently() -> None:
151     props = replace(APPENDIX2, d0=7e-7) # fast drying for the test
152     spec = ProblemSpec(props, Chamber.constant(50.0, 0.05), Radius.constant())
153     sol = solve_until_dry(spec, 20, 60.0, horizon=3600.0, display_decimals=None)
154     assert sol.t_dry is not None and sol.dry_profile is not None
155     checks = verify.drying_checks(sol.t, sol.moist, sol.t_dry, sol.dry_profile)
156     assert checks["rows_before_all_wet"] and checks["rows_after_all_dry"]
157     assert checks["profile_max_error_at_t_dry"] < 1e-6
158     assert sol.t[-1] == pytest.approx(60.0 * math.ceil(sol.t_dry / 60.0))
159     # with the display rule (MDR-0005) the last row must show every value below the target at four decimals
160     shown = solve_until_dry(spec, 20, 60.0, horizon=3600.0)
161     assert shown.t[-1] >= 60.0 * math.ceil(shown.t_dry / 60.0)
162     assert round(float(shown.moist[-1].max()), 4) < C_TARGET
163     assert round(float(shown.moist[-2].max()), 4) >= C_TARGET or shown.t[-1] == 60.0 * math.ceil(shown.t_dry / 60.0)
164
165
166 def test_subgrid_and_physical_sampling() -> None:

```

```

167     idx = subgrid_indices(160)
168     assert len(idx) == 21 and idx[1] == 8
169     xi = np.linspace(0, 1, 41)
170     profile = 1.0 - xi**2
171     out = sample_physical(profile, xi, 0.015, np.array([0.0, 0.0075, 0.015, 0.016]))
172     assert out[0] == pytest.approx(1.0) and out[1] == pytest.approx(0.75, abs=1e-9)
173     assert out[2] == pytest.approx(0.0, abs=1e-9) and np.isnan(out[3])
174
175
176 def test_piecewise_integration_is_exact_for_piecewise_linear_forcing() -> None:
177     """y' = -y + f(t) with f piecewise linear (kinks every 60 s): restarting at the kinks keeps BDF accurate."""
178     from pipelines.a.solver import integrate_piecewise
179
180     knots = np.arange(0.0, 601.0, 60.0)
181     values = np.array([0, 1, 0, 2, 0, 3, 0, 4, 0, 5, 0], dtype=float)
182
183     def fun(t: float, y: np.ndarray) -> np.ndarray:
184         return -y + np.interp(t, knots, values)
185
186     t_out = np.arange(0.0, 601.0, 30.0)
187     states, stats, event_t, _ = integrate_piecewise(
188         fun, np.array([0.0]), t_out, knots[1:], method="BDF", rtol=1e-10, atol=1e-12, jac_sparsity=None
189     )
190     # exact solution: y(t) = int_0^t e^{-(t-s)} f(s) ds, evaluated with a fine trapezoid rule on each segment
191     fine = np.linspace(0.0, 600.0, 600001)
192     f_fine = np.interp(fine, knots, values)
193     exact = [np.trapezoid(np.exp(-(t - fine[fine <= t])) * f_fine[fine <= t], fine[fine <= t]) for t in t_out]
194     assert stats["segments"] == 10 and event_t is None
195     np.testing.assert_allclose(states[:, 0], exact, atol=1e-7)
196
197
198 # ---- verifiers -----
199
200
201 @given(st.floats(min_value=0.05, max_value=50.0))
202 def test_robin_eigenvalues_satisfy_the_characteristic_equation(bi: float) -> None:
203     lam = verify.robin_eigenvalues(bi, 6)
204     assert np.all(np.diff(lam) > 0)
205     np.testing.assert_allclose(lam * j1(lam) - bi * j0(lam), 0.0, atol=1e-9)
206
207
208 def test_analytic_series_has_the_right_limits() -> None:
209     xi = np.linspace(0, 1, 11)
210     theta = verify.analytic_robin_cylinder(xi, np.array([1e-4, 5.0]), 1.0)
211     assert np.all(theta[0, :-2] > 0.99) # essentially unchanged interior at tiny Fourier number
212     assert np.all(np.abs(theta[1]) < 1e-3) # fully equilibrated
213
214
215 def test_convergence_table_reports_second_order() -> None:
216     levels = [{"k": k, "dr_cm": 0.1 / 2**k, "v": np.array([1.0 + (0.1 / 2**k) ** 2])} for k in range(4)]
217     rows = verify.convergence_table(levels, "v")
218     assert rows[0]["observed_order"] == pytest.approx(2.0, abs=0.2)
219
220
221 def test_axisymmetric_model_with_insulated_ends_reproduces_the_radial_scheme() -> None:
222     chamber = Chamber.constant(50.0, 0.05)
223     t_out = np.array([0.0, 300.0, 900.0])
224     one_d = solve(ProblemSpec(APPENDIX3, chamber, Radius.constant()), 20, t_out)
225     two_d = verify.solve_axisymmetric(APPENDIX3, chamber, t_out, nr=20, nz=3, end_bc="insulated")
226     assert np.max(np.abs(two_d["moist"][:, :, 0] - one_d.moist)) < 1e-7
227     assert np.max(np.abs(two_d["temp"][:, :, 0] - one_d.temp)) < 1e-5
228     assert np.max(np.abs(two_d["moist"] - two_d["moist"][:, :, :1])) < 1e-9

```

tests/unit/test_extension.py

```
1 """Tests of the model-evaluation extension (MDR-0008): psychrometrics, the energy flux cap, the C_eq driving force."""
2
3 from __future__ import annotations
4
5 from dataclasses import replace
6
7 import numpy as np
8 import pytest
9
10 from pipelines.a import verify
11 from pipelines.a.common import spec_from_recipe, spec_to_recipe
12 from pipelines.a.extension import latent_diagnostic, slug
13 from pipelines.a.physics import (
14     APPENDIX2,
15     APPENDIX3,
16     C_INIT,
17     Chamber,
18     FluxCap,
19     Radius,
20     dry_solid_density,
21     humidity_ratio,
22     latent_heat,
23     relative_humidity,
24     saturation_pressure,
25     wet_bulb_temperature,
26 )
27 from pipelines.a.solver import ProblemSpec, solve, solve_until_dry
28
29
30 def test_psychrometrics_reproduce_reference_values() -> None:
31     assert saturation_pressure(50.0) == pytest.approx(12352.0, rel=0.01) # steam tables: 12.35 kPa
32     assert saturation_pressure(20.0) == pytest.approx(2339.0, rel=0.01)
33     assert relative_humidity(50.0, 0.05) == pytest.approx(0.61, abs=0.02)
34     tw = wet_bulb_temperature(50.0, 0.05)
35     assert 40.5 < tw < 43.0
36     w_sat = float(humidity_ratio(saturation_pressure(30.0)))
37     assert wet_bulb_temperature(30.0, w_sat) == pytest.approx(30.0) # saturated air: wet bulb = dry bulb
38     assert wet_bulb_temperature(50.0, 0.02) < wet_bulb_temperature(50.0, 0.05) < 50.0
39     assert latent_heat(50.0) == pytest.approx(2.383e6, rel=0.005) # steam tables: 2382 kJ/kg
40     assert dry_solid_density(APPENDIX3, C_INIT) == pytest.approx((650 + 128 * 2.55) / 3.55)
41
42
43 def test_flux_cap_limits_early_loss_and_keeps_the_balance_exact() -> None:
44     chamber = Chamber.constant(50.0, 0.05)
45     rho_s = float(dry_solid_density(APPENDIX3, C_INIT))
46     cap = FluxCap.wet_bulb(chamber, APPENDIX3.h, rho_s)
47     assert 5e-5 < cap.plateau < 1.5e-4 # ~ 25 W/(m^2 K) x 8 K / 2.4 MJ/kg
48     assert cap.value(1e9) == pytest.approx(cap.plateau)
49     base = ProblemSpec(APPENDIX3, chamber, Radius.constant())
50     capped = replace(base, flux_cap=cap)
51     t_out = np.arange(0.0, 3601.0, 60.0)
52     sol_b = solve(base, 20, t_out)
53     sol_c = solve(capped, 20, t_out)
54     vol = verify.control_volumes(sol_b.xi)
55     assert sol_c.moist[-1] @ vol > sol_b.moist[-1] @ vol # less water leaves when the flux is capped
56     flux = capped.surface_moisture_loss(sol_c.t, sol_c.moist[:, -1], sol_c.air_hum)
57     assert np.all(flux <= cap.plateau / rho_s * (1 + 1e-12))
58     mb = verify.moisture_balance(
59         sol_c.t,
60         sol_c.xi,
61         sol_c.moist,
62         sol_c.radius,
63         sol_c.air_hum,
64         APPENDIX3.hm,
65         cum_loss=sol_c.cum_moist_loss,
66         flux=flux,
67     )
68     assert mb["exact"]["max_relative_residual"] < 1e-8
```

```

69     # below the release moisture the loss law is the problem's own
70     c_release = 0.05 + cap.plateau / (rho_s * APPENDIX3.hm)
71     low = np.array([0.5 * c_release])
72     assert capped.surface_moisture_loss(0.0, low, 0.05) == pytest.approx(APPENDIX3.hm * (low - 0.05))
73     # the cap survives the worker recipe round trip
74     again = spec_from_recipe(spec_to_recipe(capped))
75     assert again.flux_cap is not None and again.flux_cap.value(30.0) == pytest.approx(cap.value(30.0))
76     assert slug("cap:1.0") == "cap_1p0"
77
78
79 def test_equilibrium_moisture_driving_force_bounds_the_surface_and_is_monotone_for_constant_d() -> None:
80     # constant D: the problem is linear, max C < 0.15 <=> theta < (0.15 - C_eq)/(2.55 - C_eq), so a larger C_eq
81     # must take longer; with the exponential D(C) of the appendices a drier surface forms a low-D crust and the
82     # ordering is not guaranteed (case hardening), which is exactly what the extension stage quantifies.
83     props = replace(APPENDIX2, d_const=2e-8)
84     base = ProblemSpec(props, Chamber.constant(50.0, 0.05), Radius.constant())
85     slow = replace(base, chamber=Chamber.constant(50.0, 0.10))
86     sol_base = solve_until_dry(base, 20, 60.0, horizon=3600.0)
87     sol_slow = solve_until_dry(slow, 20, 60.0, horizon=3600.0)
88     assert sol_base.t_dry is not None and sol_slow.t_dry is not None
89     assert sol_slow.t_dry > sol_base.t_dry
90     assert sol_slow.moist[:, -1].min() >= 0.10 - 1e-9 # the surface never drops below the driving-force floor
91     assert sol_slow.dry_profile is not None and sol_slow.dry_profile.max() == pytest.approx(0.15, abs=1e-6)
92
93
94 def test_latent_diagnostic_scales_with_the_evaporation_flux() -> None:
95     spec = ProblemSpec(APPENDIX3, Chamber.constant(50.0, 0.05), Radius.constant())
96     sol = solve(spec, 20, np.arange(0.0, 3601.0, 60.0))
97     rho_s = float(dry_solid_density(APPENDIX3, C_INIT))
98     hist = {
99         "t": sol.t,
100         "surface_temp": sol.temp[:, -1],
101         "air_temp": sol.air_temp,
102         "surface_moist": sol.moist[:, -1],
103         "air_hum": sol.air_hum,
104         "mean_moist": 2.0 * (sol.moist @ verify.control_volumes(sol.xi)),
105     }
106     diag = latent_diagnostic(hist, spec, rho_s)
107     j0 = rho_s * APPENDIX3.hm * (C_INIT - 0.05)
108     assert diag["summary"]["j_w_initial"] == pytest.approx(j0)
109     assert diag["summary"]["dt_lat_peak_K"] == pytest.approx(float(latent_heat(28.0)) * j0 / APPENDIX3.h)
110     assert diag["arrays"]["e_lat"][-1] > 0 and np.all(np.diff(diag["arrays"]["e_lat"]) >= 0)
111     assert diag["summary"]["water_removed_kg_per_m"] > 0

```

tests/unit/test_hashing.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from hypothesis import given
6 from hypothesis import strategies as st
7
8 from forge.hashing import hash_tree, sha256_bytes, sha256_json, tree_digest
9
10
11 @given(st.binary())
12 def test_sha256_bytes_is_deterministic(data: bytes) -> None:
13     assert sha256_bytes(data) == sha256_bytes(data)
14     assert len(sha256_bytes(data)) == 64
15
16
17 @given(
18     st.dictionaries(
19         st.text(min_size=1, max_size=8), st.integers() | st.floats(allow_nan=False) | st.text(max_size=5), max_size=6
20     )
21 )
22 def test_sha256_json_ignores_key_order(mapping: dict) -> None:

```

```

23     reversed_mapping = dict(reversed(list(mapping.items())))
24     assert sha256_json(mapping) == sha256_json(reversed_mapping)
25
26
27 def test_hash_tree_ignores_records_and_is_sorted(tmp_path: Path) -> None:
28     (tmp_path / "b.txt").write_text("b")
29     (tmp_path / "a.txt").write_text("a")
30     (tmp_path / "manifest.json").write_text("{}")
31     (tmp_path / "__pycache__").mkdir()
32     (tmp_path / "__pycache__" / "x.pyc").write_bytes(b"\x00")
33     tree = hash_tree(tmp_path, ignore=("manifest.json", "__pycache__"))
34     assert list(tree) == ["a.txt", "b.txt"]
35     digest = tree_digest(tree)
36     (tmp_path / "manifest.json").write_text('{"changed": true}')
37     assert tree_digest(hash_tree(tmp_path, ignore=("manifest.json", "__pycache__"))) == digest

```

tests/unit/test_manifest.py

```

1  from __future__ import annotations
2
3  from pathlib import Path
4
5  from forge.manifest import Manifest
6
7
8  def test_manifest_lifecycle_and_verification(tmp_path: Path) -> None:
9      stage_dir = tmp_path / "stage"
10     stage_dir.mkdir()
11     (stage_dir / "out.json").write_text("{}")
12     (stage_dir / "events.jsonl").write_text("{}\n")
13     manifest = Manifest.start("run1", "stage", {"p": 1}, {"git_sha": "abc"}, {"cfg", {"ingest": "d"}, {}))
14     assert manifest.status == "running" and manifest.runtime["python"]
15     manifest.complete(stage_dir, {"m": 1.5})
16     assert manifest.status == "completed"
17     assert list(manifest.outputs) == ["out.json"]
18     manifest.write(stage_dir / "manifest.json")
19     loaded = Manifest.read(stage_dir / "manifest.json")
20     assert loaded.outputs_digest == manifest.outputs_digest
21     assert loaded.verify_outputs(stage_dir) == []
22     (stage_dir / "out.json").write_text('{"x": 1}')
23     assert loaded.verify_outputs(stage_dir) == [{"sha256 mismatch": "out.json"}]
24
25
26 def test_manifest_failure_records_traceback() -> None:
27     manifest = Manifest.start("run1", "stage", {}, {}, {"cfg", {}, {}))
28     try:
29         raise ValueError("boom")
30     except ValueError as exc:
31         manifest.fail(exc)
32     assert manifest.status == "failed"
33     assert manifest.error and manifest.error["type"] == "ValueError"
34     assert manifest.duration_s is not None

```

tests/unit/test_packaging_tex.py

```

1  from __future__ import annotations
2
3  from pathlib import Path
4
5  from forge.packaging import build_zip, code_members, run_members
6  from forge.tex import aux_page, summarize_log, tex_escape
7
8
9  def test_zip_is_deterministic(tmp_path: Path) -> None:

```

```

10     src = tmp_path / "src"
11     src.mkdir()
12     (src / "b.py").write_text("print(2)")
13     (src / "a.py").write_text("print(1)")
14     (src / "__pycache__").mkdir()
15     (src / "__pycache__" / "a.pyc").write_bytes(b"x")
16     members = code_members(src, ["**/*.py"])
17     assert [m[0] for m in members] == ["a.py", "b.py"]
18     one = build_zip(members, tmp_path / "one.zip")
19     two = build_zip(list(reversed(members)), tmp_path / "two.zip")
20     assert one["sha256"] == two["sha256"] and one["members"] == 2
21
22
23 def test_run_members_prefix(tmp_path: Path) -> None:
24     run = tmp_path / "runs" / "r1"
25     (run / "q1").mkdir(parents=True)
26     (run / "q1" / "out.json").write_text("{}")
27     assert run_members(run, ["q1", "missing"]) == [(("runs/r1/q1/out.json", run / "q1" / "out.json"))]
28
29
30 def test_log_summary_and_aux_page(tmp_path: Path) -> None:
31     log = (
32         "LaTeX Warning: Reference `fig:x' on page 3 undefined on input line 10.\n"
33         "Overfull \hbox (12.5pt too wide) in paragraph\n"
34         "Missing character: There is no  in font Latin Modern!\n"
35         "! Undefined control sequence.\n"
36         "Output written on main.pdf (7 pages).\n"
37     )
38     summary = summarize_log(log)
39     assert summary["undefined_references"] == ["fig:x"]
40     assert summary["overfull_max_pt"] == 12.5 and summary["missing_characters"] == 1
41     assert summary["errors"] == ["Undefined control sequence."] and summary["pages_from_log"] == 7
42     aux = tmp_path / "main.aux"
43     aux.write_text("\newlabel{page:body-end}{{27}}{page:body-end}}\n\newlabel{sec:a}{{2.1}{5}}{}}\n")
44     assert aux_page(aux, "page:body-end") == 27 and aux_page(aux, "sec:a") == 5 and aux_page(aux, "nope") is None
45
46
47 def test_tex_escape() -> None:
48     assert tex_escape("a_b & 100% #1") == r"a\_b \& 100\% \#1"

```

tests/unit/test_runner.py

```

1 from __future__ import annotations
2
3 import json
4 from pathlib import Path
5
6 import pytest
7
8 from forge import runner
9 from forge.context import StageContext
10
11
12 @pytest.fixture(autouse=True)
13 def isolated_registry(monkeypatch: pytest.MonkeyPatch) -> None:
14     monkeypatch.setattr(runner, "STAGES", {})
15     monkeypatch.setattr(runner, "discover", lambda repo: None)
16
17
18 def test_execute_records_manifest_skips_reruns_and_enforces_deps(tmp_repo: Path, tmp_path: Path) -> None:
19     vol = tmp_path / "vol"
20
21     @runner.stage("first", description="first stage")
22     def first(ctx: StageContext) -> dict:
23         ctx.out("a.txt").write_text("hello")
24         ctx.number("Answer", 42)
25         return {"n": 1}
26

```

```

27 @runner.stage("second", deps=("first",))
28 def second(ctx: StageContext) -> dict:
29     assert (ctx.dep("first") / "a.txt").read_text() == "hello"
30     return {"ok": True}
31
32 with pytest.raises(RuntimeError):
33     runner.execute("second", "r1", {}, False, tmp_repo, vol)
34 result = runner.execute("first", "r1", {"code_ref": {"git_sha": "x"}}, False, tmp_repo, vol)
35 assert result["status"] == "completed" and result["metrics"] == {"n": 1}
36 manifest = json.loads((vol / "runs" / "r1" / "first" / "manifest.json").read_text())
37 assert (
38     manifest["status"] == "completed" and "a.txt" in manifest["outputs"] and "numbers.json" in manifest["outputs"]
39 )
40 assert json.loads((vol / "runs" / "r1" / "first" / "numbers.json").read_text()) == {"Answer": "42"}
41 assert runner.execute("first", "r1", {}, False, tmp_repo, vol)["status"] == "skipped"
42 assert runner.execute("second", "r1", {}, False, tmp_repo, vol)["status"] == "completed"
43 combined = runner.execute("all", "r1", {"stages": ["first", "second"]}, False, tmp_repo, vol)
44 assert combined["status"] == "completed" and [r["status"] for r in combined["results"]] == ["skipped", "skipped"]
45
46
47 def test_failed_stage_writes_failed_manifest(tmp_repo: Path, tmp_path: Path) -> None:
48     vol = tmp_path / "vol"
49
50     @runner.stage("bad")
51     def bad(ctx: StageContext) -> dict:
52         raise ValueError("boom")
53
54     with pytest.raises(ValueError):
55         runner.execute("bad", "r1", {}, False, tmp_repo, vol)
56     manifest = json.loads((vol / "runs" / "r1" / "bad" / "manifest.json").read_text())
57     assert manifest["status"] == "failed" and manifest["error"]["message"] == "boom"
58
59
60 def test_from_run_copies_dependency_outputs(tmp_repo: Path, tmp_path: Path) -> None:
61     vol = tmp_path / "vol"
62
63     @runner.stage("base")
64     def base(ctx: StageContext) -> dict:
65         ctx.out("x.txt").write_text("x")
66         return {}
67
68     @runner.stage("child", deps=("base",))
69     def child(ctx: StageContext) -> dict:
70         return {"seen": (ctx.dep("base") / "x.txt").read_text()}
71
72     runner.execute("base", "old", {}, False, tmp_repo, vol)
73     result = runner.execute("child", "new", {"from_run": "old"}, False, tmp_repo, vol)
74     assert result["metrics"] == {"seen": "x"}

```

tests/unit/test_xlsx.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import openpyxl
6 import pytest
7
8 from forge.xlsx import SheetContract, WorkbookContract, check_workbook, read_sheets, sheet_headers, write_result
9
10
11 def _template(path: Path) -> Path:
12     wb = openpyxl.Workbook()
13     ws = wb.active
14     ws.title = "温度"
15     ws.append(["时间\\到药材中心的距离", 0, 0.1, "...", 2])
16     ws.append([1, None, None, None, None])
17     ws2 = wb.create_sheet("水分浓度")

```

```

18     ws2.append(["时间\\到药材中心的距离", 0, 0.1, "...", 2])
19     wb.save(path)
20     return path
21
22
23 def test_write_result_keeps_all_rounds_and_passes_contract(tmp_path: Path) -> None:
24     template = _template(tmp_path / "tpl.xlsx")
25     header = ["ignored", 0.0, 0.1, 0.2]
26     rows = [[t, 28.123456, 28.2, 28.3] for t in range(1, 6)]
27     info = write_result(template, tmp_path / "out.xlsx", {"温度": (header, rows), "水分浓度": (header, rows)})
28     assert info["sheets"]["温度"]["rows"] == 5
29     headers = sheet_headers(tmp_path / "out.xlsx")
30     assert headers["温度"][0] == "时间\\到药材中心的距离"
31     assert headers["温度"][1:] == [0.0, 0.1, 0.2]
32     frames = read_sheets(tmp_path / "out.xlsx")
33     assert float(frames["温度"].iloc[0, 1]) == pytest.approx(28.1235)
34     contract = WorkbookContract(
35         "out.xlsx",
36         "tpl.xlsx",
37         (
38             SheetContract("温度", min_rows=5, max_rows=5, header_len=4, numeric_from_col=0),
39             SheetContract("水分浓度", min_rows=5, header_len=4, numeric_from_col=0),
40         ),
41     )
42     report = check_workbook(tmp_path / "out.xlsx", contract, template)
43     assert report["ok"], report["errors"]
44
45
46 def test_check_workbook_flags_blank_and_text_cells(tmp_path: Path) -> None:
47     template = _template(tmp_path / "tpl.xlsx")
48     rows = [[1, 1.0, None, "x"], [2, 2.0, 2.0, 2.0]]
49     write_result(template, tmp_path / "out.xlsx", {"温度": ([ "h", 0, 0.1, 0.2], rows)})
50     contract = WorkbookContract("out.xlsx", "tpl.xlsx", (SheetContract("温度", min_rows=2, numeric_from_col=0),))
51     report = check_workbook(tmp_path / "out.xlsx", contract, template)
52     joined = " ".join(report["errors"])
53     assert not report["ok"] and "blank" in joined and "non-numeric" in joined
54
55
56 def test_write_result_rejects_unknown_sheet(tmp_path: Path) -> None:
57     template = _template(tmp_path / "tpl.xlsx")
58     with pytest.raises(KeyError):
59         write_result(template, tmp_path / "out.xlsx", {"nope": ([ "h", ], [])})
60
61
62 def test_write_only_path_for_large_sheets(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) -> None:
63     import forge.xlsx as mod
64
65     monkeypatch.setattr(mod, "BIG_SHEET_CELLS", 10)
66     template = _template(tmp_path / "tpl.xlsx")
67     rows = [[t, 1.0, 2.0] for t in range(1, 8)]
68     info = write_result(template, tmp_path / "big.xlsx", {"温度": ([ "h", 0, 0.1], rows)})
69     assert info["write_only"]
70     headers = sheet_headers(tmp_path / "big.xlsx")
71     assert list(headers) == ["温度", "水分浓度"]
72     assert headers["温度"][0] == "时间\\到药材中心的距离"

```

tools/_init_.py

tools/cli.py

```
1 #!/usr/bin/env python3
2 """Forge control plane (local). Never computes: submits cloud jobs, downloads, verifies hashes.
3
4 Examples
5     python3 tools/cli.py run ingest                # new run id, small container
6     python3 tools/cli.py run validate --run-id <id> # continue the same run
7     python3 tools/cli.py run q1,q2 --size medium    # sequential stages in one container
8     python3 tools/cli.py run q3 --size large --spawn # detached; poll with `wait`/`status`
9     python3 tools/cli.py exec --code 'print(sorted(p.name for p in RUN_DIR.iterdir()))'
10    python3 tools/cli.py status                    # manifests of the last run
11    python3 tools/cli.py download --stage results   # into artifacts/, verified against manifests
12    python3 tools/cli.py release --version v1.0.0   # deliverables into releases/v1.0.0/
13 """
14
15 from __future__ import annotations
16
17 import argparse
18 import hashlib
19 import json
20 import os
21 import shutil
22 import subprocess
23 import sys
24 from datetime import datetime
25 from pathlib import Path
26
27 REPO = Path(__file__).resolve().parents[1]
28 PROJECT = json.loads((REPO / "configs" / "project.json").read_text(encoding="utf-8"))
29 APP_FILE = REPO / "forge" / "modal_app.py"
30 STATE_DIR = REPO / ".forge"
31 LAST_RUN = STATE_DIR / "last_run"
32 MODAL = shutil.which("modal") or str(Path.home() / ".local" / "bin" / "modal")
33
34
35 def _sh(cmd: list[str], **kw) -> subprocess.CompletedProcess:
36     print("$ " + " ".join(cmd), flush=True)
37     return subprocess.run(cmd, cwd=REPO, check=False, **kw)
38
39
40 def git_ref() -> dict:
41     def out(*args: str) -> str:
42         try:
43             return subprocess.run(["git", *args], cwd=REPO, capture_output=True, text=True, check=False).stdout.strip()
44         except OSError:
45             return ""
46
47     sha = out("rev-parse", "HEAD") or "nogit"
48     return {
49         "git_sha": sha,
50         "branch": out("rev-parse", "--abbrev-ref", "HEAD"),
51         "dirty": bool(out("status", "--porcelain")),
52     }
53
54
55 def new_run_id() -> str:
56     return datetime.now().strftime("%Y%m%d-%H%M%S") + "-" + git_ref()["git_sha"][:7]
57
58
59 def remember(run_id: str) -> None:
60     STATE_DIR.mkdir(exist_ok=True)
61     LAST_RUN.write_text(run_id, encoding="utf-8")
62
63
64 def resolve_run_id(explicit: str | None, create: bool) -> str:
65     if explicit:
66         remember(explicit)
67         return explicit
68     if create or not LAST_RUN.exists():
```

```

69     run_id = new_run_id()
70     remember(run_id)
71     return run_id
72 return LAST_RUN.read_text(encoding="utf-8").strip()
73
74
75 def parse_params(items: list[str] | None) -> dict:
76     params: dict = {}
77     for item in items or []:
78         key, _, raw = item.partition("=")
79         try:
80             params[key] = json.loads(raw)
81         except json.JSONDecodeError:
82             params[key] = raw
83     return params
84
85
86 def modal_run(args: list[str], detach: bool = False) -> int:
87     cmd = [MODAL, "run"] + (["--detach"] if detach else []) + [str(APP_FILE)] + args
88     return _sh(cmd).returncode
89
90
91 def cmd_run(ns: argparse.Namespace) -> int:
92     stages = [s for s in ns.stage.split(",") if s]
93     run_id = resolve_run_id(ns.run_id, create=ns.new_run)
94     params = parse_params(ns.param)
95     params.update(
96         {"code_ref": git_ref(), "size": ns.size, "submitted_at": datetime.now().isoformat(timespec="seconds")}
97     )
98     if ns.from_run:
99         params["from_run"] = ns.from_run
100     if ns.profile:
101         params["profile"] = ns.profile
102     stage = stages[0] if len(stages) == 1 else "all"
103     if stage == "all":
104         params["stages"] = stages
105     args = ["--stage", stage, "--run-id", run_id, "--params", json.dumps(params, ensure_ascii=False), "--size", ns.size
106 ↪ ]
107     if ns.force:
108         args.append("--force")
109     if ns.spawn:
110         args.append("--spawn")
111     print(f"run_id={run_id} stages={stages} size={ns.size} spawn={ns.spawn}")
112     return modal_run(args, detach=ns.spawn)
113
114 def cmd_exec(ns: argparse.Namespace) -> int:
115     code = Path(ns.file).read_text(encoding="utf-8") if ns.file else ns.code
116     if not code:
117         print("provide --code or --file", file=sys.stderr)
118         return 2
119     run_id = resolve_run_id(ns.run_id, create=False)
120     return modal_run(
121         ["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": code}, ensure_ascii=False)]
122     )
123
124
125 def cmd_provision(_: argparse.Namespace) -> int:
126     return modal_run(["--stage", "provision"])
127
128
129 STATUS_SNIPPET = r"""
130 import json
131 from pathlib import Path
132 run = RUN_DIR
133 if not run.exists():
134     print("run directory missing:", run)
135 else:
136     rows = []
137     for m in sorted(run.glob("*/manifest.json")):
138         d = json.loads(m.read_text())

```

```

139     err = (d.get("error") or {}).get("message", "")[:80]
140     digest = d.get("outputs_digest", "")[:12]
141     rows.append((d["stage"], d["status"], d.get("duration_s"), digest, len(d.get("outputs", {})), err))
142     print(f"{'stage':14s} {'status':10s} {'seconds':>9s} {'digest':12s} {'files':>5s} error")
143     for r in rows:
144         print(f"{r[0]:14s} {r[1]:10s} {str(r[2]):>9s} {r[3]:12s} {r[4]:5d} {r[5]}")
145     print("run_dir:", run)
146 """
147
148
149 def cmd_status(ns: argparse.Namespace) -> int:
150     run_id = resolve_run_id(ns.run_id, create=False)
151     return modal_run(["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": STATUS_SNIPPET})])
152
153
154 def cmd_runs(_: argparse.Namespace) -> int:
155     return _sh([MODAL, "volume", "ls", PROJECT["volume"], "runs"]).returncode
156
157
158 def cmd_logs(ns: argparse.Namespace) -> int:
159     run_id = resolve_run_id(ns.run_id, create=False)
160     code = (
161         "from pathlib import Path\n"
162         f"p = RUN_DIR / {ns.stage!r} / 'events.jsonl'\n"
163         "print(p.read_text()[-%d:] if p.exists() else 'no events for stage')\n" % ns.tail
164     )
165     return modal_run(["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": code})])
166
167
168 def sha256(path: Path) -> str:
169     h = hashlib.sha256()
170     with path.open("rb") as fh:
171         for chunk in iter(lambda: fh.read(1 << 20), b""):
172             h.update(chunk)
173     return h.hexdigest()
174
175
176 def verify_dir(root: Path) -> int:
177     """Verify every downloaded stage directory against its manifest (transport check only)."""
178     problems = 0
179     manifests = sorted(root.rglob("manifest.json"))
180     if not manifests:
181         print(f"no manifests under {root}")
182         return 1
183     for manifest in manifests:
184         data = json.loads(manifest.read_text(encoding="utf-8"))
185         stage_dir = manifest.parent
186         bad = []
187         for rel, meta in data.get("outputs", {}).items():
188             file = stage_dir / rel
189             if not file.exists():
190                 bad.append(f"missing {rel}")
191             elif sha256(file) != meta["sha256"]:
192                 bad.append(f"sha256 mismatch {rel}")
193         status = "OK " if not bad else "BAD"
194         print(
195             f"[{status}] {data.get('stage'):14s} {data.get('status'):10s} "
196             f"files={len(data.get('outputs', {}))} {stage_dir}"
197         )
198         for b in bad:
199             print("    " + b)
200         problems += len(bad)
201     return 0 if problems == 0 else 1
202
203
204 def volume_get(remote: str, local_parent: Path) -> int:
205     """Download a remote directory into ``local_parent/<basename>`` (replacing any previous copy)."""
206     target = local_parent / Path(remote).name
207     shutil.rmtree(target, ignore_errors=True)
208     local_parent.mkdir(parents=True, exist_ok=True)
209     return _sh([MODAL, "volume", "get", "--force", PROJECT["volume"], remote, str(local_parent) + "/" ]).returncode

```

```

210
211
212 def cmd_download(ns: argparse.Namespace) -> int:
213     run_id = resolve_run_id(ns.run_id, create=False)
214     if ns.stage:
215         remote, parent = f"runs/{run_id}/{ns.stage}", REPO / ns.dest / "runs" / run_id
216     else:
217         remote, parent = f"runs/{run_id}", REPO / ns.dest / "runs"
218     rc = volume_get(remote, parent)
219     if rc != 0:
220         return rc
221     return verify_dir(parent / Path(remote).name)
222
223
224 def cmd_verify(ns: argparse.Namespace) -> int:
225     return verify_dir(Path(ns.path))
226
227
228 def cmd_release(ns: argparse.Namespace) -> int:
229     run_id = resolve_run_id(ns.run_id, create=False)
230     dest = REPO / "releases" / ns.version
231     if dest.exists() and not ns.force:
232         print(f"{dest} exists; pass --force to overwrite", file=sys.stderr)
233         return 2
234     staging = REPO / "artifacts" / "release_staging" / run_id
235     rc = volume_get(f"runs/{run_id}/release", staging)
236     if rc != 0:
237         return rc
238     shutil.rmtree(dest, ignore_errors=True)
239     shutil.copytree(staging / "release", dest)
240     manifest = json.loads((dest / "release_manifest.json").read_text(encoding="utf-8"))
241     bad = 0
242     for name, meta in manifest["files"].items():
243         file = dest / name
244         ok = file.exists() and sha256(file) == meta["sha256"]
245         bad += 0 if ok else 1
246         print(f"[{'OK' if ok else 'BAD'}] {name} {meta['bytes']} bytes sha256={meta['sha256'][:16]}")
247     print(f"release {ns.version} from run {run_id}: {'verified' if not bad else f'{bad} problems'} -> {dest}")
248     return 0 if not bad else 1
249
250
251 def cmd_wait(ns: argparse.Namespace) -> int:
252     interpreter = (
253         Path(MODAL).read_text(encoding="utf-8", errors="ignore").splitlines()[0].lstrip("#!").strip()
254         if Path(MODAL).exists()
255         else sys.executable
256     )
257     code = (
258         "import json, modal, sys\n"
259         f"call = modal.FunctionCall.from_id({ns.call_id!r})\n"
260         "print('STAGE_RESULT ' + json.dumps(call.get(), ensure_ascii=False, default=str))\n"
261     )
262     return _sh([interpreter, "-c", code]).returncode
263
264
265 def cmd_fmt(ns: argparse.Namespace) -> int:
266     """Format in the cloud, then copy the exported files back into the working tree."""
267     run_id = resolve_run_id(ns.run_id, create=False)
268     params = {"code_ref": git_ref(), "size": "small"}
269     rc = modal_run(["--stage", "fmt", "--run-id", run_id, "--params", json.dumps(params), "--force"])
270     if rc != 0:
271         return rc
272     dest = REPO / "artifacts" / "runs" / run_id / "fmt"
273     rc = volume_get(f"runs/{run_id}/fmt", dest.parent)
274     if rc != 0:
275         return rc
276     files = dest / "files"
277     changed = sorted(p for p in files.rglob("*") if p.is_file()) if files.exists() else []
278     for src in changed:
279         rel = src.relative_to(files)
280         target = REPO / rel

```

```

281         target.parent.mkdir(parents=True, exist_ok=True)
282         shutil.copy2(src, target)
283         print(f"updated {rel}")
284     report = json.loads((dest / "fmt_report.json").read_text(encoding="utf-8"))
285     print(f"{len(changed)} files formatted; remaining ruff findings: {report.get('remaining')}")
286     return 0
287
288
289 def cmd_new_run(_: argparse.Namespace) -> int:
290     run_id = new_run_id()
291     remember(run_id)
292     print(run_id)
293     return 0
294
295
296 def build_parser() -> argparse.ArgumentParser:
297     p = argparse.ArgumentParser(prog="forge", description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter
298     ↪ )
299     sub = p.add_subparsers(dest="command", required=True)
300
301     r = sub.add_parser("run", help="run one stage or a comma-separated sequence in one container")
302     r.add_argument("stage")
303     r.add_argument("--run-id")
304     r.add_argument("--new-run", action="store_true", help="start a fresh run id")
305     r.add_argument("--size", default="small", choices=["small", "medium", "large"])
306     r.add_argument("--force", action="store_true", help="re-run even if the stage already completed")
307     r.add_argument("--spawn", action="store_true", help="detach: return immediately, poll with status/wait")
308     r.add_argument("--from-run", help="copy missing dependency outputs from this earlier run")
309     r.add_argument("--profile", help="overlay configs/<profile>.toml")
310     r.add_argument("--param", action="append", help="key=value (JSON-parsed) forwarded to the stage")
311     r.set_defaults(fn=cmd_run)
312
313     e = sub.add_parser("exec", help="run an ad-hoc Python snippet in the cloud against the run directory")
314     e.add_argument("--code")
315     e.add_argument("--file")
316     e.add_argument("--run-id")
317     e.set_defaults(fn=cmd_exec)
318
319     sub.add_parser("provision", help="verify the cloud toolchain").set_defaults(fn=cmd_provision)
320
321     s = sub.add_parser("status", help="print manifests of a run")
322     s.add_argument("--run-id")
323     s.set_defaults(fn=cmd_status)
324
325     lg = sub.add_parser("logs", help="tail the event log of a stage")
326     lg.add_argument("stage")
327     lg.add_argument("--run-id")
328     lg.add_argument("--tail", type=int, default=6000)
329     lg.set_defaults(fn=cmd_logs)
330
331     sub.add_parser("runs", help="list runs on the volume").set_defaults(fn=cmd_runs)
332
333     d = sub.add_parser("download", help="download a run (or one stage) into artifacts/ and verify hashes")
334     d.add_argument("--run-id")
335     d.add_argument("--stage")
336     d.add_argument("--dest", default="artifacts")
337     d.set_defaults(fn=cmd_download)
338
339     v = sub.add_parser("verify", help="verify a downloaded directory against its manifests")
340     v.add_argument("path")
341     v.set_defaults(fn=cmd_verify)
342
343     rel = sub.add_parser("release", help="download the release stage into releases/<version> and verify")
344     rel.add_argument("--version", required=True)
345     rel.add_argument("--run-id")
346     rel.add_argument("--force", action="store_true")
347     rel.set_defaults(fn=cmd_release)
348
349     w = sub.add_parser("wait", help="block on a spawned function call id and print its result")
350     w.add_argument("call_id")
351     w.set_defaults(fn=cmd_wait)

```

```

351
352     f = sub.add_parser("fmt", help="ruff --fix + format in the cloud and copy the results back")
353     f.add_argument("--run-id")
354     f.set_defaults(fn=cmd_fmt)
355
356     sub.add_parser("new-run", help="mint a run id").set_defaults(fn=cmd_new_run)
357     return p
358
359
360 def main(argv: list[str] | None = None) -> int:
361     os.chdir(REPO)
362     ns = build_parser().parse_args(argv)
363     return int(ns.fn(ns))
364
365
366 if __name__ == "__main__":
367     sys.exit(main())

```

tools/run_local.py

```

1  #!/usr/bin/env python3
2  """Run the Problem-A pipeline on this machine, without Modal.
3
4  The reference execution of this repository is in the cloud (`tools/cli.py`), but an evaluator has no
5  account there. This script is the local equivalent: it calls exactly the same stage functions, in the
6  same order, through the same runner, writing into a local directory instead of a cloud volume.
7
8      python3 tools/run_local.py                # ingest -> ... -> tables, into ./local_runs/<id>
9      python3 tools/run_local.py --stages ingest,validate --out /tmp/a
10     python3 tools/run_local.py --quick        # reduced budgets: minutes rather than hours
11
12  Requirements (pip install): pandas, numpy, scipy, openpyxl, pyarrow, matplotlib.
13  The organisers' attachments must be present in ``附件/`` exactly as delivered; nothing else is needed.
14  Typesetting stages (`paper`/`qa`/`package`/`release`) need a TeX Live installation with XeLaTeX
15  and the CJK fonts, so they are not part of the default list here — the scientific stages are.
16  """
17
18  from __future__ import annotations
19
20  import argparse
21  import json
22  import sys
23  import time
24  from datetime import UTC, datetime
25  from pathlib import Path
26
27  REPO = Path(_file_).resolve().parents[1]
28  DEFAULT_STAGES = (
29      "ingest,validate,q1,q2,q3,q4,convergence,verification,sensitivity,extension,derived,results,figures,tables"
30  )
31  # --quick lowers the radial refinement and the ODE tolerances through ``config_overrides`` (the same
32  # mechanism as ``--profile``), so the whole chain finishes in minutes. Structure of every table and figure
33  # is reproduced; the fourth decimal is not — the published numbers used refine = 5 (docs/RUNBOOK.md).
34  _QUICK_CONFIG = {
35      "a": {
36          "grid": {"refine": 3, "convergence_levels": 4},
37          "time": {"rtol": 1e-7, "atol": 1e-9},
38          "verification": {"axisym_refine": 2, "axisym_nz": 24},
39      }
40  }
41  QUICK = {
42      s: {"config_overrides": _QUICK_CONFIG}
43      for s in (
44          "q1",
45          "q2",
46          "q3",
47          "q4",
48          "convergence",

```

```

49     "verification",
50     "sensitivity",
51     "extension",
52     "derived",
53     "results",
54     "figures",
55     "tables",
56 )
57 }
58
59
60 def main() -> int:
61     ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
62     ap.add_argument("--stages", default=DEFAULT_STAGES, help=f"comma-separated (default: {DEFAULT_STAGES})")
63     ap.add_argument("--out", default=str(REPO / "local_runs"), help="directory that plays the role of the volume")
64     ap.add_argument("--run-id", default=None, help="reuse an existing run id instead of creating one")
65     ap.add_argument("--quick", action="store_true", help="small solver budgets (minutes instead of hours)")
66     ap.add_argument("--force", action="store_true", help="re-run stages that already completed")
67     ap.add_argument("--param", action="append", default=[], help="stage param, e.g. workers=4 or all:workers=4")
68     ns = ap.parse_args()
69
70     sys.path.insert(0, str(REPO))
71     import pipelines.a.stages # noqa: F401 - registers the stages
72     from forge.runner import execute
73
74     vol = Path(ns.out).resolve()
75     run_id = ns.run_id or datetime.now(UTC).strftime("%Y%m%d-%H%M%S-local")
76     (vol / "runs" / run_id).mkdir(parents=True, exist_ok=True)
77     extra: dict[str, dict[str, object]] = {}
78     for item in ns.param:
79         scope, _, rest = item.partition(":") if ":" in item else ("all", "", item)
80         key, _, value = rest.partition("=")
81         try:
82             parsed = json.loads(value)
83         except json.JSONDecodeError:
84             parsed = value
85         extra.setdefault(scope, {})[key] = parsed
86
87     print(f"repo={REPO}\nvolume={vol}\nrun_id={run_id}")
88     for stage in [s.strip() for s in ns.stages.split(",") if s.strip()]:
89         params: dict[str, object] = {"size": "local"}
90         if ns.quick:
91             params.update(QUICK.get(stage, {}))
92         params.update(extra.get("all", {}))
93         params.update(extra.get(stage, {}))
94         t0 = time.monotonic()
95         result = execute(stage_name=stage, run_id=run_id, params=params, force=ns.force, repo=REPO, vol=vol)
96         status = result.get("status")
97         print(f"[{time.monotonic() - t0:8.1f}s] {stage:18s} {status}")
98         if status == "failed":
99             print(json.dumps(result.get("error", {}), ensure_ascii=False, indent=2))
100         return 1
101     print(f"\nresults: {vol / 'runs' / run_id / 'results'} (result1-4.xlsx)")
102     print(f"figures: {vol / 'runs' / run_id / 'figures' / 'figures'}")
103     print(f"tables: {vol / 'runs' / run_id / 'tables' / 'tables'}")
104     return 0
105
106
107 if __name__ == "__main__":
108     raise SystemExit(main())

```